```
========================================
FILE: nphies_integration_test.js
========================================

/**
 * nphies_integration_test.js
 * Integration tests for NPHIES HL7 FHIR Bundle generation, mapping, and API client calls.
 */
'use strict';

const Database = require('better-sqlite3');
const express = require('express');
const http = require('http');
const N = require('./nphies_client');

let pass = 0, fail = 0;
function ok(name, cond) {
    if (cond) {
        pass++;
        console.log(`  PASS: ${name}`);
    } else {
        fail++;
        console.error(`  FAIL: ${name}`);
    }
}

function buildMockPool(db) {
    return {
        query(sql, params = []) {
            let sqliteSql = sql;
            if (params.length > 0) {
                sqliteSql = sql.replace(/\$(\d+)/g, '?');
            }
            try {
                if (sql.trim().toLowerCase().startsWith('select')) {
                    const stmt = db.prepare(sqliteSql);
                    const rows = stmt.all(...params);
                    return Promise.resolve({ rows });
                } else {
                    const stmt = db.prepare(sqliteSql);
                    const info = stmt.run(...params);
                    return Promise.resolve({ rows: [], lastInsertRowid: info.lastInsertRowid, affectedRows: in
fo.changes });
                }
            } catch (err) {
                return Promise.reject(err);
            }
        },
        connect() {
            return Promise.resolve({
                query: (sql, params = []) => this.query(sql, params),
                release: () => {}
```

```
            });
        }
    };
}

async function req(port, method, path, body = null, headers = {}) {
    return new Promise((resolve) => {
        const options = {
            host: '127.0.0.1',
            port,
            method,
            path,
            headers: {
                'Content-Type': 'application/json',
                ...headers
            }
        };
        const r = http.request(options, (res) => {
            let resBody = '';
            res.on('data', d => resBody += d);
            res.on('end', () => {
                let json;
                try { json = JSON.parse(resBody); } catch { json = {}; }
                resolve({ status: res.statusCode, json });
            });
        });
        if (body) {
            r.write(JSON.stringify(body));
        }
        r.end();
    });
}

(async () => {
    console.log('Running NPHIES HL7 FHIR Integration Tests...');

    // 1. Setup DB
    const db = new Database(':memory:');

    db.exec(`
        CREATE TABLE IF NOT EXISTS integration_settings (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            tenant_id INTEGER,
            integration_name TEXT,
            provider TEXT,
            api_key TEXT,
            api_secret TEXT,
            endpoint_url TEXT,
            is_enabled INTEGER DEFAULT 0,
            config_json TEXT DEFAULT '{}',
            last_sync TEXT
        )
    `);
```

```
db.exec(`
    CREATE TABLE IF NOT EXISTS patients (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        name_en TEXT,
        national_id TEXT,
        gender TEXT,
        dob TEXT,
        tenant_id INTEGER
    )
`);

db.exec(`
    CREATE TABLE IF NOT EXISTS insurance_companies (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        name_en TEXT,
        contact_info TEXT,
        tenant_id INTEGER
    )
`);

db.exec(`
    CREATE TABLE IF NOT EXISTS insurance_eligibility_checks (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        tenant_id INTEGER NOT NULL,
        patient_id INTEGER,
        insurance_company_id INTEGER,
        policy_number TEXT DEFAULT '',
        status TEXT NOT NULL DEFAULT 'pending',
        coverage_amount REAL DEFAULT 0,
        nphies_request_json TEXT DEFAULT '',
        nphies_response_json TEXT DEFAULT '',
        checked_by INTEGER,
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
    )
`);

db.exec(`
    CREATE TABLE IF NOT EXISTS insurance_pre_authorizations (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        tenant_id INTEGER NOT NULL,
        patient_id INTEGER,
        admission_id INTEGER,
        insurance_company_id INTEGER,
        requested_amount REAL DEFAULT 0,
        approved_amount REAL DEFAULT 0,
        auth_status TEXT NOT NULL DEFAULT 'requested',
        auth_number TEXT DEFAULT '',
        clinical_justification TEXT DEFAULT '',
        nphies_request_json TEXT DEFAULT '',
        nphies_response_json TEXT DEFAULT '',
        requested_by INTEGER,
        decided_at TIMESTAMP,
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
    )
```

```
    `);

    db.exec(`
        CREATE TABLE IF NOT EXISTS insurance_claims (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            tenant_id INTEGER NOT NULL,
            patient_id INTEGER,
            insurance_company_id INTEGER,
            claim_amount REAL DEFAULT 0,
            lifecycle_status TEXT NOT NULL DEFAULT 'draft',
            nphies_request_json TEXT DEFAULT '',
            nphies_response_json TEXT DEFAULT '',
            submitted_at TIMESTAMP,
            created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
        )
    `);

    db.exec(`
        CREATE TABLE IF NOT EXISTS insurance_claim_lines (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            tenant_id INTEGER NOT NULL,
            claim_id INTEGER,
            service_id INTEGER,
            description TEXT DEFAULT '',
            quantity REAL DEFAULT 1,
            unit_price REAL DEFAULT 0,
            line_amount REAL DEFAULT 0,
            approved_amount REAL DEFAULT 0
        )
    `);

    // Insert mock rows
    db.prepare("INSERT INTO patients (name_en, national_id, gender, dob, tenant_id) VALUES (?, ?, ?, ?, ?)").r
un('Ahmad', '1000000001', 'male', '1990-01-01', 1);
    db.prepare("INSERT INTO insurance_companies (name_en, contact_info, tenant_id) VALUES (?, ?, ?)").run('Taw
uniya', '920001234', 1);
    db.prepare("INSERT INTO insurance_claims (tenant_id, patient_id, insurance_company_id, claim_amount, lifec
ycle_status) VALUES (?, ?, ?, ?, ?)").run(1, 1, 1, 250.00, 'draft');
    db.prepare("INSERT INTO insurance_claim_lines (tenant_id, claim_id, description, quantity, unit_price, lin
e_amount) VALUES (?, ?, ?, ?, ?, ?)").run(1, 1, 'Consultation', 1, 250.00, 250.00);

    // 2. Setup Express App
    const app = express();
    app.use(express.json());

    const requireAuth = (req, res, next) => {
        req.session = { user: { id: 10, display_name: 'Test Admin', role: 'Admin' } };
        next();
    };

    const requireTenantScope = (req, res, next) => {
        req.tenantId = 1;
        next();
    };
```

```
    const E11_INS_ROLES = ['insurance', 'finance'];
    const e11RequireTenant = (req) => req.tenantId;
    const e11IntId = (v) => parseInt(v, 10);
    const e11Money = (v) => parseFloat(v);
    const e11Err = (res, e) => res.status(500).json({ error: e.message });
    const logAudit = () => {};
    const e11NphiesEnabled = () => true;

    const e11Engine = {
        canTransitionClaim: (from, to) => from === 'draft' && to === 'submitted'
    };

    const pool = buildMockPool(db);

    // ELIGIBILITY POST ROUTE (copied from server.js modifications)
    app.post('/api/insurance/eligibility', requireAuth, requireTenantScope, async (req, res) => {
        try {
            const tenantId = e11RequireTenant(req);
            const patientId = e11IntId(req.body.patient_id);
            const companyId = e11IntId(req.body.insurance_company_id);
            const policyNumber = String(req.body.policy_number || '');

            const ins = await pool.query(
                `INSERT INTO insurance_eligibility_checks (tenant_id, patient_id, insurance_company_id, policy
_number, status, nphies_request_json, checked_by)
                 VALUES ($1,$2,$3,$4,'pending',$5,$6) RETURNING id`,
                [tenantId, patientId, companyId, policyNumber, '{}', req.session.user.id]);
            const checkId = ins.rows[0]?.id || 1; // RETURNING not fully supported in in-memory simple mock, l
et's fallback to last row id if needed
            const realCheckId = ins.lastInsertRowid || checkId;

            const settings = (await pool.query('SELECT * FROM integration_settings WHERE tenant_id=$1 AND inte
gration_name=$2', [tenantId, 'NPHIES'])).rows[0];
            const isNphiesEnabled = e11NphiesEnabled() && settings && settings.is_enabled === 1 && settings.ap
i_key && settings.api_secret && settings.endpoint_url;

            if (!isNphiesEnabled) {
                return res.status(503).json({ error: 'NPHIES integration disabled', gated: true, eligibility_i
d: realCheckId, status: 'pending' });
            }

            const patient = patientId ? (await pool.query('SELECT * FROM patients WHERE id=$1 AND tenant_id=$2
', [patientId, tenantId])).rows[0] : null;
            const company = companyId ? (await pool.query('SELECT * FROM insurance_companies WHERE id=$1 AND t
enant_id=$2', [companyId, tenantId])).rows[0] : null;

            const bundle = N.buildEligibilityBundle({ patient, company, policy: policyNumber });
            const mockFetch = async () => ({
                ok: true,
                status: 200,
                text: async () => JSON.stringify({ outcome: 'complete', disposition: 'Eligible' })
            });
```

```
            const client = new N.NphiesClient({
                endpointUrl: settings.endpoint_url,
                apiKey: settings.api_key,
                apiSecret: settings.api_secret,
                enabled: true,
                fetchImpl: mockFetch
            });

            const result = await client.checkEligibility(bundle);
            const finalStatus = result.ok ? 'eligible' : 'ineligible';
            const responseJson = JSON.stringify(result.body);

            await pool.query(
                `UPDATE insurance_eligibility_checks
                 SET status=$1, nphies_request_json=$2, nphies_response_json=$3
                 WHERE id=$4 AND tenant_id=$5`,
                [finalStatus, JSON.stringify(bundle), responseJson, realCheckId, tenantId]
            );

            res.json({ success: result.ok, eligibility_id: realCheckId, status: finalStatus, response: result.
body });
        } catch (e) { e11Err(res, e); }
    });

    // PRE-AUTH POST ROUTE (copied from server.js modifications)
    app.post('/api/insurance/pre-auth', requireAuth, requireTenantScope, async (req, res) => {
        try {
            const tenantId = e11RequireTenant(req);
            const patientId = e11IntId(req.body.patient_id);
            const companyId = e11IntId(req.body.insurance_company_id);
            const requested = e11Money(req.body.requested_amount) || 0;

            const ins = await pool.query(
                `INSERT INTO insurance_pre_authorizations (tenant_id, patient_id, insurance_company_id, reques
ted_amount, auth_status, clinical_justification, nphies_request_json, requested_by)
                 VALUES ($1,$2,$3,$4,'requested','', '[]', $5) RETURNING id`,
                [tenantId, patientId, companyId, requested, req.session.user.id]);
            const realPaId = ins.lastInsertRowid || 1;

            const settings = (await pool.query('SELECT * FROM integration_settings WHERE tenant_id=$1 AND inte
gration_name=$2', [tenantId, 'NPHIES'])).rows[0];
            const isNphiesEnabled = e11NphiesEnabled() && settings && settings.is_enabled === 1 && settings.ap
i_key && settings.api_secret && settings.endpoint_url;

            if (!isNphiesEnabled) {
                return res.status(503).json({ error: 'NPHIES integration disabled', gated: true, pre_auth_id:
realPaId, auth_status: 'requested' });
            }

            const patient = patientId ? (await pool.query('SELECT * FROM patients WHERE id=$1 AND tenant_id=$2
', [patientId, tenantId])).rows[0] : null;
            const company = companyId ? (await pool.query('SELECT * FROM insurance_companies WHERE id=$1 AND t
enant_id=$2', [companyId, tenantId])).rows[0] : null;
```

```
            const preAuthRecord = { id: realPaId, requested_amount: requested };
            const bundle = N.buildPreAuthBundle({ patient, company, preAuth: preAuthRecord });

            const mockFetch = async () => ({
                ok: true,
                status: 200,
                text: async () => JSON.stringify({ outcome: 'complete', disposition: 'Approved' })
            });

            const client = new N.NphiesClient({
                endpointUrl: settings.endpoint_url,
                apiKey: settings.api_key,
                apiSecret: settings.api_secret,
                enabled: true,
                fetchImpl: mockFetch
            });

            const result = await client.requestPreAuth(bundle);
            const authStatus = result.ok ? 'approved' : 'denied';
            const responseJson = JSON.stringify(result.body);

            await pool.query(
                `UPDATE insurance_pre_authorizations
                 SET auth_status=$1, nphies_request_json=$2, nphies_response_json=$3, decided_at=CURRENT_TIMES
TAMP
                 WHERE id=$4 AND tenant_id=$5`,
                [authStatus, JSON.stringify(bundle), responseJson, realPaId, tenantId]
            );

            res.json({ success: result.ok, pre_auth_id: realPaId, auth_status: authStatus, response: result.bo
dy });
        } catch (e) { e11Err(res, e); }
    });

    // CLAIM SUBMISSION POST ROUTE (copied from server.js modifications)
    app.post('/api/nphies/submit-claim/:id', requireAuth, requireTenantScope, async (req, res) => {
        const client = await pool.connect();
        try {
            const tenantId = e11RequireTenant(req);
            const claimId = e11IntId(req.params.id);
            if (!claimId) return res.status(400).json({ error: 'Invalid claim id' });

            const cur = await client.query('SELECT id, lifecycle_status FROM insurance_claims WHERE id=$1 AND
tenant_id=$2', [claimId, tenantId]);
            if (!cur.rows.length) { return res.status(404).json({ error: 'Claim not found' }); }
            if (!e11Engine.canTransitionClaim(cur.rows[0].lifecycle_status, 'submitted')) {
                return res.status(409).json({ error: `Cannot submit from ${cur.rows[0].lifecycle_status}` });
            }
            const intent = JSON.stringify({ claim_id: claimId, ts: new Date().toISOString() });
            await client.query('UPDATE insurance_claims SET nphies_request_json=$1 WHERE id=$2 AND tenant_id=$
3', [intent, claimId, tenantId]);

            const settings = (await client.query('SELECT * FROM integration_settings WHERE tenant_id=$1 AND in
tegration_name=$2', [tenantId, 'NPHIES'])).rows[0];
```

```
            const isNphiesEnabled = e11NphiesEnabled() && settings && settings.is_enabled === 1 && settings.ap
i_key && settings.api_secret && settings.endpoint_url;

            if (!isNphiesEnabled) {
                logAudit(req.session.user.id, req.session.user.display_name, 'NPHIES_SUBMIT_GATED', 'Insurance
', `Claim #${claimId} submit intent (NPHIES off)`, req.ip);
                return res.status(503).json({ error: 'NPHIES integration disabled', gated: true, claim_id: cla
imId });
            }

            const claim = (await client.query('SELECT * FROM insurance_claims WHERE id=$1 AND tenant_id=$2', [
claimId, tenantId])).rows[0];
            const patient = (await client.query('SELECT * FROM patients WHERE id=$1 AND tenant_id=$2', [claim.
patient_id, tenantId])).rows[0];
            const company = (await client.query('SELECT * FROM insurance_companies WHERE id=$1 AND tenant_id=$
2', [claim.insurance_company_id, tenantId])).rows[0];
            const lines = (await client.query('SELECT * FROM insurance_claim_lines WHERE claim_id=$1 AND tenan
t_id=$2', [claimId, tenantId])).rows;

            const bundle = N.buildClaimBundle({ patient, company, claim, lines });

            const mockFetch = async () => ({
                ok: true,
                status: 200,
                text: async () => JSON.stringify({ outcome: 'complete', disposition: 'Claim Accepted' })
            });

            const nClient = new N.NphiesClient({
                endpointUrl: settings.endpoint_url,
                apiKey: settings.api_key,
                apiSecret: settings.api_secret,
                enabled: true,
                fetchImpl: mockFetch
            });

            const result = await nClient.submitClaim(bundle);
            const nextStatus = result.ok ? 'submitted' : 'denied';
            const responseJson = JSON.stringify(result.body);

            await client.query(
                `UPDATE insurance_claims
                 SET lifecycle_status=$1, submitted_at=CURRENT_TIMESTAMP, nphies_request_json=$2, nphies_respo
nse_json=$3
                 WHERE id=$4 AND tenant_id=$5`,
                [nextStatus, JSON.stringify(bundle), responseJson, claimId, tenantId]
            );

            res.json({ success: result.ok, claim_id: claimId, lifecycle_status: nextStatus, response: result.b
ody });
        } catch (e) { return e11Err(res, e); }
        finally { client.release(); }
    });

    // Start server
```

```
    const srv = app.listen(0);
    await new Promise(r => srv.once('listening', r));
    const port = srv.address().port;

    // Test 1: Check Eligibility with disabled settings (Fallback gated stub)
    let resEligMock = await req(port, 'POST', '/api/insurance/eligibility', { patient_id: 1, insurance_company
_id: 1, policy_number: 'POL-123' });
    ok('Eligibility returns 503 with mock fallback', resEligMock.status === 503);
    ok('Eligibility gated is true', resEligMock.json.gated === true);

    // Seed NPHIES settings
    db.prepare(`
        INSERT INTO integration_settings (tenant_id, integration_name, provider, api_key, api_secret, endpoint
_url, is_enabled)
        VALUES (?, ?, ?, ?, ?, ?, ?)
    `).run(1, 'NPHIES', 'Mock NPHIES Platform', 'nphies-key', 'nphies-secret', 'http://localhost/fhir', 1);

    // Test 2: Check Eligibility with active settings (Real FHIR mapping & client call)
    let resEligReal = await req(port, 'POST', '/api/insurance/eligibility', { patient_id: 1, insurance_company
_id: 1, policy_number: 'POL-123' });
    ok('Eligibility returns 200 with active settings', resEligReal.status === 200);
    ok('Eligibility status is eligible', resEligReal.json.status === 'eligible');

    // Test 3: Pre-auth request with active settings
    let resPaReal = await req(port, 'POST', '/api/insurance/pre-auth', { patient_id: 1, insurance_company_id:
1, requested_amount: 1500.00 });
    ok('Pre-auth returns 200 with active settings', resPaReal.status === 200);
    ok('Pre-auth status is approved', resPaReal.json.auth_status === 'approved');

    // Test 4: Submit Claim with active settings
    let resClaimReal = await req(port, 'POST', '/api/nphies/submit-claim/1', {});
    ok('Submit Claim returns 200 with active settings', resClaimReal.status === 200);
    ok('Claim lifecycle_status is submitted', resClaimReal.json.lifecycle_status === 'submitted');

    // Retrieve database records to check outputs
    const dbElig = db.prepare("SELECT * FROM insurance_eligibility_checks WHERE id = 2").get();
    ok('Eligibility check request JSON mapped to FHIR Patient & Coverage', JSON.parse(dbElig.nphies_request_js
on).resourceType === 'Bundle');
    ok('Eligibility check response JSON saved', JSON.parse(dbElig.nphies_response_json).disposition === 'Eligi
ble');

    const dbPa = db.prepare("SELECT * FROM insurance_pre_authorizations WHERE id = 1").get();
    ok('Pre-auth request JSON mapped to FHIR PreAuth Bundle', JSON.parse(dbPa.nphies_request_json).resourceTyp
e === 'Bundle');
    ok('Pre-auth status updated to approved in DB', dbPa.auth_status === 'approved');

    const dbClaim = db.prepare("SELECT * FROM insurance_claims WHERE id = 1").get();
    ok('Claim request JSON mapped to FHIR Claim Bundle', JSON.parse(dbClaim.nphies_request_json).resourceType
=== 'Bundle');
    ok('Claim status updated to submitted in DB', dbClaim.lifecycle_status === 'submitted');

    srv.close();
    console.log(`NPHIES HL7 FHIR Integration Tests: ${pass} passed, ${fail} failed`);
    process.exit(fail === 0 ? 0 : 1);
```

```
})();



========================================
FILE: nphies_ksa_test.js
========================================

/**
 * nphies_ksa_test.js ? PURE unit test for Gate 8 (NPHIES KSA-conformant FHIR message bundles).
 * Run: node nphies_ksa_test.js   (no DB, no network; deterministic via injected `now`).
 *
 * Verifies the STRUCTURAL KSA-messaging requirements the previous minimal `collection`
 * bundles were missing: Bundle.type=message, a MessageHeader focus resource with an
 * NPHIES event code, urn:uuid fullUrls with references resolved to them, meta.profile on
 * every resource, and NPHIES/ICD-10-AM/SBS terminology systems. (Exact profile version
 * pins must still be confirmed against the live NPHIES IG before go-live ? the builder
 * documents this.)
 */
'use strict';
const N = require('./nphies_client');

const GREEN = '\x1b[32m', RED = '\x1b[31m', RESET = '\x1b[0m', BOLD = '\x1b[1m';
let passed = 0, failed = 0; const fails = [];
function assert(cond, name, det = '') {
  if (cond) { console.log(`  ${GREEN}PASS${RESET} ${name}`); passed++; }
  else { console.log(`  ${RED}FAIL${RESET} ${name}${det ? ' | ' + det : ''}`); failed++; fails.push(name); }
}
const NOW = '2026-07-03T09:00:00.000Z';
const patient = { id: 42, national_id: '1012345678', name_en: 'Sara', gender: 'female', dob: '1992-05-01' };
const company = { id: 7, name_en: 'Bupa Arabia' };
const entryOfType = (b, t) => (b.entry || []).find(e => e.resource && e.resource.resourceType === t);

console.log(`${BOLD}Gate 8 ? NPHIES KSA message-bundle conformance (pure unit test)${RESET}\n`);

// ---- Eligibility message ----
console.log('[1] buildEligibilityMessage ? message bundle shape');
let b = N.buildEligibilityMessage({ patient, company, policy: 'POL-1', now: NOW });
assert(b.resourceType === 'Bundle' && b.type === 'message', 'type=message', b.type);
assert(b.timestamp === NOW, 'bundle.timestamp set from now', b.timestamp);
assert(b.identifier && typeof b.identifier.value === 'string', 'bundle.identifier present');
assert(b.entry[0].resource.resourceType === 'MessageHeader', 'first entry is MessageHeader', b.entry[0].resour
ce.resourceType);
let mh = b.entry[0].resource;
assert(mh.eventCoding && /ksa-message-events/.test(mh.eventCoding.system) && mh.eventCoding.code === 'eligibil
ity-request',
  'MessageHeader eventCoding = eligibility-request', JSON.stringify(mh.eventCoding));
assert(mh.source && mh.source.endpoint, 'MessageHeader has source.endpoint');
assert(Array.isArray(mh.destination) && mh.destination.length >= 1, 'MessageHeader has destination');
// every entry has a urn:uuid fullUrl
assert(b.entry.every(e => typeof e.fullUrl === 'string' && e.fullUrl.startsWith('urn:uuid:')), 'all entries ha
ve urn:uuid fullUrl');
// MessageHeader.focus points at the CoverageEligibilityRequest fullUrl
const cerEntry = b.entry.find(e => e.resource.resourceType === 'CoverageEligibilityRequest');
```

```
assert(mh.focus && mh.focus[0].reference === cerEntry.fullUrl, 'MessageHeader.focus -> CER fullUrl', JSON.stri
ngify(mh.focus));
// references resolved to fullUrl (not "Patient/42")
const patEntry = entryOfType(b, 'Patient');
assert(cerEntry.resource.patient.reference === patEntry.fullUrl, 'CER.patient reference is the Patient fullUrl
');
assert(!/^Patient\//.test(cerEntry.resource.patient.reference), 'reference is NOT relative ResourceType/id');
// every resource carries meta.profile
assert(b.entry.every(e => e.resource.meta && Array.isArray(e.resource.meta.profile) && e.resource.meta.profile
.length > 0),
  'every resource has meta.profile');
assert(/nphies\.sa/.test(patEntry.resource.meta.profile[0]), 'profile URL is an nphies.sa StructureDefinition'
);
// KSA identifier systems
assert(/nphies\.sa/.test(patEntry.resource.identifier[0].system), 'patient national id system is nphies.sa');
const provEntry = (b.entry.map(e => e.resource).filter(r => r.resourceType === 'Organization'))
  .find(o => (o.identifier || []).some(i => /provider/.test(i.system)));
assert(!!provEntry, 'provider Organization has provider-license identifier');

// determinism
let b2 = N.buildEligibilityMessage({ patient, company, policy: 'POL-1', now: NOW });
assert(JSON.stringify(b) === JSON.stringify(b2), 'deterministic for same inputs+now (stable fullUrls)');

// ---- PreAuth message ----
console.log('\n[2] buildPreAuthMessage ? priorauth event + Claim focus');
b = N.buildPreAuthMessage({ patient, company, preAuth: { id: 5, amount: 1200 }, now: NOW });
mh = b.entry[0].resource;
assert(b.type === 'message' && mh.eventCoding.code === 'priorauth-request', 'event=priorauth-request', JSON.st
ringify(mh.eventCoding));
const claimP = entryOfType(b, 'Claim');
assert(claimP && claimP.resource.use === 'preauthorization', "Claim.use=preauthorization", claimP && claimP.re
source.use);
assert(mh.focus[0].reference === claimP.fullUrl, 'focus -> Claim fullUrl');
assert(/nphies\.sa\/terminology/.test(claimP.resource.type.coding[0].system), 'claim type uses NPHIES system')
;

// ---- Claim message with diagnoses + SBS items ----
console.log('\n[3] buildClaimMessage ? ICD-10-AM diagnoses + SBS items');
b = N.buildClaimMessage({
  patient, company,
  claim: { id: 9, claim_amount: 500 },
  lines: [{ description: 'CBC', quantity: 1, unit_price: 500, line_amount: 500, sbs_code: '90001-00-10' }],
  diagnoses: [{ icd10: 'J06.9', description: 'Acute URI' }],
  now: NOW
});
mh = b.entry[0].resource;
assert(b.type === 'message' && mh.eventCoding.code === 'claim-request', 'event=claim-request', JSON.stringify(
mh.eventCoding));
const claimC = entryOfType(b, 'Claim').resource;
assert(claimC.use === 'claim', 'Claim.use=claim');
assert(Array.isArray(claimC.diagnosis) && /icd-10-am/.test(claimC.diagnosis[0].diagnosisCodeableConcept.coding
[0].system),
  'diagnosis uses ICD-10-AM system', JSON.stringify(claimC.diagnosis && claimC.diagnosis[0]));
assert(claimC.diagnosis[0].diagnosisCodeableConcept.coding[0].code === 'J06.9', 'diagnosis code carried throug
```

```
h');
assert(/procedures|sbs/i.test(claimC.item[0].productOrService.coding[0].system), 'item productOrService uses S
BS/procedures system',
  JSON.stringify(claimC.item[0].productOrService));
assert(claimC.item[0].productOrService.coding[0].code === '90001-00-10', 'SBS code carried through');
assert(claimC.total.currency === 'SAR' && claimC.total.value === 500, 'total in SAR');
// items missing an sbs_code must NOT fabricate a wrong code ? fall back to text only (fail-safe)
b = N.buildClaimMessage({ patient, company, claim: { id: 9, claim_amount: 100 },
  lines: [{ description: 'Unmapped service', quantity: 1, unit_price: 100, line_amount: 100 }], diagnoses: [],
 now: NOW });
const item0 = entryOfType(b, 'Claim').resource.item[0];
assert(!item0.productOrService.coding && item0.productOrService.text === 'Unmapped service',
  'no SBS code -> text only, no fabricated coding', JSON.stringify(item0.productOrService));

// ---- back-compat: old collection builders still exported ----
console.log('\n[4] legacy collection builders remain (back-compat)');
assert(typeof N.buildEligibilityBundle === 'function' && typeof N.buildClaimBundle === 'function', 'legacy bui
lders still exported');

console.log(`\n${BOLD}Result:${RESET} ${passed} passed, ${failed} failed`);
if (failed) { console.log(`${RED}FAILURES:${RESET} ${fails.join(', ')}`); process.exit(1); }
process.exit(0);
```

```
========================================
FILE: nursing_braden_morse_test.js
========================================
```

```
/**
 * nursing_braden_morse_test.js ? Integration test for Braden Scale & Morse Fall Risk assessments.
 */
'use strict';

process.env.NODE_ENV = 'staging';

const { spawn } = require('child_process');
const http = require('http');
const assert = require('assert');
const bcrypt = require('bcryptjs');
const { pool } = require('./db_postgres');

const TEST_PORT = 3022;
const TEST_USERNAME = 'nursing_tester';
const TEST_PASSWORD = 'NURSE_PASSWORD_PLACEHOLDER';

let serverProcess;

function makeRequest(method, path, body, headers = {}) {
    return new Promise((resolve, reject) => {
        const payload = body ? JSON.stringify(body) : '';
        const req = http.request({
            hostname: 'localhost',
            port: TEST_PORT,
```

```
            path,
            method,
            headers: {
                'Content-Type': 'application/json',
                'Content-Length': Buffer.byteLength(payload),
                ...headers
            }
        }, (res) => {
            let data = '';
            res.on('data', chunk => data += chunk);
            res.on('end', () => {
                try {
                    const parsed = data ? JSON.parse(data) : {};
                    resolve({ statusCode: res.statusCode, headers: res.headers, body: parsed });
                } catch (e) {
                    resolve({ statusCode: res.statusCode, headers: res.headers, rawBody: data });
                }
            });
        });
        req.on('error', reject);
        if (payload) req.write(payload);
        req.end();
    });
}

async function runTests() {
    console.log('--- STARTING NURSING RISK ASSESSMENT INTEGRATION TESTS ---');

    const patientId = 9951;
    const nurseUserId = 9952;
    const client = await pool.connect();

    try {
        console.log('Setting up test data...');
        await client.query("SET app.tenant_id = '1'");

        // Clean up old test data
        await client.query('DELETE FROM nursing_risk_assessments WHERE patient_id = $1', [patientId]);
        await client.query('DELETE FROM patients WHERE id = $1', [patientId]);
        await client.query('DELETE FROM user_tenants WHERE user_id = $1', [nurseUserId]);
        await client.query('DELETE FROM system_users WHERE id = $1', [nurseUserId]);

        // Insert patient
        await client.query(
            'INSERT INTO patients (id, name_en, name_ar, tenant_id) VALUES ($1, $2, $3, 1)',
            [patientId, 'Nursing Risk Patient', '???? ????? ???????']
        );

        // Insert nurse user (Role: Nurse)
        const hashedPassword = await bcrypt.hash(TEST_PASSWORD, 10);
        await client.query(
            "INSERT INTO system_users (id, username, password_hash, display_name, role, permissions, is_active
) VALUES ($1, $2, $3, $4, $5, $6, 1)",
            [nurseUserId, TEST_USERNAME, hashedPassword, 'Head Nurse Aisha', 'Nurse', '["patients"]']
```

```
);

// Associate nurse with tenant 1
await client.query(
    'INSERT INTO user_tenants (user_id, tenant_id, is_active) VALUES ($1, 1, true)',
    [nurseUserId]
);

console.log('Spawning test server...');
serverProcess = spawn('node', ['server.js'], {
    env: { ...process.env, PORT: TEST_PORT, NODE_ENV: 'staging', SKIP_DB_INIT: 'true' }
});

serverProcess.stdout.on('data', (data) => {
    if (process.env.DEBUG_TESTS) console.log(`[Server STDOUT] ${data.toString().trim()}`);
});

// Wait for server to start
await new Promise(resolve => setTimeout(resolve, 3000));

console.log('Logging in...');
const loginRes = await makeRequest('POST', '/api/auth/login', {
    username: TEST_USERNAME,
    password: TEST_PASSWORD
});
assert.strictEqual(loginRes.statusCode, 200, 'Login should succeed');
const cookie = loginRes.headers['set-cookie'][0];

// Test 1: Create Braden Scale Risk Assessment
console.log('Testing create Braden Scale assessment...');
const details = {
    sensory_perception: 3,
    moisture: 4,
    activity: 2,
    mobility: 3,
    nutrition: 3,
    friction_shear: 2
};
const createRes = await makeRequest('POST', '/api/nursing/risk-assessment', {
    patient_id: patientId,
    admission_id: 101,
    assessment_type: 'Braden Scale',
    total_score: 17,
    risk_level: 'Mild Risk',
    details
}, { Cookie: cookie });

assert.strictEqual(createRes.statusCode, 200);
assert.strictEqual(createRes.body.assessment_type, 'Braden Scale');
assert.strictEqual(createRes.body.total_score, 17);
assert.strictEqual(createRes.body.risk_level, 'Mild Risk');
console.log(`? Braden Scale risk assessment created. ID: ${createRes.body.id}`);

// Test 2: Get Patient Risk Assessments
```

```
        console.log('Testing get patient risk assessments...');
        const getRes = await makeRequest('GET', `/api/nursing/risk-assessments/${patientId}`, null, { Cookie:
cookie });
        assert.strictEqual(getRes.statusCode, 200);
        assert.ok(getRes.body.length > 0);
        assert.strictEqual(getRes.body[0].total_score, 17);
        console.log('? Risk assessments retrieved successfully.');

        console.log('Cleaning up test data...');
        await client.query('DELETE FROM nursing_risk_assessments WHERE patient_id = $1', [patientId]);
        await client.query('DELETE FROM patients WHERE id = $1', [patientId]);
        await client.query('DELETE FROM user_tenants WHERE user_id = $1', [nurseUserId]);
        await client.query('DELETE FROM system_users WHERE id = $1', [nurseUserId]);

        console.log('Killing test server...');
        serverProcess.kill();
        console.log('? Nursing Risk Assessment Integration Tests passed successfully!');
        process.exit(0);

    } catch (e) {
        console.error('? Test failed:', e);
        if (serverProcess) serverProcess.kill();
        process.exit(1);
    } finally {
        client.release();
    }
}

runTests();



========================================
FILE: nursing_scores.js
========================================

/**
 * nursing_scores.js ? E6 NURSING SCORES (pure, server-side, unit-testable).
 * No DB / no HTTP / no I/O. These are deterministic clinical-scoring functions used by the
 * POST /api/nursing/scores route AND by nursing_scores_unit_test.js. Keeping them pure means the
 * server computes scores authoritatively (the client cannot forge a "score" ? it sends raw
 * observations and the server derives + bands the result).
 *
 * Implemented:
 *   - computeMorseFallRisk(...)  ? Morse Fall Scale (0?125), bands Low/Moderate/High.
 *   - computeBraden(...)         ? Braden pressure-ulcer scale (6?23), bands by risk.
 *   - computeNEWS(...)           ? NEWS2 (National Early Warning Score 2) from vitals.
 *   - computePainBand(score)     ? 0?10 numeric pain ? band.
 *
 * IMPORTANT-1 ? NEWS vs NEWS2: this engine implements NEWS2 (RCP 2017). We use the NEWS2
 *   thresholds (SpO2 Scale 1: 96+ =0, 94?95 =1, 92?93 =2, <=91 =3) and add +2 for supplemental
 *   oxygen, plus +3 for any non-'A' (V/P/U or new confusion) consciousness level. LIMITATION: we
 *   model SpO2 *Scale 1* only. We do NOT implement the NEWS2 *Scale 2* SpO2 ranges used for
 *   patients with target-range 88?92% (hypercapnic respiratory failure / chronic T2RF). For those
```

```
 *   patients the on_oxygen +2 and the Scale-1 SpO2 points will OVER-score; a Scale-2 toggle is a
 *   documented follow-up. Aggregate banding (Low/Medium/High) and the single-parameter-3 escalation
 *   to at-least-Medium follow NEWS2.
 *
 * IMPORTANT-2 / FIX item 8 ? computeBraden fail-CLOSED on incomplete input: if any of the 6
 *   subscales is missing/null/NaN we return { score: null, band: 'Incomplete', error: '...' } rather
 *   than silently defaulting to the safest (highest) subscale value. The /api/nursing/scores route
 *   rejects an Incomplete Braden with 422 (a partial pressure-ulcer score must never be stored as
 *   authoritative ? under-scoring risk is a patient-safety hazard).
 *
 * ???????? ????????? (???? ?? ??? ?????? ? ?? ??? ??? "score" ?? ??????).
 */

'use strict';

function clampInt(v, lo, hi, dflt) {
    let n = parseInt(v, 10);
    if (!Number.isFinite(n)) n = dflt;
    if (n < lo) n = lo;
    if (n > hi) n = hi;
    return n;
}

// ===== Morse Fall Scale =====
// Inputs are the 6 standard Morse items. Returns { score, band }.
//   history_of_falling: 0|25 ; secondary_diagnosis: 0|15 ; ambulatory_aid: 0|15|30 ;
//   iv_therapy: 0|20 ; gait: 0|10|20 ; mental_status: 0|15
// We accept booleans / category strings and map to the canonical points (fail-closed: unknown => 0).
function computeMorseFallRisk(o = {}) {
    const histories = o.history_of_falling ? 25 : 0;
    const secondary = o.secondary_diagnosis ? 15 : 0;
    // ambulatory aid: 'none'/'bedrest'/'nurse'=0 ; 'crutches'/'cane'/'walker'=15 ; 'furniture'=30
    let aid = 0;
    const a = String(o.ambulatory_aid || 'none').toLowerCase();
    if (a === 'furniture') aid = 30;
    else if (a === 'crutches' || a === 'cane' || a === 'walker') aid = 15;
    else aid = 0;
    const iv = o.iv_therapy ? 20 : 0;
    // gait: 'normal'/'bedrest'/'immobile'=0 ; 'weak'=10 ; 'impaired'=20
    let gait = 0;
    const g = String(o.gait || 'normal').toLowerCase();
    if (g === 'impaired') gait = 20;
    else if (g === 'weak') gait = 10;
    else gait = 0;
    // mental status: 'oriented'=0 ; 'forgets'/'overestimates'=15
    const mental = (String(o.mental_status || 'oriented').toLowerCase() === 'oriented') ? 0 : 15;

    const score = histories + secondary + aid + iv + gait + mental;
    let band;
    if (score >= 45) band = 'High';
    else if (score >= 25) band = 'Moderate';
    else band = 'Low';
    return { score, band };
}
```

```
// ===== Braden Scale (pressure ulcer risk) =====
// 6 subscales. sensory(1-4), moisture(1-4), activity(1-4), mobility(1-4), nutrition(1-4),
// friction(1-3). Total 6?23. LOWER = higher risk.
//
// FIX item 8 (fail-CLOSED): a Braden total is only valid when ALL 6 subscales are present and
// in range. If ANY subscale is null/undefined/NaN/empty/out-of-range we return
// { score: null, band: 'Incomplete', error: 'Missing subscale: <name>' } ? we DO NOT default the
// missing subscale to its safest (highest) value, because that silently UNDER-scores risk
// (a patient-safety hazard). The route rejects an Incomplete result with 422.
function strictSubscale(v, lo, hi) {
    if (v === null || v === undefined || v === '') return null;       // missing
    const n = (typeof v === 'number') ? v : parseInt(v, 10);
    if (!Number.isFinite(n)) return null;                              // NaN / non-numeric
    if (n < lo || n > hi) return null;                                 // out of range
    return n;
}
function computeBraden(o = {}) {
    const fields = [
        ['sensory', 1, 4], ['moisture', 1, 4], ['activity', 1, 4],
        ['mobility', 1, 4], ['nutrition', 1, 4], ['friction', 1, 3],
    ];
    const vals = {};
    for (const [name, lo, hi] of fields) {
        const parsed = strictSubscale(o[name], lo, hi);
        if (parsed === null) {
            return { score: null, band: 'Incomplete', error: 'Missing subscale: ' + name };
        }
        vals[name] = parsed;
    }
    const score = vals.sensory + vals.moisture + vals.activity + vals.mobility + vals.nutrition + vals.frictio
n;
    let band;
    if (score <= 9) band = 'Very High';
    else if (score <= 12) band = 'High';
    else if (score <= 14) band = 'Moderate';
    else if (score <= 18) band = 'Mild';
    else band = 'None';
    return { score, band };
}

// ===== NEWS (National Early Warning Score) =====
// Vitals ? component points (0?3 each) ? aggregate. Returns { score, band, components }.
//   resp_rate (breaths/min), o2_sat (%), on_oxygen (bool), temp (°C),
//   systolic_bp (mmHg), pulse (bpm), consciousness ('A' alert | 'V'|'P'|'U' = altered).
function newsResp(rr) {
    if (rr == null || isNaN(rr)) return 0;
    if (rr <= 8) return 3;
    if (rr <= 11) return 1;
    if (rr <= 20) return 0;
    if (rr <= 24) return 2;
    return 3;
}
function newsSpo2(s) {
```

```
    if (s == null || isNaN(s)) return 0;
    if (s <= 91) return 3;
    if (s <= 93) return 2;
    if (s <= 95) return 1;
    return 0;
}
function newsTemp(t) {
    if (t == null || isNaN(t) || t === 0) return 0;
    if (t <= 35.0) return 3;
    if (t <= 36.0) return 1;
    if (t <= 38.0) return 0;
    if (t <= 39.0) return 1;
    return 2;
}
function newsSystolic(bp) {
    if (bp == null || isNaN(bp) || bp === 0) return 0;
    if (bp <= 90) return 3;
    if (bp <= 100) return 2;
    if (bp <= 110) return 1;
    if (bp <= 219) return 0;
    return 3;
}
function newsPulse(p) {
    if (p == null || isNaN(p) || p === 0) return 0;
    if (p <= 40) return 3;
    if (p <= 50) return 1;
    if (p <= 90) return 0;
    if (p <= 110) return 1;
    if (p <= 130) return 2;
    return 3;
}
function parseSystolicFromBp(bpStr) {
    if (bpStr == null) return null;
    if (typeof bpStr === 'number') return bpStr;
    const m = String(bpStr).match(/(\d+)\s*\/\s*\d+/);
    if (m) return parseInt(m[1], 10);
    const n = parseInt(bpStr, 10);
    return Number.isFinite(n) ? n : null;
}
function computeNEWS(v = {}) {
    const respPts = newsResp(v.resp_rate != null ? Number(v.resp_rate) : null);
    const spo2Pts = newsSpo2(v.o2_sat != null ? Number(v.o2_sat) : null);
    const oxyPts = v.on_oxygen ? 2 : 0;
    const tempPts = newsTemp(v.temp != null ? Number(v.temp) : null);
    const sys = v.systolic_bp != null ? Number(v.systolic_bp) : parseSystolicFromBp(v.bp);
    const bpPts = newsSystolic(sys != null && Number.isFinite(sys) ? sys : null);
    const pulsePts = newsPulse(v.pulse != null ? Number(v.pulse) : null);
    const consciousness = String(v.consciousness || 'A').toUpperCase();
    const conPts = (consciousness === 'A') ? 0 : 3;

    const components = { resp: respPts, spo2: spo2Pts, oxygen: oxyPts, temp: tempPts, bp: bpPts, pulse: pulseP
ts, consciousness: conPts };
    const score = respPts + spo2Pts + oxyPts + tempPts + bpPts + pulsePts + conPts;
    const anyThree = Object.values(components).some(p => p >= 3);
```

```
    let band;
    if (score >= 7) band = 'High';
    else if (score >= 5 || anyThree) band = 'Medium';
    else if (score >= 1) band = 'Low';
    else band = 'None';
    return { score, band, components };
}

function computePainBand(score) {
    const n = clampInt(score, 0, 10, 0);
    let band;
    if (n >= 7) band = 'Severe';
    else if (n >= 4) band = 'Moderate';
    else if (n >= 1) band = 'Mild';
    else band = 'None';
    return { score: n, band };
}

module.exports = {
    computeMorseFallRisk,
    computeBraden,
    computeNEWS,
    computePainBand,
    // exposed for tests
    newsResp, newsSpo2, newsTemp, newsSystolic, newsPulse, parseSystolicFromBp, strictSubscale,
};
```

```
========================================
FILE: nursing_scores_unit_test.js
========================================

/**
 * nursing_scores_unit_test.js ? E6 nursing-score ENGINE unit tests (DB-free, no HTTP, no PHI).
 * Run: node nursing_scores_unit_test.js   (NODE_PATH may point at the main namaweb/node_modules,
 * but this test only requires the local ./nursing_scores engine, so it has no external deps.)
 *
 * Proves the band BOUNDARIES and the fail-closed behaviours the reviewer demanded:
 *   - Morse 44 (Moderate) vs 45 (High) boundary.
 *   - Braden 9 (Very High) vs 10 (High) boundary.
 *   - NEWS single 3-point component escalates to at-least-Medium even with low aggregate.
 *   - computeBraden INCOMPLETE-input rejection (item 8): any missing subscale => band 'Incomplete',
 *     score null, error 'Missing subscale: <name>' (never silently defaults to the safest score).
 */
'use strict';
const ns = require('./nursing_scores');

let pass = 0, fail = 0;
const failures = [];
const ok = (c, m) => { if (c) { pass++; console.log('  \x1b[32mPASS\x1b[0m', m); } else { fail++; failures.pus
h(m); console.log('  \x1b[31mFAIL\x1b[0m', m); } };

console.log('\n\x1b[1m\x1b[34m============================================================\x1b[0m');
```

```
console.log('\x1b[1m\x1b[34m  E6 ? nursing_scores engine unit tests (boundaries + fail-closed)\x1b[0m');
console.log('\x1b[1m\x1b[34m============================================================\x1b[0m\n');

// ---------------------------------------------------------------------------
console.log('\x1b[1m[ 1 ] Morse Fall Scale ? 44 (Moderate) vs 45 (High) boundary\x1b[0m');
// Build exactly 45: history(25) + secondary(15) + gait weak(... use values to hit 45).
// 25 (history) + 20 (iv) = 45 exactly => High.
{
  const m45 = ns.computeMorseFallRisk({ history_of_falling: true, iv_therapy: true }); // 25 + 20 = 45
  ok(m45.score === 45 && m45.band === 'High', `Morse 45 => High (got ${m45.score}/${m45.band})`);
  // 44 is unreachable from canonical Morse increments; verify the 44/45 THRESHOLD directly:
  // a score that lands at 40 (Moderate) and another at 45 (High) bracket the >=45 cutoff.
  const m40 = ns.computeMorseFallRisk({ history_of_falling: true, ambulatory_aid: 'cane' }); // 25 + 15 = 40
  ok(m40.score === 40 && m40.band === 'Moderate', `Morse 40 => Moderate (below the 45 High cutoff) (got ${m40.
score}/${m40.band})`);
  // 25 => exactly the Moderate floor.
  const m25 = ns.computeMorseFallRisk({ history_of_falling: true }); // 25
  ok(m25.score === 25 && m25.band === 'Moderate', `Morse 25 => Moderate (lower boundary) (got ${m25.score}/${m
25.band})`);
  const m24 = ns.computeMorseFallRisk({ secondary_diagnosis: true, gait: 'weak' }); // 15 + 10 = 25? -> use 15
 only
  const mLow = ns.computeMorseFallRisk({ secondary_diagnosis: true }); // 15 => Low
  ok(mLow.score === 15 && mLow.band === 'Low', `Morse 15 => Low (got ${mLow.score}/${mLow.band})`);
  void m24;
}

// ---------------------------------------------------------------------------
console.log('\n\x1b[1m[ 2 ] Braden ? 9 (Very High) vs 10 (High) boundary\x1b[0m');
// Braden total 6..23, LOWER = higher risk. <=9 Very High; 10..12 High.
{
  // score 9: pick subscales summing to 9 (e.g. 1+1+1+1+1+... friction 1..3). 1+1+1+1+1 =5 (five 1's) +? frict
ion must be 1..3.
  // 5 subscales(1..4) + friction(1..3). To get 9: sensory1 moisture1 activity1 mobility1 nutrition2 friction3
 = 9.
  const b9 = ns.computeBraden({ sensory: 1, moisture: 1, activity: 1, mobility: 1, nutrition: 2, friction: 3 }
);
  ok(b9.score === 9 && b9.band === 'Very High', `Braden 9 => Very High (got ${b9.score}/${b9.band})`);
  // score 10: bump nutrition to 3.
  const b10 = ns.computeBraden({ sensory: 1, moisture: 1, activity: 1, mobility: 1, nutrition: 3, friction: 3
});
  ok(b10.score === 10 && b10.band === 'High', `Braden 10 => High (just above the Very-High cutoff) (got ${b10.
score}/${b10.band})`);
  // sanity: a max score => None
  const b23 = ns.computeBraden({ sensory: 4, moisture: 4, activity: 4, mobility: 4, nutrition: 4, friction: 3
});
  ok(b23.score === 23 && b23.band === 'None', `Braden 23 => None (got ${b23.score}/${b23.band})`);
}

// ---------------------------------------------------------------------------
console.log('\n\x1b[1m[ 3 ] NEWS ? a single 3-point component escalates to >= Medium\x1b[0m');
{
  // All vitals normal EXCEPT respiratory rate 7 (<=8 => 3 points). Aggregate = 3, but a single
  // 3-point parameter must escalate the band to at-least-Medium (NEWS2 clinical-response rule).
  const news = ns.computeNEWS({ resp_rate: 7, o2_sat: 98, on_oxygen: false, temp: 37, systolic_bp: 120, pulse:
```

```
 70, consciousness: 'A' });
  ok(news.components.resp === 3, `NEWS resp_rate 7 => 3 points (got ${news.components.resp})`);
  ok(news.score === 3, `NEWS aggregate = 3 (single component) (got ${news.score})`);
  ok(news.band === 'Medium', `NEWS single-3 escalates to Medium even at low aggregate (got ${news.band})`);
  // Contrast: an aggregate of 3 spread across THREE 1-point params (no single 3) stays Low.
  const spread = ns.computeNEWS({ resp_rate: 11, o2_sat: 95, on_oxygen: false, temp: 35.5, systolic_bp: 120, p
ulse: 70, consciousness: 'A' });
  ok(spread.score === 3 && !Object.values(spread.components).some(p => p >= 3) && spread.band === 'Low',
    `NEWS aggregate 3 with NO single-3 => Low (got ${spread.score}/${spread.band})`);
  // All normal => None / 0.
  const zero = ns.computeNEWS({ resp_rate: 16, o2_sat: 98, on_oxygen: false, temp: 37, systolic_bp: 120, pulse
: 70, consciousness: 'A' });
  ok(zero.score === 0 && zero.band === 'None', `NEWS all-normal => 0/None (got ${zero.score}/${zero.band})`);
  // High aggregate (>=7) => High.
  const high = ns.computeNEWS({ resp_rate: 7, o2_sat: 90, on_oxygen: true, temp: 39.5, systolic_bp: 88, pulse:
 135, consciousness: 'V' });
  ok(high.score >= 7 && high.band === 'High', `NEWS heavy derangement => High (got ${high.score}/${high.band})
`);
}

// ---------------------------------------------------------------------------
console.log('\n\x1b[1m[ 4 ] computeBraden INCOMPLETE-input rejection (item 8, fail-closed)\x1b[0m');
{
  // Missing nutrition => Incomplete (NOT silently defaulted to the safest subscale value).
  const miss = ns.computeBraden({ sensory: 2, moisture: 2, activity: 2, mobility: 2, friction: 2 });
  ok(miss.score === null && miss.band === 'Incomplete', `missing subscale => score null + band Incomplete (got
 ${miss.score}/${miss.band})`);
  ok(/Missing subscale: nutrition/.test(miss.error || ''), `error names the missing subscale (got "${miss.erro
r}")`);
  // null/NaN subscale => Incomplete.
  const nullSub = ns.computeBraden({ sensory: null, moisture: 2, activity: 2, mobility: 2, nutrition: 2, frict
ion: 2 });
  ok(nullSub.band === 'Incomplete' && nullSub.score === null, `null subscale => Incomplete (got ${nullSub.band
})`);
  const nanSub = ns.computeBraden({ sensory: 'x', moisture: 2, activity: 2, mobility: 2, nutrition: 2, frictio
n: 2 });
  ok(nanSub.band === 'Incomplete', `non-numeric subscale => Incomplete (got ${nanSub.band})`);
  // out-of-range subscale (friction = 4, valid 1..3) => Incomplete (fail-closed, not clamped).
  const oor = ns.computeBraden({ sensory: 2, moisture: 2, activity: 2, mobility: 2, nutrition: 2, friction: 4
});
  ok(oor.band === 'Incomplete', `out-of-range subscale => Incomplete (got ${oor.band})`);
  // empty object => Incomplete on the FIRST subscale (sensory), not a max default.
  const empty = ns.computeBraden({});
  ok(empty.band === 'Incomplete' && /sensory/.test(empty.error || ''), `empty observations => Incomplete on se
nsory (got ${empty.band}/"${empty.error}")`);
}

// ---------------------------------------------------------------------------
console.log('\n\x1b[1m[ 5 ] Pain band boundaries\x1b[0m');
{
  ok(ns.computePainBand(0).band === 'None', 'pain 0 => None');
  ok(ns.computePainBand(3).band === 'Mild', 'pain 3 => Mild');
  ok(ns.computePainBand(4).band === 'Moderate', 'pain 4 => Moderate');
  ok(ns.computePainBand(6).band === 'Moderate', 'pain 6 => Moderate');
```

```
  ok(ns.computePainBand(7).band === 'Severe', 'pain 7 => Severe (boundary)');
  ok(ns.computePainBand(10).band === 'Severe', 'pain 10 => Severe');
}

console.log(`\n\x1b[1m\x1b[34m============================================================\x1b[0m`);
console.log(`  \x1b[32mPASS\x1b[0m: ${pass}   \x1b[31mFAIL\x1b[0m: ${fail}`);
if (fail > 0) { console.log('\n\x1b[31mFailures:\x1b[0m'); failures.forEach(f => console.log('  - ' + f)); }
console.log(`${fail === 0 ? '\x1b[1m\x1b[32mALL PASS\x1b[0m' : '\x1b[1m\x1b[31mFAILED\x1b[0m'}: ${pass} passed
, ${fail} failed`);
process.exit(fail === 0 ? 0 : 1);



========================================
FILE: obgyn_peds_wave3_engine.js
========================================

// ============================================================================
// OBGYN & Pediatrics ? Wave 3 PURE ENGINE (no DB, no I/O)
// Converts Enterprise_Blueprint_2026 Wave 3 C-classified department specs into
// real, unit-testable decision-support gates. Same conventions as prior waves:
// pure functions, fail-closed, each rule is the EXACT stated blueprint rule.
// ============================================================================

'use strict';

// 1) Gynecologic Surgery ? high-risk adnexal mass routing.
// Blueprint rule: auto-route high-risk adnexal masses to Gynecologic Oncology instead of
// general gynecologic surgery.
function checkAdnexalMassRouting({ malignancyRiskLevel } = {}) {
    const highRisk = malignancyRiskLevel === 'high';
    return {
        routeToGynOncology: highRisk,
        reason: highRisk ? 'high-risk adnexal mass ? routed to Gynecologic Oncology' : 'routed to general gyne
cologic surgery'
    };
}

// 2) IVF/Fertility ? OHSS freeze-all recommendation.
// Blueprint rule: auto-recommend "Freeze-All" cycle (no fresh transfer) if OHSS risk is high.
function checkOhssFreezeAllRecommendation({ ohssRiskLevel } = {}) {
    const highRisk = ohssRiskLevel === 'high';
    return {
        freezeAllRecommended: highRisk,
        reason: highRisk ? 'high OHSS risk ? freeze-all cycle recommended, no fresh transfer' : 'fresh transfe
r may proceed as planned'
    };
}

// 3) Adolescent Gynecology ? pubertal timing referral trigger.
// Blueprint rule: auto-flag precocious puberty (before age 8) or primary amenorrhea
// (no menses by 15-16) for pediatric endocrinology referral.
function checkPubertalTimingReferral({ ageYears, pubertyOnset, hasMenses } = {}) {
    if (ageYears === undefined || ageYears === null) {
```

```
        return { endocrineReferralRequired: false, reason: 'age missing ? cannot assess' };
    }
    const precociousPuberty = pubertyOnset === true && Number(ageYears) < 8;
    const primaryAmenorrhea = hasMenses === false && Number(ageYears) >= 15;
    const referral = precociousPuberty || primaryAmenorrhea;
    return {
        endocrineReferralRequired: referral,
        reason: referral
            ? (precociousPuberty ? 'precocious puberty (onset before age 8)' : 'primary amenorrhea (no menses
by 15-16)')
            : 'pubertal timing within normal range'
    };
}

// 4) Menopause ? HRT contraindication hard-block.
// Blueprint rule: hard-block HRT prescription if absolute contraindications present
// (active breast cancer, unexplained bleeding, active VTE).
function checkHrtEligibility({ activeBreastCancer, unexplainedBleeding, activeVte } = {}) {
    const contraindicated = activeBreastCancer === true || unexplainedBleeding === true || activeVte === true;
    return {
        hrtAllowed: !contraindicated,
        reason: contraindicated ? 'BLOCKED ? absolute HRT contraindication present' : 'no absolute contraindic
ations identified'
    };
}

// 5) Cosmetic Gynecology (Urogynecology) ? functional-symptom routing gate.
// Blueprint rule: route functional urogynecologic symptoms (not purely aesthetic) to full
// urodynamic workup before any cosmetic procedure.
function checkUrogynecologyRouting({ functionalSymptomsPresent } = {}) {
    const requiresWorkup = functionalSymptomsPresent === true;
    return {
        urodynamicWorkupRequiredBeforeCosmetic: requiresWorkup,
        reason: requiresWorkup
            ? 'functional urogynecologic symptoms present ? urodynamic workup required before any cosmetic pro
cedure'
            : 'purely aesthetic request ? cosmetic procedure may proceed'
    };
}

// 6) Pediatric Genetics ? expanded carrier screening trigger.
// Blueprint rule: auto-suggest expanded carrier screening for consanguineous families
// (e.g. with a child showing dysmorphic features).
function checkExpandedCarrierScreening({ consanguineousFamily, dysmorphicFeatures } = {}) {
    const suggested = consanguineousFamily === true && dysmorphicFeatures === true;
    return {
        expandedCarrierScreeningSuggested: suggested,
        reason: suggested ? 'consanguineous family with dysmorphic features ? expanded carrier screening sugge
sted' : 'criteria not met'
    };
}

// 7) Pediatric Nutrition ? severe acute malnutrition (SAM) classification.
// Blueprint rule: WHO classification; weight-for-height z-score of -4 (<= -3 threshold for
```

```
// SAM) -> urgent inpatient nutritional rehabilitation flag.
function checkMalnutritionClassification({ weightForHeightZScore } = {}) {
    if (weightForHeightZScore === undefined || weightForHeightZScore === null || !Number.isFinite(Number(weigh
tForHeightZScore))) {
        return { classification: 'unknown', urgentRehabFlag: false, reason: 'z-score missing ? cannot classify
' };
    }
    const z = Number(weightForHeightZScore);
    let classification;
    if (z <= -3) classification = 'severe_acute_malnutrition';
    else if (z <= -2) classification = 'moderate_acute_malnutrition';
    else classification = 'normal';
    const urgent = classification === 'severe_acute_malnutrition';
    return {
        classification,
        urgentRehabFlag: urgent,
        reason: urgent ? `z-score ${z} <= -3 (WHO SAM threshold) ? urgent inpatient nutritional rehabilitation
` : `z-score ${z} ? ${classification}`
    };
}

// 8) Developmental Pediatrics ? regression high-priority flag.
// Blueprint rule: auto-flag developmental regression as high-priority (faster-tracked than routine delay).
function checkDevelopmentalRegressionFlag({ lostPreviouslyAcquiredSkills } = {}) {
    const highPriority = lostPreviouslyAcquiredSkills === true;
    return {
        highPriorityFastTrack: highPriority,
        reason: highPriority
            ? 'developmental regression detected (lost previously acquired skills) ? high-priority fast-track
referral'
            : 'no regression detected ? routine developmental delay pathway if applicable'
    };
}

// 9) Pediatric Subspecialties (shared) ? weight/BSA-based dosing hard-block.
// Blueprint rule: shared pediatric_dose_guard hard-blocks any medication order missing
// weight/BSA-based calculation.
function checkPediatricDoseGuard({ weightKg, bsaM2, doseCalculationProvided } = {}) {
    const hasBasis = (weightKg !== undefined && weightKg !== null && Number(weightKg) > 0)
        || (bsaM2 !== undefined && bsaM2 !== null && Number(bsaM2) > 0);
    const allowed = hasBasis && doseCalculationProvided === true;
    return {
        orderAllowed: allowed,
        reason: allowed
            ? 'weight/BSA-based dose calculation provided'
            : 'BLOCKED ? pediatric medication order missing weight/BSA-based dose calculation'
    };
}

module.exports = {
    checkAdnexalMassRouting,
    checkOhssFreezeAllRecommendation,
    checkPubertalTimingReferral,
    checkHrtEligibility,
```

```
    checkUrogynecologyRouting,
    checkExpandedCarrierScreening,
    checkMalnutritionClassification,
    checkDevelopmentalRegressionFlag,
    checkPediatricDoseGuard
};



========================================
FILE: obgyn_peds_wave3_engine_unit_test.js
========================================

// ============================================================================
// obgyn_peds_wave3_engine_unit_test.js ? DB-free unit tests for the 9 Wave 3
// (OBGYN/Pediatrics) safety/alert gates.
// ============================================================================
'use strict';
const assert = require('assert');
const eng = require('./obgyn_peds_wave3_engine');

function runTests() {
    // 1) Gynecologic Surgery ? high-risk mass must route to Gyn-Onc.
    assert.strictEqual(eng.checkAdnexalMassRouting({ malignancyRiskLevel: 'high' }).routeToGynOncology, true);
    assert.strictEqual(eng.checkAdnexalMassRouting({ malignancyRiskLevel: 'low' }).routeToGynOncology, false);

    // 2) IVF/Fertility ? high OHSS risk must recommend freeze-all.
    assert.strictEqual(eng.checkOhssFreezeAllRecommendation({ ohssRiskLevel: 'high' }).freezeAllRecommended, t
rue);
    assert.strictEqual(eng.checkOhssFreezeAllRecommendation({ ohssRiskLevel: 'low' }).freezeAllRecommended, fa
lse);

    // 3) Adolescent Gynecology ? precocious puberty/primary amenorrhea must trigger referral.
    assert.strictEqual(eng.checkPubertalTimingReferral({ ageYears: 7, pubertyOnset: true }).endocrineReferralR
equired, true);
    assert.strictEqual(eng.checkPubertalTimingReferral({ ageYears: 16, hasMenses: false }).endocrineReferralRe
quired, true);
    assert.strictEqual(eng.checkPubertalTimingReferral({ ageYears: 12, pubertyOnset: true }).endocrineReferral
Required, false);

    // 4) Menopause ? absolute contraindication must block HRT.
    assert.strictEqual(eng.checkHrtEligibility({ activeBreastCancer: true }).hrtAllowed, false);
    assert.strictEqual(eng.checkHrtEligibility({}).hrtAllowed, true);

    // 5) Cosmetic Gynecology ? functional symptoms must require urodynamic workup first.
    assert.strictEqual(eng.checkUrogynecologyRouting({ functionalSymptomsPresent: true }).urodynamicWorkupRequ
iredBeforeCosmetic, true);
    assert.strictEqual(eng.checkUrogynecologyRouting({ functionalSymptomsPresent: false }).urodynamicWorkupReq
uiredBeforeCosmetic, false);

    // 6) Pediatric Genetics ? consanguinity + dysmorphic features must suggest screening.
    assert.strictEqual(eng.checkExpandedCarrierScreening({ consanguineousFamily: true, dysmorphicFeatures: tru
e }).expandedCarrierScreeningSuggested, true);
    assert.strictEqual(eng.checkExpandedCarrierScreening({ consanguineousFamily: true, dysmorphicFeatures: fal
```

```
se }).expandedCarrierScreeningSuggested, false);

    // 7) Pediatric Nutrition ? z-score -4 must classify as SAM with urgent flag.
    assert.strictEqual(eng.checkMalnutritionClassification({ weightForHeightZScore: -4 }).classification, 'sev
ere_acute_malnutrition');
    assert.strictEqual(eng.checkMalnutritionClassification({ weightForHeightZScore: -4 }).urgentRehabFlag, tru
e);
    assert.strictEqual(eng.checkMalnutritionClassification({ weightForHeightZScore: -1 }).urgentRehabFlag, fal
se);

    // 8) Developmental Pediatrics ? regression must be high-priority fast-tracked.
    assert.strictEqual(eng.checkDevelopmentalRegressionFlag({ lostPreviouslyAcquiredSkills: true }).highPriori
tyFastTrack, true);
    assert.strictEqual(eng.checkDevelopmentalRegressionFlag({ lostPreviouslyAcquiredSkills: false }).highPrior
ityFastTrack, false);

    // 9) Pediatric Subspecialties ? order without weight/BSA dose calc must be blocked.
    assert.strictEqual(eng.checkPediatricDoseGuard({ weightKg: 20, doseCalculationProvided: false }).orderAllo
wed, false);
    assert.strictEqual(eng.checkPediatricDoseGuard({ weightKg: 20, doseCalculationProvided: true }).orderAllow
ed, true);
    assert.strictEqual(eng.checkPediatricDoseGuard({ doseCalculationProvided: true }).orderAllowed, false);

    console.log('obgyn_peds_wave3_engine_unit_test: all 9 department gates verified');
}

if (require.main === module) {
    runTests();
}

module.exports = { runTests };
```

```
========================================
FILE: obgyn_peds_wave3_routes_static_test.js
========================================

/**
 * obgyn_peds_wave3_routes_static_test.js ? DB-free static check verifying the 9
 * Wave 3 (OBGYN/Pediatrics) stateless routes are wired correctly in server.js.
 */
'use strict';
const fs = require('fs');
const path = require('path');
const assert = require('assert');

const src = fs.readFileSync(path.join(__dirname, 'server.js'), 'utf8');

const ROUTES = [
    { route: '/api/gyn-surg/mass-assessment', fn: 'checkAdnexalMassRouting' },
    { route: '/api/ivf/ohss-risk-check', fn: 'checkOhssFreezeAllRecommendation' },
    { route: '/api/adolescent-gyn/pubertal-assessment', fn: 'checkPubertalTimingReferral' },
    { route: '/api/menopause/hrt-eligibility', fn: 'checkHrtEligibility' },
```

```
    { route: '/api/urogyn/routing-check', fn: 'checkUrogynecologyRouting' },
    { route: '/api/peds-genetics/carrier-screening-check', fn: 'checkExpandedCarrierScreening' },
    { route: '/api/peds-nutrition/malnutrition-classification', fn: 'checkMalnutritionClassification' },
    { route: '/api/dev-peds/regression-check', fn: 'checkDevelopmentalRegressionFlag' },
    { route: '/api/peds-subspecialty/dose-guard-check', fn: 'checkPediatricDoseGuard' },
];

function runTests() {
    assert.ok(src.includes("require('./obgyn_peds_wave3_engine')"), 'server.js must require obgyn_peds_wave3_e
ngine');

    for (const r of ROUTES) {
        const declStr = `app.post('${r.route}'`;
        const routeIdx = src.indexOf(declStr);
        assert.ok(routeIdx !== -1, `route must exist: ${r.route}`);

        const routeEnd = src.indexOf('});', routeIdx);
        const block = src.slice(routeIdx, routeEnd === -1 ? routeIdx + 300 : routeEnd);
        assert.ok(block.includes('requireAuth'), `${r.route} must require auth`);
        assert.ok(block.includes("requireRole('patients', 'prescriptions')"), `${r.route} must require patient
s/prescriptions role`);
        assert.ok(block.includes(`obgynPedsWave3Engine.${r.fn}(`), `${r.route} must call obgynPedsWave3Engine.
${r.fn}`);
    }

    console.log(`obgyn_peds_wave3_routes_static_test: all ${ROUTES.length} routes verified`);
}

if (require.main === module) {
    runTests();
}

module.exports = { runTests };
```

```
========================================
FILE: obgyn_workflow_test.js
========================================

/**
 * obgyn_workflow_test.js ? Epic E14 OB/Maternity business-workflow test (DB-free).
 * Simulates the full maternity journey against a mock pool and the real ob_engine:
 *   register pregnancy (GPAL+EDD) -> antenatal visits (GA + risk flags) ->
 *   ultrasound (biometry GA + EFW band) -> partogram timepoints ->
 *   delivery (state flip + server APGAR) -> neonatal (server APGAR) ->
 *   re-delivery on same pregnancy rejected (409).
 *   NODE_PATH=...\namaweb\node_modules node obgyn_workflow_test.js
 */
'use strict';
const E = require('./ob_engine');

let passed = 0, failed = 0;
function assert(cond, name, det = '') {
```

```
    if (cond) { console.log('  PASS ? ' + name); passed++; }
    else { console.log('  FAIL ? ' + name + (det ? ' | ' + det : '')); failed++; }
}

// ---- mock state ----
const TENANT = 1;
let seq = { preg: 0, visit: 0, us: 0, pg: 0, del: 0, neo: 0, nst: 0 };
const store = { pregnancies: [], visits: [], ultrasounds: [], partogram: [], deliveries: [], neonatal: [], nst
: [], audit: [] };
function audit(action, details) { store.audit.push({ action, details }); }

// ---- route-logic simulations (mirror server.js handlers) ----
function registerPregnancy(b) {
    const edd = E.computeEDD(b.lmp);
    const g = E.computeGPAL(b);
    if (!g.ok) return { status: 422, error: g.error };
    const row = { id: ++seq.preg, tenant_id: TENANT, patient_id: b.patient_id, status: 'Active',
        lmp: b.lmp, edd, gravida: g.gravida, para: g.para, abortions: g.abortion, living_children: g.living };
    store.pregnancies.push(row);
    audit('CREATE_PREGNANCY', `#${row.id} G${g.gravida}P${g.para}A${g.abortion}L${g.living}`);
    return { status: 200, row };
}
function antenatalVisit(pregId, b) {
    const preg = store.pregnancies.find(p => p.id === pregId && p.tenant_id === TENANT);
    if (!preg) return { status: 404 };
    const flags = E.antenatalRiskFlags(b);
    const ga = E.gestationalAgeFromLMP(preg.lmp, b.visit_date);
    const row = { id: ++seq.visit, pregnancy_id: pregId, ga: ga ? ga.label : '', risk_flags: flags.join(', '),
        visit_number: store.visits.filter(v => v.pregnancy_id === pregId).length + 1 };
    store.visits.push(row);
    audit('ANTENATAL_VISIT', `#${pregId} ${row.ga}`);
    return { status: 200, row, flags };
}
function ultrasound(pregId, b) {
    const preg = store.pregnancies.find(p => p.id === pregId && p.tenant_id === TENANT);
    if (!preg) return { status: 404 };
    const ga = E.gaFromBiometry(b);
    const band = ga.ok ? E.efwPercentileBand(ga.gaWeeks, b.efw) : null;
    const row = { id: ++seq.us, pregnancy_id: pregId, ga: ga.ok ? `${ga.gaWeeks}+${ga.gaDays} weeks` : '', efw
_percentile: band ? band.band : '' };
    store.ultrasounds.push(row);
    return { status: 200, row, band };
}
function partogram(pregId, b) {
    const preg = store.pregnancies.find(p => p.id === pregId && p.tenant_id === TENANT);
    if (!preg) return { status: 404 };
    const flags = E.antenatalRiskFlags({ fetal_heart_rate: b.fetal_heart_rate_baseline });
    const row = { id: ++seq.pg, pregnancy_id: pregId, dilation: b.cervical_dilation, alert_flags: flags.join('
, ') };
    store.partogram.push(row);
    return { status: 200, row };
}
function recordDelivery(pregId, b) {
    const preg = store.pregnancies.find(p => p.id === pregId && p.tenant_id === TENANT);
```

```
    if (!preg) return { status: 404 };
    const t = E.deliveryTransitionAllowed(preg.status);   // state machine
    if (!t.ok) return { status: 409, error: t.error };
    const a1 = E.computeAPGAR(b.apgar_1min_components);
    const a5 = E.computeAPGAR(b.apgar_5min_components);
    if (!a1.ok) return { status: 422, error: 'apgar1' };
    if (!a5.ok) return { status: 422, error: 'apgar5' };
    const row = { id: ++seq.del, tenant_id: TENANT, pregnancy_id: pregId, patient_id: preg.patient_id,
        apgar_1min: a1.total, apgar_5min: a5.total, delivery_type: b.delivery_type };
    store.deliveries.push(row);
    preg.status = 'Delivered';                            // commit transition
    audit('RECORD_DELIVERY', `#${pregId} APGAR ${a1.total}/${a5.total}`);
    return { status: 200, row };
}
function recordNST(pregId, b) {
    const preg = store.pregnancies.find(p => p.id === pregId && p.tenant_id === TENANT);
    if (!preg) return { status: 404 };
    const row = { id: ++seq.nst, pregnancy_id: pregId, patient_id: preg.patient_id,
        result: b.result || 'Reactive', baseline_fhr: b.baseline_fhr || 0, duration_minutes: b.duration_minute
s || 20 };
    store.nst.push(row);
    // F3: audit must always be recorded (mirrors the server.js fix)
    audit('NST_ENTRY', `NST pregnancy #${pregId}: result=${row.result} fhr=${row.baseline_fhr} dur=${row.durat
ion_minutes}min`);
    return { status: 200, row };
}
function recordNeonatal(deliveryId, b) {
    const del = store.deliveries.find(d => d.id === deliveryId && d.tenant_id === TENANT);
    if (!del) return { status: 404 };
    const a1 = E.computeAPGAR(b.apgar_1min_components);
    const a5 = E.computeAPGAR(b.apgar_5min_components);
    if (!a1.ok || !a5.ok) return { status: 422 };
    const row = { id: ++seq.neo, delivery_id: deliveryId, apgar_1min: a1.total, apgar_5min: a5.total, birth_we
ight_grams: b.birth_weight_grams };
    store.neonatal.push(row);
    audit('RECORD_NEONATAL', `del#${deliveryId} APGAR ${a1.total}/${a5.total}`);
    return { status: 200, row };
}

console.log('\n=== E14 OB/Maternity workflow simulation ===\n');

// 1) Register pregnancy
const reg = registerPregnancy({ patient_id: 201, lmp: '2026-01-15', gravida: 3, para: 1, abortion: 1, living:
1 });
assert(reg.status === 200, 'register pregnancy ok');
assert(reg.row.edd === '2026-10-22', 'EDD = LMP+280 (server-derived)', reg.row.edd);
assert(reg.row.living_children === 1, 'GPAL living children derived/validated');
const pregId = reg.row.id;

// 2) Antenatal visits ? normal then high-risk
const v1 = antenatalVisit(pregId, { systolic: 120, diastolic: 80, hemoglobin: 12, fetal_heart_rate: 140, visit
_date: '2026-04-15' });
assert(v1.status === 200 && v1.flags.length === 0, 'antenatal visit 1: normal, no risk flags');
assert(/weeks$/.test(v1.row.ga), 'antenatal GA label server-derived from LMP', v1.row.ga);
```

```
const v2 = antenatalVisit(pregId, { systolic: 150, diastolic: 95, proteinuria: '++', hemoglobin: 9, visit_date
: '2026-07-01' });
assert(v2.status === 200 && v2.flags.includes('Pre-eclampsia risk') && v2.flags.includes('Anemia'), 'antenatal
 visit 2: pre-eclampsia + anemia flagged server-side');
assert(v2.row.visit_number === 2, 'visit_number increments per pregnancy');

// 3) Ultrasound
const us = ultrasound(pregId, { bpd: 75, fl: 56, hc: 280, ac: 250, efw: 1500 });
assert(us.status === 200 && /weeks$/.test(us.row.ga), 'ultrasound GA from biometry');
assert(us.band !== null, 'ultrasound EFW percentile band computed');

// 4) Partogram timepoints
assert(partogram(pregId, { cervical_dilation: 4, fetal_heart_rate_baseline: 140 }).status === 200, 'partogram
entry 1 (4cm)');
const pg2 = partogram(pregId, { cervical_dilation: 8, fetal_heart_rate_baseline: 175 });
assert(pg2.status === 200 && pg2.row.alert_flags.includes('Abnormal FHR'), 'partogram entry 2 flags abnormal F
HR (175)');

// 5) Delivery (state flip + server APGAR)
const goodApgar = { appearance: 'pink', pulse: 'above_100', grimace: 'cry', activity: 'active', respiration: '
good' };
const del = recordDelivery(pregId, { delivery_type: 'NVD', apgar_1min_components: goodApgar, apgar_5min_compon
ents: goodApgar });
assert(del.status === 200 && del.row.apgar_1min === 10 && del.row.apgar_5min === 10, 'delivery recorded, APGAR
 10/10 server-computed');
assert(store.pregnancies.find(p => p.id === pregId).status === 'Delivered', 'pregnancy status flipped to Deliv
ered');
const delId = del.row.id;

// 6) Re-delivery rejected by state machine
const reDel = recordDelivery(pregId, { delivery_type: 'NVD', apgar_1min_components: goodApgar, apgar_5min_comp
onents: goodApgar });
assert(reDel.status === 409, 'second delivery on same pregnancy -> 409 (state machine)');

// 7) Delivery with incomplete APGAR -> 422 (fail-closed)
const reg2 = registerPregnancy({ patient_id: 201, lmp: '2026-02-01', gravida: 1, para: 0, abortion: 0 });
const badApgar = { appearance: 'pink', pulse: 'above_100', grimace: 'cry', activity: 'active' }; // missing re
spiration
assert(recordDelivery(reg2.row.id, { apgar_1min_components: goodApgar, apgar_5min_components: badApgar }).stat
us === 422, 'delivery with incomplete APGAR -> 422 (no false 10)');

// 8) Neonatal attached to delivery
const neo = recordNeonatal(delId, { birth_weight_grams: 3200, apgar_1min_components: goodApgar, apgar_5min_com
ponents: goodApgar });
assert(neo.status === 200 && neo.row.apgar_5min === 10, 'neonatal record attached, APGAR server-computed');
assert(recordNeonatal(99999, { apgar_1min_components: goodApgar, apgar_5min_components: goodApgar }).status ==
= 404, 'neonatal on unknown delivery -> 404');

// 9) NST records and audit (F3)
const nst1 = recordNST(pregId, { result: 'Reactive', baseline_fhr: 145, duration_minutes: 20 });
assert(nst1.status === 200, 'NST reactive recorded ok');
const nstNR = recordNST(pregId, { result: 'Non-Reactive', baseline_fhr: 95, duration_minutes: 20 });
assert(nstNR.status === 200, 'NST non-reactive recorded ok');
assert(store.audit.some(a => a.action === 'NST_ENTRY'), 'NST_ENTRY audit always written (F3 fix)');
```

```
const nstAudits = store.audit.filter(a => a.action === 'NST_ENTRY');
assert(nstAudits.length === 2, 'NST_ENTRY audit written for each NST (reactive and non-reactive)');

// 10) Audit coverage
assert(store.audit.some(a => a.action === 'RECORD_DELIVERY'), 'delivery audited');
assert(store.audit.some(a => a.action === 'RECORD_NEONATAL'), 'neonatal audited');
assert(store.audit.some(a => a.action === 'CREATE_PREGNANCY'), 'pregnancy creation audited');

// 11) F1: delivery handler ? pool client release exactly once (mock-pool assertion)
// The delivery handler uses: const client = await pool.connect(); try { ... } finally { client.release(); }
// With all inline client.release() calls removed (F1 fix), release count must be exactly 1
// regardless of which exit path fires (success, 403, 400, 404, 409, 422).
async function testDeliveryClientRelease() {
    function makeClient(txOk) {
        let releases = 0;
        const client = {
            query: async (sql) => {
                if (sql.startsWith('SELECT * FROM obgyn_pregnancies') && !txOk) return { rows: [] };
                if (sql.startsWith('SELECT * FROM obgyn_pregnancies')) return { rows: [{ id: 1, patient_id: 99
, status: 'Active', tenant_id: 1 }] };
                if (sql.startsWith('INSERT INTO obgyn_deliveries')) return { rows: [{ id: 1 }] };
                return { rows: [] };
            },
            release: () => { releases++; }
        };
        return { client, getReleases: () => releases };
    }

    // simulate: success path (COMMIT -> finally release)
    const m1 = makeClient(true);
    await (async () => {
        const c = m1.client;
        try {
            await c.query("SELECT set_config('app.tenant_id', $1, true)");
            await c.query('BEGIN');
            const row = (await c.query('SELECT * FROM obgyn_pregnancies WHERE id=$1 AND tenant_id=$2 FOR UPDAT
E')).rows[0];
            if (!row) { await c.query('ROLLBACK'); return; }
            const t = { ok: true };
            if (!t.ok) { await c.query('ROLLBACK'); return; }
            await c.query('INSERT INTO obgyn_deliveries');
            await c.query('COMMIT');
        } catch (e) { try { await c.query('ROLLBACK'); } catch (_) {} }
        finally { c.release(); }
    })();
    assert(m1.getReleases() === 1, 'F1: client.release() exactly once on SUCCESS path');

    // simulate: 404 early-exit after BEGIN (pregRow missing)
    const m2 = makeClient(false);
    await (async () => {
        const c = m2.client;
        try {
            await c.query("SELECT set_config('app.tenant_id', $1, true)");
            await c.query('BEGIN');
```

```
            const row = (await c.query('SELECT * FROM obgyn_pregnancies WHERE id=$1 AND tenant_id=$2 FOR UPDAT
E')).rows[0];
            if (!row) { await c.query('ROLLBACK'); return; }
        } catch (e) { try { await c.query('ROLLBACK'); } catch (_) {} }
        finally { c.release(); }
    })();
    assert(m2.getReleases() === 1, 'F1: client.release() exactly once on 404 early-exit path');

    // simulate: pre-BEGIN early-exit (tenantId null) ? no ROLLBACK, finally still releases
    const m3 = makeClient(false);
    await (async () => {
        const c = m3.client;
        try {
            // tenantId === null -> early return (no BEGIN reached)
            return;
        } catch (e) { try { await c.query('ROLLBACK'); } catch (_) {} }
        finally { c.release(); }
    })();
    assert(m3.getReleases() === 1, 'F1: client.release() exactly once on pre-BEGIN early-exit path');
}

testDeliveryClientRelease().then(() => {
    console.log('\n=== Results: ' + passed + ' passed, ' + failed + ' failed ===\n');
    process.exit(failed === 0 ? 0 : 1);
});
```

```
========================================
FILE: ob_engine.js
========================================

/**
 * ob_engine.js ? Epic E14 (OB / Maternity) pure clinical computation engine.
 *
 * ALL authority fields for the OB/Maternity module are computed here, server-side,
 * from raw clinical inputs. Clients MUST NOT be trusted to submit EDD, GA, GPAL
 * derivations, APGAR totals, biometry-derived GA/percentile, or risk classification
 * (anti-spoof ? HARD REQUIREMENT #5). These functions are pure (no I/O, no DB, no
 * Date.now side effects except where an explicit reference date is passed) so they
 * are deterministically unit-testable without a database.
 *
 * Fail-CLOSED on incomplete/invalid critical input: helpers return a structured
 * { ok:false, error } or a null authority value rather than a falsely-reassuring
 * default (HARD REQUIREMENT #7/#8).
 */

'use strict';

const MS_PER_DAY = 24 * 60 * 60 * 1000;

/** Parse a YYYY-MM-DD (or ISO) date string into a UTC Date, or null if invalid. */
function parseISODate(s) {
    if (s == null) return null;
```

```
    if (s instanceof Date) return isNaN(s.getTime()) ? null : s;
    const str = String(s).trim();
    if (!str) return null;
    // accept YYYY-MM-DD or full ISO; normalise to date-only UTC midnight
    const m = /^(\d{4})-(\d{2})-(\d{2})/.exec(str);
    if (!m) return null;
    const y = parseInt(m[1], 10), mo = parseInt(m[2], 10), d = parseInt(m[3], 10);
    if (mo < 1 || mo > 12 || d < 1 || d > 31) return null;
    const dt = new Date(Date.UTC(y, mo - 1, d));
    if (dt.getUTCFullYear() !== y || dt.getUTCMonth() !== mo - 1 || dt.getUTCDate() !== d) return null;
    return dt;
}

function toISODate(dt) {
    return dt.toISOString().split('T')[0];
}

/**
 * computeEDD ? Estimated Date of Delivery via Naegele's rule (LMP + 280 days).
 * Returns ISO date string, or null if LMP is missing/invalid (fail-closed; never a guess).
 * @param {string|Date} lmp Last Menstrual Period.
 * @returns {string|null}
 */
function computeEDD(lmp) {
    const d = parseISODate(lmp);
    if (!d) return null;
    return toISODate(new Date(d.getTime() + 280 * MS_PER_DAY));
}

/**
 * gestationalAgeFromLMP ? GA at a reference date, derived from LMP.
 * Returns { weeks, days, totalDays, label } or null if inputs invalid.
 * Negative GA (reference before LMP) is rejected (returns null) ? incomplete, not reassuring.
 * @param {string|Date} lmp
 * @param {string|Date} [refDate] defaults to today (UTC) when omitted.
 */
function gestationalAgeFromLMP(lmp, refDate) {
    const l = parseISODate(lmp);
    if (!l) return null;
    const ref = refDate ? parseISODate(refDate) : new Date(Date.UTC(
        new Date().getUTCFullYear(), new Date().getUTCMonth(), new Date().getUTCDate()));
    if (!ref) return null;
    const totalDays = Math.floor((ref.getTime() - l.getTime()) / MS_PER_DAY);
    if (totalDays < 0 || totalDays > 320) return null; // out of plausible pregnancy range
    const weeks = Math.floor(totalDays / 7);
    const days = totalDays % 7;
    return { weeks, days, totalDays, label: `${weeks}+${days} weeks` };
}

/**
 * computeGPAL ? validate & derive obstetric history (Gravida/Para/Abortion/Living).
 * GPAL is an authority field: living children is DERIVED, never trusted from client.
 * Invariants enforced (fail-closed on violation):
 *   - all non-negative integers
```

```
 *   - gravida >= 1 for an active/recorded pregnancy
 *   - para + abortion <= gravida - (current ongoing pregnancy counts toward gravida)
 *     i.e. para + abortion <= gravida  (current pregnancy may still be ongoing)
 *   - living children cannot exceed total births carried to viability (para), and
 *     cannot be negative; computed = min(provided living, para) capped, but if the
 *     provided value is inconsistent we FLAG rather than silently coerce.
 * @returns {{ok:true, gravida, para, abortion, living, flags:string[]}|{ok:false,error:string}}
 */
function computeGPAL({ gravida, para, abortion, living } = {}) {
    const g = toInt(gravida), p = toInt(para), a = toInt(abortion);
    if (g === null || p === null || a === null) {
        return { ok: false, error: 'GPAL requires integer gravida/para/abortion' };
    }
    if (g < 1) return { ok: false, error: 'gravida must be >= 1' };
    if (p < 0 || a < 0) return { ok: false, error: 'para/abortion must be >= 0' };
    if (p + a > g) return { ok: false, error: 'para + abortion cannot exceed gravida' };
    const flags = [];
    // living children: derive a safe ceiling (cannot exceed live births i.e. para count
    // of pregnancies carried past viability; multiples could exceed but we treat the
    // provided value as advisory and flag inconsistency rather than trust it blindly).
    let livingProvided = toInt(living);
    let livingComputed;
    if (livingProvided === null) {
        livingComputed = p; // best estimate: assume each viable delivery yielded a living child
    } else if (livingProvided < 0) {
        return { ok: false, error: 'living children must be >= 0' };
    } else {
        livingComputed = livingProvided;
        if (livingProvided > p * 3) { // implausible even with triplets every delivery
            flags.push('living_children_implausible');
        }
    }
    return { ok: true, gravida: g, para: p, abortion: a, living: livingComputed, flags };
}

/**
 * APGAR ? server-side scoring from the 5 component enums (0/1/2 each), total 0-10.
 * Components: appearance, pulse, grimace, activity, respiration.
 * Each component accepts either an explicit 0|1|2, or a recognised enum string.
 * Incomplete input (any missing/unrecognised component) => fail-closed:
 * returns { ok:false } (NEVER a falsely-reassuring 10).
 */
const APGAR_COMPONENTS = ['appearance', 'pulse', 'grimace', 'activity', 'respiration'];
const APGAR_ENUM = {
    appearance: { 'blue': 0, 'pale': 0, 'blue_pale': 0, 'acrocyanotic': 1, 'pink_extremities_blue': 1, 'pink':
 2, 'completely_pink': 2 },
    pulse:      { 'absent': 0, 'none': 0, 'below_100': 1, 'slow': 1, 'above_100': 2, 'normal': 2 },
    grimace:    { 'no_response': 0, 'none': 0, 'grimace': 1, 'weak': 1, 'cry': 2, 'cough': 2, 'sneeze': 2 },
    activity:   { 'limp': 0, 'flaccid': 0, 'none': 0, 'some_flexion': 1, 'flexed': 1, 'active': 2, 'active_mot
ion': 2 },
    respiration:{ 'absent': 0, 'none': 0, 'slow': 1, 'irregular': 1, 'weak_cry': 1, 'good': 2, 'strong_cry': 2
, 'crying': 2 }
};
```

```
function apgarComponentScore(component, value) {
    if (value === null || value === undefined || value === '') return null;
    // explicit numeric 0/1/2
    if (typeof value === 'number' || /^[0-2]$/.test(String(value).trim())) {
        const n = toInt(value);
        if (n === 0 || n === 1 || n === 2) return n;
        return null;
    }
    const key = String(value).trim().toLowerCase().replace(/[\s-]+/g, '_');
    const map = APGAR_ENUM[component];
    if (map && Object.prototype.hasOwnProperty.call(map, key)) return map[key];
    return null;
}

/**
 * computeAPGAR ? total 0-10 from the 5 components. Anti-spoof: any client-supplied
 * "total" field is ignored entirely; total is computed here.
 * @returns {{ok:true, total:number, components:object}|{ok:false,error:string,missing:string[]}}
 */
function computeAPGAR(input = {}) {
    const components = {};
    const missing = [];
    for (const c of APGAR_COMPONENTS) {
        const s = apgarComponentScore(c, input[c]);
        if (s === null) { missing.push(c); } else { components[c] = s; }
    }
    if (missing.length) {
        return { ok: false, error: 'incomplete APGAR components', missing };
    }
    const total = APGAR_COMPONENTS.reduce((sum, c) => sum + components[c], 0);
    return { ok: true, total, components };
}

/**
 * gaFromBiometry ? derive gestational age (weeks) from ultrasound biometry.
 * Uses Hadlock-style approximations on individual params, averaging available ones.
 * Accepts mm for BPD/HC/AC and mm for FL. Returns { ok, gaWeeks, gaDays, method, used }
 * or fail-closed { ok:false } when no usable biometry provided.
 *
 * These are clinical approximations sufficient for trend/percentile screening; not a
 * substitute for a validated fetal-biometry library, but server-authoritative & testable.
 */
function gaFromBiometry({ bpd, hc, ac, fl } = {}) {
    const used = [];
    const estimates = [];
    const b = toNum(bpd), h = toNum(hc), a = toNum(ac), f = toNum(fl);
    // BPD (mm) -> GA weeks: approx Hadlock; valid roughly 20-100mm
    if (b !== null && b > 10 && b < 120) { estimates.push(9.54 + 1.482 * (b / 10) + 0.1676 * (b / 10) * (b / 1
0)); used.push('bpd'); }
    // FL (mm) -> GA weeks
    if (f !== null && f > 5 && f < 90) { estimates.push(10.35 + 2.460 * (f / 10) + 0.170 * (f / 10) * (f / 10)
); used.push('fl'); }
    // AC (mm) -> GA weeks (coarser)
    if (a !== null && a > 30 && a < 400) { estimates.push(8.14 + 0.753 * (a / 10) + 0.0036 * (a / 10) * (a / 1
```

```
0)); used.push('ac'); }
    // HC (mm) -> GA weeks
    if (h !== null && h > 30 && h < 400) { estimates.push(8.96 + 0.540 * (h / 10) + 0.0003 * (h / 10) * (h / 1
0)); used.push('hc'); }
    if (!estimates.length) return { ok: false, error: 'no usable biometry' };
    const avg = estimates.reduce((s, v) => s + v, 0) / estimates.length;
    if (avg < 5 || avg > 45) return { ok: false, error: 'biometry out of plausible range' };
    const gaWeeks = Math.floor(avg);
    const gaDays = Math.round((avg - gaWeeks) * 7);
    return { ok: true, gaWeeks, gaDays, gaDecimal: Math.round(avg * 10) / 10, method: 'hadlock-avg', used };
}

/**
 * efwPercentile ? crude EFW (estimated fetal weight, grams) percentile band for a GA week.
 * Returns a band label (<10th, 10-90th, >90th) for screening. Fail-closed null if inputs bad.
 * Reference 50th-centile EFW grams by completed week (Hadlock population approximation).
 */
const EFW_REF = { // week: [p10, p50, p90]
    20: [240, 300, 380], 24: [530, 660, 850], 28: [900, 1150, 1500],
    30: [1180, 1500, 1950], 32: [1500, 1900, 2450], 34: [1900, 2400, 3050],
    36: [2350, 2900, 3650], 38: [2750, 3300, 4100], 40: [3000, 3600, 4400]
};
function efwPercentileBand(gaWeeks, efwGrams) {
    const w = toInt(gaWeeks), efw = toNum(efwGrams);
    if (w === null || efw === null || efw <= 0) return null;
    // find nearest reference week
    const weeks = Object.keys(EFW_REF).map(Number).sort((x, y) => x - y);
    let ref = null, best = Infinity;
    for (const wk of weeks) { const d = Math.abs(wk - w); if (d < best) { best = d; ref = EFW_REF[wk]; } }
    if (!ref) return null;
    if (efw < ref[0]) return { band: '<10th', flag: 'SGA_risk', refWeek: weeks.reduce((p, c) => Math.abs(c - w
) < Math.abs(p - w) ? c : p) };
    if (efw > ref[2]) return { band: '>90th', flag: 'LGA_risk', refWeek: weeks.reduce((p, c) => Math.abs(c - w
) < Math.abs(p - w) ? c : p) };
    return { band: '10-90th', flag: null, refWeek: weeks.reduce((p, c) => Math.abs(c - w) < Math.abs(p - w) ?
c : p) };
}

/**
 * antenatalRiskFlags ? server-side risk classification for an antenatal visit.
 * Pure function over discrete vitals; returns array of flag strings (anti-spoof:
 * risk_flags submitted by client are ignored). Incomplete values simply produce no
 * flag for that axis (we never invent a reassuring "no risk" for missing data ? the
 * absence of a flag is not asserted as safety).
 */
function antenatalRiskFlags({ systolic, diastolic, proteinuria, hemoglobin, fetal_heart_rate, fetal_movement }
 = {}) {
    const flags = [];
    const sys = toInt(systolic), dia = toInt(diastolic), hb = toNum(hemoglobin), fhr = toInt(fetal_heart_rate)
;
    const protPos = proteinuria != null && String(proteinuria).trim() !== '' &&
        !/^neg/i.test(String(proteinuria).trim());
    const htn = (sys !== null && sys >= 140) || (dia !== null && dia >= 90);
    if (htn) flags.push('Hypertension');
```

```
    if (htn && protPos) flags.push('Pre-eclampsia risk');
    if (hb !== null && hb > 0 && hb < 10) flags.push('Anemia');
    if (fhr !== null && fhr > 0 && (fhr < 110 || fhr > 160)) flags.push('Abnormal FHR');
    if (fetal_movement != null && /absent|reduced|decreased|none/i.test(String(fetal_movement))) flags.push('R
educed fetal movement');
    return flags;
}

/**
 * deliveryTransitionAllowed ? server-side state machine guard.
 * A delivery may only be recorded for a pregnancy currently in 'Active' status.
 * Already-terminal pregnancies (Delivered/Miscarriage/Ectopic/Terminated) reject.
 * @returns {{ok:true}|{ok:false,error:string}}
 */
const TERMINAL_PREGNANCY_STATUSES = ['Delivered', 'Miscarriage', 'Ectopic', 'Terminated'];
function deliveryTransitionAllowed(currentStatus) {
    const s = (currentStatus == null ? '' : String(currentStatus)).trim();
    if (s === '' ) return { ok: false, error: 'unknown pregnancy status' };
    if (s === 'Active') return { ok: true };
    if (TERMINAL_PREGNANCY_STATUSES.includes(s)) {
        return { ok: false, error: `pregnancy already in terminal status: ${s}` };
    }
    return { ok: false, error: `invalid pregnancy status for delivery: ${s}` };
}

// ===== helpers =====
function toInt(v) {
    if (v === null || v === undefined || v === '') return null;
    if (typeof v === 'number') return Number.isInteger(v) ? v : (Number.isFinite(v) ? Math.trunc(v) : null);
    const s = String(v).trim();
    if (!/^-?\d+$/.test(s)) return null;
    const n = parseInt(s, 10);
    return Number.isInteger(n) ? n : null;
}
function toNum(v) {
    if (v === null || v === undefined || v === '') return null;
    const n = Number(v);
    return Number.isFinite(n) ? n : null;
}

module.exports = {
    computeEDD,
    gestationalAgeFromLMP,
    computeGPAL,
    computeAPGAR,
    apgarComponentScore,
    gaFromBiometry,
    efwPercentileBand,
    antenatalRiskFlags,
    deliveryTransitionAllowed,
    TERMINAL_PREGNANCY_STATUSES,
    // exported for tests
    _internal: { parseISODate, toISODate, toInt, toNum }
};
```

```
========================================
FILE: ob_engine_unit_test.js
========================================

/**
 * ob_engine_unit_test.js ? Epic E14 pure-engine unit test (DB-free).
 * Verifies the server-side authority calculators: EDD (Naegele), GA, GPAL derivation,
 * APGAR scoring + fail-closed on incomplete input, biometry GA, EFW percentile,
 * antenatal risk classification, and the delivery state machine.
 *   NODE_PATH=...\namaweb\node_modules node ob_engine_unit_test.js
 */
'use strict';
const E = require('./ob_engine');

let passed = 0, failed = 0;
function assert(cond, name, det = '') {
    if (cond) { console.log('  PASS ? ' + name); passed++; }
    else { console.log('  FAIL ? ' + name + (det ? ' | ' + det : '')); failed++; }
}

console.log('\n=== E14 OB Engine ? pure unit tests ===\n');

// ---- computeEDD (Naegele: LMP + 280 days) ----
assert(E.computeEDD('2026-01-15') === '2026-10-22', 'computeEDD: 2026-01-15 -> 2026-10-22 (LMP+280)', E.comput
eEDD('2026-01-15'));
assert(E.computeEDD('2025-01-01') === '2025-10-08', 'computeEDD: 2025-01-01 -> 2025-10-08', E.computeEDD('2025
-01-01'));
assert(E.computeEDD(null) === null, 'computeEDD: null LMP -> null (fail-closed, no guess)');
assert(E.computeEDD('not-a-date') === null, 'computeEDD: invalid string -> null');
assert(E.computeEDD('2026-13-40') === null, 'computeEDD: out-of-range date -> null');

// ---- gestationalAgeFromLMP ----
const ga = E.gestationalAgeFromLMP('2026-01-01', '2026-03-12'); // 70 days = 10+0
assert(ga && ga.weeks === 10 && ga.days === 0 && ga.label === '10+0 weeks', 'GA: 70 days -> 10+0 weeks', JSON.
stringify(ga));
assert(E.gestationalAgeFromLMP('2026-03-01', '2026-01-01') === null, 'GA: reference before LMP -> null (no neg
ative GA)');
assert(E.gestationalAgeFromLMP(null, '2026-03-12') === null, 'GA: null LMP -> null');

// ---- computeGPAL ----
let g = E.computeGPAL({ gravida: 3, para: 1, abortion: 1, living: 1 });
assert(g.ok && g.gravida === 3 && g.para === 1 && g.abortion === 1 && g.living === 1, 'GPAL: valid 3/1/1/1 ok'
);
g = E.computeGPAL({ gravida: 2, para: 1, abortion: 0 });
assert(g.ok && g.living === 1, 'GPAL: living derived = para when omitted');
g = E.computeGPAL({ gravida: 1, para: 1, abortion: 1 });
assert(!g.ok, 'GPAL: para+abortion > gravida -> fail-closed (invalid)');
g = E.computeGPAL({ gravida: 0, para: 0, abortion: 0 });
assert(!g.ok, 'GPAL: gravida < 1 -> invalid');
g = E.computeGPAL({ gravida: 'x', para: 1, abortion: 0 });
assert(!g.ok, 'GPAL: non-integer gravida -> invalid (no string coercion)');
```

```
// ---- computeAPGAR (anti-spoof, fail-closed) ----
let a = E.computeAPGAR({ appearance: 'pink', pulse: 'above_100', grimace: 'cry', activity: 'active', respirati
on: 'good' });
assert(a.ok && a.total === 10, 'APGAR: all best -> 10', JSON.stringify(a));
a = E.computeAPGAR({ appearance: 'blue', pulse: 'absent', grimace: 'no_response', activity: 'limp', respiratio
n: 'absent' });
assert(a.ok && a.total === 0, 'APGAR: all worst -> 0');
a = E.computeAPGAR({ appearance: 1, pulse: 2, grimace: 1, activity: 2, respiration: 1 });
assert(a.ok && a.total === 7, 'APGAR: numeric components -> 7');
a = E.computeAPGAR({ appearance: 'pink', pulse: 'above_100', grimace: 'cry', activity: 'active' });
assert(!a.ok && a.missing.includes('respiration'), 'APGAR: missing component -> fail-closed (NOT a reassuring
10)');
a = E.computeAPGAR({ appearance: 'pink', pulse: 'above_100', grimace: 'cry', activity: 'active', respiration:
'good', total: 0 });
assert(a.ok && a.total === 10, 'APGAR: client-supplied total IGNORED (anti-spoof)');
a = E.computeAPGAR({ appearance: 'bogus', pulse: 'above_100', grimace: 'cry', activity: 'active', respiration:
 'good' });
assert(!a.ok, 'APGAR: unrecognised enum -> fail-closed');

// ---- gaFromBiometry ----
let b = E.gaFromBiometry({ bpd: 75, fl: 56, hc: 280, ac: 250 });
assert(b.ok && b.gaWeeks >= 28 && b.gaWeeks <= 34, 'biometry: typical 3rd-trimester biometry -> ~30wk', JSON.s
tringify(b));
assert(E.gaFromBiometry({}).ok === false, 'biometry: empty -> fail-closed');
assert(E.gaFromBiometry({ bpd: 0 }).ok === false, 'biometry: zero/invalid -> fail-closed');

// ---- efwPercentileBand ----
assert(E.efwPercentileBand(32, 1900).band === '10-90th', 'EFW: 1900g @32wk -> normal band');
assert(E.efwPercentileBand(32, 1000).flag === 'SGA_risk', 'EFW: 1000g @32wk -> SGA flag');
assert(E.efwPercentileBand(32, 3000).flag === 'LGA_risk', 'EFW: 3000g @32wk -> LGA flag');
assert(E.efwPercentileBand(null, 3000) === null, 'EFW: null GA -> null');

// ---- antenatalRiskFlags ----
assert(E.antenatalRiskFlags({ systolic: 150, diastolic: 95 }).includes('Hypertension'), 'risk: BP 150/95 -> Hy
pertension');
assert(E.antenatalRiskFlags({ systolic: 150, diastolic: 95, proteinuria: '++' }).includes('Pre-eclampsia risk'
), 'risk: HTN + proteinuria -> Pre-eclampsia');
assert(E.antenatalRiskFlags({ hemoglobin: 9 }).includes('Anemia'), 'risk: Hb 9 -> Anemia');
assert(E.antenatalRiskFlags({ fetal_heart_rate: 170 }).includes('Abnormal FHR'), 'risk: FHR 170 -> Abnormal FH
R');
assert(E.antenatalRiskFlags({ systolic: 120, diastolic: 80, hemoglobin: 12, fetal_heart_rate: 140 }).length ==
= 0, 'risk: normal -> no flags');

// ---- deliveryTransitionAllowed (state machine) ----
assert(E.deliveryTransitionAllowed('Active').ok === true, 'state: Active -> delivery allowed');
assert(E.deliveryTransitionAllowed('Delivered').ok === false, 'state: Delivered -> rejected (terminal)');
assert(E.deliveryTransitionAllowed('Miscarriage').ok === false, 'state: Miscarriage -> rejected');
assert(E.deliveryTransitionAllowed('').ok === false, 'state: empty status -> rejected (incomplete)');

console.log('\n=== Results: ' + passed + ' passed, ' + failed + ' failed ===\n');
process.exit(failed === 0 ? 0 : 1);
```

```
======================================
FILE: onboarding.js
======================================

// ============================================================
// onboarding.js ? E0 Facility Onboarding Wizard (server side)
// Self-contained module: archetype -> enabled module-index mapping, input validation,
// and a mountable POST /api/admin/facilities/provision route.
//
// Security posture (must NOT be weakened):
//  - Route guarded by requireAuth + requireRole('settings') + INLINE super-admin guard
//    (role !== 'Admin' -> 403 + audit BLOCKED_), mirroring the deployed Gate-4 / user-create pattern.
//  - Single DB transaction on a raw pooled client (BEGIN/COMMIT/ROLLBACK). After the tenant row is
//    created, app.tenant_id is set (transaction-local) so RLS-protected INSERTs (facilities,
//    facility_modules) pass WITH CHECK for the NEW tenant.
//  - NO default passwords: a strong random password is generated if none is supplied; the plaintext
//    is returned ONCE in the 201 body for out-of-band delivery and is NEVER logged.
//  - Integrations are GATED: only name + enabled flag + non-secret config are recorded. No API keys,
//    secrets, certificates, OTP, CSR, or outbound calls during provisioning.
//  - audit_trail written via logAudit('FACILITY_PROVISIONED').
// ============================================================
'use strict';

const bcrypt = require('bcryptjs');
const crypto = require('crypto');

// NAV_ITEMS index space is 0..42 (mirror of public/js/app.js NAV_ITEMS; index 0=Dashboard, 42=Settings).
const MODULE_INDEX_MIN = 0;
const MODULE_INDEX_MAX = 42;
const ALL_MODULE_INDICES = Array.from({ length: MODULE_INDEX_MAX + 1 }, (_, i) => i);

// Explicit subsets, consistent with the existing FACILITY_ALLOWED in public/js/app.js:
//   health_center (app.js): [0,1,2,3,4,5,6,7,8,9,11,12,13,14,15,20,21,30,33,34,35,41,42]
//   clinic/polyclinic (app.js): [0,1,2,3,4,6,7,8,9,11,12,13,14,15,20,30,34,42]
const HEALTH_CENTER_MODULES = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 11, 12, 13, 14, 15, 20, 21, 30, 33, 34, 35, 41, 4
2];
const POLYCLINIC_MODULES = [0, 1, 2, 3, 4, 6, 7, 8, 9, 11, 12, 13, 14, 15, 20, 30, 34, 42];
// general_hospital: clinical core + inpatient + lab/rad/pharmacy + finance/insurance + nursing/waiting/accoun
ts.
// 0 Dashboard,1 Reception,2 Appointments,3 Doctor,4 Lab,5 Radiology,6 Pharmacy,7 HR,8 Finance,9 Insurance,
// 10 Inventory,11 Nursing,12 Waiting,13 Patient Accounts,14 Reports,15 Messaging,16 Catalog,17 Dept Requests,
// 18 Surgery,19 Blood Bank,20 Consent,21 Emergency,22 Inpatient ADT,23 ICU,24 CSSD,30 Medical Records,
// 33 ZATCA,34 Telemedicine,42 Settings.
const GENERAL_HOSPITAL_MODULES = [
    0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 30, 33, 34, 42
];

// archetype -> enabled module indices. medical_city & large_hospital = ALL (FACILITY_ALLOWED null).
const ARCHETYPE_MODULES = {
    medical_city: ALL_MODULE_INDICES,
    large_hospital: ALL_MODULE_INDICES,
    general_hospital: GENERAL_HOSPITAL_MODULES,
    polyclinic: POLYCLINIC_MODULES,
```

```
    health_center: HEALTH_CENTER_MODULES
};

const VALID_ARCHETYPES = Object.keys(ARCHETYPE_MODULES);

// Archetypes that support inpatient beds (others must be bed-less).
const BED_ARCHETYPES = new Set(['medical_city', 'large_hospital', 'general_hospital']);

// Allow-list of integration providers that MAY be recorded (gated, no secrets).
const ALLOWED_INTEGRATIONS = new Set(['MOH', 'CBAHI', 'NPHIES', 'ZATCA', 'SCFHS', 'PACS']);

/**
 * Resolve the enabled module-index set for an archetype.
 * @returns {number[]} sorted, de-duplicated, in-range indices. [] for unknown archetype.
 */
function modulesForArchetype(archetype) {
    const list = ARCHETYPE_MODULES[archetype];
    if (!list) return [];
    return [...new Set(list)]
        .filter(i => Number.isInteger(i) && i >= MODULE_INDEX_MIN && i <= MODULE_INDEX_MAX)
        .sort((a, b) => a - b);
}

/**
 * Generate a strong random password (NO default / NO guessable value).
 * URL-safe base64, ~24 bytes of entropy. Never logged.
 */
function generateStrongPassword() {
    return crypto.randomBytes(24).toString('base64').replace(/[+/=]/g, '').slice(0, 28);
}

/**
 * Validate the provisioning payload. Pure function (no I/O) so it is unit-testable.
 * @returns {{ ok: boolean, errors: string[], value?: object }}
 */
function validateProvisionInput(body) {
    const errors = [];
    body = body && typeof body === 'object' ? body : {};

    const archetype = String(body.archetype || '').trim();
    if (!VALID_ARCHETYPES.includes(archetype)) {
        errors.push('archetype must be one of: ' + VALID_ARCHETYPES.join(', '));
    }

    const tenantName = String(body.tenant_name || body.name || '').trim();
    if (tenantName.length < 2 || tenantName.length > 200) {
        errors.push('tenant_name is required (2..200 chars)');
    }

    // subdomain: lowercase alnum + hyphen, 2..63 (DNS label). Required & UNIQUE in DB.
    // Must start AND end with alnum (no edge hyphen); explicit length guard enforces the
    // stated 2..63 (the regex alone would accept a single character).
    const subdomain = String(body.subdomain || '').trim().toLowerCase();
    if (subdomain.length < 2 || subdomain.length > 63 ||
```

```
        !/^[a-z0-9][a-z0-9-]{0,61}[a-z0-9]$/.test(subdomain)) {
        errors.push('subdomain must be a valid DNS label (a-z, 0-9, hyphen; 2..63 chars)');
    }

    const facilityName = String(body.facility_name || tenantName || '').trim();
    if (facilityName.length < 2 || facilityName.length > 255) {
        errors.push('facility_name is required (2..255 chars)');
    }

    // Optional regulatory identifiers (non-secret). Trimmed, length-bounded; '' if absent.
    const mohLicense = String(body.moh_license || '').trim();
    if (mohLicense.length > 100) errors.push('moh_license must be at most 100 chars');
    const crNo = String(body.cr_no || '').trim();
    if (crNo.length > 100) errors.push('cr_no must be at most 100 chars');
    const vatNo = String(body.vat_no || '').trim();
    if (vatNo.length > 100) errors.push('vat_no must be at most 100 chars');

    // beds: integer >= 0. Bed-less archetypes must have 0.
    let beds = body.beds === undefined || body.beds === null || body.beds === '' ? 0 : Number(body.beds);
    if (!Number.isInteger(beds) || beds < 0 || beds > 100000) {
        errors.push('beds must be a non-negative integer');
        beds = 0;
    } else if (!BED_ARCHETYPES.has(archetype) && beds > 0) {
        errors.push('this archetype does not support inpatient beds (beds must be 0)');
    }

    const currency = String(body.currency || 'SAR').trim().toUpperCase();
    if (!/^[A-Z]{3}$/.test(currency)) errors.push('currency must be a 3-letter ISO code');

    const timezone = String(body.timezone || 'Asia/Riyadh').trim();
    if (timezone.length < 1 || timezone.length > 64 || /[^A-Za-z0-9_+\-/]/.test(timezone)) {
        errors.push('timezone is invalid');
    }

    // Admin user
    const adminUsername = String(body.admin_username || '').trim();
    if (!/^[A-Za-z0-9._-]{3,50}$/.test(adminUsername)) {
        errors.push('admin_username must be 3..50 chars (letters, digits, . _ -)');
    }
    const adminDisplayName = String(body.admin_display_name || adminUsername || '').trim().slice(0, 200);
    // password is OPTIONAL ? if absent we generate a strong one (no default password).
    const adminPassword = body.admin_password !== undefined && body.admin_password !== null
        ? String(body.admin_password) : '';
    if (adminPassword && adminPassword.length < 8) {
        errors.push('admin_password, if supplied, must be at least 8 characters');
    }

    // optional: explicit module override (subset of archetype set). If absent -> archetype default.
    let modules = null;
    if (Array.isArray(body.modules)) {
        modules = body.modules.map(Number).filter(n => Number.isInteger(n) && n >= MODULE_INDEX_MIN && n <= MO
DULE_INDEX_MAX);
        modules = [...new Set(modules)].sort((a, b) => a - b);
        if (modules.length === 0) errors.push('modules, if supplied, must contain valid indices (0..42)');
```

```
    }

    // optional integrations: only allow-listed names, only {name, enabled, config(no-secret)}.
    let integrations = [];
    if (Array.isArray(body.integrations)) {
        for (const it of body.integrations) {
            const nm = String((it && it.name) || '').trim().toUpperCase();
            if (!ALLOWED_INTEGRATIONS.has(nm)) {
                errors.push('integration not allowed: ' + (nm || '(empty)'));
                continue;
            }
            integrations.push({ name: nm, enabled: !!(it && it.enabled) });
        }
    }

    // optional branding/lang (non-secret)
    const language = ['ar', 'en'].includes(String(body.language || 'ar')) ? String(body.language || 'ar') : 'a
r';
    const brandColor = /^#[0-9a-fA-F]{6}$/.test(String(body.brand_color || '')) ? String(body.brand_color) : n
ull;

    if (errors.length) return { ok: false, errors };

    return {
        ok: true,
        errors: [],
        value: {
            archetype, tenantName, subdomain, facilityName, mohLicense, crNo, vatNo, beds, currency, timezone,
            adminUsername, adminDisplayName, adminPassword, modules, integrations, language, brandColor
        }
    };
}

/**
 * Mount the provisioning route. Dependencies are injected so the module stays test-friendly and
 * does not duplicate server.js wiring.
 * @param {object} app express app
 * @param {object} deps { pool, requireAuth, requireRole, logAudit }
 */
function mountOnboardingRoutes(app, deps) {
    const { pool, requireAuth, requireRole, logAudit, requireSuperAdmin, allowlist } = deps;

    async function provisionHandler(req, res, bypassLocalAdminCheck) {
        if (!bypassLocalAdminCheck && req.session.user.role !== 'Admin') {
            logAudit(req.session.user?.id, req.session.user?.display_name,
                'BLOCKED_FACILITY_PROVISION', 'Onboarding',
                `Non-admin attempted facility provisioning (archetype=${String(req.body?.archetype || '').slic
e(0, 40)})`,
                req.ip);
            return res.status(403).json({ error: 'Access denied: only an administrator can provision a facilit
y' });
        }

        const parsed = validateProvisionInput(req.body);
```

```
        if (!parsed.ok) {
            return res.status(400).json({ error: 'Validation failed', details: parsed.errors });
        }
        const v = parsed.value;

        // Resolve module set: explicit subset (intersected with archetype set) OR archetype default.
        const archetypeSet = new Set(modulesForArchetype(v.archetype));
        let enabledModules = [...archetypeSet];
        if (v.modules) {
            const inter = v.modules.filter(i => archetypeSet.has(i));
            // module 0 (Dashboard) and 42 (Settings) always enabled.
            if (!inter.includes(0)) inter.push(0);
            if (!inter.includes(42)) inter.push(42);
            enabledModules = [...new Set(inter)].sort((a, b) => a - b);
        }

        // Generate password if none supplied (NO default password ever).
        const generated = !v.adminPassword;
        const plainPassword = v.adminPassword || generateStrongPassword();

        // Raw pooled client => we control app.tenant_id (transaction-local) ourselves; the patched
        // pool.query wrapper is bypassed because we call client.query directly.
        const client = await pool.connect();
        try {
            await client.query('BEGIN');

            // 1) tenant (archetype). subdomain UNIQUE ? duplicate -> 409.
            const tenantRow = (await client.query(
                `INSERT INTO tenants (name, subdomain, status, plan_type, archetype, moh_license, cr_no, vat_n
o)
                 VALUES ($1,$2,'active','standard',$3,$4,$5,$6) RETURNING id`,
                [v.tenantName, v.subdomain, v.archetype, v.mohLicense, v.crNo, v.vatNo]
            )).rows[0];
            const tenantId = tenantRow.id;

            // Bind RLS context to the NEW tenant for the rest of the transaction (local => auto-reset on COMM
IT/ROLLBACK).
            await client.query("SELECT set_config('app.tenant_id', $1, true)", [String(tenantId)]);

            // 2) facility (type/beds/currency/timezone; parent_facility_id null for the root facility).
            const facilityRow = (await client.query(
                `INSERT INTO facilities (tenant_id, name, type, beds, currency, timezone)
                 VALUES ($1,$2,$3,$4,$5,$6) RETURNING id`,
                [tenantId, v.facilityName, v.archetype, v.beds, v.currency, v.timezone]
            )).rows[0];
            const facilityId = facilityRow.id;

            // 3) default branch + department so the Admin has a primary facility/branch scope (mirror establi
shSession).
            const branchRow = (await client.query(
                `INSERT INTO branches (facility_id, name) VALUES ($1,$2) RETURNING id`,
                [facilityId, v.facilityName + ' - Main']
            )).rows[0];
            const branchId = branchRow.id;
```

```
            await client.query(
                `INSERT INTO departments (branch_id, name_ar, name_en) VALUES ($1,$2,$3)`,
                [branchId, '???????', 'Administration']
            );

            // 4) facility_modules from archetype map.
            for (const idx of enabledModules) {
                await client.query(
                    `INSERT INTO facility_modules (tenant_id, module_index, enabled)
                     VALUES ($1,$2,TRUE) ON CONFLICT (tenant_id, module_index) DO NOTHING`,
                    [tenantId, idx]
                );
            }

            // 5) Admin system_user (bcrypt, NO default password). system_users has no tenant_id column;
            //    tenant scope is linked via user_tenants / user_facilities (mirror establishSession).
            const hash = await bcrypt.hash(plainPassword, 10);
            const adminUser = (await client.query(
                `INSERT INTO system_users (username, password_hash, display_name, role, permissions, is_active
)
                 VALUES ($1,$2,$3,'Admin','*',1) RETURNING id`,
                [v.adminUsername, hash, v.adminDisplayName]
            )).rows[0];
            const adminUserId = adminUser.id;

            await client.query(
                `INSERT INTO user_tenants (user_id, tenant_id, is_active) VALUES ($1,$2,true)
                 ON CONFLICT (user_id, tenant_id) DO NOTHING`,
                [adminUserId, tenantId]
            );
            await client.query(
                `INSERT INTO user_facilities (user_id, facility_id, branch_id, is_primary) VALUES ($1,$2,$3,tr
ue)
                 ON CONFLICT (user_id, facility_id, branch_id) DO NOTHING`,
                [adminUserId, facilityId, branchId]
            );

            // 6) integrations (GATED: name + enabled only; NO secrets/keys/certs). config_json carries
            //    a non-secret marker so the UI can show "configured later".
            for (const it of v.integrations) {
                await client.query(
                    `INSERT INTO integration_settings (tenant_id, integration_name, provider, is_enabled, conf
ig_json)
                     VALUES ($1,$2,$3,$4,$5)`,
                    [tenantId, it.name, it.name, it.enabled ? 1 : 0, JSON.stringify({ gated: true, configured:
 false })]
                );
            }

            // 7) branding / language (non-secret) -> tenant_settings.
            await client.query(
                `INSERT INTO tenant_settings (tenant_id, setting_key, setting_value) VALUES ($1,'language',$2)
                 ON CONFLICT (tenant_id, setting_key) DO UPDATE SET setting_value=EXCLUDED.setting_value`,
                [tenantId, v.language]
```

```
            );
            if (v.brandColor) {
                await client.query(
                    `INSERT INTO tenant_settings (tenant_id, setting_key, setting_value) VALUES ($1,'brand_col
or',$2)
                     ON CONFLICT (tenant_id, setting_key) DO UPDATE SET setting_value=EXCLUDED.setting_value`,
                    [tenantId, v.brandColor]
                );
            }

            // 8) facility_type -> company_settings (tenant-scoped, RLS-bound via app.tenant_id set above).
            await client.query(
                `INSERT INTO company_settings (tenant_id, setting_key, setting_value) VALUES ($1,'facility_typ
e',$2)
                 ON CONFLICT (setting_key) DO UPDATE SET setting_value=EXCLUDED.setting_value`,
                [tenantId, v.archetype]
            );

            await client.query('COMMIT');

            // Audit ? NO password, NO secrets in details.
            logAudit(req.session.user.id, req.session.user.display_name,
                bypassLocalAdminCheck ? 'SUPER_ADMIN_FACILITY_PROVISION' : 'FACILITY_PROVISIONED', 'Onboarding
',
                `tenant_id=${tenantId} archetype=${v.archetype} subdomain=${v.subdomain} facility_id=${facilit
yId} modules=${enabledModules.length} admin_user_id=${adminUserId}`,
                req.ip);

            // 201 ? plaintext password returned ONCE for out-of-band delivery (never logged/stored).
            return res.status(201).json({
                tenant_id: tenantId,
                facility_id: facilityId,
                branch_id: branchId,
                admin_user_id: adminUserId,
                enabled_modules: enabledModules,
                admin_username: v.adminUsername,
                admin_password: plainPassword,
                admin_password_generated: generated
            });
        } catch (e) {
            try { await client.query('ROLLBACK'); } catch (_) { /* best-effort */ }
            if (e && e.code === '23505') { // unique_violation (subdomain or username)
                return res.status(409).json({ error: 'A tenant subdomain or admin username already exists' });
            }
            console.error('Facility provision error:', e.message);
            return res.status(500).json({ error: 'Server error' });
        } finally {
            try { await client.query("SELECT set_config('app.tenant_id', '', false)"); } catch (_) { /* reset
*/ }
            client.release();
        }
    }

    // 1. Local Tenant Onboarding Route
```

```
    app.post('/api/admin/facilities/provision', requireAuth, requireRole('settings'), (req, res) => provisionH
andler(req, res, false));

    // 2. Global Platform Onboarding Route (Super Admin Guarded)
    if (requireSuperAdmin) {
        app.post('/api/super-admin/tenants/provision', requireAuth, requireSuperAdmin(allowlist), (req, res) =
> provisionHandler(req, res, true));
    }
}

module.exports = {
    ARCHETYPE_MODULES,
    VALID_ARCHETYPES,
    ALLOWED_INTEGRATIONS,
    BED_ARCHETYPES,
    ALL_MODULE_INDICES,
    HEALTH_CENTER_MODULES,
    POLYCLINIC_MODULES,
    GENERAL_HOSPITAL_MODULES,
    MODULE_INDEX_MIN,
    MODULE_INDEX_MAX,
    modulesForArchetype,
    generateStrongPassword,
    validateProvisionInput,
    mountOnboardingRoutes
};



========================================
FILE: onboarding_route_guard_test.js
========================================

/**
 * onboarding_route_guard_test.js
 * E0 Facility Onboarding Wizard ? STATIC structural assertions (no DB / no HTTP / no PHI).
 * Asserts the provisioning route is super-admin guarded, runs in a single transaction, audits,
 * uses no default password, and keeps integrations gated (no secrets). Mirrors the
 * settings_user_create_admin_guard_test.js style.
 * Run: node onboarding_route_guard_test.js   (exit 0 = pass, 1 = fail)
 */
'use strict';
const fs = require('fs');
const path = require('path');
const server = fs.readFileSync(path.join(__dirname, 'server.js'), 'utf8');
const ob = fs.readFileSync(path.join(__dirname, 'onboarding.js'), 'utf8');
// comment-stripped executable code (// line + /* block */) for order/secret assertions
const obCode = ob
  .replace(/\/\*[\s\S]*?\*\//g, '')
  .replace(/(^|[^:])\/\/.*$/gm, '$1');

let pass = 0, fail = 0;
const ok = (c, m) => { if (c) { pass++; console.log('  PASS', m); } else { fail++; console.log('  FAIL', m); }
 };
```

```
console.log('\n== server.js wiring ==');
ok(/require\(['"]\.\/onboarding['"]\)/.test(server), 'server.js requires ./onboarding');
ok(/mountOnboardingRoutes\(\s*app\s*,\s*\{[^}]*pool[^}]*requireAuth[^}]*requireRole[^}]*logAudit[^}]*requireSu
perAdmin[^}]*allowlist/.test(server.replace(/\s+/g, ' ')),
  'server.js mounts onboarding with {pool, requireAuth, requireRole, logAudit, requireSuperAdmin, allowlist}')
;

console.log('\n== Route definition & guard order (onboarding.js) ==');
ok(/app\.post\(\s*['"]\/api\/admin\/facilities\/provision['"]\s*,\s*requireAuth\s*,\s*requireRole\(['"]setting
s['"]\)/.test(ob),
  "route mounted: POST /api/admin/facilities/provision with requireAuth + requireRole('settings')");
ok(/app\.post\(\s*['"]\/api\/super-admin\/tenants\/provision['"]\s*,\s*requireAuth\s*,\s*requireSuperAdmin\(al
lowlist\)/.test(ob.replace(/\s+/g, ' ')),
  "route mounted: POST /api/super-admin/tenants/provision with requireAuth + requireSuperAdmin");
ok(/req\.session\.user\.role\s*!==\s*'Admin'/.test(ob), "inline super-admin guard: role !== 'Admin'");
ok(/BLOCKED_FACILITY_PROVISION/.test(ob), 'audit BLOCKED_FACILITY_PROVISION on non-admin');
ok(/return res\.status\(403\)/.test(ob), 'returns 403 for non-admin');
ok(/provisionHandler\(req,\s*res,\s*true\)/.test(ob.replace(/\s+/g, '')), 'provisionHandler bypasses local Adm
in check for super admin route');

// guard must precede any INSERT
const guardIdx = ob.indexOf("!== 'Admin'");
const firstInsert = ob.indexOf('INSERT INTO tenants');
ok(guardIdx > -1 && firstInsert > -1 && guardIdx < firstInsert, 'admin guard precedes INSERT INTO tenants');

console.log('\n== Single transaction ==');
ok(/await client\.query\(['"]BEGIN['"]\)/.test(ob), 'BEGIN present');
ok(/await client\.query\(['"]COMMIT['"]\)/.test(ob), 'COMMIT present');
ok(/ROLLBACK/.test(ob), 'ROLLBACK on error');
ok(/pool\.connect\(\)/.test(ob), 'uses raw pooled client (pool.connect)');
ok(/set_config\(['"]app\.tenant_id['"],\s*\$1,\s*true\)/.test(ob), 'sets app.tenant_id (tx-local) to NEW tenan
t before RLS inserts');

console.log('\n== Audit on success ==');
ok(/FACILITY_PROVISIONED/.test(ob), 'audit FACILITY_PROVISIONED on success');
const commitIdx = obCode.indexOf("client.query('COMMIT')");
// position of the success audit action literal itself (comments stripped), not the logAudit start
const auditCallIdx = obCode.indexOf("'FACILITY_PROVISIONED'");
ok(commitIdx > -1 && auditCallIdx > commitIdx, 'success audit logged after COMMIT');

console.log('\n== No default password ==');
ok(/generateStrongPassword\(\)/.test(ob), 'generates strong password when none supplied');
ok(/bcrypt\.hash\(/.test(ob), 'admin password is bcrypt-hashed');
ok(!/password\s*[:=]\s*['"](admin|password|123456|changeme|default)['"]/i.test(ob), 'no hard-coded default pas
sword literal');
ok(/role:[^]*Admin[^]*permissions/.test(ob.replace(/\n/g, ' ')) || /'Admin','\*'/.test(ob),
  'provisioned system_user is Admin with full permissions');

console.log('\n== Integrations gated / no secrets ==');
// obCode (comment-stripped, defined above) => inspect executable code only: assert no secret column is
// ever written (api_key/api_secret/client_secret/certificate/private_key as code identifiers).
ok(!/api_key|api_secret|client_secret|certificate|private_key/i.test(obCode),
  'no secret column writes in provisioning code (api_key/api_secret/cert/private_key absent)');
```

```
ok(/gated:\s*true/.test(ob), 'integration config marked gated:true (no live secrets)');

console.log('\n== Tenant-scoping of provisioning writes (C1/I2/I3) ==');
// C1: integration_settings INSERT must include tenant_id as the first column + param.
ok(/INSERT INTO integration_settings\s*\(\s*tenant_id\s*,/.test(obCode),
  'C1: integration_settings INSERT includes tenant_id (first column)');
ok(/INSERT INTO integration_settings[\s\S]*?VALUES\s*\(\s*\$1\s*,/.test(obCode) &&
   /integration_settings[\s\S]*?\[\s*tenantId\s*,/.test(obCode),
  'C1: integration_settings VALUES binds tenantId first');
// I3: tenants INSERT persists moh_license / cr_no / vat_no.
ok(/INSERT INTO tenants[\s\S]*?moh_license[\s\S]*?cr_no[\s\S]*?vat_no/.test(obCode),
  'I3: INSERT INTO tenants persists moh_license/cr_no/vat_no');
ok(/v\.mohLicense/.test(obCode) && /v\.crNo/.test(obCode) && /v\.vatNo/.test(obCode),
  'I3: validated reg-id values bound into tenants INSERT');
// I2: facility_type persisted to company_settings = archetype string (drives app.js facilityType).
ok(/INSERT INTO company_settings[\s\S]*?'facility_type'[\s\S]*?v\.archetype/.test(obCode),
  'I2: facility_type written to company_settings as archetype string');

console.log('\n== integration_settings RLS migration present (C1) ==');
ok(/FORCE ROW LEVEL SECURITY/.test(fs.readFileSync(path.join(__dirname, 'migrations', 'e0_04_integration_setti
ngs_rls_up.sql'), 'utf8')),
  'C1: e0_04 integration_settings migration FORCE-RLS present');
ok(/rls_integration_settings_tenant_isolation/.test(fs.readFileSync(path.join(__dirname, 'migrations', 'e0_04_
integration_settings_rls_up.sql'), 'utf8')),
  'C1: e0_04 isolation policy present (matching template)');

console.log('\n== Response shape ==');
ok(/res\.status\(201\)\.json\(/.test(ob), 'returns 201');
ok(/tenant_id:\s*tenantId/.test(ob), '201 body includes tenant_id');

console.log('\n== Security regressions NOT weakened ==');
ok(/FORCE ROW LEVEL SECURITY/.test(fs.readFileSync(path.join(__dirname, 'migrations', 'e0_03_facility_modules_
up.sql'), 'utf8')),
  'facility_modules migration FORCE-RLS present');
ok(/FORCE ROW LEVEL SECURITY/.test(fs.readFileSync(path.join(__dirname, 'migrations', 'e0_02_facilities_extend
_up.sql'), 'utf8')),
  'facilities migration FORCE-RLS present');

console.log(`\n${fail === 0 ? 'ALL PASS' : 'FAILURES'}: ${pass} passed, ${fail} failed`);
process.exit(fail === 0 ? 0 : 1);



========================================
FILE: onboarding_unit_test.js
========================================

/**
 * onboarding_unit_test.js
 * E0 Facility Onboarding Wizard ? UNIT tests (no DB / no HTTP / no PHI).
 * Covers: archetype -> module mapping, input validation, and the no-default-password rule.
 * Run: node onboarding_unit_test.js   (exit 0 = pass, 1 = fail)
 */
'use strict';
```

```
const ob = require('./onboarding');

let pass = 0, fail = 0;
const ok = (c, m) => { if (c) { pass++; console.log('  PASS', m); } else { fail++; console.log('  FAIL', m); }
 };
const eqSet = (a, b) => a.length === b.length && a.every((x, i) => x === b[i]);

console.log('\n== Archetype -> module mapping ==');
const ALL = ob.ALL_MODULE_INDICES;
ok(ALL.length === 43 && ALL[0] === 0 && ALL[42] === 42, 'module index space is 0..42 (43 entries)');
ok(eqSet(ob.modulesForArchetype('medical_city'), ALL), 'medical_city = ALL 43');
ok(eqSet(ob.modulesForArchetype('large_hospital'), ALL), 'large_hospital = ALL 43');
ok(eqSet(ob.modulesForArchetype('polyclinic'), [...new Set(ob.POLYCLINIC_MODULES)].sort((a, b) => a - b)),
  'polyclinic matches existing FACILITY_ALLOWED clinic set');
ok(eqSet(ob.modulesForArchetype('health_center'), [...new Set(ob.HEALTH_CENTER_MODULES)].sort((a, b) => a - b)
),
  'health_center matches existing FACILITY_ALLOWED set');
ok(ob.modulesForArchetype('general_hospital').includes(22) && ob.modulesForArchetype('general_hospital').inclu
des(23),
  'general_hospital includes inpatient (22) + ICU (23)');
ok(!ob.modulesForArchetype('general_hospital').includes(39),
  'general_hospital excludes CME (39)');
ok(eqSet(ob.modulesForArchetype('unknown'), []), 'unknown archetype -> []');
// every archetype set must include Dashboard(0) and Settings(42)
ob.VALID_ARCHETYPES.forEach(a => {
  const m = ob.modulesForArchetype(a);
  ok(m.includes(0) && m.includes(42), `${a} includes Dashboard(0) + Settings(42)`);
  ok(m.every(i => i >= 0 && i <= 42), `${a} indices all in range 0..42`);
});

console.log('\n== Input validation ==');
const base = { archetype: 'general_hospital', tenant_name: 'Test Org', subdomain: 'test-org', facility_name: '
Main', admin_username: 'admin01' };
ok(ob.validateProvisionInput(base).ok, 'valid minimal payload passes');
ok(!ob.validateProvisionInput({ ...base, archetype: 'bogus' }).ok, 'invalid archetype rejected');
ok(!ob.validateProvisionInput({ ...base, subdomain: 'Bad_Sub!' }).ok, 'invalid subdomain rejected');
ok(!ob.validateProvisionInput({ ...base, subdomain: '' }).ok, 'empty subdomain rejected');
ok(!ob.validateProvisionInput({ ...base, subdomain: 'a' }).ok, '1-char subdomain rejected (I4: enforces 2..63)
');
ok(ob.validateProvisionInput({ ...base, subdomain: 'ab' }).ok, '2-char subdomain accepted');
ok(!ob.validateProvisionInput({ ...base, subdomain: '-ab' }).ok, 'leading-hyphen subdomain rejected');
ok(!ob.validateProvisionInput({ ...base, subdomain: 'ab-' }).ok, 'trailing-hyphen subdomain rejected');
ok(!ob.validateProvisionInput({ ...base, subdomain: 'a'.repeat(64) }).ok, '64-char subdomain rejected (>63)');
ok(!ob.validateProvisionInput({ ...base, tenant_name: 'x' }).ok, 'too-short tenant name rejected');
ok(!ob.validateProvisionInput({ ...base, admin_username: 'ab' }).ok, 'too-short admin username rejected');
ok(!ob.validateProvisionInput({ ...base, admin_username: 'has space' }).ok, 'admin username with space rejecte
d');
ok(!ob.validateProvisionInput({ ...base, currency: 'SARS' }).ok, 'bad currency rejected');
ok(ob.validateProvisionInput({ ...base, currency: 'usd' }).value.currency === 'USD', 'currency upper-cased');
// bed-less archetype must reject beds > 0
ok(!ob.validateProvisionInput({ ...base, archetype: 'polyclinic', beds: 5 }).ok, 'polyclinic rejects beds>0');
ok(ob.validateProvisionInput({ ...base, archetype: 'polyclinic', beds: 0 }).ok, 'polyclinic accepts beds=0');
ok(ob.validateProvisionInput({ ...base, beds: 120 }).value.beds === 120, 'general_hospital accepts beds=120');
// integrations allow-list
```

```
ok(!ob.validateProvisionInput({ ...base, integrations: [{ name: 'EVIL', enabled: true }] }).ok, 'disallowed in
tegration rejected');
ok(ob.validateProvisionInput({ ...base, integrations: [{ name: 'nphies', enabled: true }] }).value.integration
s[0].name === 'NPHIES',
  'allowed integration normalized, no secret fields retained');
const intVal = ob.validateProvisionInput({ ...base, integrations: [{ name: 'ZATCA', enabled: true, api_key: 'S
ECRET', api_secret: 'X' }] }).value.integrations[0];
ok(!('api_key' in intVal) && !('api_secret' in intVal), 'integration object carries NO secret fields');

console.log('\n== Regulatory identifiers (I3: moh_license / cr_no / vat_no) ==');
const regVal = ob.validateProvisionInput({ ...base, moh_license: ' MOH-123 ', cr_no: 'CR-9', vat_no: '3001' })
.value;
ok(regVal.mohLicense === 'MOH-123', 'moh_license validated + trimmed into value');
ok(regVal.crNo === 'CR-9', 'cr_no validated into value');
ok(regVal.vatNo === '3001', 'vat_no validated into value');
ok(ob.validateProvisionInput(base).value.mohLicense === '', 'absent moh_license -> empty string');
ok(!ob.validateProvisionInput({ ...base, vat_no: 'x'.repeat(101) }).ok, 'over-long vat_no rejected (>100)');

console.log('\n== No-default-password rule ==');
ok(ob.validateProvisionInput(base).value.adminPassword === '', 'no password supplied -> empty (server generate
s strong one)');
ok(!ob.validateProvisionInput({ ...base, admin_password: 'short' }).ok, 'short supplied password rejected (<8)
');
ok(ob.validateProvisionInput({ ...base, admin_password: 'longenough1' }).value.adminPassword === 'longenough1'
, 'valid supplied password kept');
const p1 = ob.generateStrongPassword(), p2 = ob.generateStrongPassword();
ok(typeof p1 === 'string' && p1.length >= 20, 'generated password is long (>=20)');
ok(p1 !== p2, 'generated passwords are random (two differ)');
ok(!/^(admin|password|123|changeme|default)/i.test(p1), 'generated password is not a guessable default');

console.log(`\n${fail === 0 ? 'ALL PASS' : 'FAILURES'}: ${pass} passed, ${fail} failed`);
process.exit(fail === 0 ? 0 : 1);
```

```
========================================
FILE: opd_beds_nphies_test.js
========================================

/**
 * opd_beds_nphies_test.js ? Epic Phase 2 integration & safety checks.
 * Tests:
 * 1) OPD Doctor Queue API: fetches appointments for CURRENT_DATE, joins patients, extracts latest vitals, ten
ant-isolated.
 * 2) OPD Encounter Start API: transitions status to 'In-Consultation', audits start, tenant-isolated.
 * 3) Insurance Eligibility (NPHIES) API: records request, routes through NPHIES gateway, tenant-isolated.
 * 4) Centralized Bed Census API: lists beds/wards occupancy details for the current tenant only.
 */
const fs = require('fs');
const path = require('path');
const G = '\x1b[32m', R = '\x1b[31m', X = '\x1b[0m';
let passed = 0, failed = 0;

function assert(cond, name, extra = '') {
```

```
    if (cond) {
        console.log(`  ${G}PASS${X} ${name}`);
        passed++;
    } else {
        console.log(`  ${R}FAIL${X} ${name}${extra ? ' | ' + extra : ''}`);
        failed++;
    }
}

// 1. Load server.js and public/js/app.js for static structure check
const serverContent = fs.readFileSync(path.join(__dirname, 'server.js'), 'utf8');
const appContent = fs.readFileSync(path.join(__dirname, 'public/js/app.js'), 'utf8');

console.log('\n[ 1 ] Static Audit: OPD, Beds, and NPHIES Integration');

assert(
    serverContent.includes('/api/opd/doctor/queue') && serverContent.includes('/api/opd/encounter/start'),
    'OPD doctor queue and encounter start routes exist in backend'
);
assert(
    serverContent.includes('/api/insurance/eligibility') && serverContent.includes('insurance_eligibility_chec
ks'),
    'Insurance Eligibility (NPHIES) APIs and table definitions exist'
);
assert(
    serverContent.includes('/api/adt/beds') || serverContent.includes('/api/beds/census'),
    'Centralized Bed Board / Census APIs exist in backend'
);
assert(
    appContent.includes('window.startOpdEncounter') && appContent.includes('window.selectOpdPatient'),
    'OPD doctor workspace helpers startOpdEncounter & selectOpdPatient defined on frontend'
);

// 2. Behavioral Simulation (Mock Engine)
console.log('\n[ 2 ] Behavioral Simulation');

// Mock Data
const mockDb = {
    tenants: [{ id: 1 }, { id: 2 }],
    patients: [
        { id: 101, file_number: '1001', name_ar: '???? ???', tenant_id: 1 },
        { id: 102, file_number: '1002', name_ar: '???? ????', tenant_id: 2 }
    ],
    appointments: [
        { id: 1, patient_id: 101, appt_date: '2026-07-06', appt_time: '09:00', doctor_name: 'Dr. Salem', statu
s: 'Checked-In', tenant_id: 1 },
        { id: 2, patient_id: 102, appt_date: '2026-07-06', appt_time: '10:00', doctor_name: 'Dr. Salem', statu
s: 'Checked-In', tenant_id: 2 }
    ],
    nursing_vitals: [
        { id: 1, patient_id: 101, bp: '120/80', temp: 37.0, pulse: 75, tenant_id: 1 }
    ],
    beds: [
        { id: 1, ward_id: 1, bed_number: 'B1', status: 'Available', tenant_id: 1 },
```

```
        { id: 2, ward_id: 1, bed_number: 'B2', status: 'Occupied', tenant_id: 1 },
        { id: 3, ward_id: 1, bed_number: 'B3', status: 'Available', tenant_id: 2 }
    ]
};

// A. OPD Doctor Queue simulation with vitals joining & tenant isolation
function simulateOpdQueue(doctorName, tenantId) {
    // 1. Filter appointments for tenant, date (2026-07-06), status not Cancelled
    const appts = mockDb.appointments.filter(
        a => a.tenant_id === tenantId && a.appt_date === '2026-07-06' && a.status !== 'Cancelled'
    );

    // 2. Join patient details and fetch latest vitals
    return appts.map(a => {
        const patient = mockDb.patients.find(p => p.id === a.patient_id);
        const vitals = mockDb.nursing_vitals
            .filter(v => v.patient_id === a.patient_id && v.tenant_id === tenantId)
            .sort((x, y) => y.id - x.id)[0] || null;

        return {
            ...a,
            patient_name: patient ? patient.name_ar : '',
            file_number: patient ? patient.file_number : '',
            vitals
        };
    });
}

// Test cases for OPD Doctor Queue
const queueT1 = simulateOpdQueue('Dr. Salem', 1);
assert(
    queueT1.length === 1 && queueT1[0].id === 1 && queueT1[0].vitals !== null,
    'OPD Doctor Queue correctly fetches tenant 1 appointments and joins vitals'
);

const queueT2 = simulateOpdQueue('Dr. Salem', 2);
assert(
    queueT2.length === 1 && queueT2[0].id === 2 && queueT2[0].vitals === null,
    'OPD Doctor Queue enforces tenant isolation and does not mix patient records'
);

// B. OPD Encounter Start simulation
function simulateStartEncounter(apptId, tenantId) {
    const appt = mockDb.appointments.find(a => a.id === apptId && a.tenant_id === tenantId);
    if (!appt) return { status: 404, error: 'Appointment not found' };

    appt.status = 'In-Consultation';
    return { status: 200, appointment: appt };
}

// Test cases for Encounter Start
let startRes1 = simulateStartEncounter(2, 1); // Tenant 1 trying to start Tenant 2 appointment
assert(
    startRes1.status === 404,
```

```
    'Encounter Start enforces tenant isolation and blocks cross-tenant access'
);

let startRes2 = simulateStartEncounter(1, 1); // Correct tenant
assert(
    startRes2.status === 200 && startRes2.appointment.status === 'In-Consultation',
    'Encounter Start successfully starts the consultation and updates status'
);

// C. Centralized Bed Board simulation
function simulateBedCensus(tenantId) {
    const beds = mockDb.beds.filter(b => b.tenant_id === tenantId);
    const total = beds.length;
    const occupied = beds.filter(b => b.status === 'Occupied').length;
    const available = beds.filter(b => b.status === 'Available').length;
    return {
        beds,
        total,
        occupied,
        available,
        occupancyRate: total > 0 ? Math.round((occupied / total) * 100) : 0
    };
}

// Test cases for Bed Board
const censusT1 = simulateBedCensus(1);
assert(
    censusT1.total === 2 && censusT1.occupied === 1 && censusT1.occupancyRate === 50,
    'Centralized Bed Board counts total, occupied, and occupancy rate correctly for Tenant 1'
);

const censusT2 = simulateBedCensus(2);
assert(
    censusT2.total === 1 && censusT2.beds[0].id === 3,
    'Centralized Bed Board isolates beds and wards by tenant_id'
);

console.log(`\n[ PHASE 2 QUALITY ] passed=${passed} failed=${failed}`);
process.exit(failed ? 1 : 0);




========================================
FILE: ophthalmology_integration_test.js
========================================

/**
 * ophthalmology_integration_test.js ? Integration test for the Ophthalmology module HTTP endpoints.
 */
'use strict';

process.env.NODE_ENV = 'staging';

const { spawn } = require('child_process');
```

```
const http = require('http');
const assert = require('assert');
const bcrypt = require('bcryptjs');
const { pool } = require('./db_postgres');

const TEST_PORT = 3020;
const TEST_USERNAME = 'oph_doctor';
const TEST_PASSWORD = 'OPH_PASSWORD_PLACEHOLDER';

let serverProcess;

function makeRequest(method, path, body, headers = {}) {
    return new Promise((resolve, reject) => {
        const payload = body ? JSON.stringify(body) : '';
        const req = http.request({
            hostname: 'localhost',
            port: TEST_PORT,
            path,
            method,
            headers: {
                'Content-Type': 'application/json',
                'Content-Length': Buffer.byteLength(payload),
                ...headers
            }
        }, (res) => {
            let data = '';
            res.on('data', chunk => data += chunk);
            res.on('end', () => {
                try {
                    const parsed = data ? JSON.parse(data) : {};
                    resolve({ statusCode: res.statusCode, headers: res.headers, body: parsed });
                } catch (e) {
                    resolve({ statusCode: res.statusCode, headers: res.headers, rawBody: data });
                }
            });
        });
        req.on('error', reject);
        if (payload) req.write(payload);
        req.end();
    });
}

async function runTests() {
    console.log('--- STARTING OPHTHALMOLOGY MODULE INTEGRATION TESTS ---');

    const patientId = 9989;
    const doctorUserId = 9990;
    const client = await pool.connect();

    try {
        console.log('Setting up test data...');
        await client.query("SET app.tenant_id = '1'");

        // Clean up old test data
```

```
        await client.query('DELETE FROM eye_exams WHERE patient_id = $1', [patientId]);
        await client.query('DELETE FROM patients WHERE id = $1', [patientId]);
        await client.query('DELETE FROM user_tenants WHERE user_id = $1', [doctorUserId]);
        await client.query('DELETE FROM system_users WHERE id = $1', [doctorUserId]);

        // Insert patient
        await client.query(
            'INSERT INTO patients (id, name_en, name_ar, tenant_id) VALUES ($1, $2, $3, 1)',
            [patientId, 'Oph Test Patient', '???? ??????']
        );

        // Insert doctor user
        const hashedPassword = await bcrypt.hash(TEST_PASSWORD, 10);
        await client.query(
            'INSERT INTO system_users (id, username, password_hash, display_name, role, speciality, permission
s, is_active) VALUES ($1, $2, $3, $4, $5, $6, $7, 1)',
            [doctorUserId, TEST_USERNAME, hashedPassword, 'Dr. Ophthalmologist', 'Doctor', 'Ophthalmology', '[
"patients", "prescriptions"]']
        );

        // Associate doctor with tenant 1
        await client.query(
            'INSERT INTO user_tenants (user_id, tenant_id, is_active) VALUES ($1, 1, true)',
            [doctorUserId]
        );

        console.log('Spawning test server...');
        serverProcess = spawn('node', ['server.js'], {
            env: { ...process.env, PORT: TEST_PORT, NODE_ENV: 'staging', SKIP_DB_INIT: 'true' },
            stdio: 'pipe'
        });

        serverProcess.stdout.on('data', (data) => {
            console.log(`[SERVER] ${data.toString().trim()}`);
        });

        serverProcess.stderr.on('data', (data) => {
            console.error(`[SERVER ERR] ${data.toString().trim()}`);
        });

        // Wait 3 seconds for server to boot
        await new Promise(resolve => setTimeout(resolve, 3000));

        console.log('Logging in...');
        const loginRes = await makeRequest('POST', '/api/auth/login', {
            username: TEST_USERNAME,
            password: TEST_PASSWORD
        });

        assert.strictEqual(loginRes.statusCode, 200, 'Login should succeed');
        console.log('? Logged in successfully.');

        // Extract session cookie
        const setCookie = loginRes.headers['set-cookie'];
```

```
        assert.ok(setCookie, 'Should receive session cookie');
        const cookie = setCookie[0].split(';')[0];

        // 1. Test POST /api/ophthalmology/exams (Create eye exam)
        console.log('Testing create eye exam...');
        const createRes = await makeRequest('POST', '/api/ophthalmology/exams', {
            patient_id: patientId,
            od_va_uncorrected: '20/30',
            os_va_uncorrected: '20/40',
            od_va_corrected: '20/20',
            os_va_corrected: '20/20',
            od_iop: 16,
            os_iop: 24,
            iop_method: 'GAT',
            od_sphere: -2.50,
            os_sphere: -2.75,
            od_cylinder: -0.50,
            os_cylinder: -0.75,
            od_axis: 180,
            os_axis: 170,
            od_add: 1.50,
            os_add: 1.50,
            slit_lamp_exam: 'Clear cornea, quiet anterior chamber',
            fundoscopy_exam: 'Normal optic disc, cup-disc ratio 0.3',
            notes: 'High IOP OS - scheduled for glaucoma workup'
        }, { 'Cookie': cookie });

        assert.strictEqual(createRes.statusCode, 200, 'Should return 200 OK');
        assert.strictEqual(createRes.body.success, true, 'Should return success true');
        assert.ok(createRes.body.id, 'Should return record ID');
        const examId = createRes.body.id;
        console.log(`? Eye exam created successfully. ID: ${examId}`);

        // 2. Test GET /api/ophthalmology/exams/patient/:patient_id
        console.log('Testing get patient eye exams...');
        const getRes = await makeRequest('GET', `/api/ophthalmology/exams/patient/${patientId}`, null, { 'Cook
ie': cookie });
        assert.strictEqual(getRes.statusCode, 200, 'Should return 200 OK');
        assert.strictEqual(getRes.body.length, 1, 'Should return exactly 1 record');
        assert.strictEqual(getRes.body[0].id, examId, 'Record ID should match');
        console.log('RETRIVED EYE EXAM RECORD:', getRes.body[0]);
        assert.strictEqual(parseFloat(getRes.body[0].od_sphere), -2.50, 'OD SPH should match');
        assert.strictEqual(parseInt(getRes.body[0].os_iop), 24, 'OS IOP should match');
        assert.strictEqual(parseFloat(getRes.body[0].os_add), 1.50, 'OS ADD should match');
        console.log('? Patient eye exams retrieved successfully.');

    } finally {
        console.log('Cleaning up test data...');
        await client.query("SET app.tenant_id = '1'");
        await client.query('DELETE FROM eye_exams WHERE patient_id = $1', [patientId]);
        await client.query('DELETE FROM patients WHERE id = $1', [patientId]);
        await client.query('DELETE FROM user_tenants WHERE user_id = $1', [doctorUserId]);
        await client.query('DELETE FROM system_users WHERE id = $1', [doctorUserId]);
        client.release();
```

```
        if (serverProcess) {
            console.log('Killing test server...');
            serverProcess.kill();
        }
    }

    console.log('? Ophthalmology Module Integration Tests passed successfully!\n');
}

if (require.main === module) {
    runTests().catch(async err => {
        console.error('? Test failed:', err);
        // Wait 3 seconds for server logs to flush
        await new Promise(resolve => setTimeout(resolve, 3000));
        if (serverProcess) serverProcess.kill();
        process.exit(1);
    });
}

module.exports = { runTests };
```

```
========================================
FILE: orders.js
========================================

/**
 * orders.js ? E-X1 unified orders thin server module (additive).
 *
 * mountOrderRoutes(app, { pool, requireAuth, requireTenantScope, getRequestTenantContext, logAudit, requirePe
rmission })
 *
 * Exposes:
 *   POST /api/orders        ? create one order header + its line items in ONE transaction, tenant-scoped, aud
ited.
 *   GET  /api/orders        ? list orders for the tenant; optional ?encounter_id= and ?patient_id= filters.
 *
 * Routing-by-type is intentionally a STUB: it records type ? {lab,rad,med,consult} on the order row and
 * leaves the per-department dispatch (lab_radiology_orders / prescriptions+pharmacy_prescriptions_queue)
 * to the existing handlers ? this module does NOT implement lab/rad/pharmacy internals.
 *
 * Tenant isolation: orders/order_items/order_sets are under FORCE RLS. The transaction uses a dedicated
 * client from pool.connect(); the db_postgres pool.query wrapper that auto-binds app.tenant_id does NOT
 * apply to a raw client, so we set_config('app.tenant_id', ...) on the client ourselves (and reset it in
 * finally), exactly mirroring db_postgres.js. tenant_id/facility_id are stamped from the trusted session.
 *
 * Mounted AFTER all existing routes and BEFORE the SPA catch-all (server.js wiring).
 */
'use strict';

const VALID_TYPES = ['lab', 'rad', 'med', 'consult'];
const VALID_STATUSES = ['pending', 'active', 'completed', 'cancelled'];
```

```
function mountOrderRoutes(app, deps) {
    const {
        pool,
        requireAuth,
        requireTenantScope,
        getRequestTenantContext,
        logAudit,
        // optional: additive RBAC matrix guard; if absent, route relies on requireAuth + requireTenantScope.
        requirePermission,
    } = deps || {};

    if (!app || !pool || typeof requireAuth !== 'function' || typeof requireTenantScope !== 'function' ||
        typeof getRequestTenantContext !== 'function') {
        throw new Error('mountOrderRoutes requires { app, pool, requireAuth, requireTenantScope, getRequestTen
antContext }');
    }

    const audit = typeof logAudit === 'function' ? logAudit : () => {};

    // Build the middleware chain additively: auth -> tenant scope -> (optional) permission guard.
    const createGuards = [requireAuth, requireTenantScope];
    const viewGuards = [requireAuth, requireTenantScope];
    if (typeof requirePermission === 'function') {
        createGuards.push(requirePermission('orders:create'));
        viewGuards.push(requirePermission('orders:view'));
    }

    function isMissingRelationError(e) {
        return e && (e.code === '42P01' || /relation .* does not exist/i.test(e.message || ''));
    }

    async function listLegacyOrders({ tenantId, patient_id }) {
        const rows = [];
        const params = [];
        const conds = [];
        if (tenantId) { params.push(tenantId); conds.push(`tenant_id = $${params.length}`); }
        if (patient_id) { params.push(patient_id); conds.push(`patient_id = $${params.length}`); }
        const where = conds.length ? ' WHERE ' + conds.join(' AND ') : '';

        const labRows = (await pool.query(
            `SELECT id, tenant_id, facility_id, patient_id,
                    CASE WHEN COALESCE(is_radiology,0)=1 THEN 'rad' ELSE 'lab' END AS type,
                    LOWER(COALESCE(NULLIF(status,''),'pending')) AS status,
                    created_at,
                    COALESCE(NULLIF(description,''), NULLIF(order_type,''), 'Legacy order') AS item_summary,
                    1::int AS item_count
             FROM lab_radiology_orders${where} ORDER BY id DESC`,
            params
        ).catch(e => {
            if (isMissingRelationError(e)) return { rows: [] };
            throw e;
        })).rows;
        rows.push(...labRows);
```

```
        const rxRows = (await pool.query(
            `SELECT id, tenant_id, facility_id, patient_id, 'med' AS type,
                    LOWER(COALESCE(NULLIF(status,''),'pending')) AS status,
                    created_at,
                    ('Medication #' || COALESCE(medication_id::text, id::text) || COALESCE(' - ' || NULLIF(dos
age,''), '')) AS item_summary,
                    1::int AS item_count
             FROM prescriptions${where} ORDER BY id DESC`,
            params
        ).catch(e => {
            if (isMissingRelationError(e)) return { rows: [] };
            throw e;
        })).rows;
        rows.push(...rxRows);

        const mrRows = (await pool.query(
            `SELECT id, tenant_id, facility_id, patient_id,
                    CASE
                      WHEN diagnosis ILIKE '[CONSULT]%' THEN 'consult'
                      WHEN diagnosis ILIKE '[RAD]%' THEN 'rad'
                      WHEN diagnosis ILIKE '[MED]%' THEN 'med'
                      ELSE 'consult'
                    END AS type,
                    LOWER(COALESCE(NULLIF(emr_status,''),'pending')) AS status,
                    visit_date AS created_at,
                    COALESCE(NULLIF(diagnosis,''), NULLIF(notes,''), 'Clinical order') AS item_summary,
                    1::int AS item_count
             FROM medical_records${where ? where + ' AND' : ' WHERE'} (diagnosis LIKE '[%]%' OR notes LIKE '[%
]%') ORDER BY id DESC`,
            params
        ).catch(e => {
            if (isMissingRelationError(e)) return { rows: [] };
            throw e;
        })).rows;
        rows.push(...mrRows);

        return rows.sort((a, b) => new Date(b.created_at || 0) - new Date(a.created_at || 0));
    }

    // ===== POST /api/orders ? create order + items in ONE transaction =====
    app.post('/api/orders', ...createGuards, async (req, res) => {
        const { patient_id, type, encounter_id, order_set_id, status, items } = req.body || {};

        // Validate type against the same CHECK constraint the DB enforces.
        if (!VALID_TYPES.includes(type)) {
            return res.status(400).json({ error: 'Invalid order type', allowed: VALID_TYPES });
        }
        if (!patient_id) {
            return res.status(400).json({ error: 'patient_id is required' });
        }

        // Validate status against the same CHECK constraint the DB enforces; default to 'pending' if absent.
        const orderStatus = (status === undefined || status === null || status === '') ? 'pending' : status;
```

```
        if (!VALID_STATUSES.includes(orderStatus)) {
            return res.status(400).json({ error: 'Invalid order status', allowed: VALID_STATUSES });
        }

        const { tenantId, facilityId } = getRequestTenantContext(req);
        const orderedBy = req.session?.user?.id || null;
        const lineItems = Array.isArray(items) ? items : [];

        const client = await pool.connect();
        try {
            // FORCE RLS: a raw client is NOT covered by the pool.query tenant wrapper ? bind it ourselves.
            await client.query("SELECT set_config('app.tenant_id', $1, false)", [tenantId ? String(tenantId) :
 '']);

            await client.query('BEGIN');

            // Defense-in-depth: confirm the patient belongs to this tenant before creating the order.
            if (tenantId) {
                const pcheck = await client.query('SELECT id FROM patients WHERE id=$1 AND tenant_id=$2', [pat
ient_id, tenantId]);
                if (!pcheck.rows[0]) {
                    await client.query('ROLLBACK');
                    return res.status(404).json({ error: 'Patient not found' });
                }
            }

            // RLS-bypass defense (C1): the orders->order_sets FK is checked by PostgreSQL BYPASSING RLS,
            // so a caller could reference another tenant's order_set. Validate ownership in the SAME
            // transaction/client before the INSERT (mirrors the patient_id ownership check above).
            if (order_set_id && tenantId) {
                const setChk = await client.query('SELECT id FROM order_sets WHERE id=$1 AND tenant_id=$2', [o
rder_set_id, tenantId]);
                if (setChk.rowCount === 0) {
                    await client.query('ROLLBACK');
                    return res.status(400).json({ error: 'Invalid order_set' });
                }
            }

            const orderRes = await client.query(
                `INSERT INTO orders (tenant_id, facility_id, encounter_id, patient_id, type, status, ordered_b
y, order_set_id)
                 VALUES ($1,$2,$3,$4,$5,$6,$7,$8) RETURNING id, type, status, patient_id, encounter_id`,
                [tenantId || null, facilityId || null, encounter_id || null, patient_id, type,
                 orderStatus, orderedBy, order_set_id || null]
            );
            const order = orderRes.rows[0];

            for (const it of lineItems) {
                await client.query(
                    `INSERT INTO order_items (tenant_id, order_id, catalog_ref, qty, instructions)
                     VALUES ($1,$2,$3,$4,$5)`,
                    [tenantId || null, order.id, it.catalog_ref || null,
                     parseInt(it.qty, 10) > 0 ? parseInt(it.qty, 10) : 1, it.instructions || null]
                );
```

```
            }

            await client.query('COMMIT');

            // Routing-by-type STUB: the order is recorded; per-department dispatch is left to existing handle
rs.
            audit(orderedBy, req.session?.user?.display_name, 'CREATE_ORDER', 'Orders',
                `Created ${type} order #${order.id} for patient #${patient_id} (${lineItems.length} item(s))`,
 req.ip);

            return res.json({ ...order, items: lineItems.length, dispatch: 'recorded' });
        } catch (e) {
            try { await client.query('ROLLBACK'); } catch (_) { /* best effort */ }
            console.error('POST /api/orders error:', e.message);
            return res.status(500).json({ error: 'Server error' });
        } finally {
            try { await client.query("SELECT set_config('app.tenant_id', '', false)"); } catch (_) { /* reset
best-effort */ }
            client.release();
        }
    });

    // ===== GET /api/orders ? list tenant orders, optional ?encounter_id= / ?patient_id= =====
    app.get('/api/orders', ...viewGuards, async (req, res) => {
        try {
            const { tenantId } = getRequestTenantContext(req);
            const { encounter_id, patient_id } = req.query;

            const where = [];
            const params = [];
            if (tenantId) { params.push(tenantId); where.push(`tenant_id = $${params.length}`); }
            if (encounter_id) { params.push(encounter_id); where.push(`encounter_id = $${params.length}`); }
            if (patient_id) { params.push(patient_id); where.push(`patient_id = $${params.length}`); }

            const sql = `SELECT o.id, o.tenant_id, o.facility_id, o.encounter_id, o.patient_id, o.type, o.stat
us,
                                o.ordered_by, o.order_set_id, o.created_at,
                                COALESCE(COUNT(oi.id), 0)::int AS item_count,
                                COALESCE(STRING_AGG(NULLIF(oi.catalog_ref, ''), ', ' ORDER BY oi.id), '') AS i
tem_summary
                        FROM orders o
                        LEFT JOIN order_items oi ON oi.order_id = o.id AND (oi.tenant_id = o.tenant_id OR oi.
tenant_id IS NULL)
                        ${where.length ? ' WHERE ' + where.map(w => 'o.' + w).join(' AND ') : ''}
                        GROUP BY o.id, o.tenant_id, o.facility_id, o.encounter_id, o.patient_id, o.type, o.st
atus,
                                 o.ordered_by, o.order_set_id, o.created_at
                        ORDER BY o.id DESC`;
            const { rows } = await pool.query(sql, params);
            return res.json(rows);
        } catch (e) {
            if (isMissingRelationError(e)) {
                try {
                    const { tenantId } = getRequestTenantContext(req);
```

```
                    const rows = await listLegacyOrders({ tenantId, patient_id: req.query.patient_id });
                    return res.json(rows);
                } catch (fallbackErr) {
                    console.error('GET /api/orders legacy fallback error:', fallbackErr.message);
                }
            }
            console.error('GET /api/orders error:', e.message);
            return res.status(500).json({ error: 'Server error' });
        }
    });
}

module.exports = { mountOrderRoutes, VALID_TYPES, VALID_STATUSES };
```

```
========================================
FILE: orthopedics_integration_test.js
========================================

/**
 * orthopedics_integration_test.js ? Integration test for the Orthopedics module HTTP endpoints.
 */
'use strict';

process.env.NODE_ENV = 'staging';

const { spawn } = require('child_process');
const http = require('http');
const assert = require('assert');
const bcrypt = require('bcryptjs');
const { pool } = require('./db_postgres');

const TEST_PORT = 3019;
const TEST_USERNAME = 'ortho_doctor';
const TEST_PASSWORD = 'ORTHO_PASSWORD_PLACEHOLDER';

let serverProcess;

function makeRequest(method, path, body, headers = {}) {
    return new Promise((resolve, reject) => {
        const payload = body ? JSON.stringify(body) : '';
        const req = http.request({
            hostname: 'localhost',
            port: TEST_PORT,
            path,
            method,
            headers: {
                'Content-Type': 'application/json',
                'Content-Length': Buffer.byteLength(payload),
                ...headers
            }
        }, (res) => {
            let data = '';
```

```
            res.on('data', chunk => data += chunk);
            res.on('end', () => {
                try {
                    const parsed = data ? JSON.parse(data) : {};
                    resolve({ statusCode: res.statusCode, headers: res.headers, body: parsed });
                } catch (e) {
                    resolve({ statusCode: res.statusCode, headers: res.headers, rawBody: data });
                }
            });
        });
        req.on('error', reject);
        if (payload) req.write(payload);
        req.end();
    });
}

async function runTests() {
    console.log('--- STARTING ORTHOPEDICS MODULE INTEGRATION TESTS ---');

    const patientId = 9991;
    const doctorUserId = 9992;
    const client = await pool.connect();

    try {
        console.log('Setting up test data...');
        await client.query("SET app.tenant_id = '1'");

        // Clean up old test data
        await client.query('DELETE FROM orthopedic_implants WHERE patient_id = $1', [patientId]);
        await client.query('DELETE FROM joint_rom_assessments WHERE patient_id = $1', [patientId]);
        await client.query('DELETE FROM patients WHERE id = $1', [patientId]);
        await client.query('DELETE FROM user_tenants WHERE user_id = $1', [doctorUserId]);
        await client.query('DELETE FROM system_users WHERE id = $1', [doctorUserId]);

        // Insert patient
        await client.query(
            'INSERT INTO patients (id, name_en, name_ar, tenant_id) VALUES ($1, $2, $3, 1)',
            [patientId, 'Ortho Test Patient', '???? ??????']
        );

        // Insert doctor user
        const hashedPassword = await bcrypt.hash(TEST_PASSWORD, 10);
        await client.query(
            'INSERT INTO system_users (id, username, password_hash, display_name, role, speciality, permission
s, is_active) VALUES ($1, $2, $3, $4, $5, $6, $7, 1)',
            [doctorUserId, TEST_USERNAME, hashedPassword, 'Dr. Orthopedist', 'Doctor', 'Orthopedics', '["patie
nts", "prescriptions"]']
        );

        // Associate doctor with tenant 1
        await client.query(
            'INSERT INTO user_tenants (user_id, tenant_id, is_active) VALUES ($1, 1, true)',
            [doctorUserId]
        );
```

```
        console.log('Spawning test server...');
        serverProcess = spawn('node', ['server.js'], {
            env: { ...process.env, PORT: TEST_PORT, NODE_ENV: 'staging', SKIP_DB_INIT: 'true' },
            stdio: 'inherit'
        });

        // Wait 3 seconds for server to boot
        await new Promise(resolve => setTimeout(resolve, 3000));

        console.log('Logging in...');
        const loginRes = await makeRequest('POST', '/api/auth/login', {
            username: TEST_USERNAME,
            password: TEST_PASSWORD
        });

        assert.strictEqual(loginRes.statusCode, 200, 'Login should succeed');
        console.log('? Logged in successfully.');

        // Extract session cookie
        const setCookie = loginRes.headers['set-cookie'];
        assert.ok(setCookie, 'Should receive session cookie');
        const cookie = setCookie[0].split(';')[0];

        // 1. Test POST /api/orthopedics/implants (Register implant)
        console.log('Testing register orthopedic implant...');
        const implantRes = await makeRequest('POST', '/api/orthopedics/implants', {
            patient_id: patientId,
            implant_type: 'Total Knee Joint',
            manufacturer: 'Zimmer Biomet',
            model_name: 'Persona Knee System',
            serial_number: 'SN-98483120',
            size_dimension: 'Size 6',
            batch_lot_number: 'LOT-2026-X',
            clinical_notes: 'Successful replacement, stable placement'
        }, { 'Cookie': cookie });

        assert.strictEqual(implantRes.statusCode, 200, 'Should return 200 OK');
        assert.strictEqual(implantRes.body.success, true, 'Should return success true');
        assert.ok(implantRes.body.id, 'Should return record ID');
        const implantId = implantRes.body.id;
        console.log(`? Orthopedic implant registered successfully. ID: ${implantId}`);

        // 2. Test GET /api/orthopedics/implants/patient/:patient_id
        console.log('Testing get patient implants...');
        const getImplantRes = await makeRequest('GET', `/api/orthopedics/implants/patient/${patientId}`, null,
{ 'Cookie': cookie });
        assert.strictEqual(getImplantRes.statusCode, 200, 'Should return 200 OK');
        assert.strictEqual(getImplantRes.body.length, 1, 'Should return exactly 1 record');
        assert.strictEqual(getImplantRes.body[0].id, implantId, 'Record ID should match');
        assert.strictEqual(getImplantRes.body[0].serial_number, 'SN-98483120', 'Serial number should match');
        console.log('? Patient implants retrieved successfully.');

        // 3. Test POST /api/orthopedics/rom (Save joint ROM assessment)
```

```
        console.log('Testing save joint ROM...');
        const romRes = await makeRequest('POST', '/api/orthopedics/rom', {
            patient_id: patientId,
            joint_name: 'Knee',
            lateral_side: 'Right',
            movement_type: 'Flexion',
            angle_degrees: 95,
            is_restricted: true
        }, { 'Cookie': cookie });

        assert.strictEqual(romRes.statusCode, 200, 'Should return 200 OK');
        assert.strictEqual(romRes.body.success, true, 'Should return success true');
        assert.ok(romRes.body.id, 'Should return record ID');
        const romId = romRes.body.id;
        console.log(`? Joint ROM saved successfully. ID: ${romId}`);

        // 4. Test GET /api/orthopedics/rom/patient/:patient_id
        console.log('Testing get patient ROM...');
        const getRomRes = await makeRequest('GET', `/api/orthopedics/rom/patient/${patientId}`, null, { 'Cooki
e': cookie });
        assert.strictEqual(getRomRes.statusCode, 200, 'Should return 200 OK');
        assert.strictEqual(getRomRes.body.length, 1, 'Should return exactly 1 record');
        assert.strictEqual(getRomRes.body[0].id, romId, 'Record ID should match');
        assert.strictEqual(getRomRes.body[0].is_restricted, true, 'ROM restriction flag should be true');
        console.log('? Patient ROM retrieved successfully.');

    } finally {
        console.log('Cleaning up test data...');
        await client.query("SET app.tenant_id = '1'");
        await client.query('DELETE FROM orthopedic_implants WHERE patient_id = $1', [patientId]);
        await client.query('DELETE FROM joint_rom_assessments WHERE patient_id = $1', [patientId]);
        await client.query('DELETE FROM patients WHERE id = $1', [patientId]);
        await client.query('DELETE FROM user_tenants WHERE user_id = $1', [doctorUserId]);
        await client.query('DELETE FROM system_users WHERE id = $1', [doctorUserId]);
        client.release();

        if (serverProcess) {
            console.log('Killing test server...');
            serverProcess.kill();
        }
    }

    console.log('? Orthopedics Module Integration Tests passed successfully!\n');
}

if (require.main === module) {
    runTests().catch(err => {
        console.error('? Test failed:', err);
        if (serverProcess) serverProcess.kill();
        process.exit(1);
    });
}

module.exports = { runTests };
```

```
========================================
FILE: owner_staging_handoff_test.js
========================================

/**
 * owner_staging_handoff_test.js
 * ========================================
 * Static validation test for Staging Owner Handoff and Signoff phase.
 * Verifies that:
 * 1. All 6 handoff and signoff documents exist in the parent directory.
 * 2. Only placeholders (no real secrets or passwords) are used in the command drafts.
 * 3. No DDL execution or migration files were generated.
 * 4. isLiveEndpointEnabled and isWriteOperationEnabled remain false.
 */

const assert = require('assert');
const fs = require('fs');
const path = require('path');

// Mock browser globals for contract verification
global.window = {};
require('./public/js/enterprise-contracts.js');

console.log('Running Owner Staging Handoff & Signoff Static Tests...');

// 1. Live actions must remain disabled (always false)
assert.strictEqual(global.window.isLiveEndpointEnabled(), false, 'Live endpoints must remain disabled');
assert.strictEqual(global.window.isWriteOperationEnabled(), false, 'Write operations must remain disabled');
console.log('? Live integration boundary guards verified.');

// 2. Ensure no SQL migration files exist in the root or new migrations in the migrations folder
const rootFiles = fs.readdirSync(__dirname);
const sqlFilesInRoot = rootFiles.filter(f => f.endsWith('.sql') && f !== 'add_admin.sql' && f !== 'categorize.
sql');
assert.strictEqual(sqlFilesInRoot.length, 0, 'No executable SQL files should be created in the root directory'
);

// 3. Verify existence of all 6 handoff and signoff documents in the parent directory
const govDir = path.join(__dirname, '..', 'docs', 'governance', 'enterprise-hospital-platform');
const requiredDocs = [
  'PHASE_OWNER_STAGING_HANDOFF_PREFLIGHT_AR.md',
  'OWNER_STAGING_EXECUTION_PACKET_AR.md',
  'OWNER_STAGING_COMMAND_CHECKLIST_DRAFT_AR.md',
  'OWNER_STAGING_RETURN_EVIDENCE_TEMPLATE_AR.md',
  'STAGING_OWNER_SIGNOFF_FORM_AR.md',
  'NEXT_PHASE_STAGING_EVIDENCE_REVIEW_PROMPT_AR.md'
];

requiredDocs.forEach(doc => {
  const docPath = path.join(govDir, doc);
  assert.ok(fs.existsSync(docPath), `Handoff document ${doc} must exist at ${docPath}`);
```

```
  // Verify UTF-8 encoding by reading the file
  const content = fs.readFileSync(docPath, 'utf8');
  assert.ok(content.length > 0, `Document ${doc} should not be empty`);

  // Verify no production IP or secrets are leaked (split passwords to avoid secret scanner false positives)
  const hasNoSecrets = !content.includes('DB_' + 'PASS' + 'WORD=') ||
                       content.includes('__CHANGE_ME__') ||
                       content.includes('<STAGING_DB_PASS' + 'WORD>');
  assert.ok(hasNoSecrets, `Document ${doc} should not contain real credentials`);
});
console.log('? All 6 Handoff & Signoff documents verified (present, non-empty, UTF-8 compliant).');

console.log('All Owner Staging Handoff & Signoff Static Tests Passed!');
process.exit(0);
```

```
========================================
FILE: P0_RLS_POST_SWITCH_STABILIZATION_AND_GOVERNANCE_CLEANUP_AR.md
========================================

# P0_RLS_POST_SWITCH_STABILIZATION_AND_GOVERNANCE_CLEANUP ? ????? ??????? ???????

## ???????

```
FINAL_STATUS: POST_SWITCH_STABILIZATION_PASS
SELECTED_PHASE: P0_RLS_POST_SWITCH_STABILIZATION_AND_GOVERNANCE_CLEANUP
USER_VISIBLE_ON_WEBSITE: YES
PRODUCTION_CHANGED: NO
DB_ROLE_CURRENT: nama_medical_app
APP_ROLE_SUPERUSER: false
APP_ROLE_BYPASSRLS: false
RLS_RUNTIME_ENFORCEMENT: YES
RLS_REVALIDATION_RESULT: PASS
PM2_STATUS: online
HEALTH_SMOKE: PASS
REDIS_STATUS: connected
ACCOUNTING_POSTING_ENABLED: OFF
JOURNAL_COUNT: 0
SECRET_BACKUP_IN_REPO_BEFORE: YES (.env.backup_before_role_switch)
SECRET_BACKUP_IN_REPO_AFTER: NO
BACKUP_FINAL_PATH: C:\Users\ice\nama_deploy_backups\rls_role_switch_final\.env.backup_before_role_switch
PARENT_REPO_STATUS: clean (gitlink committed and pushed)
NAMAWEB_REPO_STATUS: clean
COMMITTED: YES
PUSHED: YES
FORCE_PUSH_USED: NO
SECRETS_PRINTED: NO
NEXT_REQUIRED_ACTION: MASTER_AUTOPILOT_RESELECT_NEXT_PHASE
```

???? ??????? ?????????
```

Gate 0 ? State Guard
- ????? ??????: `nama_medical_app` (??? superuser? ??? bypassrls)
- 121 ????? RLS ????
- PM2 ?????? PID 38032? ?? crash loop
- Health: 200 `{"status":"UP"}`
- ???????? ?????? ?? ???? ?????

Gate 1 ? Secret Artifact Hygiene
- ??? `.env.backup_before_role_switch` ?? ???? ?? ???? repo ??? ???? ??? ?????
- ACL ???? ??? ????? ??????? (ice + Administrators ???)
- ?? ??? ???? ?? `git status`
- ?? ????? ?? ??

Gate 2 ? Post-switch Monitoring
- PM2: online ????? (10+ ????? ??? ???????)
- `/api/health`: 200
- `/login`: 200
- ???????? ??????? ???? ????: 401 ?
- Redis: ???? ?
- ?? ????? auth ?? ???????
- ?? ????? RLS ??? ??????

Gate 3 ? RLS Revalidation
- `current_user = nama_medical_app`
- `rolsuper = false`
- `rolbypassrls = false`
- ???? ???? tenant: 0 ???? ?
- `tenant_id=1`: patients=3, invoices=3, audit=45 ?
- `tenant_id=999`: 0 ???? ?
- ????? ?????: ????? ?????? RLS ?
- ??? audit_trail: ?

Gate 4 ? Git Governance
- namaweb repo: ????
- parent repo: gitlink ?????? commit + push ???? force
- ?? `.env` ?? ????? ???? ?? staging

?? ?? ??? ??????

??????	??????
DB role	?? ?????? (nama_medical_app)
DDL	?? ?????
??????	?? ??????
.env	?? ?????? (??? ???? backup ???? repo)
ACCOUNTING_POSTING_ENABLED	OFF ? ?? ??????
journal entries	0 ? ?? ?????
force push	?? ???????
?????	?? ?????

==========

FILE: P0_RLS_RUNTIME_ROLE_SWITCH_CONTROLLED_EXECUTION_FINAL_CLOSEOUT_AR.md
==========

P0_RLS_RUNTIME_ROLE_SWITCH_CONTROLLED_EXECUTION ? ????? ??????? ???????

???????

```
FINAL_STATUS: PRODUCTION_DEPLOYED_PASS
SELECTED_PHASE: P0_RLS_RUNTIME_ROLE_SWITCH_CONTROLLED_EXECUTION
USER_VISIBLE_ON_WEBSITE: YES
PRODUCTION_DEPLOYED: YES
DB_ROLE_BEFORE: postgres
DB_ROLE_AFTER: nama_medical_app
APP_ROLE_SUPERUSER: false
APP_ROLE_BYPASSRLS: false
RLS_FORCE_COUNT: 121
RLS_RUNTIME_ENFORCEMENT: YES
RLS_ENFORCEMENT_RESULT: PASS
TENANT_ISOLATION_TEST: PASS
APP_TENANT_ID_CONNECTION_SCOPE_RESULT: PASS
AUDIT_TRAIL_POLICY_RESULT: PASS
LOGAUDIT_STAMPING_RESULT: PASS
HEALTH_SMOKE: PASS
PM2_STATUS: online
REDIS_STATUS: connected
DDL_EXECUTED: NO
DATA_CHANGED: NO
ACCOUNTING_POSTING_ENABLED: OFF
JOURNAL_CREATED: NO
ENV_BACKUP_PATH: namaweb/.env.backup_before_role_switch
ROLLBACK_READY: YES
ROLLBACK_USED: NO
SECRETS_FOUND: NO
SECRETS_PRINTED: NO
FORCE_PUSH_USED: NO
NEXT_REQUIRED_ACTION: MASTER_AUTOPILOT_RESELECT_NEXT_PHASE
```

???? ???????

?? ????? ????? ??????? ?? ??? `postgres` (superuser) ??? ??? `nama_medical_app` ??????? ????????? ?????.

?? ?? ??????

???????	?????	???????
Gate 3	Secret Probe ? ????? ?????? ??? ????? ?????	? PASS
Gate 4	Atomic Env Switch ? ????? DB_USER ? DB_PASSWORD	? PASS
Gate 5	Controlled Restart ? ????? ????? PM2 ??????	? PASS
Gate 6	Smoke Checks ? ??? HTTP endpoints	? PASS
Gate 7	Runtime Role Proof ? ????? ????? ???????	? PASS
Gate 8	RLS Enforcement ? ?????? ?? ????? ???? ??????	? PASS
Gate 9	Regression ? ??? ?????????	? PASS

?????? ????? RLS

- **???? ???? tenant**: 0 ???? ? ?
- **tenant_id=1**: ?????? ???????? ????? ????? ? ?
- **tenant_id=999**: 0 ???? ? ?
- **??? ??? ???????????**: tenant 999 ?? ??? ?????? tenant 1 ? ?
- **????? audit_trail ????? ?????**: ? ???
- **????? audit_trail ????? (tenant ?????)**: ? ????? ?????? RLS
- **SELECT audit_trail ?????**: ?
- **??????? RLS**: 121 ????? ????
- **???? ???????**: 4 ?????? ??? ??????? ???????? (patients, invoices, audit_trail)

?????? ?? ??????

- `rolsuper = false` ? ?? ??????? ?????
- `rolbypassrls = false` ? ?? ????? ?? RLS
- `ACCOUNTING_POSTING_ENABLED = OFF` ? ???????? ?????
- `journal_count = 0` ? ?? ???? ?????
- ?? ??? ????? DDL
- ?? ??? ????? ??????

?????? ?????? (Smoke)

??????	???????
`GET /`	200
`GET /api/health`	200 `{"status":"UP"}`
`GET /login`	200
`GET /api/patients` (???? ????)	401
`GET /api/invoices` (???? ????)	401
`POST /api/visits` (???? ????)	401
`POST /api/invoices/1/refund` (???? ????)	401

?????????

- PM2: online? PID ?????? ?? crash loop
- Redis: ????
- ?? ????? auth ?? ???????
- ?? ????? permission
- ??????? (facility) ????

==========
FILE: package-lock.json
==========


```
{
  "name": "nama-medical-web",
  "version": "1.0.0",
  "lockfileVersion": 3,
  "requires": true,
  "packages": {
```

```
    "": {
      "name": "nama-medical-web",
      "version": "1.0.0",
      "hasInstallScript": true,
      "dependencies": {
        "bcrypt": "^6.0.0",
        "bcryptjs": "^3.0.3",
        "better-sqlite3": "^11.0.0",
        "compression": "^1.8.1",
        "connect-redis": "^9.0.0",
        "cors": "^2.8.5",
        "dotenv": "^17.3.1",
        "express": "^4.21.0",
        "express-rate-limit": "^8.2.1",
        "express-session": "^1.18.0",
        "helmet": "^8.1.0",
        "multer": "^2.0.2",
        "pg": "^8.18.0",
        "redis": "^6.0.0"
      },
      "devDependencies": {
        "@tailwindcss/container-queries": "^0.1.1",
        "@tailwindcss/forms": "^0.5.7",
        "tailwindcss": "^3.4.1"
      }
    },
    "node_modules/@alloc/quick-lru": {
      "version": "5.2.0",
      "resolved": "https://registry.npmjs.org/@alloc/quick-lru/-/quick-lru-5.2.0.tgz",
      "integrity": "sha512-UrcABB+4bUrFABwbluTIBErXwvbsU/V7TZWfmbgJfbkwiBuziS9gxdODUyuiecfdGQ85jglMW6juS3+z5TsKLw==",
      "dev": true,
      "license": "MIT",
      "engines": {
        "node": ">=10"
      },
      "funding": {
        "url": "https://github.com/sponsors/sindresorhus"
      }
    },
    "node_modules/@jridgewell/gen-mapping": {
      "version": "0.3.13",
      "resolved": "https://registry.npmjs.org/@jridgewell/gen-mapping/-/gen-mapping-0.3.13.tgz",
      "integrity": "sha512-2kkt/7niJ6MgEPxF0bYdQ6etZaA+fQvDcLKckhy1yIQOzaoKjBBjSj63/aLVjYE3qhRt5dvM+uUyfCg6UKCBbA==",
      "dev": true,
      "license": "MIT",
      "dependencies": {
        "@jridgewell/sourcemap-codec": "^1.5.0",
        "@jridgewell/trace-mapping": "^0.3.24"
      }
    },
    "node_modules/@jridgewell/resolve-uri": {
      "version": "3.1.2",
```

```
      "resolved": "https://registry.npmjs.org/@jridgewell/resolve-uri/-/resolve-uri-3.1.2.tgz",
      "integrity": "sha512-bRISgCIjP20/tbWSPWMEi54QVPRZExkuD9lJL+UIxUKtwVJA8wW1Trb1jMs1RFXo1CBTNZ/5hpC9QvmKWdopKw==",
      "dev": true,
      "license": "MIT",
      "engines": {
        "node": ">=6.0.0"
      }
    },
    "node_modules/@jridgewell/sourcemap-codec": {
      "version": "1.5.5",
      "resolved": "https://registry.npmjs.org/@jridgewell/sourcemap-codec/-/sourcemap-codec-1.5.5.tgz",
      "integrity": "sha512-cYQ9310grqxueWbl+WuIUIaiUaDcj7WOq5fVhEljNVgRfOUhY9fy2zTvfoqWsnebh8Sl70VScFbICvJnLKB0Og==",
      "dev": true,
      "license": "MIT"
    },
    "node_modules/@jridgewell/trace-mapping": {
      "version": "0.3.31",
      "resolved": "https://registry.npmjs.org/@jridgewell/trace-mapping/-/trace-mapping-0.3.31.tgz",
      "integrity": "sha512-zzNR+SdQSDJzc8joaeP8QQoCQr8NuYx2dIIytl1QeBEZHJ9uW6hebsrYgbz8hJwUQao3TWCMtmfV8Nu1twOLAw==",
      "dev": true,
      "license": "MIT",
      "dependencies": {
        "@jridgewell/resolve-uri": "^3.1.0",
        "@jridgewell/sourcemap-codec": "^1.4.14"
      }
    },
    "node_modules/@nodelib/fs.scandir": {
      "version": "2.1.5",
      "resolved": "https://registry.npmjs.org/@nodelib/fs.scandir/-/fs.scandir-2.1.5.tgz",
      "integrity": "sha512-vq24Bq3ym5HEQm2NKCr3yXDwjc7vTsEThRDnkp2DK9p1uqLR+DHurm/NOTo0KG7HYHU7eppKZj3MyqYuMBf62g==",
      "dev": true,
      "license": "MIT",
      "dependencies": {
        "@nodelib/fs.stat": "2.0.5",
        "run-parallel": "^1.1.9"
      },
      "engines": {
        "node": ">= 8"
      }
    },
    "node_modules/@nodelib/fs.stat": {
      "version": "2.0.5",
      "resolved": "https://registry.npmjs.org/@nodelib/fs.stat/-/fs.stat-2.0.5.tgz",
      "integrity": "sha512-RkhPPp2zrqDAQA/2jNhnztcPAlv64XdhIp7a7454A5ovI7Bukxgt7MX7udwAu3zg1DcpPU0rz3VV1SeaqvY4+A==",
      "dev": true,
      "license": "MIT",
      "engines": {
        "node": ">= 8"
      }
```

```
    },
    "node_modules/@nodelib/fs.walk": {
      "version": "1.2.8",
      "resolved": "https://registry.npmjs.org/@nodelib/fs.walk/-/fs.walk-1.2.8.tgz",
      "integrity": "sha512-oGB+UxlgWcgQkgwo8GcEGwemoTFt3FIO9ababBmaGwXIoBKZ+GTy0pP185beGg7Llih/NSHSV2XAs1lnznocSg==",
      "dev": true,
      "license": "MIT",
      "dependencies": {
        "@nodelib/fs.scandir": "2.1.5",
        "fastq": "^1.6.0"
      },
      "engines": {
        "node": ">= 8"
      }
    },
    "node_modules/@redis/bloom": {
      "version": "6.0.0",
      "resolved": "https://registry.npmjs.org/@redis/bloom/-/bloom-6.0.0.tgz",
      "integrity": "sha512-P0n5NkV9IIdT6nYXOfMHG83sho8pE7Nay7yw27wOGVLv4DthgvzebpGz6m7VuMTizeJmw3LPw2Xek5wFUhGpVw==",
      "license": "MIT",
      "engines": {
        "node": ">= 20.0.0"
      },
      "peerDependencies": {
        "@redis/client": "^6.0.0"
      }
    },
    "node_modules/@redis/client": {
      "version": "6.0.0",
      "resolved": "https://registry.npmjs.org/@redis/client/-/client-6.0.0.tgz",
      "integrity": "sha512-NS4iIT25r24sAjNQ2nSRdCW5jPJoV0rxkBee27oTeR+RXaOu89cjIsrww5rPBaYVGVdL1QCx9uz9141gZiSKdQ==",
      "license": "MIT",
      "dependencies": {
        "cluster-key-slot": "1.1.2"
      },
      "engines": {
        "node": ">= 20.0.0"
      },
      "peerDependencies": {
        "@node-rs/xxhash": "^1.1.0",
        "@opentelemetry/api": ">=1 <2"
      },
      "peerDependenciesMeta": {
        "@node-rs/xxhash": {
          "optional": true
        },
        "@opentelemetry/api": {
          "optional": true
        }
      }
    },
```

```
    "node_modules/@redis/json": {
      "version": "6.0.0",
      "resolved": "https://registry.npmjs.org/@redis/json/-/json-6.0.0.tgz",
      "integrity": "sha512-F+eqFfgPcy57Zs1KW7UtLnBtRk6lxAUIoe7dyZerpm6e+ssYXG/dWJrbrHFYs0b7tt6QBtYpVuukBuM9XqhUAg==",
      "license": "MIT",
      "engines": {
        "node": ">= 20.0.0"
      },
      "peerDependencies": {
        "@redis/client": "^6.0.0"
      }
    },
    "node_modules/@redis/search": {
      "version": "6.0.0",
      "resolved": "https://registry.npmjs.org/@redis/search/-/search-6.0.0.tgz",
      "integrity": "sha512-VHuCJ2W0YWFixGZh/l//8JiyOsD4gN+NhjdRAGIoUe0UQ4mtq1NyY2ZJ973XT+vYhaU21XdK8r8oNrd5n7wbzQ==",
      "license": "MIT",
      "engines": {
        "node": ">= 20.0.0"
      },
      "peerDependencies": {
        "@redis/client": "^6.0.0"
      }
    },
    "node_modules/@redis/time-series": {
      "version": "6.0.0",
      "resolved": "https://registry.npmjs.org/@redis/time-series/-/time-series-6.0.0.tgz",
      "integrity": "sha512-QWhkYsg+3lhBrBf+cbzybtV8LQcSrk7iXIgTaGU+pHNFTkql7TpVRE24ROS6M2ybVIV6O/zxTqfxgxxYiqyw0Q==",
      "license": "MIT",
      "engines": {
        "node": ">= 20.0.0"
      },
      "peerDependencies": {
        "@redis/client": "^6.0.0"
      }
    },
    "node_modules/@tailwindcss/container-queries": {
      "version": "0.1.1",
      "resolved": "https://registry.npmjs.org/@tailwindcss/container-queries/-/container-queries-0.1.1.tgz",
      "integrity": "sha512-p18dswChx6WnTSaJCSGx6lTmrGzNNvm2FtXmiO6AuA1V4U5REyoqwmT6kgAsIMdjo07QdAfYXHJ4hnMtfHzWgA==",
      "dev": true,
      "license": "MIT",
      "peerDependencies": {
        "tailwindcss": ">=3.2.0"
      }
    },
    "node_modules/@tailwindcss/forms": {
      "version": "0.5.11",
      "resolved": "https://registry.npmjs.org/@tailwindcss/forms/-/forms-0.5.11.tgz",
      "integrity": "sha512-h9wegbZDPurxG22xZSoWtdzc41/OlNEUQERNqI/0fOwa2aVlWGu7C35E/x6LDyD3lgtztFSSjKZyuVM0hxh
```

```
bgA==",
      "dev": true,
      "license": "MIT",
      "dependencies": {
        "mini-svg-data-uri": "^1.2.3"
      },
      "peerDependencies": {
        "tailwindcss": ">=3.0.0 || >= 3.0.0-alpha.1 || >= 4.0.0-alpha.20 || >= 4.0.0-beta.1"
      }
    },
    "node_modules/accepts": {
      "version": "1.3.8",
      "resolved": "https://registry.npmjs.org/accepts/-/accepts-1.3.8.tgz",
      "integrity": "sha512-PYAthTa2m2VKxuvSD3DPC/Gy+U+sOA1LAuT8mkmRuvw+NACSaeXEQ+NHcVF7rONl6qcaxV3Uuemwawk+7+S
JLw==",
      "license": "MIT",
      "dependencies": {
        "mime-types": "~2.1.34",
        "negotiator": "0.6.3"
      },
      "engines": {
        "node": ">= 0.6"
      }
    },
    "node_modules/any-promise": {
      "version": "1.3.0",
      "resolved": "https://registry.npmjs.org/any-promise/-/any-promise-1.3.0.tgz",
      "integrity": "sha512-7UvmKalWRt1wgjL1RrGxoSJW/0QZFIegpeGvZG9kjp8vrRu55XTHbwnqq2GpXm9uLbcuhxm3IqX9OB4MZR1
b2A==",
      "dev": true,
      "license": "MIT"
    },
    "node_modules/anymatch": {
      "version": "3.1.3",
      "resolved": "https://registry.npmjs.org/anymatch/-/anymatch-3.1.3.tgz",
      "integrity": "sha512-KMReFUr0B4t+D+OBkjR3KYqvocp2XaSzO55UcB6mgQMd3KbcE+mWTyvVV7D/zsdEbNnV6acZUutkiHQXvTr
1Rw==",
      "dev": true,
      "license": "ISC",
      "dependencies": {
        "normalize-path": "^3.0.0",
        "picomatch": "^2.0.4"
      },
      "engines": {
        "node": ">= 8"
      }
    },
    "node_modules/append-field": {
      "version": "1.0.0",
      "resolved": "https://registry.npmjs.org/append-field/-/append-field-1.0.0.tgz",
      "integrity": "sha512-klpgFSWLW1ZEs8svjfb7g4qWY0YS5imI82dTg+QahUvJ8YqAY0P10Uk8tTyh9ZGuYEZEMaeJYCF5BFuX552
hsw==",
      "license": "MIT"
    },
```

```
    "node_modules/arg": {
      "version": "5.0.2",
      "resolved": "https://registry.npmjs.org/arg/-/arg-5.0.2.tgz",
      "integrity": "sha512-PYjyFOLKQ9y57JvQ6QLo8dAgNqswh8M1RMJYdQduT6xbWSgK36P/Z/v+p888pM69jMMfS8Xd8F6I1kQ/I9HUGg==",
      "dev": true,
      "license": "MIT"
    },
    "node_modules/array-flatten": {
      "version": "1.1.1",
      "resolved": "https://registry.npmjs.org/array-flatten/-/array-flatten-1.1.1.tgz",
      "integrity": "sha512-PCVAQswWemu6UdxsDFFX/+gVeYqKAod3D3UVm91jHwynguOwAvYPhx8nNlM++NqRcK6CxxpUafjmhIdKiHibqg==",
      "license": "MIT"
    },
    "node_modules/base64-js": {
      "version": "1.5.1",
      "resolved": "https://registry.npmjs.org/base64-js/-/base64-js-1.5.1.tgz",
      "integrity": "sha512-AKpaYlHn8t4SVbOHCy+b5+KKgvR4vrsD8vbvrbiQJps7fKDTkjkDry6ji0rUJjC0kzbNePLwzxq8iypo41qeWA==",
      "funding": [
        {
          "type": "github",
          "url": "https://github.com/sponsors/feross"
        },
        {
          "type": "patreon",
          "url": "https://www.patreon.com/feross"
        },
        {
          "type": "consulting",
          "url": "https://feross.org/support"
        }
      ],
      "license": "MIT"
    },
    "node_modules/bcrypt": {
      "version": "6.0.0",
      "resolved": "https://registry.npmjs.org/bcrypt/-/bcrypt-6.0.0.tgz",
      "integrity": "sha512-cU8v/EGSrnH+HnxV2z0J7/blxH8gq7Xh2JFT6Aroax7UohdmiJJlxApMxtKfuI7z68NvvVcmR78k2LbT6efhRg==",
      "hasInstallScript": true,
      "license": "MIT",
      "dependencies": {
        "node-addon-api": "^8.3.0",
        "node-gyp-build": "^4.8.4"
      },
      "engines": {
        "node": ">= 18"
      }
    },
    "node_modules/bcryptjs": {
      "version": "3.0.3",
      "resolved": "https://registry.npmjs.org/bcryptjs/-/bcryptjs-3.0.3.tgz",
```

```
      "integrity": "sha512-GlF5wPWnSa/X5LKM1o0wz0suXIINz1iHRLvTS+sLyi7XPbe5ycmYI3DlZqVGZZtDgl4DmasFg7gOB3JYbphV5g==",
      "license": "BSD-3-Clause",
      "bin": {
        "bcrypt": "bin/bcrypt"
      }
    },
    "node_modules/better-sqlite3": {
      "version": "11.10.0",
      "resolved": "https://registry.npmjs.org/better-sqlite3/-/better-sqlite3-11.10.0.tgz",
      "integrity": "sha512-EwhOpyXiOEL/lKzHz9AW1msWFNzGc/z+LzeB3/jnFJpxu+th2yqvzsSWas1v9jgs9+xiXJcD5A8CJxAG2TaghQ==",
      "hasInstallScript": true,
      "license": "MIT",
      "dependencies": {
        "bindings": "^1.5.0",
        "prebuild-install": "^7.1.1"
      }
    },
    "node_modules/binary-extensions": {
      "version": "2.3.0",
      "resolved": "https://registry.npmjs.org/binary-extensions/-/binary-extensions-2.3.0.tgz",
      "integrity": "sha512-Ceh+7ox5qe7LJuLHoY0feh3pHuUDHAcRUeyL2VYghZwfpkNIy/+8Ocg0a3UuSoYzavmylwuLWQOf3hl0jjMMIw==",
      "dev": true,
      "license": "MIT",
      "engines": {
        "node": ">=8"
      },
      "funding": {
        "url": "https://github.com/sponsors/sindresorhus"
      }
    },
    "node_modules/bindings": {
      "version": "1.5.0",
      "resolved": "https://registry.npmjs.org/bindings/-/bindings-1.5.0.tgz",
      "integrity": "sha512-p2q/t/mhvuOj/UeLlV6566GD/guowlr0hHxClI0W9m7MWYkL1F0hLo+0Aexs9HSPCtR1SXQ0TD3MMKrXZajbiQ==",
      "license": "MIT",
      "dependencies": {
        "file-uri-to-path": "1.0.0"
      }
    },
    "node_modules/bl": {
      "version": "4.1.0",
      "resolved": "https://registry.npmjs.org/bl/-/bl-4.1.0.tgz",
      "integrity": "sha512-1W07cM9gS6DcLperZfFSj+bWLtaPGSOHWhPiGzXmvVJbRLdG82sH/Kn8EtW1VqWVA54AKf2h5k5BbnIbwF3h6w==",
      "license": "MIT",
      "dependencies": {
        "buffer": "^5.5.0",
        "inherits": "^2.0.4",
        "readable-stream": "^3.4.0"
      }
```

```
    },
    "node_modules/body-parser": {
      "version": "1.20.5",
      "resolved": "https://registry.npmjs.org/body-parser/-/body-parser-1.20.5.tgz",
      "integrity": "sha512-3grm+/2tUOvu2cjJkvsIxrv/wVpfXQW4PsQHYm7yk4vfpu7Ekl6nEsYBoJUL6qDwZUx8wUhQ8tR2qz+ad9c
9OA==",
      "license": "MIT",
      "dependencies": {
        "bytes": "~3.1.2",
        "content-type": "~1.0.5",
        "debug": "2.6.9",
        "depd": "2.0.0",
        "destroy": "~1.2.0",
        "http-errors": "~2.0.1",
        "iconv-lite": "~0.4.24",
        "on-finished": "~2.4.1",
        "qs": "~6.15.1",
        "raw-body": "~2.5.3",
        "type-is": "~1.6.18",
        "unpipe": "~1.0.0"
      },
      "engines": {
        "node": ">= 0.8",
        "npm": "1.2.8000 || >= 1.4.16"
      }
    },
    "node_modules/braces": {
      "version": "3.0.3",
      "resolved": "https://registry.npmjs.org/braces/-/braces-3.0.3.tgz",
      "integrity": "sha512-yQbXgO/OSZVD2IsiLlro+7Hf6Q18EJrKSEsdoMzKePKXct3gvD8oLcOQdIzGupr5Fj+EDe8gO/lxc1BzfMp
xvA==",
      "dev": true,
      "license": "MIT",
      "dependencies": {
        "fill-range": "^7.1.1"
      },
      "engines": {
        "node": ">=8"
      }
    },
    "node_modules/buffer": {
      "version": "5.7.1",
      "resolved": "https://registry.npmjs.org/buffer/-/buffer-5.7.1.tgz",
      "integrity": "sha512-EHcyIPBQ4BSGlvjB16k5KgAJ27CIsHY/2JBmCRReo48y9rQ3MaUzWX3KVlBa4U7MyX02HdVj0K7C3WaB3ju
7FQ==",
      "funding": [
        {
          "type": "github",
          "url": "https://github.com/sponsors/feross"
        },
        {
          "type": "patreon",
          "url": "https://www.patreon.com/feross"
        },
```

```
        {
          "type": "consulting",
          "url": "https://feross.org/support"
        }
      ],
      "license": "MIT",
      "dependencies": {
        "base64-js": "^1.3.1",
        "ieee754": "^1.1.13"
      }
    },
    "node_modules/buffer-from": {
      "version": "1.1.2",
      "resolved": "https://registry.npmjs.org/buffer-from/-/buffer-from-1.1.2.tgz",
      "integrity": "sha512-E+XQCRwSbaaiChtv6k6Dwgc+bx+Bs6vuKJHHl5kox/BaKbhiXzqQOwK4cO22yElGp2OCmjwVhT3HmxgyPGn
JfQ==",
      "license": "MIT"
    },
    "node_modules/busboy": {
      "version": "1.6.0",
      "resolved": "https://registry.npmjs.org/busboy/-/busboy-1.6.0.tgz",
      "integrity": "sha512-8SFQbg/0hQ9xy3UNTB0YEnsNBbWfhf7RtnzpL7TkBiTBRfrQ9Fxcnz7VJsleJpyp6rVLvXiuORqjlHi5q+P
YuA==",
      "dependencies": {
        "streamsearch": "^1.1.0"
      },
      "engines": {
        "node": ">=10.16.0"
      }
    },
    "node_modules/bytes": {
      "version": "3.1.2",
      "resolved": "https://registry.npmjs.org/bytes/-/bytes-3.1.2.tgz",
      "integrity": "sha512-/Nf7TyzTx6S3yRJObOAV7956r8cr2+Oj8AC5dt8wSP3BQAoeX58NoHyCU8P8zGkNXStjTSi6fzO6F0pBdcY
bEg==",
      "license": "MIT",
      "engines": {
        "node": ">= 0.8"
      }
    },
    "node_modules/call-bind-apply-helpers": {
      "version": "1.0.2",
      "resolved": "https://registry.npmjs.org/call-bind-apply-helpers/-/call-bind-apply-helpers-1.0.2.tgz",
      "integrity": "sha512-Sp1ablJ0ivDkSzjcaJdxEunN5/XvksFJ2sMBFfq6x0ryhQV/2b/KwFe21cMpmHtPOSij8K99/wSfoEuTObm
uMQ==",
      "license": "MIT",
      "dependencies": {
        "es-errors": "^1.3.0",
        "function-bind": "^1.1.2"
      },
      "engines": {
        "node": ">= 0.4"
      }
    },
```

```
    "node_modules/call-bound": {
      "version": "1.0.4",
      "resolved": "https://registry.npmjs.org/call-bound/-/call-bound-1.0.4.tgz",
      "integrity": "sha512-+ys997U96po4Kx/ABpBCqhA9EuxJaQWDQg7295H4hBphv3IZg0boBKuwYpt4YXp6MZ5AmZQnU/tyMTlRpaSejg==",
      "license": "MIT",
      "dependencies": {
        "call-bind-apply-helpers": "^1.0.2",
        "get-intrinsic": "^1.3.0"
      },
      "engines": {
        "node": ">= 0.4"
      },
      "funding": {
        "url": "https://github.com/sponsors/ljharb"
      }
    },
    "node_modules/camelcase-css": {
      "version": "2.0.1",
      "resolved": "https://registry.npmjs.org/camelcase-css/-/camelcase-css-2.0.1.tgz",
      "integrity": "sha512-QOSvevhslijgYwRx6Rv7zKdMF8lbRmx+uQGx2+vDc+KI/eBnsy9kit5aj23AgGu3pa4t9AgwbnXWqS+iOY+2aA==",
      "dev": true,
      "license": "MIT",
      "engines": {
        "node": ">= 6"
      }
    },
    "node_modules/chokidar": {
      "version": "3.6.0",
      "resolved": "https://registry.npmjs.org/chokidar/-/chokidar-3.6.0.tgz",
      "integrity": "sha512-7VT13fmjotKpGipCW9JEQAusEPE+Ei8nl6/g4FBAmIm0GOOLMua9NDDo/DWp0ZAxCr3cPq5ZpBqmPAQgDda2Pw==",
      "dev": true,
      "license": "MIT",
      "dependencies": {
        "anymatch": "~3.1.2",
        "braces": "~3.0.2",
        "glob-parent": "~5.1.2",
        "is-binary-path": "~2.1.0",
        "is-glob": "~4.0.1",
        "normalize-path": "~3.0.0",
        "readdirp": "~3.6.0"
      },
      "engines": {
        "node": ">= 8.10.0"
      },
      "funding": {
        "url": "https://paulmillr.com/funding/"
      },
      "optionalDependencies": {
        "fsevents": "~2.3.2"
      }
    },
```

```
    "node_modules/chokidar/node_modules/glob-parent": {
      "version": "5.1.2",
      "resolved": "https://registry.npmjs.org/glob-parent/-/glob-parent-5.1.2.tgz",
      "integrity": "sha512-AOIgSQCepiJYwP3ARnGx+5VnTu2HBYdzbGP45eLw1vr3zB3vZLeyed1sC9hnbcOc9/SrMyM5RPQrkGz4aS9
Zow==",
      "dev": true,
      "license": "ISC",
      "dependencies": {
        "is-glob": "^4.0.1"
      },
      "engines": {
        "node": ">= 6"
      }
    },
    "node_modules/chownr": {
      "version": "1.1.4",
      "resolved": "https://registry.npmjs.org/chownr/-/chownr-1.1.4.tgz",
      "integrity": "sha512-jJ0bqzaylmJtVnNgzTeSOs8DPavpbYgEr/b0YL8/2GO3xJEhInFmhKMUnEJQjZumK7KXGFhUy89PrsJWlak
BVg==",
      "license": "ISC"
    },
    "node_modules/cluster-key-slot": {
      "version": "1.1.2",
      "resolved": "https://registry.npmjs.org/cluster-key-slot/-/cluster-key-slot-1.1.2.tgz",
      "integrity": "sha512-RMr0FhtfXemyinomL4hrWcYJxmX6deFdCxpJzhDttxgO1+bcCnkk+9drydLVDmAMG7NE6aN/fl4F7ucU/90
gAA==",
      "license": "Apache-2.0",
      "engines": {
        "node": ">=0.10.0"
      }
    },
    "node_modules/commander": {
      "version": "4.1.1",
      "resolved": "https://registry.npmjs.org/commander/-/commander-4.1.1.tgz",
      "integrity": "sha512-NOKm8xhkzAjzFx8B2v5OAHT+u5pRQc2UCa2Vq9jYL/31o2wi9mxBA7LIFs3sV5VSC49z6pEhfbMULvShKj2
6WA==",
      "dev": true,
      "license": "MIT",
      "engines": {
        "node": ">= 6"
      }
    },
    "node_modules/compressible": {
      "version": "2.0.18",
      "resolved": "https://registry.npmjs.org/compressible/-/compressible-2.0.18.tgz",
      "integrity": "sha512-AF3r7P5dWxL8MxyITRMlORQNaOA2IkAFaTr4k7BUumjPtRpGDTZpl0Pb1XCO6JeDCBdp126Cgs9sMxqSjgY
yRg==",
      "license": "MIT",
      "dependencies": {
        "mime-db": ">= 1.43.0 < 2"
      },
      "engines": {
        "node": ">= 0.6"
      }
```

```
    },
    "node_modules/compression": {
      "version": "1.8.1",
      "resolved": "https://registry.npmjs.org/compression/-/compression-1.8.1.tgz",
      "integrity": "sha512-9mAqGPHLakhCLeNyxPkK4xVo746zQ/czLH1Ky+vkitMnWfWZps8r0qXuwhwizagCRttsL4lfG4pIOvaWLpA
P0w==",
      "license": "MIT",
      "dependencies": {
        "bytes": "3.1.2",
        "compressible": "~2.0.18",
        "debug": "2.6.9",
        "negotiator": "~0.6.4",
        "on-headers": "~1.1.0",
        "safe-buffer": "5.2.1",
        "vary": "~1.1.2"
      },
      "engines": {
        "node": ">= 0.8.0"
      }
    },
    "node_modules/compression/node_modules/negotiator": {
      "version": "0.6.4",
      "resolved": "https://registry.npmjs.org/negotiator/-/negotiator-0.6.4.tgz",
      "integrity": "sha512-myRT3DiWPHqho5PrJaIRyaMv2kgYf0mUVgBNOYMuCH5Ki1yEiQaf/ZJuQ62nvpc44wL5WDbTX7yGJi1Neev
w8w==",
      "license": "MIT",
      "engines": {
        "node": ">= 0.6"
      }
    },
    "node_modules/concat-stream": {
      "version": "2.0.0",
      "resolved": "https://registry.npmjs.org/concat-stream/-/concat-stream-2.0.0.tgz",
      "integrity": "sha512-MWufYdFw53ccGjCA+Ol7XJYpAlW6/prSMzuPOTRnJGcGzuhLn4Scrz7qf6o8bROZ514ltazcIFJZevcfbo0
x7A==",
      "engines": [
        "node >= 6.0"
      ],
      "license": "MIT",
      "dependencies": {
        "buffer-from": "^1.0.0",
        "inherits": "^2.0.3",
        "readable-stream": "^3.0.2",
        "typedarray": "^0.0.6"
      }
    },
    "node_modules/connect-redis": {
      "version": "9.0.0",
      "resolved": "https://registry.npmjs.org/connect-redis/-/connect-redis-9.0.0.tgz",
      "integrity": "sha512-QwzyvUePTMvEzG1hy45gZYw3X3YHrjmEdSkayURlcZft7hqadQ3X39wYkmCqblK2rGlw+XItELYt6GnyG6D
EIQ==",
      "license": "MIT",
      "engines": {
        "node": ">=18"
```

```
      },
      "peerDependencies": {
        "express-session": ">=1",
        "redis": ">=5"
      }
    },
    "node_modules/content-disposition": {
      "version": "0.5.4",
      "resolved": "https://registry.npmjs.org/content-disposition/-/content-disposition-0.5.4.tgz",
      "integrity": "sha512-FveZTNuGw04cxlAiWbzi6zTAL/lhehaWbTtgluJh4/E95DqMwTmha3KZN1aAWA8cFIhHzMZUvLevkw5Rqk+
tSQ==",
      "license": "MIT",
      "dependencies": {
        "safe-buffer": "5.2.1"
      },
      "engines": {
        "node": ">= 0.6"
      }
    },
    "node_modules/content-type": {
      "version": "1.0.5",
      "resolved": "https://registry.npmjs.org/content-type/-/content-type-1.0.5.tgz",
      "integrity": "sha512-nTjqfcBFEipKdXCv4YDQWCfmcLZKm81ldF0pAopTvyrFGVbcR6P/VAAd5G7N+0tTr8QqiU0tFadD6FK4NtJ
wOA==",
      "license": "MIT",
      "engines": {
        "node": ">= 0.6"
      }
    },
    "node_modules/cookie": {
      "version": "0.7.2",
      "resolved": "https://registry.npmjs.org/cookie/-/cookie-0.7.2.tgz",
      "integrity": "sha512-yki5XnKuf750l50uGTllt6kKILY4nQ1eNIQatoXEByZ5dWgnKqbnqmTrBE5B4N7lrMJKQ2ytWMiTO2o0v6E
w/w==",
      "license": "MIT",
      "engines": {
        "node": ">= 0.6"
      }
    },
    "node_modules/cookie-signature": {
      "version": "1.0.7",
      "resolved": "https://registry.npmjs.org/cookie-signature/-/cookie-signature-1.0.7.tgz",
      "integrity": "sha512-NXdYc3dLr47pBkpUCHtKSwIOQXLVn8dZEuywboCOJY/osA0wFSLlSawr3KN8qXJEyX66FcONTH8EIlVuK0y
yFA==",
      "license": "MIT"
    },
    "node_modules/cors": {
      "version": "2.8.6",
      "resolved": "https://registry.npmjs.org/cors/-/cors-2.8.6.tgz",
      "integrity": "sha512-tJtZBBHA6vjIAaF6EnIaq6laBBP9aq/Y3ouVJjEfoHbRBcHBAHYcMh/w8LDrk2PvIMMq8gmopa5D4V8Rmbr
xGw==",
      "license": "MIT",
      "dependencies": {
        "object-assign": "^4",
```

```
        "vary": "^1"
      },
      "engines": {
        "node": ">= 0.10"
      },
      "funding": {
        "type": "opencollective",
        "url": "https://opencollective.com/express"
      }
    },
    "node_modules/cssesc": {
      "version": "3.0.0",
      "resolved": "https://registry.npmjs.org/cssesc/-/cssesc-3.0.0.tgz",
      "integrity": "sha512-/Tb/JcjK111nNScGob5MNtsntNM1aCNUDipB/TkwZFhyDrrE47SOx/18wF2bbjgc3ZzCSKW1T5nt5EbFoAz
/Vg==",
      "dev": true,
      "license": "MIT",
      "bin": {
        "cssesc": "bin/cssesc"
      },
      "engines": {
        "node": ">=4"
      }
    },
    "node_modules/debug": {
      "version": "2.6.9",
      "resolved": "https://registry.npmjs.org/debug/-/debug-2.6.9.tgz",
      "integrity": "sha512-bC7ElrdJaJnPbAP+1EotYvqZsb3ecl5wi6Bfi6BJTUcNowp6cvspg0jXznRTKDjm/E7AdgFBVeAPVMNcKGs
HMA==",
      "license": "MIT",
      "dependencies": {
        "ms": "2.0.0"
      }
    },
    "node_modules/decompress-response": {
      "version": "6.0.0",
      "resolved": "https://registry.npmjs.org/decompress-response/-/decompress-response-6.0.0.tgz",
      "integrity": "sha512-aW35yZM6Bb/4oJlZncMH2LCoZtJXTRxES17vE3hoRiowU2kWHaJKFkSBDnDR+cm9J+9QhXmREyIfv0pji9e
jCQ==",
      "license": "MIT",
      "dependencies": {
        "mimic-response": "^3.1.0"
      },
      "engines": {
        "node": ">=10"
      },
      "funding": {
        "url": "https://github.com/sponsors/sindresorhus"
      }
    },
    "node_modules/deep-extend": {
      "version": "0.6.0",
      "resolved": "https://registry.npmjs.org/deep-extend/-/deep-extend-0.6.0.tgz",
      "integrity": "sha512-LOHxIOaPYdHlJRtCQfDIVZtfw/ufM8+rVj649RIHzcm/vGwQRXFt6OPqIFWsm2XEMrNIEtWR64sY1LEKD2v
```

```
AOA==",
      "license": "MIT",
      "engines": {
        "node": ">=4.0.0"
      }
    },
    "node_modules/depd": {
      "version": "2.0.0",
      "resolved": "https://registry.npmjs.org/depd/-/depd-2.0.0.tgz",
      "integrity": "sha512-g7nH6P6dyDioJogAAGprGpCtVImJhpPk/roCzdb3fIh61/s/nPsfR6onyMwkCAR/OlC3yBC0lESvUoQEAss
Irw==",
      "license": "MIT",
      "engines": {
        "node": ">= 0.8"
      }
    },
    "node_modules/destroy": {
      "version": "1.2.0",
      "resolved": "https://registry.npmjs.org/destroy/-/destroy-1.2.0.tgz",
      "integrity": "sha512-2sJGJTaXIIaR1w4iJSNoN0hnMY7Gpc/n8D4qSCJw8QqFWXf7cuAgnEHxBpweaVcPevC2l3KpjYCx3NypQQg
aJg==",
      "license": "MIT",
      "engines": {
        "node": ">= 0.8",
        "npm": "1.2.8000 || >= 1.4.16"
      }
    },
    "node_modules/detect-libc": {
      "version": "2.1.2",
      "resolved": "https://registry.npmjs.org/detect-libc/-/detect-libc-2.1.2.tgz",
      "integrity": "sha512-Btj2BOOO83o3WyH59e8MgXsxEQVcarkUOpEYrubB0urwnN10yQ364rsiByU11nZlqWYZm05i/of7io4mzih
BtQ==",
      "license": "Apache-2.0",
      "engines": {
        "node": ">=8"
      }
    },
    "node_modules/didyoumean": {
      "version": "1.2.2",
      "resolved": "https://registry.npmjs.org/didyoumean/-/didyoumean-1.2.2.tgz",
      "integrity": "sha512-gxtyfqMg7GKyhQmb056K7M3xszy/myH8w+B4RT+QXBQsvAOdc3XymqDDPHx1BgPgsdAA5SIifona89YtRAT
Dzw==",
      "dev": true,
      "license": "Apache-2.0"
    },
    "node_modules/dlv": {
      "version": "1.1.3",
      "resolved": "https://registry.npmjs.org/dlv/-/dlv-1.1.3.tgz",
      "integrity": "sha512-+HlytyjlPKnIG8XuRG8WvmBP8xs8P71y+SKKS6ZXWoEgLuePxtDoUEiH7WkdePWrQ5JBpE6aoVqfZfJUQkj
XwA==",
      "dev": true,
      "license": "MIT"
    },
    "node_modules/dotenv": {
```

```
      "version": "17.3.1",
      "resolved": "https://registry.npmjs.org/dotenv/-/dotenv-17.3.1.tgz",
      "integrity": "sha512-IO8C/dzEb6O3F9/twg6ZLXz164a2fhTnEWb95H23Dm4OuN+92NmEAlTrupP9VW6Jm3sO26tQlqyvyi4CsnY9GA==",
      "license": "BSD-2-Clause",
      "engines": {
        "node": ">=12"
      },
      "funding": {
        "url": "https://dotenvx.com"
      }
    },
    "node_modules/dunder-proto": {
      "version": "1.0.1",
      "resolved": "https://registry.npmjs.org/dunder-proto/-/dunder-proto-1.0.1.tgz",
      "integrity": "sha512-KIN/nDJBQRcXw0MLVhZE9iQHmG68qAVIBg9CqmUYjmQIhgij9U5MFvrqkUL5FbtyyzZuOeOt0zdeRe4UY7ct+A==",
      "license": "MIT",
      "dependencies": {
        "call-bind-apply-helpers": "^1.0.1",
        "es-errors": "^1.3.0",
        "gopd": "^1.2.0"
      },
      "engines": {
        "node": ">= 0.4"
      }
    },
    "node_modules/ee-first": {
      "version": "1.1.1",
      "resolved": "https://registry.npmjs.org/ee-first/-/ee-first-1.1.1.tgz",
      "integrity": "sha512-WMwm9LhRUo+WUaRN+vRuETqG89IgZphVSNkdFgeb6sS/E4OrDIN7t48CAewSHXc6C8lefD8KKfr5vY61brQlow==",
      "license": "MIT"
    },
    "node_modules/encodeurl": {
      "version": "2.0.0",
      "resolved": "https://registry.npmjs.org/encodeurl/-/encodeurl-2.0.0.tgz",
      "integrity": "sha512-Q0n9HRi4m6JuGIV1eFlmvJB7ZEVxu93IrMyiMsGC0lrMJMWzRgx6WGquyfQgZVb31vhGgXnfmPNNXmxnOkRBrg==",
      "license": "MIT",
      "engines": {
        "node": ">= 0.8"
      }
    },
    "node_modules/end-of-stream": {
      "version": "1.4.5",
      "resolved": "https://registry.npmjs.org/end-of-stream/-/end-of-stream-1.4.5.tgz",
      "integrity": "sha512-ooEGc6HP26xXq/N+GCGOT0JKCLDGrq2bQUZrQ7gyrJiZANJ/8YDTxTpQBXGMn+WbIQXNVpyWymm7KYVICQnyOg==",
      "license": "MIT",
      "dependencies": {
        "once": "^1.4.0"
      }
    },
```

```
    "node_modules/es-define-property": {
      "version": "1.0.1",
      "resolved": "https://registry.npmjs.org/es-define-property/-/es-define-property-1.0.1.tgz",
      "integrity": "sha512-e3nRfgfUZ4rNGL232gUgX06QNyyez04KdjFrF+LTRoOXmrOgFKDg4BCdsjW8EnT69eqdYGmRpJwiPVYNrCa
W3g==",
      "license": "MIT",
      "engines": {
        "node": ">= 0.4"
      }
    },
    "node_modules/es-errors": {
      "version": "1.3.0",
      "resolved": "https://registry.npmjs.org/es-errors/-/es-errors-1.3.0.tgz",
      "integrity": "sha512-Zf5H2Kxt2xjTvbJvP2ZWLEICxA6j+hAmMzIlypy4xcBg1vKVnx89Wy0GbS+kf5cwCVFFzdCFh2XSCFNULS6
csw==",
      "license": "MIT",
      "engines": {
        "node": ">= 0.4"
      }
    },
    "node_modules/es-object-atoms": {
      "version": "1.1.2",
      "resolved": "https://registry.npmjs.org/es-object-atoms/-/es-object-atoms-1.1.2.tgz",
      "integrity": "sha512-HWcBoN6NileqtSydK2FqHbS/LoDd2pqrnQHLyJzBj4kOp/ky2MWMN694xOfkK8/SnUsW2DH7EfyVlydKCsm
1Zw==",
      "license": "MIT",
      "dependencies": {
        "es-errors": "^1.3.0"
      },
      "engines": {
        "node": ">= 0.4"
      }
    },
    "node_modules/escape-html": {
      "version": "1.0.3",
      "resolved": "https://registry.npmjs.org/escape-html/-/escape-html-1.0.3.tgz",
      "integrity": "sha512-NiSupZ4OeuGwr68lGIeym/ksIZMJodUGOSCZ/FSnTxcrekbvqrgdUxlJOMpijaKZVjAJrWrGs/6Jy8OMuyj
9ow==",
      "license": "MIT"
    },
    "node_modules/etag": {
      "version": "1.8.1",
      "resolved": "https://registry.npmjs.org/etag/-/etag-1.8.1.tgz",
      "integrity": "sha512-aIL5Fx7mawVa300al2BnEE4iNvo1qETxLrPI/o05L7z6go7fCw1J6EQmbK4FmJ2AS7kgVF/KEZWufBfdClM
cPg==",
      "license": "MIT",
      "engines": {
        "node": ">= 0.6"
      }
    },
    "node_modules/expand-template": {
      "version": "2.0.3",
      "resolved": "https://registry.npmjs.org/expand-template/-/expand-template-2.0.3.tgz",
      "integrity": "sha512-XYfuKMvj4O35f/pOXLObndIRvyQ+/+6AhODh+OKWj9S9498pHHn/IMszH+gt0fBCRWMNfk1ZSp5x3AifmnI
```

```
2vg==",
      "license": "(MIT OR WTFPL)",
      "engines": {
        "node": ">=6"
      }
    },
    "node_modules/express": {
      "version": "4.22.2",
      "resolved": "https://registry.npmjs.org/express/-/express-4.22.2.tgz",
      "integrity": "sha512-IuL+Elrou2ZvCFHs18/CIzy2Nzvo25nZ1/D2eIZlz7c+QUayAcYoiM2BthCjs+EBHVpjYjcuLDAiCWgeIX3
X1Q==",
      "license": "MIT",
      "dependencies": {
        "accepts": "~1.3.8",
        "array-flatten": "1.1.1",
        "body-parser": "~1.20.5",
        "content-disposition": "~0.5.4",
        "content-type": "~1.0.4",
        "cookie": "~0.7.1",
        "cookie-signature": "~1.0.6",
        "debug": "2.6.9",
        "depd": "2.0.0",
        "encodeurl": "~2.0.0",
        "escape-html": "~1.0.3",
        "etag": "~1.8.1",
        "finalhandler": "~1.3.1",
        "fresh": "~0.5.2",
        "http-errors": "~2.0.0",
        "merge-descriptors": "1.0.3",
        "methods": "~1.1.2",
        "on-finished": "~2.4.1",
        "parseurl": "~1.3.3",
        "path-to-regexp": "~0.1.12",
        "proxy-addr": "~2.0.7",
        "qs": "~6.15.1",
        "range-parser": "~1.2.1",
        "safe-buffer": "5.2.1",
        "send": "~0.19.0",
        "serve-static": "~1.16.2",
        "setprototypeof": "1.2.0",
        "statuses": "~2.0.1",
        "type-is": "~1.6.18",
        "utils-merge": "1.0.1",
        "vary": "~1.1.2"
      },
      "engines": {
        "node": ">= 0.10.0"
      },
      "funding": {
        "type": "opencollective",
        "url": "https://opencollective.com/express"
      }
    },
    "node_modules/express-rate-limit": {
```

```
      "version": "8.5.2",
      "resolved": "https://registry.npmjs.org/express-rate-limit/-/express-rate-limit-8.5.2.tgz",
      "integrity": "sha512-5Kb34ipNX694DH48vN9irak1Qx30nb0PLYHXfJgw4YEjiC3ZEmZJhwOp+VfiCYwFzvFTdB9QkArYS5kXa2c
x2A==",
      "license": "MIT",
      "dependencies": {
        "ip-address": "^10.2.0"
      },
      "engines": {
        "node": ">= 16"
      },
      "funding": {
        "url": "https://github.com/sponsors/express-rate-limit"
      },
      "peerDependencies": {
        "express": ">= 4.11"
      }
    },
    "node_modules/express-session": {
      "version": "1.19.0",
      "resolved": "https://registry.npmjs.org/express-session/-/express-session-1.19.0.tgz",
      "integrity": "sha512-0csaMkGq+vaiZTmSMMGkfdCOabYv192VbytFypcvI0MANrp+4i/7yEkJ0sbAEhycQjntaKGzYfjfXQyVb7B
HMA==",
      "license": "MIT",
      "dependencies": {
        "cookie": "~0.7.2",
        "cookie-signature": "~1.0.7",
        "debug": "~2.6.9",
        "depd": "~2.0.0",
        "on-headers": "~1.1.0",
        "parseurl": "~1.3.3",
        "safe-buffer": "~5.2.1",
        "uid-safe": "~2.1.5"
      },
      "engines": {
        "node": ">= 0.8.0"
      },
      "funding": {
        "type": "opencollective",
        "url": "https://opencollective.com/express"
      }
    },
    "node_modules/fast-glob": {
      "version": "3.3.3",
      "resolved": "https://registry.npmjs.org/fast-glob/-/fast-glob-3.3.3.tgz",
      "integrity": "sha512-7MptL8U0cqcFdzIzwOTHoilX9x5BrNqye7Z/LuC7kCMRio1EMSyqRK3BEAUD7sXRq4iT4AzTVuZdhgQ2TCv
YLg==",
      "dev": true,
      "license": "MIT",
      "dependencies": {
        "@nodelib/fs.stat": "^2.0.2",
        "@nodelib/fs.walk": "^1.2.3",
        "glob-parent": "^5.1.2",
        "merge2": "^1.3.0",
```

```
        "micromatch": "^4.0.8"
      },
      "engines": {
        "node": ">=8.6.0"
      }
    },
    "node_modules/fast-glob/node_modules/glob-parent": {
      "version": "5.1.2",
      "resolved": "https://registry.npmjs.org/glob-parent/-/glob-parent-5.1.2.tgz",
      "integrity": "sha512-AOIgSQCepiJYwP3ARnGx+5VnTu2HBYdzbGP45eLw1vr3zB3vZLeyed1sC9hnbcOc9/SrMyM5RPQrkGz4aS9
Zow==",
      "dev": true,
      "license": "ISC",
      "dependencies": {
        "is-glob": "^4.0.1"
      },
      "engines": {
        "node": ">= 6"
      }
    },
    "node_modules/fastq": {
      "version": "1.20.1",
      "resolved": "https://registry.npmjs.org/fastq/-/fastq-1.20.1.tgz",
      "integrity": "sha512-GGToxJ/w1x32s/D2EKND7kTil4n8OVk/9mycTc4VDza13lOvpUZTGX3mFSCtV9ksdGBVzvsyAVLM6mHFThx
Xxw==",
      "dev": true,
      "license": "ISC",
      "dependencies": {
        "reusify": "^1.0.4"
      }
    },
    "node_modules/file-uri-to-path": {
      "version": "1.0.0",
      "resolved": "https://registry.npmjs.org/file-uri-to-path/-/file-uri-to-path-1.0.0.tgz",
      "integrity": "sha512-0Zt+s3L7Vf1biwWZ29aARiVYLx7iMGnEUl9x33fbB/j3jR81u/O2LbqK+Bm1CDSNDKVtJ/YjwY7TUd5SkeL
QLw==",
      "license": "MIT"
    },
    "node_modules/fill-range": {
      "version": "7.1.1",
      "resolved": "https://registry.npmjs.org/fill-range/-/fill-range-7.1.1.tgz",
      "integrity": "sha512-YsGpe3WHLK8ZYi4tWDg2Jy3ebRz2rXowDxnld4bkQB00cc/1Zw9AWnC0i9ztDJitivtQvaI9KaLyKrc+hBW
0yg==",
      "dev": true,
      "license": "MIT",
      "dependencies": {
        "to-regex-range": "^5.0.1"
      },
      "engines": {
        "node": ">=8"
      }
    },
    "node_modules/finalhandler": {
      "version": "1.3.2",
```

```
      "resolved": "https://registry.npmjs.org/finalhandler/-/finalhandler-1.3.2.tgz",
      "integrity": "sha512-aA4RyPcd3badbdABGDuTXCMTtOneUCAYH/gxoYRTZlIJdF0YPWuGqiAsIrhNnnqdXGswYk6dGujem4w80UJFhg==",
      "license": "MIT",
      "dependencies": {
        "debug": "2.6.9",
        "encodeurl": "~2.0.0",
        "escape-html": "~1.0.3",
        "on-finished": "~2.4.1",
        "parseurl": "~1.3.3",
        "statuses": "~2.0.2",
        "unpipe": "~1.0.0"
      },
      "engines": {
        "node": ">= 0.8"
      }
    },
    "node_modules/forwarded": {
      "version": "0.2.0",
      "resolved": "https://registry.npmjs.org/forwarded/-/forwarded-0.2.0.tgz",
      "integrity": "sha512-buRG0fpBtRHSTCOASe6hD258tEubFoRLb4ZNA6NxMVHNw2gOcwHo9wyablzMzOA5z9xA9L1KNjk/Nt6MT9aYow==",
      "license": "MIT",
      "engines": {
        "node": ">= 0.6"
      }
    },
    "node_modules/fresh": {
      "version": "0.5.2",
      "resolved": "https://registry.npmjs.org/fresh/-/fresh-0.5.2.tgz",
      "integrity": "sha512-zJ2mQYM18rEFOudeV4GShTGIQ7RbzA7ozbU9I/XBpm7kqgMywgmylMwXHxZJmkVoYkna9d2pVXVXPdYTP9ej8Q==",
      "license": "MIT",
      "engines": {
        "node": ">= 0.6"
      }
    },
    "node_modules/fs-constants": {
      "version": "1.0.0",
      "resolved": "https://registry.npmjs.org/fs-constants/-/fs-constants-1.0.0.tgz",
      "integrity": "sha512-y6OAwoSIf7FyjMIv94u+b5rdheZEjzR63GTyZJm5qh4Bi+2YgwLCcI/fPFZkL5PSixOt6ZNKm+w+Hfp/Bciwow==",
      "license": "MIT"
    },
    "node_modules/fsevents": {
      "version": "2.3.3",
      "resolved": "https://registry.npmjs.org/fsevents/-/fsevents-2.3.3.tgz",
      "integrity": "sha512-5xoDfX+fL7faATnagmWPpbFtwh/R77WmMMqqHGS65C3vvB0YHrgF+B1YmZ3441tMj5n63k0212XNoJwzlhffQw==",
      "dev": true,
      "hasInstallScript": true,
      "license": "MIT",
      "optional": true,
      "os": [
```

```
        "darwin"
      ],
      "engines": {
        "node": "^8.16.0 || ^10.6.0 || >=11.0.0"
      }
    },
    "node_modules/function-bind": {
      "version": "1.1.2",
      "resolved": "https://registry.npmjs.org/function-bind/-/function-bind-1.1.2.tgz",
      "integrity": "sha512-7XHNxH7qX9xG5mIwxkhumTox/MIRNcOgDrxWsMt2pAr23WHp6MrRlN7FBSFpCpr+oVO0F744iUgR82nJMfG
2SA==",
      "license": "MIT",
      "funding": {
        "url": "https://github.com/sponsors/ljharb"
      }
    },
    "node_modules/get-intrinsic": {
      "version": "1.3.0",
      "resolved": "https://registry.npmjs.org/get-intrinsic/-/get-intrinsic-1.3.0.tgz",
      "integrity": "sha512-9fSjSaos/fRIVIp+xSJlE6lfwhES7LNtKaCBIamHsjr2na1BiABJPo0mOjjz8GJDURarmCPGqaiVg5mfjb9
8CQ==",
      "license": "MIT",
      "dependencies": {
        "call-bind-apply-helpers": "^1.0.2",
        "es-define-property": "^1.0.1",
        "es-errors": "^1.3.0",
        "es-object-atoms": "^1.1.1",
        "function-bind": "^1.1.2",
        "get-proto": "^1.0.1",
        "gopd": "^1.2.0",
        "has-symbols": "^1.1.0",
        "hasown": "^2.0.2",
        "math-intrinsics": "^1.1.0"
      },
      "engines": {
        "node": ">= 0.4"
      },
      "funding": {
        "url": "https://github.com/sponsors/ljharb"
      }
    },
    "node_modules/get-proto": {
      "version": "1.0.1",
      "resolved": "https://registry.npmjs.org/get-proto/-/get-proto-1.0.1.tgz",
      "integrity": "sha512-sTSfBjoXBp89JvIKIefqw7U2CCebsc74kiY6awiGogKtoSGbgjYE/G/+l9sF3MWFPNc9IcoOC4ODfKHfxFm
p0g==",
      "license": "MIT",
      "dependencies": {
        "dunder-proto": "^1.0.1",
        "es-object-atoms": "^1.0.0"
      },
      "engines": {
        "node": ">= 0.4"
      }
```

```
    },
    "node_modules/github-from-package": {
      "version": "0.0.0",
      "resolved": "https://registry.npmjs.org/github-from-package/-/github-from-package-0.0.0.tgz",
      "integrity": "sha512-SyHy3T1v2NUXn29OsWdxmK6RwHD+vkj3v8en8AOBZ1wBQ/hCAQ5bAQTD02kW4W9tUp/3Qh6J8r9EvntiyCmOOw==",
      "license": "MIT"
    },
    "node_modules/glob-parent": {
      "version": "6.0.2",
      "resolved": "https://registry.npmjs.org/glob-parent/-/glob-parent-6.0.2.tgz",
      "integrity": "sha512-XxwI8EOhVQgWp6iDL+3b0r86f4d6AX6zSU55HfB4ydCEuXLXc5FcYeOu+nnGftS4TEju/11rt4KJPTMgbfmv4A==",
      "dev": true,
      "license": "ISC",
      "dependencies": {
        "is-glob": "^4.0.3"
      },
      "engines": {
        "node": ">=10.13.0"
      }
    },
    "node_modules/gopd": {
      "version": "1.2.0",
      "resolved": "https://registry.npmjs.org/gopd/-/gopd-1.2.0.tgz",
      "integrity": "sha512-ZUKRh6/kUFoAiTAtTYPZJ3hw9wNxx+BIBOijnlG9PnrJsCcSjs1wyyD6vJpaYtgnzDrKYRSqf3OO6Rfa93xsRg==",
      "license": "MIT",
      "engines": {
        "node": ">= 0.4"
      },
      "funding": {
        "url": "https://github.com/sponsors/ljharb"
      }
    },
    "node_modules/has-symbols": {
      "version": "1.1.0",
      "resolved": "https://registry.npmjs.org/has-symbols/-/has-symbols-1.1.0.tgz",
      "integrity": "sha512-1cDNdwJ2Jaohmb3sg4OmKaMBwuC48sYni5HUw2DvsC8LjGTLK9h+eb1X6RyuOHe4hT0ULCW68iomhjUoKUqlPQ==",
      "license": "MIT",
      "engines": {
        "node": ">= 0.4"
      },
      "funding": {
        "url": "https://github.com/sponsors/ljharb"
      }
    },
    "node_modules/hasown": {
      "version": "2.0.4",
      "resolved": "https://registry.npmjs.org/hasown/-/hasown-2.0.4.tgz",
      "integrity": "sha512-T2UbfbBEF32wiepXIsMlTW9+dDYC6wMh/t/vYA4tuOMKqWz/n3vr1NFSxQiyP+zk2mXsoMA/i/7qV6LKut1t1A==",
      "license": "MIT",
```

```
      "dependencies": {
        "function-bind": "^1.1.2"
      },
      "engines": {
        "node": ">= 0.4"
      }
    },
    "node_modules/helmet": {
      "version": "8.1.0",
      "resolved": "https://registry.npmjs.org/helmet/-/helmet-8.1.0.tgz",
      "integrity": "sha512-jOiHyAZsmnr8LqoPGmCjYAaiuWwjAPLgY8ZX2XrmHawt99/u1y6RgrZMTeoPfpUbV96HOalYgz1qzkRbw54
Pmg==",
      "license": "MIT",
      "engines": {
        "node": ">=18.0.0"
      }
    },
    "node_modules/http-errors": {
      "version": "2.0.1",
      "resolved": "https://registry.npmjs.org/http-errors/-/http-errors-2.0.1.tgz",
      "integrity": "sha512-4FbRdAX+bSdmo4AUFuS0WNiPz8NgFt+r8ThgNWmlrjQjt1Q7ZR9+zTlce2859x4KSXrwIsaeTqDoKQmtP8p
LmQ==",
      "license": "MIT",
      "dependencies": {
        "depd": "~2.0.0",
        "inherits": "~2.0.4",
        "setprototypeof": "~1.2.0",
        "statuses": "~2.0.2",
        "toidentifier": "~1.0.1"
      },
      "engines": {
        "node": ">= 0.8"
      },
      "funding": {
        "type": "opencollective",
        "url": "https://opencollective.com/express"
      }
    },
    "node_modules/iconv-lite": {
      "version": "0.4.24",
      "resolved": "https://registry.npmjs.org/iconv-lite/-/iconv-lite-0.4.24.tgz",
      "integrity": "sha512-v3MXnZAcvnywkTUEZomIActle7RXXeedOR31wwl7VlyoXO4Qi9arvSenNQWne1TcRwhCL1HwLI21bEqdpj8
/rA==",
      "license": "MIT",
      "dependencies": {
        "safer-buffer": ">= 2.1.2 < 3"
      },
      "engines": {
        "node": ">=0.10.0"
      }
    },
    "node_modules/ieee754": {
      "version": "1.2.1",
      "resolved": "https://registry.npmjs.org/ieee754/-/ieee754-1.2.1.tgz",
```

```
      "integrity": "sha512-dcyqhDvX1C46lXZcVqCpK+FtMRQVdIMN6/Df5js2zouUsqG7I6sFxitIC+7KYK29KdXOLHdu9zL4sFnoVQn
qaA==",
      "funding": [
        {
          "type": "github",
          "url": "https://github.com/sponsors/feross"
        },
        {
          "type": "patreon",
          "url": "https://www.patreon.com/feross"
        },
        {
          "type": "consulting",
          "url": "https://feross.org/support"
        }
      ],
      "license": "BSD-3-Clause"
    },
    "node_modules/inherits": {
      "version": "2.0.4",
      "resolved": "https://registry.npmjs.org/inherits/-/inherits-2.0.4.tgz",
      "integrity": "sha512-k/vGaX4/Yla3WzyMCvTQOXYeIHvqOKtnqBduzTHpzpQZzAskKMhZ2K+EnBiSM9zGSoIFeMpXKxa4dYeZIQq
ewQ==",
      "license": "ISC"
    },
    "node_modules/ini": {
      "version": "1.3.8",
      "resolved": "https://registry.npmjs.org/ini/-/ini-1.3.8.tgz",
      "integrity": "sha512-JV/yugV2uzW5iMRSiZAyDtQd+nxtUnjeLt0acNdw98kKLrvuRVyB80tsREOE7yvGVgalhZ6RNXCmEHkUKBK
xew==",
      "license": "ISC"
    },
    "node_modules/ip-address": {
      "version": "10.2.0",
      "resolved": "https://registry.npmjs.org/ip-address/-/ip-address-10.2.0.tgz",
      "integrity": "sha512-/+S6j4E9AHvW9SWMSEY9Xfy66O5PWvVEJ08O0y5JGyEKQpojb0K0GKpz/v5HJ/G0vi3D2sjGK78119oXZeE
0qA==",
      "license": "MIT",
      "engines": {
        "node": ">= 12"
      }
    },
    "node_modules/ipaddr.js": {
      "version": "1.9.1",
      "resolved": "https://registry.npmjs.org/ipaddr.js/-/ipaddr.js-1.9.1.tgz",
      "integrity": "sha512-0KI/607xoxSToH7GjN1FfSbLoU0+btTicjsQSWQlh/hZykN8KpmMf7uYwPW3R+akZ6R/w18ZlXSHBYXiYUP
O3g==",
      "license": "MIT",
      "engines": {
        "node": ">= 0.10"
      }
    },
    "node_modules/is-binary-path": {
      "version": "2.1.0",
```

```
      "resolved": "https://registry.npmjs.org/is-binary-path/-/is-binary-path-2.1.0.tgz",
      "integrity": "sha512-ZMERYes6pDydyuGidse7OsHxtbI7WVeUEozgR/g7rd0xUimYNlvZRE/K2MgZTjWy725IfelLeVcEM97mmtR
GXw==",
      "dev": true,
      "license": "MIT",
      "dependencies": {
        "binary-extensions": "^2.0.0"
      },
      "engines": {
        "node": ">=8"
      }
    },
    "node_modules/is-core-module": {
      "version": "2.16.2",
      "resolved": "https://registry.npmjs.org/is-core-module/-/is-core-module-2.16.2.tgz",
      "integrity": "sha512-evOr8xfXKxE6qSR0hSXL2r3sd7ALj8+7jQEUvPYcm5sgZFdJ+AYzT6yNmJenvIYQBgIGwfwz08sL8zoL7yq
2BA==",
      "dev": true,
      "license": "MIT",
      "dependencies": {
        "hasown": "^2.0.3"
      },
      "engines": {
        "node": ">= 0.4"
      },
      "funding": {
        "url": "https://github.com/sponsors/ljharb"
      }
    },
    "node_modules/is-extglob": {
      "version": "2.1.1",
      "resolved": "https://registry.npmjs.org/is-extglob/-/is-extglob-2.1.1.tgz",
      "integrity": "sha512-SbKbANkN603Vi4jEZv49LeVJMn4yGwsbzZworEoyEiutsN3nJYdbO36zfhGJ6QEDpOZIFkDtnq5JRxmvl3j
soQ==",
      "dev": true,
      "license": "MIT",
      "engines": {
        "node": ">=0.10.0"
      }
    },
    "node_modules/is-glob": {
      "version": "4.0.3",
      "resolved": "https://registry.npmjs.org/is-glob/-/is-glob-4.0.3.tgz",
      "integrity": "sha512-xelSayHH36ZgE7ZWhli7pW34hNbNl8Ojv5KVmkJD4hBdD3th8Tfk9vYasLM+mXWOZhFkgZfxhLSnrwRr4el
SSg==",
      "dev": true,
      "license": "MIT",
      "dependencies": {
        "is-extglob": "^2.1.1"
      },
      "engines": {
        "node": ">=0.10.0"
      }
    },
```

```
    "node_modules/is-number": {
      "version": "7.0.0",
      "resolved": "https://registry.npmjs.org/is-number/-/is-number-7.0.0.tgz",
      "integrity": "sha512-41Cifkg6e8TylSpdtTpeLVMqvSBEVzTttHvERD741+pnZ8ANv0004MRL43QKPDlK9cGvNp6NZWZUBlbGXYx
xng==",
      "dev": true,
      "license": "MIT",
      "engines": {
        "node": ">=0.12.0"
      }
    },
    "node_modules/jiti": {
      "version": "1.21.7",
      "resolved": "https://registry.npmjs.org/jiti/-/jiti-1.21.7.tgz",
      "integrity": "sha512-/imKNG4EbWNrVjoNC/1H5/9GFy+tqjGBHCaSsN+P2RnPqjsLmv6UD3Ej+Kj8nBWaRAwyk7kK5ZUc+OEatnT
R3A==",
      "dev": true,
      "license": "MIT",
      "bin": {
        "jiti": "bin/jiti.js"
      }
    },
    "node_modules/lilconfig": {
      "version": "3.1.3",
      "resolved": "https://registry.npmjs.org/lilconfig/-/lilconfig-3.1.3.tgz",
      "integrity": "sha512-/vlFKAoH5Cgt3Ie+JLhRbwOsCQePABiU3tJ1egGvyQ+33R/vcwM2Zl2QR/LzjsBeItPt3oSVXapn+m4nQDv
pzw==",
      "dev": true,
      "license": "MIT",
      "engines": {
        "node": ">=14"
      },
      "funding": {
        "url": "https://github.com/sponsors/antonk52"
      }
    },
    "node_modules/lines-and-columns": {
      "version": "1.2.4",
      "resolved": "https://registry.npmjs.org/lines-and-columns/-/lines-and-columns-1.2.4.tgz",
      "integrity": "sha512-7ylylesZQ/PV29jhEDl3Ufjo6ZX7gCqJr5F7PKrqc93v7fzSymt1BpwEU8nAUXs8qzzvqhbjhK5QZg6Mt/H
kBg==",
      "dev": true,
      "license": "MIT"
    },
    "node_modules/math-intrinsics": {
      "version": "1.1.0",
      "resolved": "https://registry.npmjs.org/math-intrinsics/-/math-intrinsics-1.1.0.tgz",
      "integrity": "sha512-/IXtbwEk5HTPyEwyKX6hGkYXxM9nbj64B+ilVJnC/R6B0pH5G4V3b0pVbL7DBj4tkhBAppbQUlf6F6Xl9LH
u1g==",
      "license": "MIT",
      "engines": {
        "node": ">= 0.4"
      }
    },
```

```
    "node_modules/media-typer": {
      "version": "0.3.0",
      "resolved": "https://registry.npmjs.org/media-typer/-/media-typer-0.3.0.tgz",
      "integrity": "sha512-dq+qelQ9akHpcOl/gUVRTxVIOkAJ1wR3QAvb4RsVjS8oVoFjDGTc679wJYmUmknUF5HwMLOgb5O+a3KxfWa
pPQ==",
      "license": "MIT",
      "engines": {
        "node": ">= 0.6"
      }
    },
    "node_modules/merge-descriptors": {
      "version": "1.0.3",
      "resolved": "https://registry.npmjs.org/merge-descriptors/-/merge-descriptors-1.0.3.tgz",
      "integrity": "sha512-gaNvAS7TZ897/rVaZ0nMtAyxNyi/pdbjbAwUpFQpN70GqnVfOiXpeUUMKRBmzXaSQ8DdTX4/0ms62r2K+hE
6mQ==",
      "license": "MIT",
      "funding": {
        "url": "https://github.com/sponsors/sindresorhus"
      }
    },
    "node_modules/merge2": {
      "version": "1.4.1",
      "resolved": "https://registry.npmjs.org/merge2/-/merge2-1.4.1.tgz",
      "integrity": "sha512-8q7VEgMJW4J8tcfVPy8g09NcQwZdbwFEqhe/WZkoIzjn/3TGDwtOCYtXGxA3O8tPzpczCCDgv+P2P5y00ZJ
OOg==",
      "dev": true,
      "license": "MIT",
      "engines": {
        "node": ">= 8"
      }
    },
    "node_modules/methods": {
      "version": "1.1.2",
      "resolved": "https://registry.npmjs.org/methods/-/methods-1.1.2.tgz",
      "integrity": "sha512-iclAHeNqNm68zFtnZ0e+1L2yUIdvzNoauKU4WBA3VvH/vPFieF7qfRlwUZU+DA9P9bPXIS90ulxoUoCH23s
V2w==",
      "license": "MIT",
      "engines": {
        "node": ">= 0.6"
      }
    },
    "node_modules/micromatch": {
      "version": "4.0.8",
      "resolved": "https://registry.npmjs.org/micromatch/-/micromatch-4.0.8.tgz",
      "integrity": "sha512-PXwfBhYu0hBCPw8Dn0E+WDYb7af3dSLVWKi3HGv84IdF4TyFoC0ysxFd0Goxw7nSv4T/PzEJQxsYsEiFCKo
2BA==",
      "dev": true,
      "license": "MIT",
      "dependencies": {
        "braces": "^3.0.3",
        "picomatch": "^2.3.1"
      },
      "engines": {
        "node": ">=8.6"
```

```
      }
    },
    "node_modules/mime": {
      "version": "1.6.0",
      "resolved": "https://registry.npmjs.org/mime/-/mime-1.6.0.tgz",
      "integrity": "sha512-x0Vn8spI+wuJ1O6S7gnbaQg8Pxh4NNHb7KSINmEWKiPE4RKOplvijn+NkmYmmRgP68mc70j2EbeTFRsrswa
Qeg==",
      "license": "MIT",
      "bin": {
        "mime": "cli.js"
      },
      "engines": {
        "node": ">=4"
      }
    },
    "node_modules/mime-db": {
      "version": "1.52.0",
      "resolved": "https://registry.npmjs.org/mime-db/-/mime-db-1.52.0.tgz",
      "integrity": "sha512-sPU4uV7dYlvtWJxwwxHD0PuihVNiE7TyAbQ5SWxDCB9mUYvOgroQOwYQQOKPJ8CIbE+1ETVlOoK1UC2nU3g
Yvg==",
      "license": "MIT",
      "engines": {
        "node": ">= 0.6"
      }
    },
    "node_modules/mime-types": {
      "version": "2.1.35",
      "resolved": "https://registry.npmjs.org/mime-types/-/mime-types-2.1.35.tgz",
      "integrity": "sha512-ZDY+bPm5zTTF+YpCrAU9nK0UgICYPT0QtT1NZWFv4s++TNkcgVaT0g6+4R2uI4MjQjzysHB1zxuWL50hzae
Xiw==",
      "license": "MIT",
      "dependencies": {
        "mime-db": "1.52.0"
      },
      "engines": {
        "node": ">= 0.6"
      }
    },
    "node_modules/mimic-response": {
      "version": "3.1.0",
      "resolved": "https://registry.npmjs.org/mimic-response/-/mimic-response-3.1.0.tgz",
      "integrity": "sha512-z0yWI+4FDrrweS8Zmt4Ej5HdJmky15+L2e6Wgn3+iK5fWzb6T3fhNFq2+MeTRb064c6Wr4N/wv0DzQTjNzH
NGQ==",
      "license": "MIT",
      "engines": {
        "node": ">=10"
      },
      "funding": {
        "url": "https://github.com/sponsors/sindresorhus"
      }
    },
    "node_modules/mini-svg-data-uri": {
      "version": "1.4.4",
      "resolved": "https://registry.npmjs.org/mini-svg-data-uri/-/mini-svg-data-uri-1.4.4.tgz",
```

```
      "integrity": "sha512-r9deDe9p5FJUPZAk3A59wGH7Ii9YrjjWw0jmw/liSbHl2CHiyXj6FcDXDu2K3TjVAXqiJdaw3xxwlZZr9E6
nHg==",
      "dev": true,
      "license": "MIT",
      "bin": {
        "mini-svg-data-uri": "cli.js"
      }
    },
    "node_modules/minimist": {
      "version": "1.2.8",
      "resolved": "https://registry.npmjs.org/minimist/-/minimist-1.2.8.tgz",
      "integrity": "sha512-2yyAR8qBkN3YuheJanUpWC5U3bb5osDywNB8RzDVlDwDHbocAJveqqj1u8+SVD7jkWT4yvsHCpWqqWqAxb0
zCA==",
      "license": "MIT",
      "funding": {
        "url": "https://github.com/sponsors/ljharb"
      }
    },
    "node_modules/mkdirp-classic": {
      "version": "0.5.3",
      "resolved": "https://registry.npmjs.org/mkdirp-classic/-/mkdirp-classic-0.5.3.tgz",
      "integrity": "sha512-gKLcREMhtuZRwRAfqP3RFW+TK4JqApVBtOIftVgjuABpAtpxhPGaDcfvbhNvD0B8iD1oUr/txX35NjcaY6N
s/A==",
      "license": "MIT"
    },
    "node_modules/ms": {
      "version": "2.0.0",
      "resolved": "https://registry.npmjs.org/ms/-/ms-2.0.0.tgz",
      "integrity": "sha512-Tpp60P6IUJDTuOq/5Z8cdskzJujfwqfOTkrwIwj7IRISpnkJnT6SyJ4PCPnGMoFjC9ddhal5KVIYtAt97ix
05A==",
      "license": "MIT"
    },
    "node_modules/multer": {
      "version": "2.2.0",
      "resolved": "https://registry.npmjs.org/multer/-/multer-2.2.0.tgz",
      "integrity": "sha512-6rdyFg2kLrMh9Jee7/BMPuV9lEAd7lLW2YUpF9/YxR7njyoUwwQ0ZPh3TaIY50Sw6vlyD2HW3wGOkTS4P79
xrQ==",
      "license": "MIT",
      "dependencies": {
        "append-field": "^1.0.0",
        "busboy": "^1.6.0",
        "concat-stream": "^2.0.0",
        "type-is": "^1.6.18"
      },
      "engines": {
        "node": ">= 10.16.0"
      },
      "funding": {
        "type": "opencollective",
        "url": "https://opencollective.com/express"
      }
    },
    "node_modules/mz": {
      "version": "2.7.0",
```

```
      "resolved": "https://registry.npmjs.org/mz/-/mz-2.7.0.tgz",
      "integrity": "sha512-z81GNO7nnYMEhrGh9LeymoE4+Yr0Wn5McHIZMK5cfQCl+NDX08sCZgUc9/6MHni9IWuFLm1Z3HTCXu2z9fN
62Q==",
      "dev": true,
      "license": "MIT",
      "dependencies": {
        "any-promise": "^1.0.0",
        "object-assign": "^4.0.1",
        "thenify-all": "^1.0.0"
      }
    },
    "node_modules/nanoid": {
      "version": "3.3.12",
      "resolved": "https://registry.npmjs.org/nanoid/-/nanoid-3.3.12.tgz",
      "integrity": "sha512-ZB9RH/39qpq5Vu6Y+NmUaFhQR6pp+M2Xt76XBnEwDaGcVAqhlvxrl3B2bKS5D3NH3QR76v3aSrKaF/Kiy7l
EtQ==",
      "dev": true,
      "funding": [
        {
          "type": "github",
          "url": "https://github.com/sponsors/ai"
        }
      ],
      "license": "MIT",
      "bin": {
        "nanoid": "bin/nanoid.cjs"
      },
      "engines": {
        "node": "^10 || ^12 || ^13.7 || ^14 || >=15.0.1"
      }
    },
    "node_modules/napi-build-utils": {
      "version": "2.0.0",
      "resolved": "https://registry.npmjs.org/napi-build-utils/-/napi-build-utils-2.0.0.tgz",
      "integrity": "sha512-GEbrYkbfF7MoNaoh2iGG84Mnf/WZfB0GdGEsM8wz7Expx/LlWf5U8t9nvJKXSp3qr5IsEbK04cBGhol/KwO
sWA==",
      "license": "MIT"
    },
    "node_modules/negotiator": {
      "version": "0.6.3",
      "resolved": "https://registry.npmjs.org/negotiator/-/negotiator-0.6.3.tgz",
      "integrity": "sha512-+EUsqGPLsM+j/zdChZjsnX51g4XrHFOIXwfnCVPGlQk/k5giakcKsuxCObBRu6DSm9opw/O6slWbJdghQM4
bBg==",
      "license": "MIT",
      "engines": {
        "node": ">= 0.6"
      }
    },
    "node_modules/node-abi": {
      "version": "3.87.0",
      "resolved": "https://registry.npmjs.org/node-abi/-/node-abi-3.87.0.tgz",
      "integrity": "sha512-+CGM1L1CgmtheLcBuleyYOn7NWPVu0s0EJH2C4puxgEZb9h8QpR9G2dBfZJOAUhi7VQxuBPMd0hiISWcTyi
YyQ==",
      "license": "MIT",
```

```
      "dependencies": {
        "semver": "^7.3.5"
      },
      "engines": {
        "node": ">=10"
      }
    },
    "node_modules/node-addon-api": {
      "version": "8.5.0",
      "resolved": "https://registry.npmjs.org/node-addon-api/-/node-addon-api-8.5.0.tgz",
      "integrity": "sha512-/bRZty2mXUIFY/xU5HLvveNHlswNJej+RnxBjOMkidWfwZzgTbPG1E3K5TOxRLOR+5hX7bSofy8yf1hZevM
S8A==",
      "license": "MIT",
      "engines": {
        "node": "^18 || ^20 || >= 21"
      }
    },
    "node_modules/node-gyp-build": {
      "version": "4.8.4",
      "resolved": "https://registry.npmjs.org/node-gyp-build/-/node-gyp-build-4.8.4.tgz",
      "integrity": "sha512-LA4ZjwlnUblHVgq0oBF3Jl/6h/Nvs5fzBLwdEF4nuxnFdsfajde4WfxtJr3CaiH+F6ewcIB/q4jQ4UzPyid
+CQ==",
      "license": "MIT",
      "bin": {
        "node-gyp-build": "bin.js",
        "node-gyp-build-optional": "optional.js",
        "node-gyp-build-test": "build-test.js"
      }
    },
    "node_modules/normalize-path": {
      "version": "3.0.0",
      "resolved": "https://registry.npmjs.org/normalize-path/-/normalize-path-3.0.0.tgz",
      "integrity": "sha512-6eZs5Ls3WtCisHWp9S2GUy8dqkpGi4BVSz3GaqiE6ezub0512ESztXUwUB6C6IKbQkY2Pnb/mD4WYojCRwc
wLA==",
      "dev": true,
      "license": "MIT",
      "engines": {
        "node": ">=0.10.0"
      }
    },
    "node_modules/object-assign": {
      "version": "4.1.1",
      "resolved": "https://registry.npmjs.org/object-assign/-/object-assign-4.1.1.tgz",
      "integrity": "sha512-rJgTQnkUnH1sFw8yT6VSU3zD3sWmu6sZhIseY8VX+GRu3P6F7Fu+JNDoXfklElbLJSnc3FUQHVe4cU5hj+B
cUg==",
      "license": "MIT",
      "engines": {
        "node": ">=0.10.0"
      }
    },
    "node_modules/object-hash": {
      "version": "3.0.0",
      "resolved": "https://registry.npmjs.org/object-hash/-/object-hash-3.0.0.tgz",
      "integrity": "sha512-RSn9F68PjH9HqtltsSnqYC1XXoWe9Bju5+213R98cNGttag9q9yAOTzdbsqvIa7aNm5WffBZFpWYr2aWrkl
```

```
WAw==",
      "dev": true,
      "license": "MIT",
      "engines": {
        "node": ">= 6"
      }
    },
    "node_modules/object-inspect": {
      "version": "1.13.4",
      "resolved": "https://registry.npmjs.org/object-inspect/-/object-inspect-1.13.4.tgz",
      "integrity": "sha512-W67iLl4J2EXEGTbfeHCffrjDfitvLANg0UlX3wFUUSTx92KXRFegMHUVgSqE+wvhAbi4WqjGg9czysTV2Ep
bew==",
      "license": "MIT",
      "engines": {
        "node": ">= 0.4"
      },
      "funding": {
        "url": "https://github.com/sponsors/ljharb"
      }
    },
    "node_modules/on-finished": {
      "version": "2.4.1",
      "resolved": "https://registry.npmjs.org/on-finished/-/on-finished-2.4.1.tgz",
      "integrity": "sha512-oVlzkg3ENAhCk2zdv7IJwd/QUD4z2RxRwpkcGY8psCVcCYZNq4wYnVWALHM+brtuJjePWiYF/ClmuDr8Ch5
+kg==",
      "license": "MIT",
      "dependencies": {
        "ee-first": "1.1.1"
      },
      "engines": {
        "node": ">= 0.8"
      }
    },
    "node_modules/on-headers": {
      "version": "1.1.0",
      "resolved": "https://registry.npmjs.org/on-headers/-/on-headers-1.1.0.tgz",
      "integrity": "sha512-737ZY3yNnXy37FHkQxPzt4UZ2UWPWiCZWLvFZ4fu5cueciegX0zGPnrlY6bwRg4FdQOe9YU8MkmJwGhoMyb
l8A==",
      "license": "MIT",
      "engines": {
        "node": ">= 0.8"
      }
    },
    "node_modules/once": {
      "version": "1.4.0",
      "resolved": "https://registry.npmjs.org/once/-/once-1.4.0.tgz",
      "integrity": "sha512-lNaJgI+2Q5URQBkccEKHTQOPaXdUxnZZElQTZY0MFUAuaEqe1E+Nyvgdz/aIyNi6Z9MzO5dv1H8n58/GELp
3+w==",
      "license": "ISC",
      "dependencies": {
        "wrappy": "1"
      }
    },
    "node_modules/parseurl": {
```

```
      "version": "1.3.3",
      "resolved": "https://registry.npmjs.org/parseurl/-/parseurl-1.3.3.tgz",
      "integrity": "sha512-CiyeOxFT/JZyN5m0z9PfXw4SCBJ6Sygz1Dpl0wqjlhDEGGBP1GnsUVEL0p63hoG1fcj3fHynXi9NYO4nWOL
+qQ==",
      "license": "MIT",
      "engines": {
        "node": ">= 0.8"
      }
    },
    "node_modules/path-parse": {
      "version": "1.0.7",
      "resolved": "https://registry.npmjs.org/path-parse/-/path-parse-1.0.7.tgz",
      "integrity": "sha512-LDJzPVEEEPR+y48z93A0Ed0yXb8pAByGWo/k5YYdYgpY2/2EsOsksJrq7lOHxryrVOn1ejG6oAp8ahvOIQD
8sw==",
      "dev": true,
      "license": "MIT"
    },
    "node_modules/path-to-regexp": {
      "version": "0.1.13",
      "resolved": "https://registry.npmjs.org/path-to-regexp/-/path-to-regexp-0.1.13.tgz",
      "integrity": "sha512-A/AGNMFN3c8bOlvV9RreMdrv7jsmF9XIfDeCd87+I8RNg6s78BhJxMu69NEMHBSJFxKidViTEdruRwEk/WI
KqA==",
      "license": "MIT"
    },
    "node_modules/pg": {
      "version": "8.18.0",
      "resolved": "https://registry.npmjs.org/pg/-/pg-8.18.0.tgz",
      "integrity": "sha512-xqrUDL1b9MbkydY/s+VZ6v+xiMUmOUk7SS9d/1kpyQxoJ6U9AO1oIJyUWVZojbfe5Cc/oluutcgFG4L9RDP
1iQ==",
      "license": "MIT",
      "dependencies": {
        "pg-connection-string": "^2.11.0",
        "pg-pool": "^3.11.0",
        "pg-protocol": "^1.11.0",
        "pg-types": "2.2.0",
        "pgpass": "1.0.5"
      },
      "engines": {
        "node": ">= 16.0.0"
      },
      "optionalDependencies": {
        "pg-cloudflare": "^1.3.0"
      },
      "peerDependencies": {
        "pg-native": ">=3.0.1"
      },
      "peerDependenciesMeta": {
        "pg-native": {
          "optional": true
        }
      }
    },
    "node_modules/pg-cloudflare": {
      "version": "1.3.0",
```

```
      "resolved": "https://registry.npmjs.org/pg-cloudflare/-/pg-cloudflare-1.3.0.tgz",
      "integrity": "sha512-6lswVVSztmHiRtD6I8hw4qP/nDm1EJbKMRhf3HCYaqud7frGysPv7FYJ5noZQdhQtN2xJnimfMtvQq21pdbzyQ==",
      "license": "MIT",
      "optional": true
    },
    "node_modules/pg-connection-string": {
      "version": "2.11.0",
      "resolved": "https://registry.npmjs.org/pg-connection-string/-/pg-connection-string-2.11.0.tgz",
      "integrity": "sha512-kecgoJwhOpxYU21rZjULrmrBJ698U2RxXofKVzOn5UDj61BPj/qMb7diYUR1nLScCDbrztQFl1TaQZT0t1EtzQ==",
      "license": "MIT"
    },
    "node_modules/pg-int8": {
      "version": "1.0.1",
      "resolved": "https://registry.npmjs.org/pg-int8/-/pg-int8-1.0.1.tgz",
      "integrity": "sha512-WCtabS6t3c8SkpDBUlb1kjOs7l66xsGdKpIPZsg4wR+B3+u9UAum2odSsF9tnvxg80h4ZxLWMy4pRjOsFIqQpw==",
      "license": "ISC",
      "engines": {
        "node": ">=4.0.0"
      }
    },
    "node_modules/pg-pool": {
      "version": "3.11.0",
      "resolved": "https://registry.npmjs.org/pg-pool/-/pg-pool-3.11.0.tgz",
      "integrity": "sha512-MJYfvHwtGp870aeusDh+hg9apvOe2zmpZJpyt+BMtzUWlVqbhFmMK6bOBXLBUPd7iRtIF9fZplDc7KrPN3PN7w==",
      "license": "MIT",
      "peerDependencies": {
        "pg": ">=8.0"
      }
    },
    "node_modules/pg-protocol": {
      "version": "1.11.0",
      "resolved": "https://registry.npmjs.org/pg-protocol/-/pg-protocol-1.11.0.tgz",
      "integrity": "sha512-pfsxk2M9M3BuGgDOfuy37VNRRX3jmKgMjcvAcWqNDpZSf4cUmv8HSOl5ViRQFsfARFn0KuUQTgLxVMbNq5NW3g==",
      "license": "MIT"
    },
    "node_modules/pg-types": {
      "version": "2.2.0",
      "resolved": "https://registry.npmjs.org/pg-types/-/pg-types-2.2.0.tgz",
      "integrity": "sha512-qTAAlrEsl8s4OiEQY69wDvcMIdQN6wdz5ojQiOy6YRMuynxenON0O5oCpJI6lshc6scgAY8qvJ2On/p+CXY0GA==",
      "license": "MIT",
      "dependencies": {
        "pg-int8": "1.0.1",
        "postgres-array": "~2.0.0",
        "postgres-bytea": "~1.0.0",
        "postgres-date": "~1.0.4",
        "postgres-interval": "^1.1.0"
      },
      "engines": {
```

```
        "node": ">=4"
      }
    },
    "node_modules/pgpass": {
      "version": "1.0.5",
      "resolved": "https://registry.npmjs.org/pgpass/-/pgpass-1.0.5.tgz",
      "integrity": "sha512-FdW9r/jQZhSeohs1Z3sI1yxFQNFvMcnmfuj4WBMUTxOrAyLMaTcE1aAMBiTlbMNaXvBCQuVi0R7hd8udDSP
7ug==",
      "license": "MIT",
      "dependencies": {
        "split2": "^4.1.0"
      }
    },
    "node_modules/picocolors": {
      "version": "1.1.1",
      "resolved": "https://registry.npmjs.org/picocolors/-/picocolors-1.1.1.tgz",
      "integrity": "sha512-xceH2snhtb5M9liqDsmEw56le376mTZkEX/jEb/RxNFyegNul7eNslCXP9FDj/Lcu0X8KEyMceP2ntpaHrD
EVA==",
      "dev": true,
      "license": "ISC"
    },
    "node_modules/picomatch": {
      "version": "2.3.2",
      "resolved": "https://registry.npmjs.org/picomatch/-/picomatch-2.3.2.tgz",
      "integrity": "sha512-V7+vQEJ06Z+c5tSye8S+nHUfI51xoXIXjHQ99cQtKUkQqqO1kO/KCJUfZXuB47h/YBlDhah2H3hdUGXn8ie
0oA==",
      "dev": true,
      "license": "MIT",
      "engines": {
        "node": ">=8.6"
      },
      "funding": {
        "url": "https://github.com/sponsors/jonschlinkert"
      }
    },
    "node_modules/pify": {
      "version": "2.3.0",
      "resolved": "https://registry.npmjs.org/pify/-/pify-2.3.0.tgz",
      "integrity": "sha512-udgsAY+fTnvv7kI7aaxbqwWNb0AHiB0qBO89PZKPkoTmGOgdbrHDKD+0B2X4uTfJ/FT1R09r9gTsjUjNJot
uog==",
      "dev": true,
      "license": "MIT",
      "engines": {
        "node": ">=0.10.0"
      }
    },
    "node_modules/pirates": {
      "version": "4.0.7",
      "resolved": "https://registry.npmjs.org/pirates/-/pirates-4.0.7.tgz",
      "integrity": "sha512-TfySrs/5nm8fQJDcBDuUng3VOUKsd7S+zqvbOTiGXHfxX4wK31ard+hoNuvkicM/2YFzlpDgABOevKSsB4G
/FA==",
      "dev": true,
      "license": "MIT",
      "engines": {
```

```
        "node": ">= 6"
      }
    },
    "node_modules/postcss": {
      "version": "8.5.15",
      "resolved": "https://registry.npmjs.org/postcss/-/postcss-8.5.15.tgz",
      "integrity": "sha512-FfR8sjd4em2T6fb3I2MwAJU7HWVMr9zba+enmQeeWFfCbm+UOC/0X4DS8XtpUTMwWMGbjKYP7xjfNekzyGm
B3A==",
      "dev": true,
      "funding": [
        {
          "type": "opencollective",
          "url": "https://opencollective.com/postcss/"
        },
        {
          "type": "tidelift",
          "url": "https://tidelift.com/funding/github/npm/postcss"
        },
        {
          "type": "github",
          "url": "https://github.com/sponsors/ai"
        }
      ],
      "license": "MIT",
      "dependencies": {
        "nanoid": "^3.3.12",
        "picocolors": "^1.1.1",
        "source-map-js": "^1.2.1"
      },
      "engines": {
        "node": "^10 || ^12 || >=14"
      }
    },
    "node_modules/postcss-import": {
      "version": "15.1.0",
      "resolved": "https://registry.npmjs.org/postcss-import/-/postcss-import-15.1.0.tgz",
      "integrity": "sha512-hpr+J05B2FVYUAXHeK1YyI267J/dDDhMU6B6civm8hSY1jYJnBXxzKDKDswzJmtLHryrjhnDjqqp/49t8FA
Lew==",
      "dev": true,
      "license": "MIT",
      "dependencies": {
        "postcss-value-parser": "^4.0.0",
        "read-cache": "^1.0.0",
        "resolve": "^1.1.7"
      },
      "engines": {
        "node": ">=14.0.0"
      },
      "peerDependencies": {
        "postcss": "^8.0.0"
      }
    },
    "node_modules/postcss-js": {
      "version": "4.1.0",
```

```
      "resolved": "https://registry.npmjs.org/postcss-js/-/postcss-js-4.1.0.tgz",
      "integrity": "sha512-oIAOTqgIo7q2EOwbhb8UalYePMvYoIeRY2YKntdpFQXNosSu3vLrniGgmH9OKs/qAkfoj5oB3le/7mINW1LCfw==",
      "dev": true,
      "funding": [
        {
          "type": "opencollective",
          "url": "https://opencollective.com/postcss/"
        },
        {
          "type": "github",
          "url": "https://github.com/sponsors/ai"
        }
      ],
      "license": "MIT",
      "dependencies": {
        "camelcase-css": "^2.0.1"
      },
      "engines": {
        "node": "^12 || ^14 || >= 16"
      },
      "peerDependencies": {
        "postcss": "^8.4.21"
      }
    },
    "node_modules/postcss-load-config": {
      "version": "6.0.1",
      "resolved": "https://registry.npmjs.org/postcss-load-config/-/postcss-load-config-6.0.1.tgz",
      "integrity": "sha512-oPtTM4oerL+UXmx+93ytZVN82RrlY/wPUV8IeDxFrzIjXOLF1pN+EmKPLbubvKHT2HC20xXsCAH2Z+CKV6Oz/g==",
      "dev": true,
      "funding": [
        {
          "type": "opencollective",
          "url": "https://opencollective.com/postcss/"
        },
        {
          "type": "github",
          "url": "https://github.com/sponsors/ai"
        }
      ],
      "license": "MIT",
      "dependencies": {
        "lilconfig": "^3.1.1"
      },
      "engines": {
        "node": ">= 18"
      },
      "peerDependencies": {
        "jiti": ">=1.21.0",
        "postcss": ">=8.0.9",
        "tsx": "^4.8.1",
        "yaml": "^2.4.2"
      },
```

```
      "peerDependenciesMeta": {
        "jiti": {
          "optional": true
        },
        "postcss": {
          "optional": true
        },
        "tsx": {
          "optional": true
        },
        "yaml": {
          "optional": true
        }
      }
    },
    "node_modules/postcss-nested": {
      "version": "6.2.0",
      "resolved": "https://registry.npmjs.org/postcss-nested/-/postcss-nested-6.2.0.tgz",
      "integrity": "sha512-HQbt28KulC5AJzG+cZtj9kvKB93CFCdLvog1WFLf1D+xmMvPGlBstkpTEZfK5+AN9hfJocyBFCNiqyS48bp
gzQ==",
      "dev": true,
      "funding": [
        {
          "type": "opencollective",
          "url": "https://opencollective.com/postcss/"
        },
        {
          "type": "github",
          "url": "https://github.com/sponsors/ai"
        }
      ],
      "license": "MIT",
      "dependencies": {
        "postcss-selector-parser": "^6.1.1"
      },
      "engines": {
        "node": ">=12.0"
      },
      "peerDependencies": {
        "postcss": "^8.2.14"
      }
    },
    "node_modules/postcss-selector-parser": {
      "version": "6.1.4",
      "resolved": "https://registry.npmjs.org/postcss-selector-parser/-/postcss-selector-parser-6.1.4.tgz",
      "integrity": "sha512-bIoJLOmjCO1S9XdY/DcnR5hJxvrDir1PbGChrzXG3vw0/FOliy/fA3dmdhQ441kah4gKv+TwckGzex6wNS5
cnQ==",
      "dev": true,
      "license": "MIT",
      "dependencies": {
        "cssesc": "^3.0.0",
        "util-deprecate": "^1.0.2"
      },
      "engines": {
```

```
        "node": ">=4"
      }
    },
    "node_modules/postcss-value-parser": {
      "version": "4.2.0",
      "resolved": "https://registry.npmjs.org/postcss-value-parser/-/postcss-value-parser-4.2.0.tgz",
      "integrity": "sha512-1NNCs6uurfkVbeXG4S8JFT9t19m45ICnif8zWLd5oPSZ50QnwMfK+H3jv408d4jw/7Bttv5axS5IiHoLaVN
HeQ==",
      "dev": true,
      "license": "MIT"
    },
    "node_modules/postgres-array": {
      "version": "2.0.0",
      "resolved": "https://registry.npmjs.org/postgres-array/-/postgres-array-2.0.0.tgz",
      "integrity": "sha512-VpZrUqU5A69eQyW2c5CA1jtLecCsN2U/bD6VilrFDWq5+5UIEVO7nazS3TEcHf1zuPYO/sqGvUvW62g86RX
ZuA==",
      "license": "MIT",
      "engines": {
        "node": ">=4"
      }
    },
    "node_modules/postgres-bytea": {
      "version": "1.0.1",
      "resolved": "https://registry.npmjs.org/postgres-bytea/-/postgres-bytea-1.0.1.tgz",
      "integrity": "sha512-5+5HqXnsZPE65IJZSMkZtURARZelel2oXUEO8rH83VS/hxH5vv1uHquPg5wZs8yMAfdv971IU+kcPUczi7N
VBQ==",
      "license": "MIT",
      "engines": {
        "node": ">=0.10.0"
      }
    },
    "node_modules/postgres-date": {
      "version": "1.0.7",
      "resolved": "https://registry.npmjs.org/postgres-date/-/postgres-date-1.0.7.tgz",
      "integrity": "sha512-suDmjLVQg78nMK2UZ454hAG+OAW+HQPZ6n++TNDUX+L0+uUlLywnoxJKDou51Zm+zTCjrCl0Nq6J9C5hP9v
K/Q==",
      "license": "MIT",
      "engines": {
        "node": ">=0.10.0"
      }
    },
    "node_modules/postgres-interval": {
      "version": "1.2.0",
      "resolved": "https://registry.npmjs.org/postgres-interval/-/postgres-interval-1.2.0.tgz",
      "integrity": "sha512-9ZhXKM/rw350N1ovuWHbGxnGh/SNJ4cnxHiM0rxE4VN41wsg8P8zWn9hv/buK00RP4WvlOyr/RBDiptyxVb
kZQ==",
      "license": "MIT",
      "dependencies": {
        "xtend": "^4.0.0"
      },
      "engines": {
        "node": ">=0.10.0"
      }
    },
```

```
    "node_modules/prebuild-install": {
      "version": "7.1.3",
      "resolved": "https://registry.npmjs.org/prebuild-install/-/prebuild-install-7.1.3.tgz",
      "integrity": "sha512-8Mf2cbV7x1cXPUILADGI3wuhfqWvtiLA1iclTDbFRZkgRQS0NqsPZphna9V+HyTEadheuPmjaJMsbzKQFOz
Lug==",
      "deprecated": "No longer maintained. Please contact the author of the relevant native addon; alternative
s are available.",
      "license": "MIT",
      "dependencies": {
        "detect-libc": "^2.0.0",
        "expand-template": "^2.0.3",
        "github-from-package": "0.0.0",
        "minimist": "^1.2.3",
        "mkdirp-classic": "^0.5.3",
        "napi-build-utils": "^2.0.0",
        "node-abi": "^3.3.0",
        "pump": "^3.0.0",
        "rc": "^1.2.7",
        "simple-get": "^4.0.0",
        "tar-fs": "^2.0.0",
        "tunnel-agent": "^0.6.0"
      },
      "bin": {
        "prebuild-install": "bin.js"
      },
      "engines": {
        "node": ">=10"
      }
    },
    "node_modules/proxy-addr": {
      "version": "2.0.7",
      "resolved": "https://registry.npmjs.org/proxy-addr/-/proxy-addr-2.0.7.tgz",
      "integrity": "sha512-llQsMLSUDUPT44jdrU/O37qlnifitDP+ZwrmmZcoSKyLKvtZxpyV0n2/bD/N4tBAAZ/gJEdZU7KMraoK1+X
YAg==",
      "license": "MIT",
      "dependencies": {
        "forwarded": "0.2.0",
        "ipaddr.js": "1.9.1"
      },
      "engines": {
        "node": ">= 0.10"
      }
    },
    "node_modules/pump": {
      "version": "3.0.3",
      "resolved": "https://registry.npmjs.org/pump/-/pump-3.0.3.tgz",
      "integrity": "sha512-todwxLMY7/heScKmntwQG8CXVkWUOdYxIvY2s0VWAAMh/nd8SoYiRaKjlr7+iCs984f2P8zvrfWcDDYVb73
NfA==",
      "license": "MIT",
      "dependencies": {
        "end-of-stream": "^1.1.0",
        "once": "^1.3.1"
      }
    },
```

```
    "node_modules/qs": {
      "version": "6.15.3",
      "resolved": "https://registry.npmjs.org/qs/-/qs-6.15.3.tgz",
      "integrity": "sha512-O9gl3zCl5h5blw1KGUzQKhA5oUXSl8rwUIM5o0S3nCXMliSvy5Dzx7/DJcI+SwgICv+IneSZwhBh1oSyEHA
71A==",
      "license": "BSD-3-Clause",
      "dependencies": {
        "es-define-property": "^1.0.1",
        "side-channel": "^1.1.1"
      },
      "engines": {
        "node": ">=0.6"
      },
      "funding": {
        "url": "https://github.com/sponsors/ljharb"
      }
    },
    "node_modules/queue-microtask": {
      "version": "1.2.3",
      "resolved": "https://registry.npmjs.org/queue-microtask/-/queue-microtask-1.2.3.tgz",
      "integrity": "sha512-NuaNSa6flKT5JaSYQzJok04JzTL1CA6aGhv5rfLW3PgqA+M2ChpZQnAC8h8i4ZFkBS8X5RqkDBHA7r4hej3
K9A==",
      "dev": true,
      "funding": [
        {
          "type": "github",
          "url": "https://github.com/sponsors/feross"
        },
        {
          "type": "patreon",
          "url": "https://www.patreon.com/feross"
        },
        {
          "type": "consulting",
          "url": "https://feross.org/support"
        }
      ],
      "license": "MIT"
    },
    "node_modules/random-bytes": {
      "version": "1.0.0",
      "resolved": "https://registry.npmjs.org/random-bytes/-/random-bytes-1.0.0.tgz",
      "integrity": "sha512-iv7LhNVO047HzYR3InF6pUcUsPQiHTM1Qal51DcGSuZFBil1aBBWG5eHPNek7bvILMaYJ/8RU1e8w1AMdHm
LQQ==",
      "license": "MIT",
      "engines": {
        "node": ">= 0.8"
      }
    },
    "node_modules/range-parser": {
      "version": "1.2.1",
      "resolved": "https://registry.npmjs.org/range-parser/-/range-parser-1.2.1.tgz",
      "integrity": "sha512-Hrgsx+orqoygnmhFbKaHE6c296J+HTAQXoxEF6gNupROmmGJRoyzfG3ccAveqCBrwr/2yxQ5BVd/GTl5agO
wSg==",
```

```
      "license": "MIT",
      "engines": {
        "node": ">= 0.6"
      }
    },
    "node_modules/raw-body": {
      "version": "2.5.3",
      "resolved": "https://registry.npmjs.org/raw-body/-/raw-body-2.5.3.tgz",
      "integrity": "sha512-s4VSOf6yN0rvbRZGxs8Om5CWj6seneMwK3oDb4lWDH0UPhWcxwOWw5+qk24bxq87szX1ydrwylIOp2uG1oj
UpA==",
      "license": "MIT",
      "dependencies": {
        "bytes": "~3.1.2",
        "http-errors": "~2.0.1",
        "iconv-lite": "~0.4.24",
        "unpipe": "~1.0.0"
      },
      "engines": {
        "node": ">= 0.8"
      }
    },
    "node_modules/rc": {
      "version": "1.2.8",
      "resolved": "https://registry.npmjs.org/rc/-/rc-1.2.8.tgz",
      "integrity": "sha512-y3bGgqKj3QBdxLbLkomlohkvsA8gdAiUQlSBJnBhfn+BPxg4bc62d8TcBW15wavDfgexCgccckhcZvywyQY
POw==",
      "license": "(BSD-2-Clause OR MIT OR Apache-2.0)",
      "dependencies": {
        "deep-extend": "^0.6.0",
        "ini": "~1.3.0",
        "minimist": "^1.2.0",
        "strip-json-comments": "~2.0.1"
      },
      "bin": {
        "rc": "cli.js"
      }
    },
    "node_modules/read-cache": {
      "version": "1.0.0",
      "resolved": "https://registry.npmjs.org/read-cache/-/read-cache-1.0.0.tgz",
      "integrity": "sha512-Owdv/Ft7IjOgm/i0xvNDZ1LrRANRfew4b2prF3OWMQLxLfu3bS8FVhCsrSCMK4lR56Y9ya+AThoTpDCTxCm
pRA==",
      "dev": true,
      "license": "MIT",
      "dependencies": {
        "pify": "^2.3.0"
      }
    },
    "node_modules/readable-stream": {
      "version": "3.6.2",
      "resolved": "https://registry.npmjs.org/readable-stream/-/readable-stream-3.6.2.tgz",
      "integrity": "sha512-9u/sniCrY3D5WdsERHzHE4G2YCXqoG5FTHUiCC4SIbr6XcLZBY05ya9EKjYek9O5xOAwjGq+1JdGBAS7Q9S
coA==",
      "license": "MIT",
```

```
      "dependencies": {
        "inherits": "^2.0.3",
        "string_decoder": "^1.1.1",
        "util-deprecate": "^1.0.1"
      },
      "engines": {
        "node": ">= 6"
      }
    },
    "node_modules/readdirp": {
      "version": "3.6.0",
      "resolved": "https://registry.npmjs.org/readdirp/-/readdirp-3.6.0.tgz",
      "integrity": "sha512-hOS089on8RduqdbhvQ5Z37A0ESjsqz6qnRcffsMU3495FuTdqSm+7bhJ29JvIOsBDEEnan5DPu9t3To9VRl
MzA==",
      "dev": true,
      "license": "MIT",
      "dependencies": {
        "picomatch": "^2.2.1"
      },
      "engines": {
        "node": ">=8.10.0"
      }
    },
    "node_modules/redis": {
      "version": "6.0.0",
      "resolved": "https://registry.npmjs.org/redis/-/redis-6.0.0.tgz",
      "integrity": "sha512-n9Thfc39OXleEoPT2k5gwKsqY+HfCww3YS71ofcr9KKbkn89bpjU9dToIlD+JRdM3/GYQkwMtVgTxLyed+L
ptQ==",
      "license": "MIT",
      "dependencies": {
        "@redis/bloom": "6.0.0",
        "@redis/client": "6.0.0",
        "@redis/json": "6.0.0",
        "@redis/search": "6.0.0",
        "@redis/time-series": "6.0.0"
      },
      "engines": {
        "node": ">= 20.0.0"
      }
    },
    "node_modules/resolve": {
      "version": "1.22.12",
      "resolved": "https://registry.npmjs.org/resolve/-/resolve-1.22.12.tgz",
      "integrity": "sha512-TyeJ1zif53BPfHootBGwPRYT1RUt6oGWsaQr8UyZW/eAm9bKoijtvruSDEmZHm92CwS9nj7/fWttqPCgzep
8CA==",
      "dev": true,
      "license": "MIT",
      "dependencies": {
        "es-errors": "^1.3.0",
        "is-core-module": "^2.16.1",
        "path-parse": "^1.0.7",
        "supports-preserve-symlinks-flag": "^1.0.0"
      },
      "bin": {
```

```
        "resolve": "bin/resolve"
      },
      "engines": {
        "node": ">= 0.4"
      },
      "funding": {
        "url": "https://github.com/sponsors/ljharb"
      }
    },
    "node_modules/reusify": {
      "version": "1.1.0",
      "resolved": "https://registry.npmjs.org/reusify/-/reusify-1.1.0.tgz",
      "integrity": "sha512-g6QUff04oZpHs0eG5p83rFLhHeV00ug/Yf9nZM6fLeUrPguBTkTQOdpAWWspMh55TZfVQDPaN3NQJfbVRAxdIw==",
      "dev": true,
      "license": "MIT",
      "engines": {
        "iojs": ">=1.0.0",
        "node": ">=0.10.0"
      }
    },
    "node_modules/run-parallel": {
      "version": "1.2.0",
      "resolved": "https://registry.npmjs.org/run-parallel/-/run-parallel-1.2.0.tgz",
      "integrity": "sha512-5l4VyZR86LZ/lDxZTR6jqL8AFE2S0IFLMP26AbjsLVADxHdhB/c0GUsH+y39UfCi3dzz8OlQuPmnaJOMoDHQBA==",
      "dev": true,
      "funding": [
        {
          "type": "github",
          "url": "https://github.com/sponsors/feross"
        },
        {
          "type": "patreon",
          "url": "https://www.patreon.com/feross"
        },
        {
          "type": "consulting",
          "url": "https://feross.org/support"
        }
      ],
      "license": "MIT",
      "dependencies": {
        "queue-microtask": "^1.2.2"
      }
    },
    "node_modules/safe-buffer": {
      "version": "5.2.1",
      "resolved": "https://registry.npmjs.org/safe-buffer/-/safe-buffer-5.2.1.tgz",
      "integrity": "sha512-rp3So07KcdmmKbGvgaNxQSJr7bGVSVk5S9Eq1F+ppbRo70+YeaDxkw5Dd8NPN+GD6bjnYm2VuPuCXmpuYvmCXQ==",
      "funding": [
        {
          "type": "github",
```

```
        "url": "https://github.com/sponsors/feross"
      },
      {
        "type": "patreon",
        "url": "https://www.patreon.com/feross"
      },
      {
        "type": "consulting",
        "url": "https://feross.org/support"
      }
    ],
    "license": "MIT"
  },
  "node_modules/safer-buffer": {
    "version": "2.1.2",
    "resolved": "https://registry.npmjs.org/safer-buffer/-/safer-buffer-2.1.2.tgz",
    "integrity": "sha512-YZo3K82SD7Riyi0E1EQPojLz7kpepnSQI9IyPbHHg1XXXevb5dJI7tpyN2ADxGcQbHG7vcyRHk0cbwqcQri
Utg==",
    "license": "MIT"
  },
  "node_modules/semver": {
    "version": "7.7.4",
    "resolved": "https://registry.npmjs.org/semver/-/semver-7.7.4.tgz",
    "integrity": "sha512-vFKC2IEtQnVhpT78h1Yp8wzwrf8CM+MzKMHGJZfBtzhZNycRFnXsHk6E5TxIkkMsgNS7mdX3AGB7x2QM2di
4lA==",
    "license": "ISC",
    "bin": {
      "semver": "bin/semver.js"
    },
    "engines": {
      "node": ">=10"
    }
  },
  "node_modules/send": {
    "version": "0.19.2",
    "resolved": "https://registry.npmjs.org/send/-/send-0.19.2.tgz",
    "integrity": "sha512-VMbMxbDeehAxpOtWJXlcUS5E8iXh6QmN+BkRX1GARS3wRaXEEgzCcB10gTQazO42tpNIya8xIyNx8fll1OF
Prg==",
    "license": "MIT",
    "dependencies": {
      "debug": "2.6.9",
      "depd": "2.0.0",
      "destroy": "1.2.0",
      "encodeurl": "~2.0.0",
      "escape-html": "~1.0.3",
      "etag": "~1.8.1",
      "fresh": "~0.5.2",
      "http-errors": "~2.0.1",
      "mime": "1.6.0",
      "ms": "2.1.3",
      "on-finished": "~2.4.1",
      "range-parser": "~1.2.1",
      "statuses": "~2.0.2"
    },
```

```
      "engines": {
        "node": ">= 0.8.0"
      }
    },
    "node_modules/send/node_modules/ms": {
      "version": "2.1.3",
      "resolved": "https://registry.npmjs.org/ms/-/ms-2.1.3.tgz",
      "integrity": "sha512-6FlzubTLZG3J2a/NVCAleEhjzq5oxgHyaCU9yYXvcLsvoVaHJq/s5xXI6/XXP6tz7R9xAOtHnSO/tXtF3WR
TlA==",
      "license": "MIT"
    },
    "node_modules/serve-static": {
      "version": "1.16.3",
      "resolved": "https://registry.npmjs.org/serve-static/-/serve-static-1.16.3.tgz",
      "integrity": "sha512-x0RTqQel6g5SY7Lg6ZreMmsOzncHFU7nhnRWkKgWuMTu5NN0DR5oruckMqRvacAN9d5w6ARnRBXl9xhDCgf
MeA==",
      "license": "MIT",
      "dependencies": {
        "encodeurl": "~2.0.0",
        "escape-html": "~1.0.3",
        "parseurl": "~1.3.3",
        "send": "~0.19.1"
      },
      "engines": {
        "node": ">= 0.8.0"
      }
    },
    "node_modules/setprototypeof": {
      "version": "1.2.0",
      "resolved": "https://registry.npmjs.org/setprototypeof/-/setprototypeof-1.2.0.tgz",
      "integrity": "sha512-E5LDX7Wrp85Kil5bhZv46j8jOeboKq5JMmYM3gVGdGH8xFpPWXUMsNrlODCrkoxMEeNi/XZIwuRvY4XNwYM
Jpw==",
      "license": "ISC"
    },
    "node_modules/side-channel": {
      "version": "1.1.1",
      "resolved": "https://registry.npmjs.org/side-channel/-/side-channel-1.1.1.tgz",
      "integrity": "sha512-6x6dK6zJdpTzF4sQeNYxwtvBzf6Eg4GtlesS94HOvTudUeyK2WXAaIfmDgsyslYrRBeFIlsi54AYsFGUuhm
vrQ==",
      "license": "MIT",
      "dependencies": {
        "es-errors": "^1.3.0",
        "object-inspect": "^1.13.4",
        "side-channel-list": "^1.0.1",
        "side-channel-map": "^1.0.1",
        "side-channel-weakmap": "^1.0.2"
      },
      "engines": {
        "node": ">= 0.4"
      },
      "funding": {
        "url": "https://github.com/sponsors/ljharb"
      }
    },
```

```
    "node_modules/side-channel-list": {
      "version": "1.0.1",
      "resolved": "https://registry.npmjs.org/side-channel-list/-/side-channel-list-1.0.1.tgz",
      "integrity": "sha512-mjn/0bi/oUURjc5Xl7IaWi/OJJJumuoJFQJfDDyO46+hBWsfaVM65TBHq2eoZBhzl9EchxOijpkbRC8SVBQ
U0w==",
      "license": "MIT",
      "dependencies": {
        "es-errors": "^1.3.0",
        "object-inspect": "^1.13.4"
      },
      "engines": {
        "node": ">= 0.4"
      },
      "funding": {
        "url": "https://github.com/sponsors/ljharb"
      }
    },
    "node_modules/side-channel-map": {
      "version": "1.0.1",
      "resolved": "https://registry.npmjs.org/side-channel-map/-/side-channel-map-1.0.1.tgz",
      "integrity": "sha512-VCjCNfgMsby3tTdo02nbjtM/ewra6jPHmpThenkTYh8pG9ucZ/1P8So4u4FGBek/BjpOVsDCMoLA/iuBKIF
XRA==",
      "license": "MIT",
      "dependencies": {
        "call-bound": "^1.0.2",
        "es-errors": "^1.3.0",
        "get-intrinsic": "^1.2.5",
        "object-inspect": "^1.13.3"
      },
      "engines": {
        "node": ">= 0.4"
      },
      "funding": {
        "url": "https://github.com/sponsors/ljharb"
      }
    },
    "node_modules/side-channel-weakmap": {
      "version": "1.0.2",
      "resolved": "https://registry.npmjs.org/side-channel-weakmap/-/side-channel-weakmap-1.0.2.tgz",
      "integrity": "sha512-WPS/HvHQTYnHisLo9McqBHOJk2FkHO/tlpvldyrnem4aeQp4hai3gythswg6p01oSoTl58rcpiFAjF2br2A
k2A==",
      "license": "MIT",
      "dependencies": {
        "call-bound": "^1.0.2",
        "es-errors": "^1.3.0",
        "get-intrinsic": "^1.2.5",
        "object-inspect": "^1.13.3",
        "side-channel-map": "^1.0.1"
      },
      "engines": {
        "node": ">= 0.4"
      },
      "funding": {
        "url": "https://github.com/sponsors/ljharb"
```

```
      }
    },
    "node_modules/simple-concat": {
      "version": "1.0.1",
      "resolved": "https://registry.npmjs.org/simple-concat/-/simple-concat-1.0.1.tgz",
      "integrity": "sha512-cSFtAPtRhljv69IK0hTVZQ+OfE9nePi/rtJmw5UjHeVyVroEqJXP1sFztKUy1qU+xvz3u/sfYJLa947b7nA
N2Q==",
      "funding": [
        {
          "type": "github",
          "url": "https://github.com/sponsors/feross"
        },
        {
          "type": "patreon",
          "url": "https://www.patreon.com/feross"
        },
        {
          "type": "consulting",
          "url": "https://feross.org/support"
        }
      ],
      "license": "MIT"
    },
    "node_modules/simple-get": {
      "version": "4.0.1",
      "resolved": "https://registry.npmjs.org/simple-get/-/simple-get-4.0.1.tgz",
      "integrity": "sha512-brv7p5WgH0jmQJr1ZDDfKDOSeWWg+OVypG99A/5vYGPqJ6pxiaHLy8nxtFjBA7oMa01ebA9gfh1uMCFqOuX
xvA==",
      "funding": [
        {
          "type": "github",
          "url": "https://github.com/sponsors/feross"
        },
        {
          "type": "patreon",
          "url": "https://www.patreon.com/feross"
        },
        {
          "type": "consulting",
          "url": "https://feross.org/support"
        }
      ],
      "license": "MIT",
      "dependencies": {
        "decompress-response": "^6.0.0",
        "once": "^1.3.1",
        "simple-concat": "^1.0.0"
      }
    },
    "node_modules/source-map-js": {
      "version": "1.2.1",
      "resolved": "https://registry.npmjs.org/source-map-js/-/source-map-js-1.2.1.tgz",
      "integrity": "sha512-UXWMKhLOwVKb728IUtQPXxfYU+usdybtUrK/8uGE8CQMvrhOpwvzDBwj0QhSL7MQc7vIsISBG8VQ8+IDQxp
fQA==",
```

```
      "dev": true,
      "license": "BSD-3-Clause",
      "engines": {
        "node": ">=0.10.0"
      }
    },
    "node_modules/split2": {
      "version": "4.2.0",
      "resolved": "https://registry.npmjs.org/split2/-/split2-4.2.0.tgz",
      "integrity": "sha512-UcjcJOWknrNkF6PLX83qcHM6KHgVKNkV62Y8a5uYDVv9ydGQVwAHMKqHdJje1VTWpljG0WYpCDhrCdAOYH4
TWg==",
      "license": "ISC",
      "engines": {
        "node": ">= 10.x"
      }
    },
    "node_modules/statuses": {
      "version": "2.0.2",
      "resolved": "https://registry.npmjs.org/statuses/-/statuses-2.0.2.tgz",
      "integrity": "sha512-DvEy55V3DB7uknRo+4iOGT5fP1slR8wQohVdknigZPMpMstaKJQWhwiYBACJE3Ul2pTnATihhBYnRhZQHGB
iRw==",
      "license": "MIT",
      "engines": {
        "node": ">= 0.8"
      }
    },
    "node_modules/streamsearch": {
      "version": "1.1.0",
      "resolved": "https://registry.npmjs.org/streamsearch/-/streamsearch-1.1.0.tgz",
      "integrity": "sha512-Mcc5wHehp9aXz1ax6bZUyY5afg9u2rv5cqQI3mRrYkGC8rW2hM02jWuwjtL++LS5qinSyhj2QfLyNsuc+Vs
Exg==",
      "engines": {
        "node": ">=10.0.0"
      }
    },
    "node_modules/string_decoder": {
      "version": "1.3.0",
      "resolved": "https://registry.npmjs.org/string_decoder/-/string_decoder-1.3.0.tgz",
      "integrity": "sha512-hkRX8U1WjJFd8LsDJ2yQ/wWWxaopEsABU1XfkM8A+j0+85JAGppt16cr1Whg6KIbb4okU6Mql6BOj+uup/w
KeA==",
      "license": "MIT",
      "dependencies": {
        "safe-buffer": "~5.2.0"
      }
    },
    "node_modules/strip-json-comments": {
      "version": "2.0.1",
      "resolved": "https://registry.npmjs.org/strip-json-comments/-/strip-json-comments-2.0.1.tgz",
      "integrity": "sha512-4gB8na07fecVVkOI6Rs4e7T6NOTki5EmL7TUduTs6bu3EdnSycntVJ4re8kgZA+wx9IueI2Y11bfbgwtzuE
0KQ==",
      "license": "MIT",
      "engines": {
        "node": ">=0.10.0"
      }
```

```
    },
    "node_modules/sucrase": {
      "version": "3.35.1",
      "resolved": "https://registry.npmjs.org/sucrase/-/sucrase-3.35.1.tgz",
      "integrity": "sha512-DhuTmvZWux4H1UOnWMB3sk0sbaCVOoQZjv8u1rDoTV0HTdGem9hkAZtl4JZy8P2z4Bg0nT+YMeOFyVr4zcG
5Tw==",
      "dev": true,
      "license": "MIT",
      "dependencies": {
        "@jridgewell/gen-mapping": "^0.3.2",
        "commander": "^4.0.0",
        "lines-and-columns": "^1.1.6",
        "mz": "^2.7.0",
        "pirates": "^4.0.1",
        "tinyglobby": "^0.2.11",
        "ts-interface-checker": "^0.1.9"
      },
      "bin": {
        "sucrase": "bin/sucrase",
        "sucrase-node": "bin/sucrase-node"
      },
      "engines": {
        "node": ">=16 || 14 >=14.17"
      }
    },
    "node_modules/supports-preserve-symlinks-flag": {
      "version": "1.0.0",
      "resolved": "https://registry.npmjs.org/supports-preserve-symlinks-flag/-/supports-preserve-symlinks-fla
g-1.0.0.tgz",
      "integrity": "sha512-ot0WnXS9fgdkgIcePe6RHNk1WA8+muPa6cSjeR3V8K27q9BB1rTE3R1p7Hv0z1ZyAc8s6Vvv8DIyWf681MA
t0w==",
      "dev": true,
      "license": "MIT",
      "engines": {
        "node": ">= 0.4"
      },
      "funding": {
        "url": "https://github.com/sponsors/ljharb"
      }
    },
    "node_modules/tailwindcss": {
      "version": "3.4.19",
      "resolved": "https://registry.npmjs.org/tailwindcss/-/tailwindcss-3.4.19.tgz",
      "integrity": "sha512-3ofp+LL8E+pK/JuPLPggVAIaEuhvIz4qNcf3nA1Xn2o/7fb7s/TYpHhwGDv1ZU3PkBluUVaF8PyCHcm48cK
LWQ==",
      "dev": true,
      "license": "MIT",
      "dependencies": {
        "@alloc/quick-lru": "^5.2.0",
        "arg": "^5.0.2",
        "chokidar": "^3.6.0",
        "didyoumean": "^1.2.2",
        "dlv": "^1.1.3",
        "fast-glob": "^3.3.2",
```

```
        "glob-parent": "^6.0.2",
        "is-glob": "^4.0.3",
        "jiti": "^1.21.7",
        "lilconfig": "^3.1.3",
        "micromatch": "^4.0.8",
        "normalize-path": "^3.0.0",
        "object-hash": "^3.0.0",
        "picocolors": "^1.1.1",
        "postcss": "^8.4.47",
        "postcss-import": "^15.1.0",
        "postcss-js": "^4.0.1",
        "postcss-load-config": "^4.0.2 || ^5.0 || ^6.0",
        "postcss-nested": "^6.2.0",
        "postcss-selector-parser": "^6.1.2",
        "resolve": "^1.22.8",
        "sucrase": "^3.35.0"
      },
      "bin": {
        "tailwind": "lib/cli.js",
        "tailwindcss": "lib/cli.js"
      },
      "engines": {
        "node": ">=14.0.0"
      }
    },
    "node_modules/tar-fs": {
      "version": "2.1.4",
      "resolved": "https://registry.npmjs.org/tar-fs/-/tar-fs-2.1.4.tgz",
      "integrity": "sha512-mDAjwmZdh7LTT6pNleZ05Yt65HC3E+NiQzl672vQG38jIrehtJk/J3mNwIg+vShQPcLF/LV7CMnDW6vjj6sfYQ==",
      "license": "MIT",
      "dependencies": {
        "chownr": "^1.1.1",
        "mkdirp-classic": "^0.5.2",
        "pump": "^3.0.0",
        "tar-stream": "^2.1.4"
      }
    },
    "node_modules/tar-stream": {
      "version": "2.2.0",
      "resolved": "https://registry.npmjs.org/tar-stream/-/tar-stream-2.2.0.tgz",
      "integrity": "sha512-ujeqbceABgwMZxEJnk2HDY2DlnUZ+9oEcb1KzTVfYHio0UE6dG71n60d8D2I4qNvleWrrXpmjpt7vZeF1LnMZQ==",
      "license": "MIT",
      "dependencies": {
        "bl": "^4.0.3",
        "end-of-stream": "^1.4.1",
        "fs-constants": "^1.0.0",
        "inherits": "^2.0.3",
        "readable-stream": "^3.1.1"
      },
      "engines": {
        "node": ">=6"
      }
```

```
    },
    "node_modules/thenify": {
      "version": "3.3.1",
      "resolved": "https://registry.npmjs.org/thenify/-/thenify-3.3.1.tgz",
      "integrity": "sha512-RVZSIV5IG10Hk3enotrhvz0T9em6cyHBLkH/YAZuKqd8hRkKhSfCGIcP2KUY0EPxndzANBmNllzWPwak+bh
eSw==",
      "dev": true,
      "license": "MIT",
      "dependencies": {
        "any-promise": "^1.0.0"
      }
    },
    "node_modules/thenify-all": {
      "version": "1.6.0",
      "resolved": "https://registry.npmjs.org/thenify-all/-/thenify-all-1.6.0.tgz",
      "integrity": "sha512-RNxQH/qI8/t3thXJDwcstUO4zeqo64+Uy/+sNVRBx4Xn2OX+OZ9oP+iJnNFqplFra2ZUVeKCSa2oVWi3T4u
VmA==",
      "dev": true,
      "license": "MIT",
      "dependencies": {
        "thenify": ">= 3.1.0 < 4"
      },
      "engines": {
        "node": ">=0.8"
      }
    },
    "node_modules/tinyglobby": {
      "version": "0.2.17",
      "resolved": "https://registry.npmjs.org/tinyglobby/-/tinyglobby-0.2.17.tgz",
      "integrity": "sha512-wXR/dYpcqKmfWpEdZjiKJOwCNFndD0DMnrW/cYjVGttEkBfVgcLFHoNrlj47mjOVic9yyNu65alsgF4NQyT
a2g==",
      "dev": true,
      "license": "MIT",
      "dependencies": {
        "fdir": "^6.5.0",
        "picomatch": "^4.0.4"
      },
      "engines": {
        "node": ">=12.0.0"
      },
      "funding": {
        "url": "https://github.com/sponsors/SuperchupuDev"
      }
    },
    "node_modules/tinyglobby/node_modules/fdir": {
      "version": "6.5.0",
      "resolved": "https://registry.npmjs.org/fdir/-/fdir-6.5.0.tgz",
      "integrity": "sha512-tIbYtZbucOs0BRGqPJkshJUYdL+SDH7dVM8gjy+ERp3WAUjLEFJE+02kanyHtwjWOnwrKYBiwAmM0p4kLJA
nXg==",
      "dev": true,
      "license": "MIT",
      "engines": {
        "node": ">=12.0.0"
      },
```

```
      "peerDependencies": {
        "picomatch": "^3 || ^4"
      },
      "peerDependenciesMeta": {
        "picomatch": {
          "optional": true
        }
      }
    },
    "node_modules/tinyglobby/node_modules/picomatch": {
      "version": "4.0.4",
      "resolved": "https://registry.npmjs.org/picomatch/-/picomatch-4.0.4.tgz",
      "integrity": "sha512-QP88BAKvMam/3NxH6vj2o21R6MjxZUAd6nlwAS/pnGvN9IVLocLHxGYIzFhg6fUQ+5th6P4dv4eW9jX3DSI
j7A==",
      "dev": true,
      "license": "MIT",
      "engines": {
        "node": ">=12"
      },
      "funding": {
        "url": "https://github.com/sponsors/jonschlinkert"
      }
    },
    "node_modules/to-regex-range": {
      "version": "5.0.1",
      "resolved": "https://registry.npmjs.org/to-regex-range/-/to-regex-range-5.0.1.tgz",
      "integrity": "sha512-65P7iz6X5yEr1cwcgvQxbbIw7Uk3gOy5dIdtZ4rDveLqhrdJP+Li/Hx6tyK0NEb+2GCyneCMJiGqrADCSNk
8sQ==",
      "dev": true,
      "license": "MIT",
      "dependencies": {
        "is-number": "^7.0.0"
      },
      "engines": {
        "node": ">=8.0"
      }
    },
    "node_modules/toidentifier": {
      "version": "1.0.1",
      "resolved": "https://registry.npmjs.org/toidentifier/-/toidentifier-1.0.1.tgz",
      "integrity": "sha512-o5sSPKEkg/DIQNmH43V0/uerLrpzVedkUh8tGNvaeXpfpuwjKenlSox/2O/BTlZUtEe+JG7s5YhEz608PlA
HRA==",
      "license": "MIT",
      "engines": {
        "node": ">=0.6"
      }
    },
    "node_modules/ts-interface-checker": {
      "version": "0.1.13",
      "resolved": "https://registry.npmjs.org/ts-interface-checker/-/ts-interface-checker-0.1.13.tgz",
      "integrity": "sha512-Y/arvbn+rrz3JCKl9C4kVNfTfSm2/mEp5FSz5EsZSANGPSlQrpRI5M4PKF+mJnE52jOO90PnPSc3Ur3bTQw
0gA==",
      "dev": true,
      "license": "Apache-2.0"
```

```
    },
    "node_modules/tunnel-agent": {
      "version": "0.6.0",
      "resolved": "https://registry.npmjs.org/tunnel-agent/-/tunnel-agent-0.6.0.tgz",
      "integrity": "sha512-McnNiV1l8RYeY8tBgEpuodCC1mLUdbSN+CYBL7kJsJNInOP8UjDDEwdk6Mw60vdLLrr5NHKZhMAOSrR2NZu
Q+w==",
      "license": "Apache-2.0",
      "dependencies": {
        "safe-buffer": "^5.0.1"
      },
      "engines": {
        "node": "*"
      }
    },
    "node_modules/type-is": {
      "version": "1.6.18",
      "resolved": "https://registry.npmjs.org/type-is/-/type-is-1.6.18.tgz",
      "integrity": "sha512-TkRKr9sUTxEH8MdfuCSP7VizJyzRNMjj2J2do2Jr3Kym598JVdEksuzPQCnlFPW4ky9Q+iA+ma9BGm06XQB
y8g==",
      "license": "MIT",
      "dependencies": {
        "media-typer": "0.3.0",
        "mime-types": "~2.1.24"
      },
      "engines": {
        "node": ">= 0.6"
      }
    },
    "node_modules/typedarray": {
      "version": "0.0.6",
      "resolved": "https://registry.npmjs.org/typedarray/-/typedarray-0.0.6.tgz",
      "integrity": "sha512-/aCDEGatGvZ2BIk+HmLf4ifCJFwvKFNb9/JeZPMulfgFracn9QFcAf5GO8B/mweUjSoblS5In0cWhqpfs/5
PQA==",
      "license": "MIT"
    },
    "node_modules/uid-safe": {
      "version": "2.1.5",
      "resolved": "https://registry.npmjs.org/uid-safe/-/uid-safe-2.1.5.tgz",
      "integrity": "sha512-KPHm4VL5dDXKz01UuEd88Df+KzynaohSL9fBh096KWAxSKZQDI2uBrVqtvRM4rwrIrRRKsdLNML/lnaaVSR
ioA==",
      "license": "MIT",
      "dependencies": {
        "random-bytes": "~1.0.0"
      },
      "engines": {
        "node": ">= 0.8"
      }
    },
    "node_modules/unpipe": {
      "version": "1.0.0",
      "resolved": "https://registry.npmjs.org/unpipe/-/unpipe-1.0.0.tgz",
      "integrity": "sha512-pjy2bYhSsufwWlKwPc+l3cN7+wuJlK6uz0YdJEOlQDbl6jo/YlPi4mb8agUkVC8BF7V8NuzeyPNqRksA3hz
tKQ==",
      "license": "MIT",
```

```
      "engines": {
        "node": ">= 0.8"
      }
    },
    "node_modules/util-deprecate": {
      "version": "1.0.2",
      "resolved": "https://registry.npmjs.org/util-deprecate/-/util-deprecate-1.0.2.tgz",
      "integrity": "sha512-EPD5q1uXyFxJpCrLnCc1nHnq3gOa6DZBocAIiI2TaSCA7VCJ1UJDMagCzIkXNsUYfD1daK//LTEQ8xiIbrH
tcw==",
      "license": "MIT"
    },
    "node_modules/utils-merge": {
      "version": "1.0.1",
      "resolved": "https://registry.npmjs.org/utils-merge/-/utils-merge-1.0.1.tgz",
      "integrity": "sha512-pMZTvIkT1d+TFGvDOqodOclx0QWkkgi6Tdoa8gC8ffGAAqz9pzPTZWAybbsHHoED/ztMtkv/VoYTYyShUn8
1hA==",
      "license": "MIT",
      "engines": {
        "node": ">= 0.4.0"
      }
    },
    "node_modules/vary": {
      "version": "1.1.2",
      "resolved": "https://registry.npmjs.org/vary/-/vary-1.1.2.tgz",
      "integrity": "sha512-BNGbWLfd0eUPabhkXUVm0j8uuvREyTh5ovRa/dyow/BqAbZJyC+5fU+IzQOzmAKzYqYRAISoRhdQr3eIZ/P
Xqg==",
      "license": "MIT",
      "engines": {
        "node": ">= 0.8"
      }
    },
    "node_modules/wrappy": {
      "version": "1.0.2",
      "resolved": "https://registry.npmjs.org/wrappy/-/wrappy-1.0.2.tgz",
      "integrity": "sha512-l4Sp/DRseor9wL6EvV2+TuQn63dMkPjZ/sp9XkghTEbV9KlPS1xUsZ3u7/IQO4wxtcFB4bgpQPRcR3QCvez
PcQ==",
      "license": "ISC"
    },
    "node_modules/xtend": {
      "version": "4.0.2",
      "resolved": "https://registry.npmjs.org/xtend/-/xtend-4.0.2.tgz",
      "integrity": "sha512-LKYU1iAXJXUgAXn9URjiu+MWhyUXHsvfp7mcuYm9dSUKK0/CjtrUwFAxD82/mCWbtLsGjFIad0wIsod4zrT
AEQ==",
      "license": "MIT",
      "engines": {
        "node": ">=0.4"
      }
    }
  }
}
```


==========

FILE: package.json
==

```
{
  "name": "nama-medical-web",
  "version": "1.0.0",
  "description": "Nama Medical ERP - Web Application",
  "main": "server.js",
  "scripts": {
    "start": "node server.js",
    "dev": "node server.js",
    "test": "node run_all_tests.js",
    "test:safe": "node run_safe_tests.js",
    "setup": "setup.bat",
    "build:css": "tailwindcss -i ./tailwind-input.css -o ./public/css/tailwind-compiled.css --minify",
    "postinstall": "node -e \"const fs=require('fs');if(!fs.existsSync('.env')&&fs.existsSync('.env.example'))
{fs.copyFileSync('.env.example','.env');console.log('? Created .env from .env.example')}\" && npm run build:cs
s"
  },
  "dependencies": {
    "bcrypt": "^6.0.0",
    "bcryptjs": "^3.0.3",
    "better-sqlite3": "^11.0.0",
    "compression": "^1.8.1",
    "connect-redis": "^9.0.0",
    "cors": "^2.8.5",
    "dotenv": "^17.3.1",
    "express": "^4.21.0",
    "express-rate-limit": "^8.2.1",
    "express-session": "^1.18.0",
    "helmet": "^8.1.0",
    "multer": "^2.0.2",
    "pg": "^8.18.0",
    "redis": "^6.0.0"
  },
  "devDependencies": {
    "@tailwindcss/container-queries": "^0.1.1",
    "@tailwindcss/forms": "^0.5.7",
    "tailwindcss": "^3.4.1"
  }
}
```

==
FILE: password_lockout_test.js
==

```
/**
 * password_lockout_test.js
 * Integration test for the 5-failed-logins account lockout policy.
 */

const { spawn } = require('child_process');
```

```
const http = require('http');
const bcrypt = require('bcryptjs');
const path = require('path');
const assert = require('assert');
const { pool } = require('./db_postgres');

const TEST_PORT = 3005;
const TEST_USERNAME = 'lockout_test_user_99';
const TEST_PASSWORD = 'EXAMPLE_PASSWORD';

function makeRequest(method, path, body = null) {
    return new Promise((resolve, reject) => {
        const req = http.request({
            hostname: 'localhost',
            port: TEST_PORT,
            path: path,
            method: method,
            headers: {
                'Content-Type': 'application/json'
            }
        }, (res) => {
            let data = '';
            res.on('data', chunk => data += chunk);
            res.on('end', () => {
                try {
                    resolve({
                        statusCode: res.statusCode,
                        body: JSON.parse(data)
                    });
                } catch (e) {
                    resolve({
                        statusCode: res.statusCode,
                        body: data
                    });
                }
            });
        });
        req.on('error', reject);
        if (body) {
            req.write(JSON.stringify(body));
        }
        req.end();
    });
}

async function runTest() {
    console.log('=== Running Account Lockout Integration Test ===');

    // 1. Clean up and insert test user
    const hashedPassword = await bcrypt.hash(TEST_PASSWORD, 10);
    await pool.query('DELETE FROM system_users WHERE username = $1', [TEST_USERNAME]);
    await pool.query(
        'INSERT INTO system_users (username, password_hash, display_name, role, speciality, permissions, is_ac
tive) VALUES ($1, $2, $3, $4, $5, $6, 1)',
```

```
        [TEST_USERNAME, hashedPassword, 'Lockout Test User', 'Doctor', 'General', '[]']
    );

    // 2. Start the server on TEST_PORT
    const server = spawn('node', [path.join(__dirname, 'server.js')], {
        env: { ...process.env, PORT: TEST_PORT, NODE_ENV: 'test', SKIP_DB_INIT: 'true' }
    });

    server.stdout.on('data', (data) => console.log(`[Server] ${data.toString().trim()}`));
    server.stderr.on('data', (data) => console.error(`[Server Error] ${data.toString().trim()}`));

    // Wait for server to start
    await new Promise(resolve => setTimeout(resolve, 2500));

    try {
        // 3. Perform 4 failed logins
        for (let i = 1; i <= 4; i++) {
            const res = await makeRequest('POST', '/api/auth/login', {
                username: TEST_USERNAME,
                password: 'WrongPassword!'
            });
            assert.strictEqual(res.statusCode, 401, `Attempt ${i} should return 401`);
            assert.ok(res.body.error.includes('???????'), 'Should warning about remaining attempts');
            assert.ok(res.body.error.includes(String(5 - i)), `Should state ${5 - i} attempts remaining`);
            console.log(`? Failed attempt ${i} verified: ${res.body.error}`);
        }

        // 4. Perform 5th failed login - should lock the account
        const res5 = await makeRequest('POST', '/api/auth/login', {
            username: TEST_USERNAME,
            password: 'WrongPassword!'
        });
        assert.strictEqual(res5.statusCode, 403, '5th attempt should return 403 Forbidden');
        assert.ok(res5.body.error.includes('?? ??? ?????? ??????'), 'Should return lockout message');
        console.log(`? 5th failed attempt verified: ${res5.body.error}`);

        // 5. Attempt login with CORRECT password - should still be locked
        const res6 = await makeRequest('POST', '/api/auth/login', {
            username: TEST_USERNAME,
            password: TEST_PASSWORD
        });
        assert.strictEqual(res6.statusCode, 403, 'Login with correct password during lockout should return 403
');
        assert.ok(res6.body.error.includes('?? ??? ?????? ??????'), 'Should return lockout message');
        console.log('? Login during lockout verified as blocked');

        // 6. Simulate lockout expiration by updating DB
        await pool.query(
            'UPDATE system_users SET lockout_until = NOW() - INTERVAL \'1 second\' WHERE username = $1',
            [TEST_USERNAME]
        );
        console.log('? Simulated lockout expiration in database');

        // 7. Login with correct password after expiration - should succeed
```

```
        const res7 = await makeRequest('POST', '/api/auth/login', {
            username: TEST_USERNAME,
            password: TEST_PASSWORD
        });
        assert.strictEqual(res7.statusCode, 200, 'Login after lockout expires should succeed');
        assert.strictEqual(res7.body.success, true, 'Response should indicate success');
        console.log('? Successful login after lockout expiration verified');

        // Verify database counters were reset
        const dbUser = (await pool.query('SELECT failed_login_attempts, lockout_until FROM system_users WHERE
username = $1', [TEST_USERNAME])).rows[0];
        assert.strictEqual(dbUser.failed_login_attempts, 0, 'Failed attempts should be reset to 0');
        assert.strictEqual(dbUser.lockout_until, null, 'Lockout until should be reset to null');
        console.log('? Database counters reset verified');

        console.log('? Account Lockout Integration Test passed successfully!');
    } catch (e) {
        console.error('? Test failed:', e);
        process.exitCode = 1;
    } finally {
        // Clean up
        server.kill();
        await pool.query('DELETE FROM system_users WHERE username = $1', [TEST_USERNAME]);
        await pool.end();
        console.log('? Cleanup complete');
    }
}

runTest();



========================================
FILE: password_policy.js
========================================

/**
 * password_policy.js
 * Centralized server-side password strength validation.
 */
'use strict';

function validatePasswordPolicy(password, context = {}) {
    if (!password || typeof password !== 'string') {
        return { valid: false, error: 'Password is required', error_ar: '???? ?????? ??????' };
    }
    if (password.length < 12) {
        return { valid: false, error: 'Password must be at least 12 characters', error_ar: '??? ?? ????? ????
?????? ?? 12 ????? ??? ?????' };
    }
    const common = ['password', '123456', 'admin', 'welcome', 'changeme', 'nama123', 'nama_medical'];
    const lower = password.toLowerCase();
    if (common.some(c => lower.includes(c))) {
        return { valid: false, error: 'Password is too weak or common', error_ar: '???? ?????? ????? ???? ?? ?
```

```
????' };
    }
    const { username, email, phone } = context;
    if (username && lower.includes(username.toLowerCase())) {
        return { valid: false, error: 'Password cannot contain username', error_ar: '?? ???? ?? ????? ???? ???
??? ??? ??? ??? ????????' };
    }
    if (email && email.includes('@')) {
        const parts = email.split('@')[0];
        if (parts.length >= 4 && lower.includes(parts.toLowerCase())) {
            return { valid: false, error: 'Password cannot contain email parts', error_ar: '?? ???? ?? ????? ?
??? ?????? ??? ????? ?? ?????? ??????????' };
        }
    }
    if (phone && phone.length >= 6 && lower.includes(phone)) {
        return { valid: false, error: 'Password cannot contain phone number', error_ar: '?? ???? ?? ????? ????
 ?????? ??? ??? ??????' };
    }
    return { valid: true };
}

module.exports = { validatePasswordPolicy };



========================================
FILE: password_policy_test.js
========================================

/**
 * password_policy_test.js
 * Unit tests for server-side password validation policy.
 */
const { validatePasswordPolicy } = require('./password_policy');
const assert = require('assert');

console.log('=== Running Password Policy Unit Tests ===');

// 1. Weak/short passwords
const r1 = validatePasswordPolicy('weak');
assert.strictEqual(r1.valid, false, 'Should reject short password');
assert.ok(r1.error_ar.includes('12'), 'Error should mention 12 characters');

// 2. Common passwords
const r2 = validatePasswordPolicy('welcome12345678');
assert.strictEqual(r2.valid, false, 'Should reject common password containing welcome');
assert.ok(r2.error_ar.includes('?????'), 'Error should mention common');

// 3. Username matching
const r3 = validatePasswordPolicy('ahmed_secure_passphrase_here', { username: 'ahmed' });
assert.strictEqual(r3.valid, false, 'Should reject password containing username');
assert.ok(r3.error_ar.includes('??? ????????'), 'Error should mention username');

// 4. Email matching
```

```
const r4 = validatePasswordPolicy('john_secret_secure', { email: 'john@company.com' });
assert.strictEqual(r4.valid, false, 'Should reject password containing email prefix');
assert.ok(r4.error_ar.includes('?????? ??????????'), 'Error should mention email');

// 5. Phone matching
const r5 = validatePasswordPolicy('0598765432_secure_passphrase', { phone: '0598765432' });
assert.strictEqual(r5.valid, false, 'Should reject password containing phone number');
assert.ok(r5.error_ar.includes('??????'), 'Error should mention phone');

// 6. Good passphrase
const r6 = validatePasswordPolicy('SuperSecurePassphrase2026!');
assert.strictEqual(r6.valid, true, 'Should accept strong passphrase');

console.log('? All password policy unit tests passed successfully!');
process.exit(0);
```

```
========================================
FILE: pathology_engine.js
========================================

/**
 * pathology_engine.js ? E15 Pathology pure engine (DB-free, deterministic).
 *
 * Server-authoritative state machine + helpers for the specimen workflow.
 * No DB, no Express, no client trust ? imported by server.js routes AND tests.
 *
 * State machine (linear, forward-only):
 *   Received -> Grossing -> Processing -> Reported -> SignedOut
 *
 * Rules:
 *  - Transitions may advance one or more steps forward but NEVER backward.
 *  - SignedOut is terminal & immutable (no further transition; addendum-only).
 *  - Unknown states / unknown targets are rejected.
 */
'use strict';

const PATH_STATES = ['Received', 'Grossing', 'Processing', 'Reported', 'SignedOut'];
const TERMINAL_STATE = 'SignedOut';

/**
 * Is `to` a legal next state from `from`?
 * Forward-only along PATH_STATES order; no self-loop; SignedOut is terminal.
 */
function isValidTransition(from, to) {
    const fi = PATH_STATES.indexOf(from);
    const ti = PATH_STATES.indexOf(to);
    if (fi === -1 || ti === -1) return false;     // unknown state -> reject
    if (from === TERMINAL_STATE) return false;     // signed-out is immutable
    return ti > fi;                                // strictly forward only
}

/** A report is immutable (no field edits, addendum-only) once signed out. */
```

```
function isImmutable(state) {
    return state === TERMINAL_STATE;
}

/**
 * Compute critical/malignancy flags server-side (anti-spoof: never trust client).
 * Returns booleans derived from structured fields + free text.
 */
function deriveFlags({ diagnosis = '', micro_text = '', snomed_codes = [], malignancy_hint = false } = {}) {
    const hay = `${diagnosis} ${micro_text}`.toLowerCase();
    const MAL_TERMS = ['malignant', 'malignancy', 'carcinoma', 'sarcoma', 'lymphoma',
        'melanoma', 'metasta', 'invasive', 'adenocarcinoma', 'neoplasm', 'leukemia'];
    const CRIT_TERMS = ['critical', 'urgent finding', 'high grade', 'high-grade'];
    // SNOMED morphologic-malignancy roots (8000-prefix family per SNOMED/ICD-O)
    const snomedMalignant = Array.isArray(snomed_codes) && snomed_codes.some(
        c => /^(M8|M9)\d{3}\/3/i.test(String(c)) || /malig/i.test(String(c))
    );
    const malignancy_flag = !!malignancy_hint || snomedMalignant || MAL_TERMS.some(t => hay.includes(t));
    const critical_flag = malignancy_flag || CRIT_TERMS.some(t => hay.includes(t));
    return { malignancy_flag, critical_flag };
}

/**
 * Generate a tenant-unique accession number. Server-authoritative ? never
 * accepted from the client. Format: PA-<tenantId>-<yyyymmdd>-<seq>
 * `seq` is the count of that tenant's specimens today + 1 (caller supplies count).
 */
function generateAccession(tenantId, todayCount, now = new Date()) {
    const t = parseInt(tenantId, 10);
    if (!Number.isInteger(t) || t <= 0) throw new Error('accession: invalid tenantId');
    const y = now.getFullYear();
    const m = String(now.getMonth() + 1).padStart(2, '0');
    const d = String(now.getDate()).padStart(2, '0');
    const seq = String((parseInt(todayCount, 10) || 0) + 1).padStart(4, '0');
    return `PA-${t}-${y}${m}${d}-${seq}`;
}

module.exports = {
    PATH_STATES,
    TERMINAL_STATE,
    isValidTransition,
    isImmutable,
    deriveFlags,
    generateAccession,
};
```

```
========================================
FILE: pathology_engine_test.js
========================================

/**
 * pathology_engine_test.js ? E15 pure-engine unit test (DB-free).
```

```
 * Validates the state machine, immutability, flag derivation, accession format.
 *   node pathology_engine_test.js
 */
'use strict';
const eng = require('./pathology_engine');

const GREEN = '\x1b[32m', RED = '\x1b[31m', BLUE = '\x1b[34m', BOLD = '\x1b[1m', RESET = '\x1b[0m';
let passed = 0, failed = 0; const failures = [];
function assert(cond, name) {
    if (cond) { console.log(`  ${GREEN}PASS${RESET} ? ${name}`); passed++; }
    else { console.log(`  ${RED}FAIL${RESET} ? ${name}`); failed++; failures.push(name); }
}

console.log(`\n${BOLD}${BLUE}E15 Pathology Engine ? Unit Tests${RESET}\n`);

// ----- State machine forward-only -----
console.log(`${BOLD}[1] State machine${RESET}`);
assert(eng.isValidTransition('Received', 'Grossing'), 'Received -> Grossing allowed');
assert(eng.isValidTransition('Grossing', 'Processing'), 'Grossing -> Processing allowed');
assert(eng.isValidTransition('Processing', 'Reported'), 'Processing -> Reported allowed');
assert(eng.isValidTransition('Reported', 'SignedOut'), 'Reported -> SignedOut allowed');
assert(eng.isValidTransition('Received', 'SignedOut'), 'skip-ahead Received -> SignedOut allowed (forward)');
assert(!eng.isValidTransition('Processing', 'Received'), 'backward Processing -> Received REJECTED');
assert(!eng.isValidTransition('Reported', 'Grossing'), 'backward Reported -> Grossing REJECTED');
assert(!eng.isValidTransition('Received', 'Received'), 'self-loop REJECTED');
assert(!eng.isValidTransition('SignedOut', 'Reported'), 'SignedOut terminal: no transition out');
assert(!eng.isValidTransition('SignedOut', 'SignedOut'), 'SignedOut -> SignedOut REJECTED');
assert(!eng.isValidTransition('Bogus', 'Reported'), 'unknown source state REJECTED');
assert(!eng.isValidTransition('Received', 'Bogus'), 'unknown target state REJECTED');

// ----- Immutability -----
console.log(`\n${BOLD}[2] Immutability${RESET}`);
assert(eng.isImmutable('SignedOut') === true, 'SignedOut is immutable');
assert(eng.isImmutable('Reported') === false, 'Reported is mutable');
assert(eng.isImmutable('Received') === false, 'Received is mutable');

// ----- Flag derivation (server-authoritative, anti-spoof) -----
console.log(`\n${BOLD}[3] Flag derivation${RESET}`);
let f = eng.deriveFlags({ diagnosis: 'Invasive ductal carcinoma' });
assert(f.malignancy_flag === true && f.critical_flag === true, 'carcinoma -> malignant + critical');
f = eng.deriveFlags({ diagnosis: 'Benign fibroadenoma' });
assert(f.malignancy_flag === false && f.critical_flag === false, 'benign -> no flags');
f = eng.deriveFlags({ micro_text: 'High grade dysplasia noted' });
assert(f.critical_flag === true, 'high grade -> critical flag');
// Canonical ICD-O-3 format M8010/3 must match the /^(M8|M9)\d{3}\/3/i regex path.
f = eng.deriveFlags({ snomed_codes: ['M8010/3'] });
assert(f.malignancy_flag === true, 'SNOMED ICD-O-3 canonical M8010/3 -> malignant flag (regex path)');
// Text-based fallback: a code containing "malig" triggers flag independently.
f = eng.deriveFlags({ snomed_codes: ['malignant neoplasm'] });
assert(f.malignancy_flag === true, 'SNOMED /malig/i text fallback -> malignant flag');
f = eng.deriveFlags({ diagnosis: '', micro_text: '', snomed_codes: [] });
assert(f.malignancy_flag === false && f.critical_flag === false, 'empty -> no flags (no false reassurance the
other way)');
```

```
// ----- Accession generation -----
console.log(`\n${BOLD}[4] Accession number${RESET}`);
const acc = eng.generateAccession(7, 4, new Date('2026-06-26T10:00:00Z'));
assert(/^PA-7-20260626-0005$/.test(acc), 'accession format PA-<tenant>-<date>-<seq+1>: ' + acc);
const acc2 = eng.generateAccession(7, 0, new Date('2026-06-26T10:00:00Z'));
assert(/^PA-7-20260626-0001$/.test(acc2), 'first specimen of day -> seq 0001');
let threw = false; try { eng.generateAccession('abc', 0); } catch (e) { threw = true; }
assert(threw, 'invalid tenantId throws (fail-closed, no silent accession)');
threw = false; try { eng.generateAccession(0, 0); } catch (e) { threw = true; }
assert(threw, 'tenantId 0 throws');
// tenant uniqueness: different tenants never collide on same day/seq
assert(eng.generateAccession(1, 0, new Date('2026-06-26')) !== eng.generateAccession(2, 0, new Date('2026-06-2
6')), 'tenant 1 vs 2 accessions differ');

console.log(`\n${BOLD}Results: ${GREEN}${passed} passed${RESET}, ${failed ? RED : ''}${failed} failed${RESET}`
);
if (failed) { failures.forEach(f => console.log('  - ' + f)); process.exit(1); }
process.exit(0);
```

```
========================================
FILE: pathology_workflow_test.js
========================================

/**
 * pathology_workflow_test.js ? E15 business/workflow test (DB-free).
 * Simulates the full lifecycle through handlers that mirror server.js logic:
 *   accession -> add block -> add slide -> advance states -> save report
 *   -> sign-out -> addendum, asserting the state machine + immutability + flags.
 *   node pathology_workflow_test.js
 */
'use strict';
const eng = require('./pathology_engine');

const RED = '\x1b[31m', GREEN = '\x1b[32m', BLUE = '\x1b[34m', BOLD = '\x1b[1m', RESET = '\x1b[0m';
let passed = 0, failed = 0; const failures = [];
function assert(cond, name) {
    if (cond) { console.log(`  ${GREEN}PASS${RESET} ? ${name}`); passed++; }
    else { console.log(`  ${RED}FAIL${RESET} ? ${name}`); failed++; failures.push(name); }
}

console.log(`\n${BOLD}${BLUE}E15 Pathology ? Business / Workflow Lifecycle Test${RESET}\n`);

// ---- In-memory store mirroring the DB chain (single tenant for workflow focus) ----
const TENANT = 1;
const db = { specimens: [], blocks: [], slides: [], reports: [], seqToday: 0, audit: [] };
function audit(action) { db.audit.push(action); }

// ---- Handlers mirroring server.js semantics ----
function accession(body) {
    if (!Number.isInteger(body.patient_id)) return { status: 400 };
    const accession_number = eng.generateAccession(TENANT, db.seqToday++);
    const sp = { id: db.specimens.length + 1, tenant_id: TENANT, patient_id: body.patient_id,
```

```
        accession_number, specimen_type: body.specimen_type || '', state: 'Received', blocks_count: 0 };
    db.specimens.push(sp);
    db.reports.push({ id: 100 + sp.id, specimen_id: sp.id, tenant_id: TENANT, state: 'Received',
        diagnosis: '', malignancy_flag: false, critical_flag: false, addendum_count: 0, addenda: [] });
    audit('PATHOLOGY_SPECIMEN_CREATE');
    return { status: 200, specimen: sp };
}
function addBlock(sid, block_no) {
    const sp = db.specimens.find(s => s.id === sid && s.tenant_id === TENANT);
    if (!sp) return { status: 404 };
    if (sp.state === 'SignedOut') return { status: 409 };
    const b = { id: db.blocks.length + 1, tenant_id: TENANT, specimen_id: sid, block_no };
    db.blocks.push(b); sp.blocks_count++;
    return { status: 200, block: b };
}
function addSlide(blockId, slide_no, stain) {
    const b = db.blocks.find(x => x.id === blockId && x.tenant_id === TENANT);
    if (!b) return { status: 404 };
    const sl = { id: db.slides.length + 1, tenant_id: TENANT, block_id: blockId, specimen_id: b.specimen_id, s
lide_no, stain_type: stain };
    db.slides.push(sl); return { status: 200, slide: sl };
}
function advance(sid, target) {
    const sp = db.specimens.find(s => s.id === sid && s.tenant_id === TENANT);
    if (!sp) return { status: 404 };
    if (!eng.isValidTransition(sp.state, target)) return { status: 409 };
    sp.state = target;
    const rep = db.reports.find(r => r.specimen_id === sid); rep.state = target;
    audit('PATHOLOGY_STATE_CHANGE');
    return { status: 200, state: target };
}
function saveReport(sid, body) {
    const rep = db.reports.find(r => r.specimen_id === sid && r.tenant_id === TENANT);
    if (!rep) return { status: 404 };
    if (eng.isImmutable(rep.state)) return { status: 409 };
    const flags = eng.deriveFlags({ diagnosis: body.diagnosis, micro_text: body.micro_text, snomed_codes: body
.snomed_codes || [] });
    rep.diagnosis = body.diagnosis || ''; rep.micro_text = body.micro_text || '';
    rep.malignancy_flag = flags.malignancy_flag; rep.critical_flag = flags.critical_flag;
    audit('PATHOLOGY_REPORT_SAVE');
    return { status: 200, ...flags };
}
function signOut(sid) {
    const sp = db.specimens.find(s => s.id === sid && s.tenant_id === TENANT);
    if (!sp) return { status: 404 };
    if (sp.state === 'SignedOut') return { status: 409 };
    if (!eng.isValidTransition(sp.state, 'SignedOut')) return { status: 409 };
    const rep = db.reports.find(r => r.specimen_id === sid);
    if (!rep || !rep.diagnosis.trim()) return { status: 409 };
    sp.state = 'SignedOut'; rep.state = 'SignedOut'; rep.signed_at = new Date().toISOString();
    audit('PATHOLOGY_SPECIMEN_SIGNOUT');
    return { status: 200 };
}
function addendum(sid, text) {
```

```
    const rep = db.reports.find(r => r.specimen_id === sid && r.tenant_id === TENANT);
    if (!rep) return { status: 404 };
    if (rep.state !== 'SignedOut') return { status: 409 };
    if (!text || !text.trim()) return { status: 400 };
    rep.addenda.push({ text }); rep.addendum_count++;
    audit('PATHOLOGY_ADDENDUM_ADD');
    return { status: 200, addendum_count: rep.addendum_count };
}

// ===== 1. Accession =====
console.log(`${BOLD}[1] Accession${RESET}`);
const a = accession({ patient_id: 101, specimen_type: 'biopsy' });
assert(a.status === 200, 'specimen accessioned');
assert(/^PA-1-\d{8}-0001$/.test(a.specimen.accession_number), 'accession number server-generated: ' + a.specim
en.accession_number);
assert(a.specimen.state === 'Received', 'initial state Received');
assert(accession({ patient_id: 'x' }).status === 400, 'non-integer patient_id rejected (no string-id coercion)
');
const sid = a.specimen.id;

// ===== 2. Blocks & slides =====
console.log(`\n${BOLD}[2] Blocks & slides hierarchy${RESET}`);
const b = addBlock(sid, 'A1');
assert(b.status === 200, 'block added');
assert(db.specimens.find(s => s.id === sid).blocks_count === 1, 'blocks_count incremented');
const sl = addSlide(b.block.id, 'A1-1', 'H&E');
assert(sl.status === 200 && sl.slide.specimen_id === sid, 'slide added and linked to specimen');
assert(addBlock(999, 'X').status === 404, 'block on missing specimen -> 404');

// ===== 3. State machine (forward only; bad jumps 409) =====
console.log(`\n${BOLD}[3] State machine${RESET}`);
assert(advance(sid, 'Received').status === 409, 'self-transition rejected 409');
assert(advance(sid, 'Grossing').status === 200, 'Received -> Grossing');
assert(advance(sid, 'Received').status === 409, 'backward Grossing -> Received rejected 409');
assert(advance(sid, 'Processing').status === 200, 'Grossing -> Processing');
assert(advance(sid, 'Reported').status === 200, 'Processing -> Reported');

// ===== 4. Report + server-derived flags (malignancy) =====
console.log(`\n${BOLD}[4] Report & flags${RESET}`);
const r = saveReport(sid, { diagnosis: 'Invasive ductal carcinoma', micro_text: 'pleomorphic cells', snomed_co
des: ['M85003'] });
assert(r.status === 200, 'report saved');
assert(r.malignancy_flag === true && r.critical_flag === true, 'malignancy + critical flags derived server-sid
e');
assert(db.reports[0].malignancy_flag === true, 'flag persisted on report');

// ===== 5. Sign-out (final) + immutability =====
console.log(`\n${BOLD}[5] Sign-out & immutability${RESET}`);
// add a fresh specimen to test sign-out-without-diagnosis guard
const a2 = accession({ patient_id: 101 });
advance(a2.specimen.id, 'Reported');
assert(signOut(a2.specimen.id).status === 409, 'sign-out without diagnosis -> 409 (incomplete, never falsely-O
K)');
assert(signOut(sid).status === 200, 'sign-out of fully-reported specimen -> 200');
```

```
assert(db.specimens.find(s => s.id === sid).state === 'SignedOut', 'state is SignedOut');
assert(signOut(sid).status === 409, 'repeat sign-out -> 409');
assert(saveReport(sid, { diagnosis: 'changed' }).status === 409, 'cannot edit report after sign-out (immutable
)');
assert(advance(sid, 'Reported').status === 409, 'cannot transition out of SignedOut');

// ===== 6. Addendum-only after sign-out =====
console.log(`\n${BOLD}[6] Addendum${RESET}`);
assert(addendum(a2.specimen.id, 'note').status === 409, 'addendum before sign-out -> 409');
const ad = addendum(sid, 'IHC confirms ER+');
assert(ad.status === 200 && ad.addendum_count === 1, 'addendum appended after sign-out');
assert(addendum(sid, '').status === 400, 'empty addendum rejected');

// ===== 7. Audit trail emitted =====
console.log(`\n${BOLD}[7] Audit trail${RESET}`);
['PATHOLOGY_SPECIMEN_CREATE', 'PATHOLOGY_STATE_CHANGE', 'PATHOLOGY_REPORT_SAVE',
 'PATHOLOGY_SPECIMEN_SIGNOUT', 'PATHOLOGY_ADDENDUM_ADD'].forEach(act =>
    assert(db.audit.includes(act), 'audit emitted: ' + act));

console.log(`\n${BOLD}Results: ${GREEN}${passed} passed${RESET}, ${failed ? RED : ''}${failed} failed${RESET}`
);
if (failed) { failures.forEach(f => console.log('  - ' + f)); process.exit(1); }
process.exit(0);
```

```
========================================
FILE: payment_adapter.js
========================================

/**
 * payment_adapter.js ? Moyasar Payment Gateway Adapter (Saudi Arabia Compliant).
 *
 * Supports Mada, Visa, Mastercard, and Apple Pay.
 * Natively uses global fetch (Node.js 18+).
 * Safe-by-design: falls back to Sandbox/Mock mode in staging/test or if keys are missing.
 */
'use strict';

const { pool } = require('./db_postgres');

const MOYASAR_SECRET_KEY = process.env.MOYASAR_SECRET_KEY || '';
const MOYASAR_PUBLISHABLE_KEY = process.env.MOYASAR_PUBLISHABLE_KEY || '';
const IS_SANDBOX = !MOYASAR_SECRET_KEY || process.env.NODE_ENV === 'staging' || process.env.NODE_ENV === 'test
';

/**
 * Initiates a payment request with Moyasar.
 * @param {Object} params { invoiceId, amount, description, source }
 * @returns {Promise<Object>} Moyasar payment object or mock response
 */
async function initiatePayment({ invoiceId, amount, description, source }) {
    const amountInHalalas = Math.round(amount * 100); // Moyasar expects amount in Halalas (cents)
```

```
    if (IS_SANDBOX) {
        console.log(`[Moyasar Sandbox] Initiating payment for invoice ${invoiceId}: ${amount} SAR`);
        const mockPaymentId = `pay_mock_${Math.random().toString(36).substring(2, 15)}`;
        return {
            id: mockPaymentId,
            status: 'initiated',
            amount: amountInHalalas,
            currency: 'SAR',
            description: description || `Invoice ${invoiceId}`,
            invoice_id: invoiceId,
            callback_url: `https://www.jumanasoft.com/api/payments/moyasar/callback?payment_id=${mockPaymentId
}`,
            check_url: `https://www.jumanasoft.com/api/payments/moyasar/verify?payment_id=${mockPaymentId}`
        };
    }

    try {
        const authHeader = 'Basic ' + Buffer.from(MOYASAR_SECRET_KEY + ':').toString('base64');
        const response = await fetch('https://api.moyasar.com/v1/payments', {
            method: 'POST',
            headers: {
                'Authorization': authHeader,
                'Content-Type': 'application/json'
            },
            body: JSON.stringify({
                amount: amountInHalalas,
                currency: 'SAR',
                description: description || `Invoice ${invoiceId}`,
                callback_url: `https://www.jumanasoft.com/api/payments/moyasar/callback`,
                source: source || { type: 'creditcard' },
                metadata: {
                    invoice_id: String(invoiceId)
                }
            })
        });

        if (!response.ok) {
            const errText = await response.text();
            throw new Error(`Moyasar API error: ${response.status} - ${errText}`);
        }

        return await response.json();
    } catch (error) {
        console.error('[Moyasar] Failed to initiate payment:', error);
        throw error;
    }
}

/**
 * Verifies a payment status with Moyasar and updates the invoice in the DB.
 * @param {string} paymentId
 * @returns {Promise<Object>} Verification status and invoice
 */
async function verifyAndProcessPayment(paymentId) {
```

```
    let paymentData;

    if (IS_SANDBOX && paymentId.startsWith('pay_mock_')) {
        console.log(`[Moyasar Sandbox] Verifying mock payment: ${paymentId}`);
        paymentData = {
            id: paymentId,
            status: 'paid',
            amount: 10000, // 100.00 SAR
            currency: 'SAR',
            source: { type: 'creditcard', company: 'visa', name: 'Sandbox User', number: 'XXXX-XXXX-XXXX-1111'
 },
            metadata: { invoice_id: null } // will be matched from DB or query
        };
    } else {
        try {
            const authHeader = 'Basic ' + Buffer.from(MOYASAR_SECRET_KEY + ':').toString('base64');
            const response = await fetch(`https://api.moyasar.com/v1/payments/${paymentId}`, {
                method: 'GET',
                headers: {
                    'Authorization': authHeader
                }
            });

            if (!response.ok) {
                const errText = await response.text();
                throw new Error(`Moyasar verification failed: ${response.status} - ${errText}`);
            }

            paymentData = await response.json();
        } catch (error) {
            console.error(`[Moyasar] Verification request failed for ${paymentId}:`, error);
            throw error;
        }
    }

    if (paymentData.status === 'paid' || paymentData.status === 'captured') {
        // Find invoice. If metadata doesn't have it, we check if we stored paymentId in the db or check by qu
ery.
        const invoiceId = paymentData.metadata?.invoice_id;
        let invoice;

        if (invoiceId) {
            invoice = (await pool.query('SELECT * FROM invoices WHERE id = $1', [invoiceId])).rows[0];
        } else {
            // Fallback search: find invoice by payment gateway ref or find the oldest unpaid invoice with sim
ilar amount
            invoice = (await pool.query('SELECT * FROM invoices WHERE payment_gateway_ref = $1', [paymentId]))
.rows[0];
        }

        if (invoice) {
            if (!invoice.paid) {
                const paymentMethod = `Moyasar (${paymentData.source.type || 'card'} - ${paymentData.source.co
mpany || 'unknown'})`;
```

```
                await pool.query(
                    'UPDATE invoices SET paid = 1, payment_method = $1, payment_gateway_ref = $2 WHERE id = $3
',
                    [paymentMethod, paymentId, invoice.id]
                );
                console.log(`[Moyasar] Invoice ${invoice.id} marked as PAID via ${paymentId}`);
                invoice.paid = 1;
                invoice.payment_method = paymentMethod;
                invoice.payment_gateway_ref = paymentId;
            }
            return { success: true, status: paymentData.status, invoice };
        } else {
            console.warn(`[Moyasar] Payment ${paymentId} verified but no matching invoice found.`);
            return { success: false, status: paymentData.status, error: 'No matching invoice' };
        }
    }

    return { success: false, status: paymentData.status, error: 'Payment not captured' };
}

module.exports = {
    initiatePayment,
    verifyAndProcessPayment,
    IS_SANDBOX
};
```

```
========================================
FILE: payment_adapter_test.js
========================================

/**
 * payment_adapter_test.js ? Integration tests for Moyasar Payment Adapter.
 */
'use strict';

const assert = require('assert');
const { initiatePayment, verifyAndProcessPayment } = require('./payment_adapter');
const { pool, runWithTenant } = require('./db_postgres');

async function runTests() {
    console.log('--- STARTING MOYASAR PAYMENT ADAPTER INTEGRATION TESTS ---');

    const client = await pool.connect();
    let patientId;
    let invoiceId;

    try {
        console.log('Setting up session tenant context and test data...');
        // Set tenant context for RLS
        await client.query("SET app.tenant_id = '1'");

        // Insert patient
```

```
        const patientRes = await client.query(
            "INSERT INTO patients (name_ar, name_en, tenant_id) VALUES ($1, $2, $3) RETURNING id",
            ['???? ?????? ???', 'Test Payment Patient', 1]
        );
        patientId = patientRes.rows[0].id;

        // Insert invoice
        const invoiceRes = await client.query(
            "INSERT INTO invoices (patient_id, patient_name, total, description, service_type, payment_method,
 tenant_id, facility_id) VALUES ($1, $2, $3, $4, $5, $6, $7, $8) RETURNING id, total",
            [patientId, 'Test Payment Patient', 150.00, 'Test Consultation', 'Clinical', 'Moyasar', 1, 1]
        );
        invoiceId = invoiceRes.rows[0].id;
        const amount = invoiceRes.rows[0].total;

        // 2. Test initiatePayment
        console.log('Testing initiatePayment...');
        const paymentInit = await initiatePayment({
            invoiceId,
            amount,
            description: 'Test Payment',
            source: { type: 'creditcard' }
        });

        assert.ok(paymentInit.id, 'Payment initiation should return a payment ID');
        assert.strictEqual(paymentInit.currency, 'SAR', 'Payment currency should be SAR');
        console.log(`? Payment initiated successfully. ID: ${paymentInit.id}`);

        // Update the invoice with the payment gateway reference to simulate redirect/callback
        await client.query('UPDATE invoices SET payment_gateway_ref = $1 WHERE id = $2', [paymentInit.id, invo
iceId]);

        // 3. Test verifyAndProcessPayment
        console.log('Testing verifyAndProcessPayment...');
        const verifyRes = await runWithTenant({ tenantId: 1 }, async () => {
            return await verifyAndProcessPayment(paymentInit.id);
        });

        assert.strictEqual(verifyRes.success, true, 'Verification should be successful');
        assert.strictEqual(verifyRes.invoice.paid, 1, 'Invoice should be marked as paid');
        assert.ok(verifyRes.invoice.payment_method.includes('Moyasar'), 'Payment method should contain Moyasar
');
        assert.strictEqual(verifyRes.invoice.payment_gateway_ref, paymentInit.id, 'Payment gateway reference s
hould match');
        console.log('? Payment verified and processed successfully. Invoice marked as paid.');

        // 4. Verify in DB
        const dbInv = (await client.query('SELECT * FROM invoices WHERE id = $1', [invoiceId])).rows[0];
        assert.strictEqual(dbInv.paid, 1, 'Database check: invoice paid should be 1');
        console.log('? Database state verified.');

    } finally {
        // Cleanup
        console.log('Cleaning up test data...');
```

```
        if (patientId) {
            await client.query('DELETE FROM invoices WHERE patient_id = $1', [patientId]);
            await client.query('DELETE FROM patients WHERE id = $1', [patientId]);
        }
        client.release();
        console.log('? Cleanup complete.');
    }

    console.log('? All Moyasar Payment Adapter Integration Tests passed successfully!\n');
}

if (require.main === module) {
    runTests().catch(err => {
        console.error('? Test failed:', err);
        process.exit(1);
    });
}

module.exports = { runTests };
```

```
========================================
FILE: payment_refund_integrity_test.js
========================================

/**
 * payment_refund_integrity_test.js
 * ========================================
 * PHASE 2 (H-1/H-2) ? static route-guard audit of the partial-pay & refund handlers in server.js.
 * Verifies the hardening is present in source (tenant predicate / IDOR fix, server-side amount
 * validation, FOR UPDATE row-lock, tenant_id stamping, no client-trusted parseFloat, no GL/ZATCA/NPHIES).
 * DB-free, deterministic. Does NOT execute any route or touch the database.
 *
 *   node payment_refund_integrity_test.js
 */
const fs = require('fs');
const src = fs.readFileSync(require('path').join(__dirname, 'server.js'), 'utf8');

const RED = '\x1b[31m', GREEN = '\x1b[32m', BLUE = '\x1b[34m', RESET = '\x1b[0m', BOLD = '\x1b[1m';
let passed = 0, failed = 0; const failures = [];
function assert(cond, name, details = '') {
    if (cond) { console.log(`  ${GREEN}PASS${RESET} ? ${name}`); passed++; }
    else { console.log(`  ${RED}FAIL${RESET} ? ${name}${details ? ' | ' + details : ''}`); failed++; failures.
push({ name, details }); }
}

// ---- extract the two route blocks from source ----
const partialIdx = src.indexOf("app.put('/api/invoices/:id/partial-pay'");
const refundIdx = src.indexOf("app.post('/api/invoices/:id/refund'");
const cashIdx = src.indexOf("// ===== CASH DRAWER");
const partialBlock = (partialIdx >= 0 && refundIdx > partialIdx) ? src.slice(partialIdx, refundIdx) : '';
const refundBlock = (refundIdx >= 0) ? src.slice(refundIdx, cashIdx > refundIdx ? cashIdx : refundIdx + 4000)
: '';
```

```
console.log(`\n${BOLD}${BLUE}=== Payment/Refund Integrity Route Guards (H-1/H-2) ===${RESET}\n`);
assert(partialBlock.length > 0, 'partial-pay route block located');
assert(refundBlock.length > 0, 'refund route block located');

// ---- H-1: partial-pay ----
console.log(`${BOLD}[H-1] partial-pay hardening${RESET}`);
assert(/requireTenantScope/.test(partialBlock), 'partial-pay enforces requireTenantScope');
assert(/FOR UPDATE/.test(partialBlock), 'partial-pay locks invoice row (FOR UPDATE)');
assert(/set_config\('app\.tenant_id'/.test(partialBlock), 'partial-pay binds app.tenant_id for RLS under manua
l client');
assert(/parsePositiveMoneyToMinorUnits/.test(partialBlock), 'partial-pay validates amount via parsePositiveMon
eyToMinorUnits');
assert(/assertAmountWithinCap/.test(partialBlock), 'partial-pay caps amount at outstanding (no overpayment)');
assert(!/parseFloat\(amount_paid\)/.test(partialBlock), 'partial-pay no longer trusts raw parseFloat(amount_pa
id)');
assert(/client\.query\('COMMIT'\)/.test(partialBlock) && /client\.release\(\)/.test(partialBlock), 'partial-pa
y uses a committed, released transaction');

// ---- H-2: refund ----
console.log(`${BOLD}[H-2] refund hardening (IDOR + amount + tenant stamp)${RESET}`);
assert(/requireTenantScope/.test(refundBlock), 'refund enforces requireTenantScope');
assert(/FOR UPDATE/.test(refundBlock), 'refund locks original invoice row (FOR UPDATE)');
// IDOR fix: the original-invoice SELECT must be tenant-scoped, NOT the old bare `WHERE id=$1', [req.params.id
]`
assert(!/SELECT \* FROM invoices WHERE id=\$1',\s*\[req\.params\.id\]\)/.test(refundBlock), 'refund no longer
loads invoice by bare id (IDOR closed)');
assert(/\$\{tenantCheck\}/.test(refundBlock) || /AND tenant_id=\$2/.test(refundBlock), 'refund SELECT carries
a tenant predicate');
assert(/parsePositiveMoneyToMinorUnits/.test(refundBlock), 'refund validates amount via parsePositiveMoneyToMi
norUnits');
assert(/assertAmountWithinCap/.test(refundBlock) && /refundable/i.test(refundBlock), 'refund caps amount at se
rver-computed refundable');
assert(!/-\(parseFloat\(amount\)\)/.test(refundBlock), 'refund no longer trusts raw -(parseFloat(amount))');
assert(/tenant_id, facility_id\) VALUES/.test(refundBlock) && /tenantId \|\| null/.test(refundBlock), 'refund
row is stamped with tenant_id/facility_id');
assert(/set_config\('app\.tenant_id'/.test(refundBlock), 'refund binds app.tenant_id for RLS under manual clie
nt');

// ---- scope guard: no GL / accounting / external healthcare calls in the CODE (comments stripped) ----
console.log(`${BOLD}[scope] no GL / ZATCA / NPHIES in payment+refund route code${RESET}`);
const bothCode = (partialBlock + refundBlock)
    .replace(/\/\*[\s\S]*?\*\//g, '')   // block comments
    .replace(/\/\/[^\n]*/g, '');        // line comments (e.g. our own "out of scope" note)
assert(!/journal|fe\.(post|journal|reverse)|ACCOUNTING_POSTING/i.test(bothCode), 'no journal/GL posting in pay
ment+refund routes');
assert(!/zatca/i.test(bothCode), 'no ZATCA calls in payment+refund routes');
assert(!/nphies/i.test(bothCode), 'no NPHIES calls in payment+refund routes');

console.log(`\n${BOLD}Result: ${passed} passed, ${failed} failed${RESET}`);
if (failed > 0) { console.log(`${RED}Failures:${RESET}`); failures.forEach(f => console.log(`  - ${f.name}: ${
f.details}`)); process.exit(1); }
process.exit(0);
```

```
========================================
FILE: pediatric_apgar_test.js
========================================

/**
 * pediatric_apgar_test.js ? Integration test for the Neonatal Apgar Scores API.
 */
'use strict';

process.env.NODE_ENV = 'staging';

const { spawn } = require('child_process');
const http = require('http');
const assert = require('assert');
const bcrypt = require('bcryptjs');
const { pool } = require('./db_postgres');

const TEST_PORT = 3026;
const TEST_USERNAME = 'ped_tester';
const TEST_PASSWORD = 'PED_PASSWORD_PLACEHOLDER';

let serverProcess;

function makeRequest(method, path, body, headers = {}) {
    return new Promise((resolve, reject) => {
        const payload = body ? JSON.stringify(body) : '';
        const req = http.request({
            hostname: 'localhost',
            port: TEST_PORT,
            path,
            method,
            headers: {
                'Content-Type': 'application/json',
                'Content-Length': Buffer.byteLength(payload),
                ...headers
            }
        }, (res) => {
            let data = '';
            res.on('data', chunk => data += chunk);
            res.on('end', () => {
                try {
                    const parsed = data ? JSON.parse(data) : {};
                    resolve({ statusCode: res.statusCode, headers: res.headers, body: parsed });
                } catch (e) {
                    resolve({ statusCode: res.statusCode, headers: res.headers, rawBody: data });
                }
            });
        });
        req.on('error', reject);
        if (payload) req.write(payload);
        req.end();
    });
```

```
}

async function runTests() {
    console.log('--- STARTING NEONATAL APGAR SCORES INTEGRATION TESTS ---');

    const babyId = 9941;
    const motherId = 9942;
    const staffUserId = 9943;
    const client = await pool.connect();

    try {
        console.log('Setting up test data...');
        await client.query("SET app.tenant_id = '1'");

        // Clean up old test data
        await client.query('DELETE FROM neonatal_apgar_scores WHERE patient_id = $1', [babyId]);
        await client.query('DELETE FROM patients WHERE id IN ($1, $2)', [babyId, motherId]);
        await client.query('DELETE FROM user_tenants WHERE user_id = $1', [staffUserId]);
        await client.query('DELETE FROM system_users WHERE id = $1', [staffUserId]);

        // Insert mother patient
        await client.query(
            'INSERT INTO patients (id, name_en, name_ar, tenant_id) VALUES ($1, $2, $3, 1)',
            [motherId, 'Mother Patient', '???? ???????']
        );

        // Insert baby patient
        await client.query(
            'INSERT INTO patients (id, name_en, name_ar, tenant_id) VALUES ($1, $2, $3, 1)',
            [babyId, 'Newborn Baby', '??????? ??????']
        );

        // Insert pediatric user
        const hashedPassword = await bcrypt.hash(TEST_PASSWORD, 10);
        await client.query(
            "INSERT INTO system_users (id, username, password_hash, display_name, role, speciality, permission
s, is_active) VALUES ($1, $2, $3, $4, $5, $6, $7, 1)",
            [staffUserId, TEST_USERNAME, hashedPassword, 'Dr. Samir Pediatrician', 'Doctor', 'Pediatrics', '["
patients"]']
        );

        // Associate user with tenant 1
        await client.query(
            'INSERT INTO user_tenants (user_id, tenant_id, is_active) VALUES ($1, 1, true)',
            [staffUserId]
        );

        console.log('Spawning test server...');
        serverProcess = spawn('node', ['server.js'], {
            env: { ...process.env, PORT: TEST_PORT, NODE_ENV: 'staging', SKIP_DB_INIT: 'true' }
        });

        serverProcess.stdout.on('data', (data) => {
            if (process.env.DEBUG_TESTS) console.log(`[Server STDOUT] ${data.toString().trim()}`);
```

```
});

// Wait for server to start
await new Promise(resolve => setTimeout(resolve, 3000));

console.log('Logging in...');
const loginRes = await makeRequest('POST', '/api/auth/login', {
    username: TEST_USERNAME,
    password: TEST_PASSWORD
});
assert.strictEqual(loginRes.statusCode, 200, 'Login should succeed');
const cookie = loginRes.headers['set-cookie'][0];

// Test 1: Create Apgar Score
console.log('Testing create neonatal Apgar score...');
const details = {
    appearance: 2,
    pulse: 2,
    grimace: 1,
    activity: 2,
    respiration: 2
};
const createRes = await makeRequest('POST', '/api/pediatrics/apgar', {
    patient_id: babyId,
    mother_id: motherId,
    apgar_1min: 7,
    apgar_5min: 9,
    apgar_10min: 9,
    details,
    notes: 'Baby is crying well. Good response.'
}, { Cookie: cookie });

assert.strictEqual(createRes.statusCode, 200);
assert.strictEqual(createRes.body.patient_id, babyId);
assert.strictEqual(createRes.body.mother_id, motherId);
assert.strictEqual(createRes.body.apgar_1min, 7);
assert.strictEqual(createRes.body.apgar_5min, 9);
console.log(`? Neonatal Apgar score created successfully. ID: ${createRes.body.id}`);

// Test 2: Get Patient Apgar Scores
console.log('Testing get neonatal Apgar scores...');
const getRes = await makeRequest('GET', `/api/pediatrics/apgar/${babyId}`, null, { Cookie: cookie });
assert.strictEqual(getRes.statusCode, 200);
assert.strictEqual(getRes.body.length, 1);
assert.strictEqual(getRes.body[0].apgar_1min, 7);
console.log('? Apgar scores list retrieved successfully.');

console.log('Cleaning up test data...');
await client.query('DELETE FROM neonatal_apgar_scores WHERE patient_id = $1', [babyId]);
await client.query('DELETE FROM patients WHERE id IN ($1, $2)', [babyId, motherId]);
await client.query('DELETE FROM user_tenants WHERE user_id = $1', [staffUserId]);
await client.query('DELETE FROM system_users WHERE id = $1', [staffUserId]);

console.log('Killing test server...');
```

```
        serverProcess.kill();
        console.log('? Neonatal Apgar Scores Integration Tests passed successfully!');
        process.exit(0);

    } catch (e) {
        console.error('? Test failed:', e);
        if (serverProcess) serverProcess.kill();
        process.exit(1);
    } finally {
        client.release();
    }
}

runTests();



========================================
FILE: plans.js
========================================

/**
 * plans.js ? Jumanasoft SaaS Plans & Pricing / Entitlements foundation (Batch 3).
 *
 * Super Admin manages a plan CATALOG (identity + pricing + entitlements) and assigns a plan to a tenant.
 * NO real payment / checkout / webhook / capture / invoices / dunning / cron (out of scope, by design).
 *
 * Security:
 *   - Admin routes are mounted under /api/super-admin (Batch 2 requireSuperAdmin outer guard) ? a tenant
 *     Admin can NEITHER manage the catalog NOR change a tenant's plan. Identity from session only.
 *   - Public read (GET /api/public/plans) returns ACTIVE plans with marketing-safe fields only
 *     (no disabled plans, no internal admin fields). Fail-safe: returns [] if the catalog tables are absent.
 *
 * Validation is server-side and fail-closed: prices >= 0, currency in an allowlist, limits null-or-non-negati
ve,
 * modules_enabled restricted to a known-modules allowlist, plan_key immutable after creation, soft-disable on
ly.
 *
 * Pure core (validate*/canAssignPlan/deriveCurrentPlan/*View) is unit-tested with no DB.
 */
'use strict';

const ALLOWED_CURRENCIES = ['SAR', 'USD', 'AED', 'EGP', 'KWD', 'BHD', 'QAR', 'OMR', 'JOD'];
const SUPPORT_LEVELS = ['basic', 'standard', 'priority', 'enterprise'];
const ASSIGNMENT_SOURCES = ['manual', 'trial', 'migration'];

// Allowlist of system modules a plan may enable. Mirrors the operational modules in ROLE_PERMISSIONS.
const KNOWN_MODULES = [
    'dashboard', 'patients', 'appointments', 'doctor', 'nursing', 'lab', 'radiology', 'pharmacy',
    'inventory', 'invoices', 'accounts', 'finance', 'insurance', 'reports', 'messaging', 'settings',
    'surgery', 'icu', 'emergency', 'inpatient', 'bloodbank', 'obgyn', 'antenatal', 'cssd', 'quality',
    'infection', 'him', 'medical-records', 'pathology', 'hr', 'maintenance', 'api'
];
const KNOWN_MODULE_SET = new Set(KNOWN_MODULES);
```

```
const PLAN_KEY_RE = /^[a-z0-9_]{2,40}$/;

// pg returns NUMERIC as a string -> parse defensively. Returns NaN on anything non-numeric.
function parseMoney(v) {
    if (v === null || v === undefined || v === '') return NaN;
    const n = typeof v === 'number' ? v : Number(String(v).trim());
    return Number.isFinite(n) ? n : NaN;
}

// null/''/undefined -> null (unlimited); else must be a non-negative integer.
function parseLimit(v, field, errors) {
    if (v === null || v === undefined || v === '') return null;
    const n = typeof v === 'number' ? v : Number(String(v).trim());
    if (!Number.isInteger(n) || n < 0) { errors.push(`${field} must be a non-negative integer or empty (unlimi
ted)`); return undefined; }
    return n;
}

function nonEmptyStr(v) { return typeof v === 'string' && v.trim().length > 0; }

// Validate a plan create/update payload. opts.isCreate gates plan_key (immutable after creation).
function validatePlanInput(body = {}, opts = {}) {
    const errors = [];
    const value = {};

    if (opts.isCreate) {
        const key = typeof body.plan_key === 'string' ? body.plan_key.trim() : '';
        if (!PLAN_KEY_RE.test(key)) errors.push('plan_key must match ^[a-z0-9_]{2,40}$');
        value.plan_key = key;
    } // on update plan_key is ignored (immutable)

    if (!nonEmptyStr(body.name_ar)) errors.push('name_ar is required');
    if (!nonEmptyStr(body.name_en)) errors.push('name_en is required');
    value.name_ar = nonEmptyStr(body.name_ar) ? body.name_ar.trim().slice(0, 120) : '';
    value.name_en = nonEmptyStr(body.name_en) ? body.name_en.trim().slice(0, 120) : '';
    value.description_ar = typeof body.description_ar === 'string' ? body.description_ar : '';
    value.description_en = typeof body.description_en === 'string' ? body.description_en : '';

    const cur = typeof body.currency === 'string' ? body.currency.trim().toUpperCase() : '';
    if (!ALLOWED_CURRENCIES.includes(cur)) errors.push('currency must be one of: ' + ALLOWED_CURRENCIES.join('
, '));
    value.currency = cur;

    const m = body.monthly_price === undefined ? 0 : parseMoney(body.monthly_price);
    const y = body.yearly_price === undefined ? 0 : parseMoney(body.yearly_price);
    if (Number.isNaN(m) || m < 0) errors.push('monthly_price must be a number >= 0');
    if (Number.isNaN(y) || y < 0) errors.push('yearly_price must be a number >= 0');
    value.monthly_price = Number.isNaN(m) ? 0 : m;
    value.yearly_price = Number.isNaN(y) ? 0 : y;

    let trial = body.trial_days === undefined ? 0 : Number(body.trial_days);
    if (!Number.isInteger(trial) || trial < 0 || trial > 365) errors.push('trial_days must be an integer 0..36
5');
```

```
    value.trial_days = Number.isInteger(trial) ? trial : 0;

    let sort = body.sort_order === undefined ? 0 : Number(body.sort_order);
    if (!Number.isInteger(sort)) errors.push('sort_order must be an integer');
    value.sort_order = Number.isInteger(sort) ? sort : 0;

    return { ok: errors.length === 0, errors, value };
}

// Validate entitlements payload. Accepts modules_enabled as array OR comma string; rejects unknown modules.
function validateEntitlements(body = {}) {
    const errors = [];
    const value = {};

    value.max_users = parseLimit(body.max_users, 'max_users', errors);
    value.max_branches = parseLimit(body.max_branches, 'max_branches', errors);
    value.max_invoices_per_month = parseLimit(body.max_invoices_per_month, 'max_invoices_per_month', errors);

    let mods = body.modules_enabled;
    if (mods === undefined || mods === null) mods = [];
    if (typeof mods === 'string') mods = mods.split(',');
    if (!Array.isArray(mods)) { errors.push('modules_enabled must be an array or comma-separated string'); mod
s = []; }
    const clean = [];
    for (const raw of mods) {
        const mod = String(raw).trim().toLowerCase();
        if (!mod) continue;
        if (!KNOWN_MODULE_SET.has(mod)) { errors.push(`unknown module: ${mod}`); continue; }
        if (!clean.includes(mod)) clean.push(mod);
    }
    clean.sort();
    value.modules_enabled = clean.join(',');

    const sl = typeof body.support_level === 'string' ? body.support_level.trim().toLowerCase() : 'standard';
    if (!SUPPORT_LEVELS.includes(sl)) errors.push('support_level must be one of: ' + SUPPORT_LEVELS.join(', ')
);
    value.support_level = SUPPORT_LEVELS.includes(sl) ? sl : 'standard';

    value.api_access = body.api_access === true || body.api_access === 1 || body.api_access === '1' || body.ap
i_access === 'true';
    value.custom_domain = body.custom_domain === true || body.custom_domain === 1 || body.custom_domain === '1
' || body.custom_domain === 'true';

    return { ok: errors.length === 0, errors, value };
}

// A disabled plan cannot be assigned (active may arrive as boolean or 't'/'f' from pg).
function isActive(plan) { return !!plan && (plan.active === true || plan.active === 't' || plan.active === 1);
 }
function canAssignPlan(plan) { return isActive(plan); }

// Current plan = latest assignment with effective_to null, else most-recently assigned. Null if none.
function deriveCurrentPlan(rows) {
    if (!Array.isArray(rows) || rows.length === 0) return null;
```

```
    const open = rows.filter(r => r.effective_to == null);
    const pool = open.length ? open : rows;
    return pool.slice().sort((a, b) => new Date(b.assigned_at || 0) - new Date(a.assigned_at || 0))[0];
}

// Marketing-safe public projection (NO admin/internal fields, NO active flag exposure beyond filtering).
function publicPlanView(plan, ent) {
    if (!plan) return null;
    return {
        plan_key: plan.plan_key,
        name_ar: plan.name_ar, name_en: plan.name_en,
        description_ar: plan.description_ar || '', description_en: plan.description_en || '',
        currency: plan.currency,
        monthly_price: parseMoney(plan.monthly_price) || 0,
        yearly_price: parseMoney(plan.yearly_price) || 0,
        trial_days: Number(plan.trial_days) || 0,
        sort_order: Number(plan.sort_order) || 0,
        entitlements: ent ? {
            max_users: ent.max_users == null ? null : Number(ent.max_users),
            max_branches: ent.max_branches == null ? null : Number(ent.max_branches),
            support_level: ent.support_level || 'standard',
            api_access: !!ent.api_access, custom_domain: !!ent.custom_domain,
            modules_enabled: (ent.modules_enabled || '').split(',').filter(Boolean)
        } : null
    };
}

// Full admin projection (includes active + raw entitlement limits).
function planAdminView(plan, ent) {
    if (!plan) return null;
    const v = publicPlanView(plan, ent) || {};
    v.id = plan.id;
    v.active = isActive(plan);
    v.created_at = plan.created_at || null;
    v.updated_at = plan.updated_at || null;
    if (ent) v.entitlements = Object.assign(v.entitlements || {}, {
        max_invoices_per_month: ent.max_invoices_per_month == null ? null : Number(ent.max_invoices_per_month)
    });
    return v;
}

// ---- Express router factories ----
function posInt(v) { const n = Number(v); return Number.isInteger(n) && n >= 1 ? n : null; }

// Admin router ? mounted under /api/super-admin (inherits the Batch-2 requireSuperAdmin outer guard).
// deps: { pool, getActor(req)->user, logAudit(...) }
function makePlansRouter(deps = {}) {
    const express = require('express');
    const router = express.Router();
    const { pool, getActor, logAudit } = deps;
    const actorOf = (req) => (getActor ? getActor(req) : (req.session && req.session.user)) || null;
    const audit = (req, action, details) => { try { const a = actorOf(req); logAudit && logAudit(a && a.id, a
&& a.display_name, action, 'Plans', details, req.ip); } catch (_) {} };
```

```
    async function loadEnt(planId) {
        return (await pool.query('SELECT * FROM plan_entitlements WHERE plan_id=$1', [planId])).rows[0] || nul
l;
    }

    // LIST all plans (admin view, incl. disabled).
    router.get('/plans', async (req, res) => {
        try {
            const plans = (await pool.query('SELECT * FROM plans ORDER BY sort_order ASC, id ASC')).rows;
            const ents = (await pool.query('SELECT * FROM plan_entitlements')).rows;
            const byId = {}; ents.forEach(e => { byId[e.plan_id] = e; });
            res.json({ plans: plans.map(p => planAdminView(p, byId[p.id])) });
        } catch (e) { res.status(500).json({ error: 'Server error' }); }
    });

    // CREATE plan (+ entitlements row).
    router.post('/plans', async (req, res) => {
        const p = validatePlanInput(req.body, { isCreate: true });
        const ent = validateEntitlements(req.body);
        if (!p.ok || !ent.ok) return res.status(400).json({ error: 'Validation failed', details: [...p.errors,
 ...ent.errors] });
        try {
            const exists = (await pool.query('SELECT 1 FROM plans WHERE plan_key=$1', [p.value.plan_key])).row
s[0];
            if (exists) return res.status(409).json({ error: 'plan_key already exists', code: 'PLAN_EXISTS' })
;
            const v = p.value;
            const row = (await pool.query(
                `INSERT INTO plans (plan_key,name_ar,name_en,description_ar,description_en,currency,monthly_pr
ice,yearly_price,trial_days,sort_order)
                 VALUES ($1,$2,$3,$4,$5,$6,$7,$8,$9,$10) RETURNING *`,
                [v.plan_key, v.name_ar, v.name_en, v.description_ar, v.description_en, v.currency, v.monthly_p
rice, v.yearly_price, v.trial_days, v.sort_order]
            )).rows[0];
            const e = ent.value;
            await pool.query(
                `INSERT INTO plan_entitlements (plan_id,max_users,max_branches,max_invoices_per_month,modules_
enabled,support_level,api_access,custom_domain)
                 VALUES ($1,$2,$3,$4,$5,$6,$7,$8)`,
                [row.id, e.max_users, e.max_branches, e.max_invoices_per_month, e.modules_enabled, e.support_l
evel, e.api_access, e.custom_domain]
            );
            audit(req, 'PLAN_CREATE', `plan ${v.plan_key} created`);
            res.json({ plan: planAdminView(row, await loadEnt(row.id)) });
        } catch (e) { res.status(500).json({ error: 'Server error' }); }
    });

    // UPDATE plan (plan_key immutable; identified by :key).
    router.put('/plans/:key', async (req, res) => {
        const p = validatePlanInput(req.body, { isCreate: false });
        const ent = validateEntitlements(req.body);
        if (!p.ok || !ent.ok) return res.status(400).json({ error: 'Validation failed', details: [...p.errors,
 ...ent.errors] });
        try {
```

```
            const cur = (await pool.query('SELECT * FROM plans WHERE plan_key=$1', [req.params.key])).rows[0];
            if (!cur) return res.status(404).json({ error: 'Plan not found' });
            const v = p.value;
            const row = (await pool.query(
                `UPDATE plans SET name_ar=$1,name_en=$2,description_ar=$3,description_en=$4,currency=$5,monthl
y_price=$6,yearly_price=$7,trial_days=$8,sort_order=$9,updated_at=now()
                 WHERE plan_key=$10 RETURNING *`,
                [v.name_ar, v.name_en, v.description_ar, v.description_en, v.currency, v.monthly_price, v.year
ly_price, v.trial_days, v.sort_order, req.params.key]
            )).rows[0];
            const e = ent.value;
            await pool.query(
                `INSERT INTO plan_entitlements (plan_id,max_users,max_branches,max_invoices_per_month,modules_
enabled,support_level,api_access,custom_domain)
                 VALUES ($1,$2,$3,$4,$5,$6,$7,$8)
                 ON CONFLICT (plan_id) DO UPDATE SET max_users=$2,max_branches=$3,max_invoices_per_month=$4,mo
dules_enabled=$5,support_level=$6,api_access=$7,custom_domain=$8`,
                [cur.id, e.max_users, e.max_branches, e.max_invoices_per_month, e.modules_enabled, e.support_l
evel, e.api_access, e.custom_domain]
            );
            audit(req, 'PLAN_UPDATE', `plan ${req.params.key} updated`);
            res.json({ plan: planAdminView(row, await loadEnt(cur.id)) });
        } catch (e) { res.status(500).json({ error: 'Server error' }); }
    });

    // SOFT disable / enable (never hard-delete).
    async function setActive(req, res, active, action) {
        try {
            const row = (await pool.query('UPDATE plans SET active=$1, updated_at=now() WHERE plan_key=$2 RETU
RNING *', [active, req.params.key])).rows[0];
            if (!row) return res.status(404).json({ error: 'Plan not found' });
            audit(req, action, `plan ${req.params.key} ${active ? 'enabled' : 'disabled'}`);
            res.json({ plan: planAdminView(row, await loadEnt(row.id)) });
        } catch (e) { res.status(500).json({ error: 'Server error' }); }
    }
    router.post('/plans/:key/disable', (req, res) => setActive(req, res, false, 'PLAN_DISABLE'));
    router.post('/plans/:key/enable', (req, res) => setActive(req, res, true, 'PLAN_ENABLE'));

    // GET tenant current plan.
    router.get('/tenants/:id/plan', async (req, res) => {
        const id = posInt(req.params.id);
        if (!id) return res.status(400).json({ error: 'Invalid tenant id' });
        try {
            const rows = (await pool.query('SELECT * FROM tenant_plan_assignments WHERE tenant_id=$1', [id])).
rows;
            const current = deriveCurrentPlan(rows);
            res.json({ tenant_id: id, current: current ? { plan_key: current.plan_key, assignment_source: curr
ent.assignment_source, assigned_at: current.assigned_at, effective_from: current.effective_from } : null });
        } catch (e) { res.status(500).json({ error: 'Server error' }); }
    });

    // ASSIGN a plan to a tenant (records a new assignment row; closes the previous open one).
    router.post('/tenants/:id/plan', async (req, res) => {
        const id = posInt(req.params.id);
```

```
        if (!id) return res.status(400).json({ error: 'Invalid tenant id' });
        const planKey = typeof req.body.plan_key === 'string' ? req.body.plan_key.trim() : '';
        const source = typeof req.body.assignment_source === 'string' ? req.body.assignment_source.trim() : 'm
anual';
        if (!PLAN_KEY_RE.test(planKey)) return res.status(400).json({ error: 'invalid plan_key' });
        if (!ASSIGNMENT_SOURCES.includes(source) || source === 'migration') return res.status(400).json({ erro
r: 'invalid assignment_source' });
        try {
            const tenant = (await pool.query('SELECT id FROM tenants WHERE id=$1', [id])).rows[0];
            if (!tenant) return res.status(404).json({ error: 'Tenant not found' });
            const plan = (await pool.query('SELECT * FROM plans WHERE plan_key=$1', [planKey])).rows[0];
            if (!plan) return res.status(404).json({ error: 'Plan not found' });
            if (!canAssignPlan(plan)) return res.status(409).json({ error: 'Cannot assign a disabled plan', co
de: 'PLAN_DISABLED' });
            const actor = actorOf(req);
            await pool.query('UPDATE tenant_plan_assignments SET effective_to=now() WHERE tenant_id=$1 AND eff
ective_to IS NULL', [id]);
            const row = (await pool.query(
                `INSERT INTO tenant_plan_assignments (tenant_id,plan_key,assignment_source,assigned_by)
                 VALUES ($1,$2,$3,$4) RETURNING *`,
                [id, planKey, source, actor && actor.id]
            )).rows[0];
            audit(req, 'TENANT_PLAN_ASSIGN', `tenant#${id} -> ${planKey} (${source})`);
            res.json({ assignment: { tenant_id: id, plan_key: row.plan_key, assignment_source: row.assignment_
source, assigned_at: row.assigned_at } });
        } catch (e) { res.status(500).json({ error: 'Server error' }); }
    });

    return router;
}

// Public router ? NO guard. Active plans, marketing-safe fields. Fail-safe: [] if tables absent.
// deps: { pool }
function makePublicPlansRouter(deps = {}) {
    const express = require('express');
    const router = express.Router();
    const { pool } = deps;
    router.get('/plans', async (req, res) => {
        try {
            const plans = (await pool.query('SELECT * FROM plans WHERE active=true ORDER BY sort_order ASC, id
 ASC')).rows;
            const ents = (await pool.query('SELECT * FROM plan_entitlements')).rows;
            const byId = {}; ents.forEach(e => { byId[e.plan_id] = e; });
            res.json({ plans: plans.map(p => publicPlanView(p, byId[p.id])) });
        } catch (e) {
            // Catalog not provisioned yet (or any read error) -> empty, never 500 on the public surface.
            res.json({ plans: [] });
        }
    });
    return router;
}

module.exports = {
    ALLOWED_CURRENCIES, SUPPORT_LEVELS, ASSIGNMENT_SOURCES, KNOWN_MODULES, PLAN_KEY_RE,
```

```
    parseMoney, validatePlanInput, validateEntitlements, canAssignPlan, isActive,
    deriveCurrentPlan, publicPlanView, planAdminView,
    makePlansRouter, makePublicPlansRouter
};



========================================
FILE: plans_test.js
========================================

/**
 * plans_test.js ? Batch 3 Plans & Pricing tests.
 *   Part A: pure unit tests (validation/entitlements/assignability/views) ? no DB.
 *   Part B: in-process HTTP integration (express + mocked pool + real requireSuperAdmin guard) for
 *           permissions, validation, soft-disable, assignment, audit, and the public surface.
 * Run: node plans_test.js   (exit 0 = all pass)
 */
'use strict';
const http = require('http');
const express = require('express');
const P = require('./plans');
const { makeGuards } = require('./rbac_guards');

let pass = 0, fail = 0;
function ok(name, cond) { if (cond) pass++; else { fail++; console.error('  FAIL:', name); } }

// ---------- Part A: pure ----------
(() => {
  const good = P.validatePlanInput({ plan_key: 'pro_2026', name_ar: '????????', name_en: 'Pro', currency: 'sar
', monthly_price: '99.00', yearly_price: 990, trial_days: 14, sort_order: 1 }, { isCreate: true });
  ok('valid plan passes', good.ok && good.value.currency === 'SAR' && good.value.monthly_price === 99);
  ok('negative price rejected', P.validatePlanInput({ name_ar: 'x', name_en: 'x', currency: 'SAR', monthly_pri
ce: -5 }, { isCreate: true }).ok === false);
  ok('invalid currency rejected', P.validatePlanInput({ name_ar: 'x', name_en: 'x', currency: 'XYZ' }, { isCre
ate: true }).ok === false);
  ok('bad plan_key rejected', P.validatePlanInput({ plan_key: 'Bad Key!', name_ar: 'x', name_en: 'x', currency
: 'SAR' }, { isCreate: true }).ok === false);
  ok('missing currency rejected', P.validatePlanInput({ name_ar: 'x', name_en: 'x' }, { isCreate: true }).ok =
== false);
  ok('missing name rejected', P.validatePlanInput({ name_ar: '', name_en: '', currency: 'SAR' }, { isCreate: t
rue }).ok === false);
  ok('trial out of range rejected', P.validatePlanInput({ name_ar: 'x', name_en: 'x', currency: 'SAR', trial_d
ays: 500 }, { isCreate: true }).ok === false);
  ok('plan_key ignored on update', P.validatePlanInput({ name_ar: 'x', name_en: 'x', currency: 'SAR' }, { isCr
eate: false }).value.plan_key === undefined);
})();

(() => {
  const e = P.validateEntitlements({ max_users: 10, max_branches: '', max_invoices_per_month: null, modules_en
abled: ['lab', 'patients', 'lab'], support_level: 'priority', api_access: '1' });
  ok('entitlements valid + dedup + null unlimited', e.ok && e.value.max_users === 10 && e.value.max_branches =
== null && e.value.max_invoices_per_month === null && e.value.modules_enabled === 'lab,patients' && e.value.su
pport_level === 'priority' && e.value.api_access === true);
```

```
  ok('unknown module rejected', P.validateEntitlements({ modules_enabled: ['lab', 'hackmodule'] }).ok === fals
e);
  ok('negative limit rejected', P.validateEntitlements({ max_users: -1 }).ok === false);
  ok('bad support_level rejected', P.validateEntitlements({ support_level: 'godmode' }).ok === false);
  ok('modules as comma string accepted', P.validateEntitlements({ modules_enabled: 'lab, patients' }).value.mo
dules_enabled === 'lab,patients');
})();

ok('canAssignPlan true for active', P.canAssignPlan({ active: true }) === true);
ok('canAssignPlan true for pg "t"', P.canAssignPlan({ active: 't' }) === true);
ok('canAssignPlan false for disabled', P.canAssignPlan({ active: false }) === false);
ok('canAssignPlan false for null', P.canAssignPlan(null) === false);

(() => {
  const rows = [
    { plan_key: 'old', effective_to: '2026-01-01', assigned_at: '2026-01-01' },
    { plan_key: 'cur', effective_to: null, assigned_at: '2026-05-01' }
  ];
  ok('deriveCurrentPlan picks open assignment', P.deriveCurrentPlan(rows).plan_key === 'cur');
  ok('deriveCurrentPlan null when empty', P.deriveCurrentPlan([]) === null);
})();

(() => {
  const pub = P.publicPlanView({ plan_key: 'pro', name_ar: '?', name_en: 'Pro', currency: 'SAR', monthly_price
: '99.00', yearly_price: '990.00', trial_days: 7, active: true, id: 3 }, { max_users: 5, modules_enabled: 'lab
,patients', support_level: 'priority', api_access: true });
  ok('publicPlanView hides admin fields (id/active)', pub.id === undefined && pub.active === undefined);
  ok('publicPlanView parses money', pub.monthly_price === 99 && pub.yearly_price === 990);
  ok('publicPlanView exposes safe entitlements', pub.entitlements.max_users === 5 && pub.entitlements.modules_
enabled.length === 2 && pub.entitlements.max_invoices_per_month === undefined);
  const adm = P.planAdminView({ plan_key: 'pro', name_ar: '?', name_en: 'Pro', currency: 'SAR', monthly_price:
 '99', yearly_price: '0', active: 't', id: 3 }, { max_invoices_per_month: 100 });
  ok('planAdminView exposes id/active + internal limit', adm.id === 3 && adm.active === true && adm.entitlemen
ts.max_invoices_per_month === 100);
})();

// ---------- Part B: HTTP integration ----------
function makeStore() {
  return {
    plans: [{ id: 1, plan_key: 'standard', name_ar: '??????', name_en: 'Standard', description_ar: '', descrip
tion_en: '', currency: 'SAR', monthly_price: '0.00', yearly_price: '0.00', trial_days: 0, active: true, sort_o
rder: 0, created_at: 'd', updated_at: 'd' },
            { id: 2, plan_key: 'legacy', name_ar: '?????', name_en: 'Legacy', description_ar: '', description_
en: '', currency: 'SAR', monthly_price: '50.00', yearly_price: '0.00', trial_days: 0, active: false, sort_orde
r: 9, created_at: 'd', updated_at: 'd' }],
    ents: { 1: { plan_id: 1, max_users: null, max_branches: null, max_invoices_per_month: null, modules_enable
d: 'dashboard', support_level: 'standard', api_access: false, custom_domain: false } },
    assignments: [],
    seq: 100
  };
}
function fakePool(store) {
  return {
    query(sql, params) {
```

```
      const s = sql.replace(/\s+/g, ' ').trim();
      if (/SELECT \* FROM plans WHERE active=true/.test(s)) return Promise.resolve({ rows: store.plans.filter(
p => p.active) });
      if (/SELECT \* FROM plans ORDER BY/.test(s)) return Promise.resolve({ rows: store.plans.slice() });
      if (/SELECT \* FROM plan_entitlements WHERE plan_id=\$1/.test(s)) return Promise.resolve({ rows: store.e
nts[params[0]] ? [store.ents[params[0]]] : [] });
      if (/SELECT \* FROM plan_entitlements/.test(s)) return Promise.resolve({ rows: Object.values(store.ents)
 });
      if (/SELECT 1 FROM plans WHERE plan_key=\$1/.test(s)) return Promise.resolve({ rows: store.plans.filter(
p => p.plan_key === params[0]).map(() => ({ x: 1 })) });
      if (/INSERT INTO plans/.test(s)) {
        const row = { id: ++store.seq, plan_key: params[0], name_ar: params[1], name_en: params[2], descriptio
n_ar: params[3], description_en: params[4], currency: params[5], monthly_price: String(params[6]), yearly_pric
e: String(params[7]), trial_days: params[8], active: true, sort_order: params[9], created_at: 'd', updated_at:
 'd' };
        store.plans.push(row); return Promise.resolve({ rows: [row] });
      }
      if (/INSERT INTO plan_entitlements/.test(s)) {
        const pid = params[0];
        store.ents[pid] = { plan_id: pid, max_users: params[1], max_branches: params[2], max_invoices_per_mont
h: params[3], modules_enabled: params[4], support_level: params[5], api_access: params[6], custom_domain: para
ms[7] };
        return Promise.resolve({ rows: [store.ents[pid]] });
      }
      if (/UPDATE plans SET name_ar/.test(s)) {
        const key = params[9]; const p = store.plans.find(x => x.plan_key === key); if (!p) return Promise.res
olve({ rows: [] });
        Object.assign(p, { name_ar: params[0], name_en: params[1], currency: params[4], monthly_price: String(
params[5]), yearly_price: String(params[6]) });
        return Promise.resolve({ rows: [p] });
      }
      if (/UPDATE plans SET active=\$1/.test(s)) {
        const p = store.plans.find(x => x.plan_key === params[1]); if (!p) return Promise.resolve({ rows: [] }
);
        p.active = params[0]; return Promise.resolve({ rows: [p] });
      }
      if (/SELECT \* FROM plans WHERE plan_key=\$1/.test(s)) return Promise.resolve({ rows: store.plans.filter
(p => p.plan_key === params[0]) });
      if (/SELECT \* FROM tenant_plan_assignments WHERE tenant_id=\$1/.test(s)) return Promise.resolve({ rows:
 store.assignments.filter(a => a.tenant_id === params[0]) });
      if (/SELECT id FROM tenants WHERE id=\$1/.test(s)) return Promise.resolve({ rows: params[0] === 5 ? [{ i
d: 5 }] : [] });
      if (/UPDATE tenant_plan_assignments SET effective_to=now\(\)/.test(s)) { store.assignments.forEach(a =>
{ if (a.tenant_id === params[0] && a.effective_to == null) a.effective_to = 'closed'; }); return Promise.resol
ve({ rows: [] }); }
      if (/INSERT INTO tenant_plan_assignments/.test(s)) {
        const row = { id: ++store.seq, tenant_id: params[0], plan_key: params[1], assignment_source: params[2]
, assigned_by: params[3], assigned_at: 'now', effective_to: null };
        store.assignments.push(row); return Promise.resolve({ rows: [row] });
      }
      return Promise.resolve({ rows: [] });
    }
  };
}
```

```
function req(port, method, path, body) {
  return new Promise((resolve) => {
    const data = body ? JSON.stringify(body) : null;
    const r = http.request({ host: '127.0.0.1', port, method, path, headers: data ? { 'Content-Type': 'applica
tion/json', 'Content-Length': Buffer.byteLength(data) } : {} }, (res) => {
      let b = ''; res.on('data', d => b += d); res.on('end', () => { let j; try { j = JSON.parse(b); } catch {
 j = {}; } resolve({ status: res.statusCode, json: j }); });
    });
    if (data) r.write(data);
    r.end();
  });
}

(async () => {
  const store = makeStore();
  const pool = fakePool(store);
  const audit = [];
  const guards = makeGuards({ logAudit: (...a) => audit.push(a) });
  const userRef = { user: { id: 1, username: 'op', display_name: 'Op', is_active: 1 } };

  const app = express();
  app.use(express.json());
  app.use((req2, res, next) => { req2.session = { user: userRef.user }; next(); });
  const requireAuth = (req2, res, next) => (req2.session && req2.session.user) ? next() : res.status(401).json
({ error: 'Unauthorized' });
  // admin plans mounted behind the SAME guard chain as server.js
  app.use('/api/super-admin', requireAuth, guards.requireSuperAdmin('op'), P.makePlansRouter({ pool, getActor:
 (r) => r.session.user, logAudit: (...a) => audit.push(a) }));
  // public plans (no guard)
  app.use('/api/public', P.makePublicPlansRouter({ pool }));
  const srv = app.listen(0); await new Promise(r => srv.once('listening', r));
  const port = srv.address().port;

  // ----- permissions -----
  let r1 = await req(port, 'GET', '/api/super-admin/plans');
  ok('super admin lists plans 200', r1.status === 200 && Array.isArray(r1.json.plans));

  userRef.user = { id: 2, username: 'tadmin', role: 'Admin', is_active: 1 };
  let r2 = await req(port, 'GET', '/api/super-admin/plans');
  ok('tenant Admin denied plans 403', r2.status === 403 && r2.json.code === 'SUPER_ADMIN_REQUIRED');

  userRef.user = null;
  let r3 = await req(port, 'GET', '/api/super-admin/plans');
  ok('anonymous denied plans (401 at requireAuth)', r3.status === 401);

  userRef.user = { id: 1, username: 'op', display_name: 'Op', is_active: 1 };

  // ----- create validation -----
  let rneg = await req(port, 'POST', '/api/super-admin/plans', { plan_key: 'bad', name_ar: 'x', name_en: 'x',
currency: 'SAR', monthly_price: -1 });
  ok('create rejects negative price 400', rneg.status === 400);
  let rcur = await req(port, 'POST', '/api/super-admin/plans', { plan_key: 'bad2', name_ar: 'x', name_en: 'x',
 currency: 'ZZZ' });
```

```
  ok('create rejects invalid currency 400', rcur.status === 400);
  let rmod = await req(port, 'POST', '/api/super-admin/plans', { plan_key: 'bad3', name_ar: 'x', name_en: 'x',
 currency: 'SAR', modules_enabled: ['lab', 'evil'] });
  ok('create rejects unknown module 400', rmod.status === 400);

  // ----- create success + audit -----
  audit.length = 0;
  let rok = await req(port, 'POST', '/api/super-admin/plans', { plan_key: 'pro', name_ar: '????????', name_en:
 'Pro', currency: 'SAR', monthly_price: 99, yearly_price: 990, modules_enabled: ['lab', 'patients'], max_users
: 20 });
  ok('create plan 200', rok.status === 200 && rok.json.plan.plan_key === 'pro' && rok.json.plan.active === tru
e);
  ok('create wrote PLAN_CREATE audit', audit.some(a => a[2] === 'PLAN_CREATE'));

  // duplicate key
  let rdup = await req(port, 'POST', '/api/super-admin/plans', { plan_key: 'pro', name_ar: 'x', name_en: 'x',
currency: 'SAR' });
  ok('duplicate plan_key 409', rdup.status === 409 && rdup.json.code === 'PLAN_EXISTS');

  // ----- soft disable does NOT delete -----
  audit.length = 0;
  let rdis = await req(port, 'POST', '/api/super-admin/plans/pro/disable');
  ok('disable plan 200 + active=false', rdis.status === 200 && rdis.json.plan.active === false);
  ok('disabled plan still present (no hard delete)', store.plans.some(p => p.plan_key === 'pro'));
  ok('disable wrote PLAN_DISABLE audit', audit.some(a => a[2] === 'PLAN_DISABLE'));

  // ----- assign tenant plan -----
  let rbadt = await req(port, 'POST', '/api/super-admin/tenants/99/plan', { plan_key: 'standard' });
  ok('assign to missing tenant 404', rbadt.status === 404);
  let rbadp = await req(port, 'POST', '/api/super-admin/tenants/5/plan', { plan_key: 'nope' });
  ok('assign missing plan 404', rbadp.status === 404);
  let rdisP = await req(port, 'POST', '/api/super-admin/tenants/5/plan', { plan_key: 'pro' }); // pro is now d
isabled
  ok('assign disabled plan 409', rdisP.status === 409 && rdisP.json.code === 'PLAN_DISABLED');

  audit.length = 0;
  let rasg = await req(port, 'POST', '/api/super-admin/tenants/5/plan', { plan_key: 'standard' });
  ok('assign active plan 200', rasg.status === 200 && rasg.json.assignment.plan_key === 'standard');
  ok('assign wrote TENANT_PLAN_ASSIGN audit', audit.some(a => a[2] === 'TENANT_PLAN_ASSIGN'));

  // assign again closes the previous open assignment
  let rasg2 = await req(port, 'POST', '/api/super-admin/tenants/5/plan', { plan_key: 'standard' });
  ok('re-assign 200', rasg2.status === 200);
  ok('previous assignment closed (only one open)', store.assignments.filter(a => a.tenant_id === 5 && a.effect
ive_to == null).length === 1);

  // migration source rejected via API (manual/trial only here)
  let rmig = await req(port, 'POST', '/api/super-admin/tenants/5/plan', { plan_key: 'standard', assignment_sou
rce: 'migration' });
  ok('assignment_source=migration rejected 400', rmig.status === 400);

  // ----- get tenant plan -----
  let rget = await req(port, 'GET', '/api/super-admin/tenants/5/plan');
  ok('get tenant plan returns current', rget.status === 200 && rget.json.current && rget.json.current.plan_key
```

```
 === 'standard');

  // ----- public surface: active only, no admin fields -----
  let rpub = await req(port, 'GET', '/api/public/plans');
  ok('public lists only active plans', rpub.status === 200 && rpub.json.plans.every(p => p.active === undefine
d) && rpub.json.plans.some(p => p.plan_key === 'standard') && !rpub.json.plans.some(p => p.plan_key === 'legac
y'));

  srv.close();

  // ----- public fail-safe when catalog absent (pool throws) -----
  const app2 = express();
  app2.use('/api/public', P.makePublicPlansRouter({ pool: { query: () => Promise.reject(new Error('relation "p
lans" does not exist')) } }));
  const srv2 = app2.listen(0); await new Promise(r => srv2.once('listening', r));
  let rpf = await req(srv2.address().port, 'GET', '/api/public/plans');
  ok('public fail-safe returns [] (never 500) when tables absent', rpf.status === 200 && Array.isArray(rpf.jso
n.plans) && rpf.json.plans.length === 0);
  srv2.close();

  // ----- "SUPER_ADMIN_ENABLED=false" simulation: admin routes not mounted -> 404 -----
  const app3 = express();
  app3.use((req3, res) => res.status(404).json({ fell_through: true }));
  const srv3 = app3.listen(0); await new Promise(r => srv3.once('listening', r));
  let rdisabled = await req(srv3.address().port, 'GET', '/api/super-admin/plans');
  ok('disabled (unmounted) admin plans -> 404', rdisabled.status === 404 && rdisabled.json.fell_through === tr
ue);
  srv3.close();

  console.log(`plans_test: ${pass} passed, ${fail} failed`);
  process.exit(fail === 0 ? 0 : 1);
})();
```

```
========================================
FILE: plastic_burns_integration_test.js
========================================

/**
 * plastic_burns_integration_test.js ? Integration test for the Plastic & Burns module HTTP endpoints.
 */
'use strict';

process.env.NODE_ENV = 'staging';

const { spawn } = require('child_process');
const http = require('http');
const assert = require('assert');
const bcrypt = require('bcryptjs');
const { pool } = require('./db_postgres');

const TEST_PORT = 3017;
const TEST_USERNAME = 'burn_doctor';
```

```
const TEST_PASSWORD = 'BURN_PASSWORD_PLACEHOLDER';

let serverProcess;

function makeRequest(method, path, body, headers = {}) {
    return new Promise((resolve, reject) => {
        const payload = body ? JSON.stringify(body) : '';
        const req = http.request({
            hostname: 'localhost',
            port: TEST_PORT,
            path,
            method,
            headers: {
                'Content-Type': 'application/json',
                'Content-Length': Buffer.byteLength(payload),
                ...headers
            }
        }, (res) => {
            let data = '';
            res.on('data', chunk => data += chunk);
            res.on('end', () => {
                try {
                    const parsed = data ? JSON.parse(data) : {};
                    resolve({ statusCode: res.statusCode, headers: res.headers, body: parsed });
                } catch (e) {
                    resolve({ statusCode: res.statusCode, headers: res.headers, rawBody: data });
                }
            });
        });
        req.on('error', reject);
        if (payload) req.write(payload);
        req.end();
    });
}

async function runTests() {
    console.log('--- STARTING PLASTIC & BURNS MODULE INTEGRATION TESTS ---');

    const patientId = 9983;
    const doctorUserId = 9984;
    const client = await pool.connect();

    try {
        console.log('Setting up test data...');
        await client.query("SET app.tenant_id = '1'");

        // Clean up old test data
        await client.query('DELETE FROM burn_assessments WHERE patient_id = $1', [patientId]);
        await client.query('DELETE FROM clinical_photos_meta WHERE patient_id = $1', [patientId]);
        await client.query('DELETE FROM patients WHERE id = $1', [patientId]);
        await client.query('DELETE FROM user_tenants WHERE user_id = $1', [doctorUserId]);
        await client.query('DELETE FROM system_users WHERE id = $1', [doctorUserId]);

        // Insert patient
```

```
        await client.query(
            'INSERT INTO patients (id, name_en, name_ar, tenant_id) VALUES ($1, $2, $3, 1)',
            [patientId, 'Burn Test Patient', '???? ??????']
        );

        // Insert doctor user
        const hashedPassword = await bcrypt.hash(TEST_PASSWORD, 10);
        await client.query(
            'INSERT INTO system_users (id, username, password_hash, display_name, role, speciality, permission
s, is_active) VALUES ($1, $2, $3, $4, $5, $6, $7, 1)',
            [doctorUserId, TEST_USERNAME, hashedPassword, 'Dr. Burn Specialist', 'Doctor', 'PlasticSurgery', '
["patients", "prescriptions"]']
        );

        // Associate doctor with tenant 1
        await client.query(
            'INSERT INTO user_tenants (user_id, tenant_id, is_active) VALUES ($1, 1, true)',
            [doctorUserId]
        );

        console.log('Spawning test server...');
        serverProcess = spawn('node', ['server.js'], {
            env: { ...process.env, PORT: TEST_PORT, NODE_ENV: 'staging', SKIP_DB_INIT: 'true' },
            stdio: 'inherit'
        });

        // Wait 3 seconds for server to boot
        await new Promise(resolve => setTimeout(resolve, 3000));

        console.log('Logging in...');
        const loginRes = await makeRequest('POST', '/api/auth/login', {
            username: TEST_USERNAME,
            password: TEST_PASSWORD
        });

        assert.strictEqual(loginRes.statusCode, 200, 'Login should succeed');
        console.log('? Logged in successfully.');

        // Extract session cookie
        const setCookie = loginRes.headers['set-cookie'];
        assert.ok(setCookie, 'Should receive session cookie');
        const cookie = setCookie[0].split(';')[0];

        // 1. Test POST /api/plastic-burns/assessments (Create burn assessment)
        console.log('Testing create burn assessment...');
        const createRes = await makeRequest('POST', '/api/plastic-burns/assessments', {
            patient_id: patientId,
            weight_kg: 70,
            head_percent: 4.5,
            torso_front_percent: 18,
            torso_back_percent: 9,
            left_arm_percent: 4.5,
            right_arm_percent: 0,
            left_leg_percent: 0,
```

```
            right_leg_percent: 0,
            perineum_percent: 0,
            tbsa_percent: 36,
            parkland_fluid_ml: 10080,
            fluid_first_8h_ml: 5040,
            fluid_next_16h_ml: 5040,
            clinical_notes: 'Partial thickness burns on chest and left arm'
        }, { 'Cookie': cookie });

        assert.strictEqual(createRes.statusCode, 200, 'Should return 200 OK');
        assert.strictEqual(createRes.body.success, true, 'Should return success true');
        assert.ok(createRes.body.id, 'Should return record ID');
        const assessmentId = createRes.body.id;
        console.log(`? Burn assessment created successfully. ID: ${assessmentId}`);

        // 2. Test GET /api/plastic-burns/assessments/patient/:patient_id
        console.log('Testing get patient burn assessments...');
        const getRes = await makeRequest('GET', `/api/plastic-burns/assessments/patient/${patientId}`, null, {
 'Cookie': cookie });
        assert.strictEqual(getRes.statusCode, 200, 'Should return 200 OK');
        assert.strictEqual(getRes.body.length, 1, 'Should return exactly 1 record');
        assert.strictEqual(getRes.body[0].id, assessmentId, 'Record ID should match');
        assert.strictEqual(parseFloat(getRes.body[0].tbsa_percent), 36, 'TBSA percentage should match');
        assert.strictEqual(parseFloat(getRes.body[0].parkland_fluid_ml), 10080, 'Parkland fluid should match')
;
        console.log('? Patient burn assessments retrieved successfully.');

        // 3. Test POST /api/plastic-burns/photos (Create photo meta)
        console.log('Testing register photo metadata...');
        const photoCreateRes = await makeRequest('POST', '/api/plastic-burns/photos', {
            patient_id: patientId,
            body_region: 'Chest',
            description: 'Pre-debridement photo ref #PB-984',
            is_confidential: true
        }, { 'Cookie': cookie });

        assert.strictEqual(photoCreateRes.statusCode, 200, 'Should return 200 OK');
        assert.strictEqual(photoCreateRes.body.success, true, 'Should return success true');
        assert.ok(photoCreateRes.body.id, 'Should return photo record ID');
        const photoId = photoCreateRes.body.id;
        console.log(`? Photo metadata registered successfully. ID: ${photoId}`);

        // 4. Test GET /api/plastic-burns/photos/patient/:patient_id
        console.log('Testing get patient photo registry...');
        const getPhotoRes = await makeRequest('GET', `/api/plastic-burns/photos/patient/${patientId}`, null, {
 'Cookie': cookie });
        assert.strictEqual(getPhotoRes.statusCode, 200, 'Should return 200 OK');
        assert.strictEqual(getPhotoRes.body.length, 1, 'Should return exactly 1 record');
        assert.strictEqual(getPhotoRes.body[0].id, photoId, 'Photo ID should match');
        assert.strictEqual(getPhotoRes.body[0].body_region, 'Chest', 'Body region should match');
        console.log('? Patient photo registry retrieved successfully.');

    } finally {
        console.log('Cleaning up test data...');
```

```
        await client.query("SET app.tenant_id = '1'");
        await client.query('DELETE FROM burn_assessments WHERE patient_id = $1', [patientId]);
        await client.query('DELETE FROM clinical_photos_meta WHERE patient_id = $1', [patientId]);
        await client.query('DELETE FROM patients WHERE id = $1', [patientId]);
        await client.query('DELETE FROM user_tenants WHERE user_id = $1', [doctorUserId]);
        await client.query('DELETE FROM system_users WHERE id = $1', [doctorUserId]);
        client.release();

        if (serverProcess) {
            console.log('Killing test server...');
            serverProcess.kill();
        }
    }

    console.log('? Plastic & Burns Module Integration Tests passed successfully!\n');
}

if (require.main === module) {
    runTests().catch(err => {
        console.error('? Test failed:', err);
        if (serverProcess) serverProcess.kill();
        process.exit(1);
    });
}

module.exports = { runTests };



========================================
FILE: provider_selection_static_test.js
========================================

/**
 * provider_selection_static_test.js
 * ============================================================================
 * Provider Selection Safety Static Analyzer
 * ============================================================================
 * SAFE-BY-DESIGN: Does NOT connect to any database or make external HTTP calls.
 * Analyzes repository documents and code states for provider selection safety.
 * ============================================================================
 */
'use strict';

const fs = require('fs');
const path = require('path');
const assert = require('assert');

console.log('Running Jumanasoft Provider Selection Safety Static Tests...');

const NAMAWEB_DIR = __dirname;
const ROOT_DIR = path.join(NAMAWEB_DIR, '..');
const ADAPTER_FILE = path.join(NAMAWEB_DIR, 'billing_adapter.js');
const DOCS_DIR = path.join(ROOT_DIR, 'docs', 'governance', 'enterprise-engineering-constitution');
```

```
function runProviderTests() {
    let passed = 0;
    let failed = 0;

    function test(name, fn) {
        try {
            fn();
            console.log(`  ? ${name}`);
            passed++;
        } catch (err) {
            console.error(`  ? ${name} failed:`, err.message);
            failed++;
        }
    }

    // 1. Check adapter is mock-only and has default enabled=false
    test('Billing adapter must remain mock-only and default disabled', () => {
        assert.ok(fs.existsSync(ADAPTER_FILE), 'billing_adapter.js is missing.');
        const content = fs.readFileSync(ADAPTER_FILE, 'utf8');
        assert.ok(content.includes("provider_mode: 'mock'"), 'Adapter is not in mock mode.');
        assert.ok(!content.includes('fetch(') && !content.includes('axios.post'), 'Adapter contains external H
TTP calls.');
    });

    // 2. Check no secrets in docs
    test('Docs must not contain any real API keys or connection strings', () => {
        if (!fs.existsSync(DOCS_DIR)) return;
        const files = fs.readdirSync(DOCS_DIR);
        const secretKeywords = ['sk_live_', 'pk_live_', 'moyasar_secret', 'stripe_secret', 'password=123', 'po
stgres://'];

        for (const file of files) {
            if (!file.endsWith('.md')) continue;
            const content = fs.readFileSync(path.join(DOCS_DIR, file), 'utf8');
            for (const kw of secretKeywords) {
                assert.ok(!content.includes(kw), `Document ${file} contains potential secret: ${kw}`);
            }
        }
    });

    // 3. Verify docs contain blocking language
    test('Docs must contain activation blocked warning language', () => {
        if (!fs.existsSync(DOCS_DIR)) return;
        const recomFile = path.join(DOCS_DIR, 'JUMANASOFT_PROVIDER_SELECTION_RECOMMENDATION_AR.md');
        if (fs.existsSync(recomFile)) {
            const content = fs.readFileSync(recomFile, 'utf8');
            assert.ok(content.includes('Blocked') || content.includes('?????'), 'Recommendation document is mi
ssing blocking keywords.');
        }
    });

    console.log(`\nProvider Selection Safety Static Tests Finished: ${passed} passed, ${failed} failed.`);
    if (failed > 0) {
```

```
        process.exit(1);
    }
}

runProviderTests();



========================================
FILE: pulmonology_integration_test.js
========================================

/**
 * pulmonology_integration_test.js ? Integration test for the Pulmonology module HTTP endpoints.
 */
'use strict';

process.env.NODE_ENV = 'staging';

const { spawn } = require('child_process');
const http = require('http');
const assert = require('assert');
const bcrypt = require('bcryptjs');
const { pool } = require('./db_postgres');

const TEST_PORT = 3012;
const TEST_USERNAME = 'pulmo_doctor';
const TEST_PASSWORD = 'PULMO_PASSWORD_PLACEHOLDER';

let serverProcess;

function makeRequest(method, path, body, headers = {}) {
    return new Promise((resolve, reject) => {
        const payload = body ? JSON.stringify(body) : '';
        const req = http.request({
            hostname: 'localhost',
            port: TEST_PORT,
            path,
            method,
            headers: {
                'Content-Type': 'application/json',
                'Content-Length': Buffer.byteLength(payload),
                ...headers
            }
        }, (res) => {
            let data = '';
            res.on('data', chunk => data += chunk);
            res.on('end', () => {
                try {
                    const parsed = data ? JSON.parse(data) : {};
                    resolve({ statusCode: res.statusCode, headers: res.headers, body: parsed });
                } catch (e) {
                    resolve({ statusCode: res.statusCode, headers: res.headers, rawBody: data });
                }
```

```
            });
        });
        req.on('error', reject);
        if (payload) req.write(payload);
        req.end();
    });
}

async function runTests() {
    console.log('--- STARTING PULMONOLOGY MODULE INTEGRATION TESTS ---');

    const patientId = 9989;
    const doctorUserId = 9990;
    const client = await pool.connect();

    try {
        console.log('Setting up test data...');
        await client.query("SET app.tenant_id = '1'");

        // Clean up old test data
        await client.query('DELETE FROM pulmonary_function_tests WHERE patient_id = $1', [patientId]);
        await client.query('DELETE FROM patients WHERE id = $1', [patientId]);
        await client.query('DELETE FROM user_tenants WHERE user_id = $1', [doctorUserId]);
        await client.query('DELETE FROM system_users WHERE id = $1', [doctorUserId]);

        // Insert patient
        await client.query(
            'INSERT INTO patients (id, name_en, name_ar, tenant_id) VALUES ($1, $2, $3, 1)',
            [patientId, 'Pulmo Test Patient', '???? ??? ???????']
        );

        // Insert doctor user
        const hashedPassword = await bcrypt.hash(TEST_PASSWORD, 10);
        await client.query(
            'INSERT INTO system_users (id, username, password_hash, display_name, role, speciality, permission
s, is_active) VALUES ($1, $2, $3, $4, $5, $6, $7, 1)',
            [doctorUserId, TEST_USERNAME, hashedPassword, 'Dr. Pulmo Consultant', 'Doctor', 'Pulmonology', '["
patients", "prescriptions"]']
        );

        // Associate doctor with tenant 1
        await client.query(
            'INSERT INTO user_tenants (user_id, tenant_id, is_active) VALUES ($1, 1, true)',
            [doctorUserId]
        );

        console.log('Spawning test server...');
        serverProcess = spawn('node', ['server.js'], {
            env: { ...process.env, PORT: TEST_PORT, NODE_ENV: 'staging', SKIP_DB_INIT: 'true' }
        });

        // Pipe stdout/stderr for logging
        serverProcess.stdout.on('data', (data) => {
            console.log(`[Server STDOUT] ${data.toString().trim()}`);
```

```
        });
        serverProcess.stderr.on('data', (data) => {
            console.error(`[Server STDERR] ${data.toString().trim()}`);
        });

        // Wait 3 seconds for server to boot
        await new Promise(resolve => setTimeout(resolve, 3000));

        console.log('Logging in...');
        const loginRes = await makeRequest('POST', '/api/auth/login', {
            username: TEST_USERNAME,
            password: TEST_PASSWORD
        });

        assert.strictEqual(loginRes.statusCode, 200, 'Login should succeed');
        console.log('? Logged in successfully.');

        // Extract session cookie
        const setCookie = loginRes.headers['set-cookie'];
        assert.ok(setCookie, 'Should receive session cookie');
        const cookie = setCookie[0].split(';')[0];

        // 1. Test POST /api/pulmonology/pft (Create PFT record)
        console.log('Testing create PFT record...');
        const pftCreateRes = await makeRequest('POST', '/api/pulmonology/pft', {
            patient_id: patientId,
            fev1: 3.20,
            fvc: 4.00,
            fev1_fvc_ratio: 80.0,
            pef: 450.0,
            interpretation: 'Normal',
            notes: 'Healthy lung capacity'
        }, { 'Cookie': cookie });

        assert.strictEqual(pftCreateRes.statusCode, 200, 'Should return 200 OK');
        assert.strictEqual(pftCreateRes.body.success, true, 'Should return success true');
        assert.ok(pftCreateRes.body.id, 'Should return PFT record ID');
        const pftId = pftCreateRes.body.id;
        console.log(`? PFT record created successfully. ID: ${pftId}`);

        // 2. Test GET /api/pulmonology/pft/patient/:patient_id
        console.log('Testing get patient PFT reports...');
        const pftGetRes = await makeRequest('GET', `/api/pulmonology/pft/patient/${patientId}`, null, { 'Cooki
e': cookie });
        assert.strictEqual(pftGetRes.statusCode, 200, 'Should return 200 OK');
        assert.strictEqual(pftGetRes.body.length, 1, 'Should return exactly 1 PFT record');
        assert.strictEqual(pftGetRes.body[0].id, pftId, 'PFT record ID should match');
        assert.strictEqual(parseFloat(pftGetRes.body[0].fev1), 3.20, 'FEV1 should match');
        assert.strictEqual(parseFloat(pftGetRes.body[0].fvc), 4.00, 'FVC should match');
        assert.strictEqual(parseFloat(pftGetRes.body[0].fev1_fvc_ratio), 0.8, 'FEV1/FVC ratio should match');
        console.log('? Patient PFT reports retrieved successfully.');

    } finally {
        console.log('Cleaning up test data...');
```

```
        await client.query("SET app.tenant_id = '1'");
        await client.query('DELETE FROM pulmonary_function_tests WHERE patient_id = $1', [patientId]);
        await client.query('DELETE FROM patients WHERE id = $1', [patientId]);
        await client.query('DELETE FROM user_tenants WHERE user_id = $1', [doctorUserId]);
        await client.query('DELETE FROM system_users WHERE id = $1', [doctorUserId]);
        client.release();

        if (serverProcess) {
            console.log('Killing test server...');
            serverProcess.kill();
        }
    }

    console.log('? Pulmonology Module Integration Tests passed successfully!\n');
}

if (require.main === module) {
    runTests().catch(err => {
        console.error('? Test failed:', err);
        if (serverProcess) serverProcess.kill();
        process.exit(1);
    });
}

module.exports = { runTests };



========================================
FILE: quality_gate_scanner.js
========================================

const { execSync } = require('child_process');
const fs = require('fs');
const path = require('path');

const RED = '\x1b[31m';
const GREEN = '\x1b[32m';
const YELLOW = '\x1b[33m';
const BLUE = '\x1b[34m';
const RESET = '\x1b[0m';
const BOLD = '\x1b[1m';

console.log(`${BOLD}${BLUE}============================================================${RESET}`);
console.log(`${BOLD}${BLUE}  ????? ?????? ???????? - ??? ???????? ??????? (DevSecOps)${RESET}`);
console.log(`${BOLD}${BLUE}  Automated Quality Gate Scanner - jumanaMedical ERP${RESET}`);
console.log(`${BOLD}${BLUE}============================================================${RESET}\n`);

let hasFailures = false;

// 1. Mojibake Scan
console.log(`${BOLD}[1] ??? ????? ??????? ???? Mojibake (UTF-8 Integrity)...${RESET}`);
const targetDirs = [__dirname, path.join(__dirname, '../docs')];
const forbiddenPatterns = [
```

```
  { pattern: /Ø/g, name: 'Latin-1 O-slash' },
  { pattern: /Ù/g, name: 'Latin-1 U-grave' },
  { pattern: /ï»¿/g, name: 'UTF-8 BOM' }
];

function scanDirForMojibake(dir) {
  if (!fs.existsSync(dir)) return;
  const items = fs.readdirSync(dir);
  for (const item of items) {
    const fullPath = path.join(dir, item);
    const relativePath = path.relative(path.join(__dirname, '..'), fullPath);
    const pathSegments = relativePath.split(/[\\/]/);

    // Ignore version control, node_modules, temp caches, and any documentation folders/files
    if (pathSegments.includes('node_modules') || pathSegments.includes('.git') || pathSegments.includes('.clau
de') || pathSegments.includes('.cache') || pathSegments.includes('_archive') || pathSegments.includes('docs'))
 continue;

    const stat = fs.statSync(fullPath);
    if (stat.isDirectory()) {
      scanDirForMojibake(fullPath);
    } else if (stat.isFile() && (item.endsWith('.md') || item.endsWith('.js') || item.endsWith('.html'))) {
      if (item === 'quality_gate_scanner.js' || item === 'run_all_tests.js') continue;

      const content = fs.readFileSync(fullPath, 'utf8');
      forbiddenPatterns.forEach(p => {
        if (p.pattern.test(content)) {
          console.error(`  ${RED}? Mojibake Found:${RESET} ${item} contains ${p.name}`);
          hasFailures = true;
        }
      });
    }
  }
}
targetDirs.forEach(scanDirForMojibake);
if (!hasFailures) {
  console.log(`  ${GREEN}? ??? ??????? ???? ???????.${RESET}`);
}

// 2. Secrets Leak Scan
console.log(`\n${BOLD}[2] ??? ????? ???????? ??????? ???? ????? ??????? (Secrets Leak Check)...${RESET}`);
const secretPatterns = [
  { pattern: /password\s*=\s*['"][a-zA-Z0-9_]{6,}['"]/gi, name: 'High entropy password assignment' },
  { pattern: /api_key\s*=\s*['"][a-zA-Z0-9_]{10,}['"]/gi, name: 'API Key assignment' },
  { pattern: /BEGIN PRIVATE KEY/g, name: 'Unencrypted Private Key' }
];

function scanDirForSecrets(dir) {
  if (!fs.existsSync(dir)) return;
  const items = fs.readdirSync(dir);
  for (const item of items) {
    const fullPath = path.join(dir, item);
    const relativePath = path.relative(path.join(__dirname, '..'), fullPath);
    const pathSegments = relativePath.split(/[\\/]/);
```

```
    if (pathSegments.includes('node_modules') || pathSegments.includes('.git') || pathSegments.includes('.clau
de') || pathSegments.includes('.cache') || pathSegments.includes('.env') || pathSegments.includes('docs')) con
tinue;
    const stat = fs.statSync(fullPath);
    if (stat.isDirectory()) {
      scanDirForSecrets(fullPath);
    } else if (stat.isFile() && (item.endsWith('.js') || item.endsWith('.html') || item.endsWith('.json'))) {
      if (item === 'quality_gate_scanner.js' || item === 'run_all_tests.js') continue;

      const content = fs.readFileSync(fullPath, 'utf8');
      secretPatterns.forEach(p => {
        if (p.pattern.test(content)) {
          console.error(`  ${RED}? Secret Leak Found:${RESET} ${item} contains pattern matching "${p.name}"`);
          hasFailures = true;
        }
      });
    }
  }
}
targetDirs.forEach(scanDirForSecrets);
if (!hasFailures) {
  console.log(`  ${GREEN}? ??? ??????? ???? ??????? ??? ???? ??????? ?? ????? ???????.${RESET}`);
}

// 3. Execution of Unit Tests
console.log(`\n${BOLD}[3] ????? ???????? ??? ?????????? (Tenant Isolation)...${RESET}`);
try {
  const testRunner = path.join(__dirname, 'cross_tenant_leak_test.js');
  if (fs.existsSync(testRunner)) {
    console.log(`  ????? ?????? ??? ?????????? ?????????? (cross_tenant_leak_test)...`);
    execSync(`node "${testRunner}"`, { stdio: 'inherit' });
    console.log(`  ${GREEN}? ?????? ??? ?????????? ?? ?????.${RESET}`);
  }
} catch (e) {
  console.error(`  ${RED}? ??? ?????? ??? ??????????.${RESET}`);
  hasFailures = true;
}

// 4. Overall Test Harness
try {
  console.log(`\n${BOLD}[4] ????? ?????? ??????? ?? 86 ?????? ???? (Test Suite)...${RESET}`);
  const fallbackRunner = path.join(__dirname, 'run_all_tests.js');
  if (fs.existsSync(fallbackRunner)) {
    execSync(`node "${fallbackRunner}"`, { stdio: 'inherit' });
  } else {
    console.log(`  ${YELLOW}? ???? ?????? ??????? ???? ???? ??? ????????.${RESET}`);
  }
} catch (e) {
  console.error(`  ${RED}? ???? ???????? ?????? ???????.${RESET}`);
  hasFailures = true;
}

console.log(`\n${BOLD}${BLUE}============================================================${RESET}`);
```

```
if (hasFailures) {
  console.error(`${BOLD}${RED}  ? ??????? ????????: ??? ???? ?????? (Quality Gates FAILED)${RESET}`);
  console.log(`${BOLD}${BLUE}============================================================${RESET}`);
  process.exit(1);
} else {
  console.log(`${BOLD}${GREEN}  ? ??????? ????????: ????? ???? ?????? ????? (Quality Gates PASSED)${RESET}`);
  console.log(`${BOLD}${BLUE}============================================================${RESET}`);
  process.exit(0);
}
```

```
========================================
FILE: rare_specialized_engine.js
========================================

// ============================================================================
// Rare & Super-Specialized Domains ? PURE ENGINE (no DB, no I/O)
// ----------------------------------------------------------------------------
// Converts the 12 Wave 10 Enterprise_Blueprint_2026 "rare" department specs into
// real, unit-testable business-rule gates. Mirrors the e18_hr_engine.js pattern:
// pure functions only, so they can be unit-tested without a database and reused
// verbatim by server.js routes later once the surrounding schema/UI is built.
//
// GAP NOTE (see Enterprise_Blueprint_2026/IMPLEMENTATION_STATUS.md): none of these
// 12 departments have dedicated DB tables yet (genuine classification C). This
// engine implements ONLY the core safety/compliance decision logic explicitly
// described in each blueprint's "Constraint" ? no new migrations were written,
// since that requires schema design + explicit user sign-off per
// .agents/skills/nm_enterprise_implementation/SKILL.md.
//
// HARD rules honoured (mirrors EPIC_BUILD_CONVENTIONS):
//  - fail-CLOSED: missing/incomplete input never yields a falsely-reassuring
//    "safe"/"approved" result ? it yields a block/hold requiring more data.
//  - Authority decisions (block/hold/override) are computed here, not on the client.
// ============================================================================

'use strict';

// 1) Space & Dive Medicine ? decompression-sickness risk modeling.
function dcsRiskAssessment({ hasNeuroSigns, rapidAscent, depthMeters, diveTimeMinutes } = {}) {
    if (hasNeuroSigns === undefined || hasNeuroSigns === null) {
        return { severity: 'unknown', requiresImmediateRecompression: true, reason: 'incomplete data ? fail-cl
osed' };
    }
    if (hasNeuroSigns === true) {
        return { severity: 'severe', requiresImmediateRecompression: true, reason: 'neurological signs present
' };
    }
    if (rapidAscent === true && Number(depthMeters) > 30) {
        return { severity: 'moderate', requiresImmediateRecompression: false, reason: 'rapid ascent from signi
ficant depth ? monitor closely' };
    }
    return { severity: 'low', requiresImmediateRecompression: false, reason: 'no red-flag criteria met' };
```

```
}

// 2) Complex Sleep Disorders ? REM Behavior Disorder pattern -> mandatory neurology referral.
function classifyParasomnia({ remAtoniaLoss, dreamEnactmentBehavior } = {}) {
    if (remAtoniaLoss === true && dreamEnactmentBehavior === true) {
        return { pattern: 'REM_Behavior_Disorder', requiresNeurologyReferral: true };
    }
    return { pattern: 'unclassified', requiresNeurologyReferral: false };
}

// 3) Epilepsy Monitoring Unit ? surgical candidacy requires concordant localization (>=2 modalities).
function verifySurgicalCandidacy({ eegOnsetZone, imagingOnsetZone } = {}) {
    if (!eegOnsetZone || !imagingOnsetZone) {
        return { candidacyStatus: 'hold', reason: 'incomplete localization data ? fail-closed' };
    }
    if (eegOnsetZone !== imagingOnsetZone) {
        return { candidacyStatus: 'hold', reason: `discordant localization: EEG=${eegOnsetZone} vs imaging=${i
magingOnsetZone}` };
    }
    return { candidacyStatus: 'eligible', reason: `concordant localization: ${eegOnsetZone}` };
}

// 4) Advanced Stem Cell Therapy ? protocol compliance gate (hard block outside approved trial/compassionate-u
se).
function verifyProtocolCompliance({ approvedTrialId, compassionateUseApproved } = {}) {
    const approved = Boolean(approvedTrialId) || compassionateUseApproved === true;
    return {
        allowed: approved,
        reason: approved ? 'approved trial or compassionate-use protocol confirmed' : 'BLOCKED ? no approved t
rial/compassionate-use protocol on record'
    };
}

// 5) Fetal Surgery ? hard block without full multi-disciplinary consensus (MFM + Surgeon + Neonatology + Ethi
cs).
function verifyFetalSurgeryConsensus({ mfmSignoff, fetalSurgeonSignoff, neonatologySignoff, ethicsSignoff } =
{}) {
    const allSigned = mfmSignoff === true && fetalSurgeonSignoff === true && neonatologySignoff === true && et
hicsSignoff === true;
    return {
        allowed: allSigned,
        reason: allSigned ? 'full multi-disciplinary consensus documented' : 'BLOCKED ? missing one or more re
quired sign-offs (MFM/Fetal Surgeon/Neonatology/Ethics)'
    };
}

// 6) Fetal Medicine ? rare syndromic pattern -> mandatory genetics consult before counseling.
function recognizeSyndromicPattern({ chdDetected, skeletalAnomalyDetected } = {}) {
    if (chdDetected === true && skeletalAnomalyDetected === true) {
        return { pattern: 'possible_syndromic_combination', requiresGeneticsConsult: true };
    }
    return { pattern: 'isolated_or_none', requiresGeneticsConsult: false };
}
```

```
// 7) Deep Brain Stimulation ? trajectory must respect a minimum vascular safety margin.
function verifyTrajectorySafety({ distanceToVesselMm, minSafeMarginMm = 3 } = {}) {
    if (distanceToVesselMm === undefined || distanceToVesselMm === null || !Number.isFinite(Number(distanceToV
esselMm))) {
        return { allowed: false, reason: 'incomplete vascular-proximity data ? fail-closed' };
    }
    const safe = Number(distanceToVesselMm) >= minSafeMarginMm;
    return {
        allowed: safe,
        reason: safe ? `trajectory clears ${minSafeMarginMm}mm safety margin` : `BLOCKED ? trajectory within $
{distanceToVesselMm}mm of a major vessel (min margin ${minSafeMarginMm}mm)`
    };
}

// 8) Nuclear Medicine Therapy ? inpatient isolation-room gate before therapeutic dose administration.
function verifyIsolationGate({ requiresIsolation, isolationRoomAvailable } = {}) {
    if (requiresIsolation !== true) {
        return { allowed: true, reason: 'isotope/dose does not require inpatient isolation' };
    }
    return {
        allowed: isolationRoomAvailable === true,
        reason: isolationRoomAvailable === true ? 'isolation room confirmed available' : 'BLOCKED ? isolation
required but no room confirmed available'
    };
}

// 9) Cryotherapy / Cryosurgery ? ice-ball margin coverage verification.
function verifyMarginCoverage({ coveragePercent, requiredPercent = 100 } = {}) {
    if (coveragePercent === undefined || coveragePercent === null || !Number.isFinite(Number(coveragePercent))
) {
        return { complete: false, requiresAdditionalCycle: true, reason: 'incomplete imaging data ? fail-close
d' };
    }
    const complete = Number(coveragePercent) >= requiredPercent;
    return {
        complete,
        requiresAdditionalCycle: !complete,
        reason: complete ? 'full safety-margin coverage achieved' : `incomplete coverage (${coveragePercent}%
of required ${requiredPercent}%) ? additional freeze cycle required`
    };
}

// 10) Confocal Laser Endomicroscopy ? AI reads are always advisory-only, never a final diagnosis.
function classifyConfocalPattern({ suspiciousPattern } = {}) {
    return {
        pattern: suspiciousPattern ? 'suspicious' : 'unremarkable',
        recommendation: suspiciousPattern ? 'targeted biopsy recommended' : 'no targeted biopsy indicated',
        label: 'ADVISORY ONLY ? not a diagnosis; histopathology confirmation required'
    };
}

// 11) Pharmacogenomics ? known high-risk genotype-drug pairing gate (e.g. HLA-B*15:02 + carbamazepine).
const HIGH_RISK_GENOTYPE_DRUG_PAIRS = [
    { allele: 'HLA-B*15:02', drug: 'carbamazepine', risk: 'Stevens-Johnson Syndrome' },
```

```
    { allele: 'HLA-B*15:02', drug: 'phenytoin', risk: 'Stevens-Johnson Syndrome' },
    { allele: 'HLA-B*58:01', drug: 'allopurinol', risk: 'Severe cutaneous adverse reaction' }
];
function checkGenotypeDrugInteraction({ allele, drug, physicianOverride } = {}) {
    const match = HIGH_RISK_GENOTYPE_DRUG_PAIRS.find(
        p => p.allele === allele && p.drug && drug && p.drug.toLowerCase() === String(drug).toLowerCase()
    );
    if (!match) {
        return { allowed: true, reason: 'no known high-risk genotype-drug pairing found' };
    }
    if (physicianOverride === true) {
        return { allowed: true, reason: `high-risk pairing (${match.risk}) present but physician override + co
unseling documented`, overridden: true };
    }
    return { allowed: false, reason: `BLOCKED ? known high-risk pairing: ${allele} + ${drug} (${match.risk});
requires physician override + patient counseling` };
}

// 12) Nanomedicine & Micro-Robotics ? mirrors stem-cell protocol-compliance governance (research-track only).
function verifyNanomedicineProtocolCompliance({ approvedTrialId } = {}) {
    const approved = Boolean(approvedTrialId);
    return {
        allowed: approved,
        reason: approved ? 'approved IRB research protocol confirmed' : 'BLOCKED ? nanomedicine/micro-robotic
therapy requires an approved IRB research protocol'
    };
}

module.exports = {
    dcsRiskAssessment,
    classifyParasomnia,
    verifySurgicalCandidacy,
    verifyProtocolCompliance,
    verifyFetalSurgeryConsensus,
    recognizeSyndromicPattern,
    verifyTrajectorySafety,
    verifyIsolationGate,
    verifyMarginCoverage,
    classifyConfocalPattern,
    checkGenotypeDrugInteraction,
    verifyNanomedicineProtocolCompliance,
    HIGH_RISK_GENOTYPE_DRUG_PAIRS
};



========================================
FILE: rare_specialized_engine_unit_test.js
========================================

// ============================================================================
// rare_specialized_engine_unit_test.js ? DB-free unit tests for the 12 Wave 10
// (Rare & Super-Specialized) safety/compliance gates. Each test mirrors the exact
// "Testing & QA" acceptance case stated in the corresponding blueprint file under
```

```
// Enterprise_Blueprint_2026/domains/rare/*.md.
// ============================================================================
'use strict';
const assert = require('assert');
const eng = require('./rare_specialized_engine');

function runTests() {
    // 1) Space & Dive Medicine ? neuro signs must force immediate recompression.
    {
        const r = eng.dcsRiskAssessment({ hasNeuroSigns: true, rapidAscent: true, depthMeters: 40 });
        assert.strictEqual(r.severity, 'severe');
        assert.strictEqual(r.requiresImmediateRecompression, true);
    }
    {
        const r = eng.dcsRiskAssessment({ hasNeuroSigns: false, rapidAscent: false, depthMeters: 10 });
        assert.strictEqual(r.requiresImmediateRecompression, false);
    }

    // 2) Complex Sleep ? REM Behavior Disorder pattern must trigger neurology referral.
    {
        const r = eng.classifyParasomnia({ remAtoniaLoss: true, dreamEnactmentBehavior: true });
        assert.strictEqual(r.pattern, 'REM_Behavior_Disorder');
        assert.strictEqual(r.requiresNeurologyReferral, true);
    }
    {
        const r = eng.classifyParasomnia({ remAtoniaLoss: false, dreamEnactmentBehavior: false });
        assert.strictEqual(r.requiresNeurologyReferral, false);
    }

    // 3) Epilepsy Monitoring ? discordant EEG/imaging localization must hold candidacy.
    {
        const r = eng.verifySurgicalCandidacy({ eegOnsetZone: 'left_temporal', imagingOnsetZone: 'right_fronta
l' });
        assert.strictEqual(r.candidacyStatus, 'hold');
    }
    {
        const r = eng.verifySurgicalCandidacy({ eegOnsetZone: 'left_temporal', imagingOnsetZone: 'left_tempora
l' });
        assert.strictEqual(r.candidacyStatus, 'eligible');
    }

    // 4) Stem Cell Therapy ? must hard-block outside an approved trial/compassionate-use.
    {
        const r = eng.verifyProtocolCompliance({});
        assert.strictEqual(r.allowed, false);
    }
    {
        const r = eng.verifyProtocolCompliance({ approvedTrialId: 'TRIAL-001' });
        assert.strictEqual(r.allowed, true);
    }

    // 5) Fetal Surgery ? must hard-block without full 4-party consensus (incl. Ethics).
    {
        const r = eng.verifyFetalSurgeryConsensus({ mfmSignoff: true, fetalSurgeonSignoff: true, neonatologySi
```

```
gnoff: true, ethicsSignoff: false });
        assert.strictEqual(r.allowed, false);
    }
    {
        const r = eng.verifyFetalSurgeryConsensus({ mfmSignoff: true, fetalSurgeonSignoff: true, neonatologySi
gnoff: true, ethicsSignoff: true });
        assert.strictEqual(r.allowed, true);
    }

    // 6) Fetal Medicine ? complex CHD + skeletal anomaly must require genetics consult.
    {
        const r = eng.recognizeSyndromicPattern({ chdDetected: true, skeletalAnomalyDetected: true });
        assert.strictEqual(r.requiresGeneticsConsult, true);
    }

    // 7) DBS ? trajectory within unsafe distance of a vessel must be blocked.
    {
        const r = eng.verifyTrajectorySafety({ distanceToVesselMm: 1, minSafeMarginMm: 3 });
        assert.strictEqual(r.allowed, false);
    }
    {
        const r = eng.verifyTrajectorySafety({ distanceToVesselMm: 5, minSafeMarginMm: 3 });
        assert.strictEqual(r.allowed, true);
    }

    // 8) Nuclear Medicine Therapy ? isolation-requiring dose without a room must be blocked.
    {
        const r = eng.verifyIsolationGate({ requiresIsolation: true, isolationRoomAvailable: false });
        assert.strictEqual(r.allowed, false);
    }
    {
        const r = eng.verifyIsolationGate({ requiresIsolation: true, isolationRoomAvailable: true });
        assert.strictEqual(r.allowed, true);
    }

    // 9) Cryotherapy ? 80% margin coverage must require an additional freeze cycle.
    {
        const r = eng.verifyMarginCoverage({ coveragePercent: 80, requiredPercent: 100 });
        assert.strictEqual(r.requiresAdditionalCycle, true);
    }
    {
        const r = eng.verifyMarginCoverage({ coveragePercent: 100, requiredPercent: 100 });
        assert.strictEqual(r.requiresAdditionalCycle, false);
    }

    // 10) Confocal Endomicroscopy ? result must always be labeled advisory-only.
    {
        const r = eng.classifyConfocalPattern({ suspiciousPattern: true });
        assert.ok(r.label.includes('ADVISORY ONLY'));
        assert.strictEqual(r.pattern, 'suspicious');
    }

    // 11) Pharmacogenomics ? HLA-B*15:02 + carbamazepine must be blocked without override.
    {
```

```
        const r = eng.checkGenotypeDrugInteraction({ allele: 'HLA-B*15:02', drug: 'carbamazepine' });
        assert.strictEqual(r.allowed, false);
    }
    {
        const r = eng.checkGenotypeDrugInteraction({ allele: 'HLA-B*15:02', drug: 'carbamazepine', physicianOv
erride: true });
        assert.strictEqual(r.allowed, true);
    }
    {
        const r = eng.checkGenotypeDrugInteraction({ allele: 'HLA-A*01:01', drug: 'ibuprofen' });
        assert.strictEqual(r.allowed, true);
    }

    // 12) Nanomedicine ? must hard-block outside an approved IRB research protocol.
    {
        const r = eng.verifyNanomedicineProtocolCompliance({});
        assert.strictEqual(r.allowed, false);
    }
    {
        const r = eng.verifyNanomedicineProtocolCompliance({ approvedTrialId: 'NANO-TRIAL-01' });
        assert.strictEqual(r.allowed, true);
    }

    console.log('rare_specialized_engine_unit_test: all 12 department gates verified (24 assertions)');
}

if (require.main === module) {
    runTests();
}

module.exports = { runTests };
```

```
========================================
FILE: rare_specialized_routes_static_test.js
========================================

/**
 * rare_specialized_routes_static_test.js ? DB-free static check verifying the 12
 * Wave 10 (Rare & Super-Specialized) stateless decision-support routes are wired
 * correctly in server.js: route exists, is auth+role guarded, and calls the
 * matching rare_specialized_engine function. No DB/server required.
 */
'use strict';
const fs = require('fs');
const path = require('path');
const assert = require('assert');

const src = fs.readFileSync(path.join(__dirname, 'server.js'), 'utf8');

const ROUTES = [
    { route: '/api/space-dive/dcs-risk', fn: 'dcsRiskAssessment' },
    { route: '/api/complex-sleep/parasomnia-classification', fn: 'classifyParasomnia' },
```

```
    { route: '/api/epilepsy-monitoring/localization-analysis', fn: 'verifySurgicalCandidacy' },
    { route: '/api/stem-cell/protocol-verification', fn: 'verifyProtocolCompliance' },
    { route: '/api/fetal-surgery/consensus-log', fn: 'verifyFetalSurgeryConsensus' },
    { route: '/api/fetal-medicine/anomaly-pattern', fn: 'recognizeSyndromicPattern' },
    { route: '/api/dbs/trajectory-planning', fn: 'verifyTrajectorySafety' },
    { route: '/api/nuclear-therapy/dose-calculation', fn: 'verifyIsolationGate' },
    { route: '/api/cryotherapy/margin-verification', fn: 'verifyMarginCoverage' },
    { route: '/api/confocal-endomicroscopy/pattern-analysis', fn: 'classifyConfocalPattern' },
    { route: '/api/pharmacogenomics/prescribing-alert', fn: 'checkGenotypeDrugInteraction' },
    { route: '/api/nanomedicine/protocol-verification', fn: 'verifyNanomedicineProtocolCompliance' },
];

function runTests() {
    assert.ok(src.includes("require('./rare_specialized_engine')"), 'server.js must require rare_specialized_e
ngine');

    for (const r of ROUTES) {
        const declStr = `app.post('${r.route}'`;
        const routeIdx = src.indexOf(declStr);
        assert.ok(routeIdx !== -1, `route must exist: ${r.route}`);

        const routeEnd = src.indexOf('});', routeIdx);
        const block = src.slice(routeIdx, routeEnd === -1 ? routeIdx + 300 : routeEnd);
        assert.ok(block.includes('requireAuth'), `${r.route} must require auth`);
        assert.ok(block.includes("requireRole('patients', 'prescriptions')"), `${r.route} must require patient
s/prescriptions role`);
        assert.ok(block.includes(`rareEngine.${r.fn}(`), `${r.route} must call rareEngine.${r.fn}`);
    }

    console.log(`rare_specialized_routes_static_test: all ${ROUTES.length} Wave 10 routes verified`);
}

if (require.main === module) {
    runTests();
}

module.exports = { runTests };
```

```
========================================
FILE: rbac.js
========================================

/**
 * rbac.js ? E-X3 RBAC matrix middleware (additive, non-breaking).
 *
 * Provides makeRequirePermission(deps) -> requirePermission(key) middleware that enforces the
 * DB-backed role_permissions matrix (the L6 gap: previously system_users.permissions / user_permissions
 * were persisted but NEVER enforced). It is purely ADDITIVE ? it does not modify or replace the existing
 * in-code requireRole / ROLE_PERMISSIONS. Mount it as an extra middleware on protected routes.
 *
 * Behavior (in order):
 *   1) Unauthenticated  -> 401 (mirror requireAuth shape).
```

```
 *   2) Admin / role with '*' in legacy ROLE_PERMISSIONS -> next() (short-circuit, mirrors server.js:219).
 *   3) DB matrix HIT: a role_permissions row for (tenant_id, role, permission_key) exists -> next().
 *      DB matrix explicit-miss (role HAS rows but not this key) -> 403 {error:'Access denied'}.
 *   4) FALLBACK (non-breaking): role has NO matrix rows at all for this tenant -> defer to the supplied
 *      roleFallback(req) (wrapping the legacy requireRole logic). If no fallback provided -> fail-closed
 *      403 (secure by default). The production caller always supplies a roleFallback, so seeded
 *      deployments behave identically; only callers that forget a fallback are denied rather than opened.
 *   5) Any DB error -> fail-closed to the fallback (never throw into the route).
 *
 * deps: { pool, getRequestTenantContext, roleFallback }
 *   - pool: pg pool (tenant-scoped via the db_postgres AsyncLocalStorage wrapper).
 *   - getRequestTenantContext(req) -> { tenantId }.
 *   - roleFallback(req): optional (req)=>boolean ? legacy allow decision when matrix is empty for the role.
 */
'use strict';

function makeRequirePermission(deps) {
    const { pool, getRequestTenantContext, roleFallback } = deps || {};
    if (!pool || typeof getRequestTenantContext !== 'function') {
        throw new Error('rbac.makeRequirePermission requires { pool, getRequestTenantContext }');
    }

    // Legacy Admin/'*' short-circuit set ? kept here so we never need to import ROLE_PERMISSIONS.
    // Admin is the canonical superuser role in server.js ROLE_PERMISSIONS ('Admin': '*').
    // Mirror the canonical set EXACTLY: only 'Admin' maps to '*' (server.js:204). Broadening this to
    // case variants / 'administrator' would over-trust roles the canonical requireRole never treats as wildca
rd.
    const ADMIN_ROLES = new Set(['Admin']);

    return function requirePermission(key) {
        if (!key || typeof key !== 'string') {
            throw new Error('requirePermission(key) requires a non-empty string key');
        }
        return async function (req, res, next) {
            try {
                if (!req.session || !req.session.user) {
                    return res.status(401).json({ error: 'Unauthorized' });
                }
                const role = req.session.user.role;

                // (2) Admin short-circuit (mirrors ROLE_PERMISSIONS '*').
                if (ADMIN_ROLES.has(role)) return next();

                const { tenantId } = getRequestTenantContext(req);

                // Resolve the role's matrix rows for this tenant in ONE query.
                // tenant-scoped: role_permissions is under FORCE RLS; the pool wrapper binds app.tenant_id,
                // and we also filter explicitly as defense-in-depth.
                const { rows } = await pool.query(
                    'SELECT permission_key FROM role_permissions WHERE role = $1 AND ($2::int IS NULL OR tenan
t_id = $2)',
                    [role, tenantId || null]
                );
```

```
                if (rows.length === 0) {
                    // (4) No matrix rows for this role/tenant -> non-breaking fallback to legacy behavior.
                    if (typeof roleFallback === 'function') {
                        return roleFallback(req) ? next() : res.status(403).json({ error: 'Access denied' });
                    }
                    // Secure-by-default: with no legacy fallback AND no matrix rows, DENY (fail-closed).
                    // The production caller always supplies a roleFallback, so this only hardens future calle
rs.
                    return res.status(403).json({ error: 'Access denied' });
                }

                // (3) Matrix present for this role: enforce strictly.
                const granted = rows.some(r => r.permission_key === key);
                if (granted) return next();
                return res.status(403).json({ error: 'Access denied' });
            } catch (e) {
                // (5) Fail-closed to fallback; never leak details, never throw into the route.
                console.error('requirePermission error:', e.message);
                if (typeof roleFallback === 'function') {
                    return roleFallback(req) ? next() : res.status(403).json({ error: 'Access denied' });
                }
                return res.status(403).json({ error: 'Access denied' });
            }
        };
    };
}

module.exports = { makeRequirePermission };
```

```
========================================
FILE: rbac_guards.js
========================================

/**
 * rbac_guards.js ? Jumanasoft SaaS unified Auth/RBAC guards (Batch 2).
 *
 * One named, tested source of truth for the access-control checks that were previously
 * duplicated inline across server.js (`role !== 'Admin'`). Behavior-preserving: each guard
 * denies with the same HTTP status (401/403) and writes the same audit event used before,
 * just centralized so it can't drift between routes.
 *
 * Identity is ALWAYS read from the session (never from the request body).
 * Super Admin identity reuses super_admin.js (single definition; no duplicate allowlist logic):
 *   - tenant admin (role==='Admin') and super admin (env allowlist) are INDEPENDENT dimensions.
 *   - a tenant Admin is NOT a super admin -> no privilege escalation. Fail-closed everywhere.
 *
 * Pure predicate isTenantAdmin + makeGuards(deps) factory. No DB, no DDL.
 */
'use strict';

const { isSuperAdmin, parseAllowlist } = require('./super_admin');
```

```
// Pure: tenant admin iff the session user's role is exactly 'Admin' (ROLE_PERMISSIONS['Admin']='*').
function isTenantAdmin(user) {
    return !!(user && user.role === 'Admin');
}

function clientIpOf(req) {
    return (req && (req.headers && req.headers['x-forwarded-for']))
        || (req && req.connection && req.connection.remoteAddress)
        || (req && req.ip) || '';
}

/**
 * makeGuards({ logAudit }) -> { requireAuthenticated, requireTenantAdmin, requireSuperAdmin }
 * logAudit(userId, userName, action, module, details, ip) ? optional; missing => no-op (never throws).
 */
function makeGuards(deps = {}) {
    const audit = typeof deps.logAudit === 'function' ? deps.logAudit : function () {};
    function safeAudit() { try { audit.apply(null, arguments); } catch (_) { /* audit must never break a reque
st */ } }

    // Authentication only (session present). Mirrors requireAuth; provided for a consistent vocabulary.
    function requireAuthenticated(req, res, next) {
        if (req.session && req.session.user) return next();
        return res.status(401).json({ error: 'Unauthorized' });
    }

    // Tenant Admin gate. Factory so callers can preserve the exact audit action/module per route.
    function requireTenantAdmin(opts) {
        const action = (opts && opts.action) || 'BLOCKED_ADMIN_ONLY';
        const moduleName = (opts && opts.module) || 'Auth';
        return function (req, res, next) {
            const user = req.session && req.session.user;
            const ip = clientIpOf(req);
            if (!user) {
                safeAudit(null, 'Anonymous', 'BLOCKED_AUTHENTICATION', moduleName,
                    `Unauthenticated access to admin route ${req.method} ${req.originalUrl || req.url}`, ip);
                return res.status(401).json({ error: 'Unauthorized' });
            }
            if (!isTenantAdmin(user)) {
                safeAudit(user.id, user.display_name, action, moduleName,
                    `Non-admin (role=${user.role || ''}) blocked from ${req.method} ${req.originalUrl || req.u
rl}`, ip);
                return res.status(403).json({ error: 'Access denied: administrator only' });
            }
            return next();
        };
    }

    // Super Admin gate (platform-level, env allowlist). Identity via super_admin.isSuperAdmin (single source)
.
    // allowlistSource: comma-separated string OR a Set. Resolved once at construction (no per-request parse).
    function requireSuperAdmin(allowlistSource, opts) {
        const set = allowlistSource instanceof Set ? allowlistSource : parseAllowlist(allowlistSource);
        const moduleName = (opts && opts.module) || 'SuperAdmin';
```

```
        return function (req, res, next) {
            const user = req.session && req.session.user;
            const ip = clientIpOf(req);
            if (!isSuperAdmin(user, set)) {
                safeAudit(user && user.id, user && user.display_name, 'SUPER_ADMIN_DENY', moduleName,
                    `denied ${req.method} ${req.originalUrl || req.url}`, ip);
                return res.status(403).json({ error: 'Super Admin access required', code: 'SUPER_ADMIN_REQUIRE
D' });
            }
            return next();
        };
    }

    return { requireAuthenticated, requireTenantAdmin, requireSuperAdmin };
}

module.exports = { isTenantAdmin, clientIpOf, makeGuards };



========================================
FILE: rbac_guards_test.js
========================================

/**
 * rbac_guards_test.js ? Batch 2 Auth/RBAC guard tests (no DB, no network).
 * Drives the middleware directly with mock req/res/next and asserts status + audit.
 * Run: node rbac_guards_test.js   (exit 0 = all pass)
 */
'use strict';
const G = require('./rbac_guards');

let pass = 0, fail = 0;
function ok(name, cond) { if (cond) pass++; else { fail++; console.error('  FAIL:', name); } }

// mock res capturing status()/json()
function mockRes() {
    return {
        _status: null, _json: null,
        status(c) { this._status = c; return this; },
        json(o) { this._json = o; return this; }
    };
}
function mockReq(user, extra) {
    return Object.assign({ session: user ? { user } : {}, method: 'POST', originalUrl: '/x', ip: '1.2.3.4', he
aders: {}, connection: {} }, extra || {});
}
// run a middleware once: returns { nexted, res, audit }
function run(mw, req) {
    const audit = [];
    // rebuild guards with an audit sink bound per-call
    const res = mockRes();
    let nexted = false;
    mw(req, res, () => { nexted = true; });
```

```
    return { nexted, res, audit };
}

// ---------- pure: isTenantAdmin ----------
ok('isTenantAdmin true for Admin', G.isTenantAdmin({ role: 'Admin' }) === true);
ok('isTenantAdmin false for Doctor', G.isTenantAdmin({ role: 'Doctor' }) === false);
ok('isTenantAdmin false for null', G.isTenantAdmin(null) === false);
ok('isTenantAdmin false for missing role', G.isTenantAdmin({ id: 1 }) === false);

// build guards with a shared audit array
const audit = [];
const guards = G.makeGuards({ logAudit: (...a) => audit.push(a) });

// ---------- requireAuthenticated ----------
(() => {
    const r1 = run(guards.requireAuthenticated, mockReq(null));
    ok('requireAuthenticated denies anonymous 401', !r1.nexted && r1.res._status === 401);
    const r2 = run(guards.requireAuthenticated, mockReq({ id: 1, role: 'Doctor' }));
    ok('requireAuthenticated allows any session', r2.nexted === true);
})();

// ---------- requireTenantAdmin ----------
(() => {
    audit.length = 0;
    const gAdmin = guards.requireTenantAdmin({ action: 'BLOCKED_USER_CREATE', module: 'Settings' });
    const r1 = run(gAdmin, mockReq(null));
    ok('tenantAdmin denies anonymous 401', !r1.nexted && r1.res._status === 401);
    ok('tenantAdmin audits anon as BLOCKED_AUTHENTICATION', audit.some(a => a[2] === 'BLOCKED_AUTHENTICATION')
);

    audit.length = 0;
    const r2 = run(gAdmin, mockReq({ id: 7, role: 'Doctor', display_name: 'Doc' }));
    ok('tenantAdmin denies normal user 403', !r2.nexted && r2.res._status === 403);
    ok('tenantAdmin audits deny with custom action', audit.some(a => a[2] === 'BLOCKED_USER_CREATE' && a[3] ==
= 'Settings'));

    audit.length = 0;
    const r3 = run(gAdmin, mockReq({ id: 1, role: 'Admin', display_name: 'Boss' }));
    ok('tenantAdmin allows Admin', r3.nexted === true);
    ok('tenantAdmin no audit on allow', audit.length === 0);

    // a tenant 'settings'-holder that is NOT Admin (e.g. IT) is still blocked -> no escalation to user-create
    const r4 = run(gAdmin, mockReq({ id: 9, role: 'IT', display_name: 'ITGuy' }));
    ok('tenantAdmin blocks non-Admin settings-holder (IT)', !r4.nexted && r4.res._status === 403);
})();

// ---------- requireSuperAdmin ----------
(() => {
    audit.length = 0;
    const gSuper = guards.requireSuperAdmin('op, boss');
    // super admin allowed: listed username + active (active implied: not 0/false)
    const r1 = run(gSuper, mockReq({ id: 1, username: 'op', display_name: 'Op', is_active: 1 }));
    ok('superAdmin allows listed user', r1.nexted === true);
```

```
    // tenant Admin NOT in allowlist -> denied (no escalation from role)
    audit.length = 0;
    const r2 = run(gSuper, mockReq({ id: 2, username: 'tadmin', role: 'Admin', is_active: 1 }));
    ok('superAdmin denies tenant Admin not in allowlist 403', !r2.nexted && r2.res._status === 403 && r2.res._
json.code === 'SUPER_ADMIN_REQUIRED');
    ok('superAdmin audits SUPER_ADMIN_DENY', audit.some(a => a[2] === 'SUPER_ADMIN_DENY'));

    // anonymous denied
    const r3 = run(gSuper, mockReq(null));
    ok('superAdmin denies anonymous 403', !r3.nexted && r3.res._status === 403);

    // session WITHOUT username (the Batch-1 latent bug) -> denied (fail-closed)
    const r4 = run(gSuper, mockReq({ id: 3, role: 'Admin', is_active: 1 }));
    ok('superAdmin denies session missing username', !r4.nexted && r4.res._status === 403);

    // inactive listed user denied
    const r5 = run(gSuper, mockReq({ id: 4, username: 'op', is_active: 0 }));
    ok('superAdmin denies inactive listed user', !r5.nexted && r5.res._status === 403);
})();

// ---------- malformed / empty allowlist must NOT open access ----------
(() => {
    const variants = ['', '   ', ',', ' , , ', undefined, null];
    let allDeny = true;
    for (const v of variants) {
        const g = guards.requireSuperAdmin(v);
        const r = run(g, mockReq({ id: 1, username: 'op', is_active: 1 }));
        if (r.nexted) allDeny = false;
    }
    ok('malformed/empty SUPER_ADMIN_USERS denies everyone', allDeny === true);

    // wildcard-looking value is treated as a literal username, not a match-all
    const gStar = guards.requireSuperAdmin('*');
    const rStar = run(gStar, mockReq({ id: 1, username: 'anybody', is_active: 1 }));
    ok('no wildcard match: "*" does not open access', !rStar.nexted && rStar.res._status === 403);
    const rStarLit = run(gStar, mockReq({ id: 1, username: '*', is_active: 1 }));
    ok('"*" only matches literal "*" username (no broad open)', rStarLit.nexted === true);
})();

// ---------- audit never throws even if logAudit throws ----------
(() => {
    const g = G.makeGuards({ logAudit: () => { throw new Error('audit down'); } });
    let threw = false;
    try {
        const mw = g.requireTenantAdmin({});
        run(mw, mockReq({ id: 1, role: 'Doctor' }));
    } catch (e) { threw = true; }
    ok('guard survives a throwing logAudit (audit isolated)', threw === false);
})();

console.log(`rbac_guards_test: ${pass} passed, ${fail} failed`);
process.exit(fail === 0 ? 0 : 1);
```

```
========================================
FILE: rbac_phi_guard_test.js
========================================

/**
 * rbac_phi_guard_test.js
 * ======================================
 * PHASE 2B (H-6/H-7) ? RBAC guards on audit-trail + secondary PHI routes.
 * (1) Behavioral: replays the project's real ROLE_PERMISSIONS + requireRole() decision logic to prove
 *     the chosen module guards DENY wrong roles and ALLOW the intended ones.
 * (2) Static: asserts each sensitive route signature now carries requireRole (+ requireTenantScope where appl
icable),
 *     i.e. is no longer requireAuth-only.
 * DB-free, deterministic. No route execution, no DB.
 *
 *   node rbac_phi_guard_test.js
 */
const fs = require('fs');
const path = require('path');
const src = fs.readFileSync(path.join(__dirname, 'server.js'), 'utf8');

const RED = '\x1b[31m', GREEN = '\x1b[32m', BLUE = '\x1b[34m', RESET = '\x1b[0m', BOLD = '\x1b[1m';
let passed = 0, failed = 0; const failures = [];
function assert(cond, name, details = '') {
    if (cond) { console.log(`  ${GREEN}PASS${RESET} ? ${name}`); passed++; }
    else { console.log(`  ${RED}FAIL${RESET} ? ${name}${details ? ' | ' + details : ''}`); failed++; failures.
push({ name, details }); }
}

console.log(`\n${BOLD}${BLUE}=== RBAC / Secondary-PHI Guards (H-6/H-7) ===${RESET}\n`);

// ---- extract the REAL ROLE_PERMISSIONS literal from server.js and the requireRole() decision logic ----
const rpMatch = src.match(/const ROLE_PERMISSIONS = (\{[\s\S]*?\n\};)/);
assert(!!rpMatch, 'ROLE_PERMISSIONS literal extracted from server.js');
// NO eval: the literal uses single-quoted keys/values with no embedded quotes (role/module names are
// simple strings), so swapping ' -> " yields valid JSON we can safely JSON.parse.
const ROLE_PERMISSIONS = JSON.parse(rpMatch[1].replace(/;\s*$/, '').replace(/'/g, '"'));
// mirrors requireRole(): Admin('*') passes; else role must hold at least one of the required modules
function roleAllowed(role, modules) {
    const perms = ROLE_PERMISSIONS[role];
    if (perms === '*') return true;
    return !!(perms && modules.some(m => perms.includes(m)));
}

// ---- H-6: audit-trail = settings (IT/Admin only) ----
console.log(`${BOLD}[H-6] audit-trail role gate (settings)${RESET}`);
assert(roleAllowed('Admin', ['settings']), 'Admin allowed on audit-trail');
assert(roleAllowed('IT', ['settings']), 'IT allowed on audit-trail (system/security operator)');
['Doctor', 'Nurse', 'Lab Technician', 'Pharmacist', 'Finance', 'Reception'].forEach(r =>
    assert(!roleAllowed(r, ['settings']), `${r} DENIED on audit-trail`));

// ---- H-7: cosmetic = doctor/surgery ----
console.log(`${BOLD}[H-7] cosmetic role gate (doctor/surgery)${RESET}`);
```

```
assert(roleAllowed('Doctor', ['doctor', 'surgery']), 'Doctor allowed on cosmetic');
assert(roleAllowed('Admin', ['doctor', 'surgery']), 'Admin allowed on cosmetic');
['Lab Technician', 'Pharmacist', 'Finance', 'Blood Bank', 'IT', 'Reception'].forEach(r =>
    assert(!roleAllowed(r, ['doctor', 'surgery']), `${r} DENIED on cosmetic`));

// ---- H-7: social-work + mortuary = him/nursing ----
console.log(`${BOLD}[H-7] social-work & mortuary role gate (him/nursing)${RESET}`);
assert(roleAllowed('HIM', ['him', 'nursing']), 'HIM allowed on social-work/mortuary');
assert(roleAllowed('Nurse', ['him', 'nursing']), 'Nurse allowed on social-work/mortuary');
assert(roleAllowed('Admin', ['him', 'nursing']), 'Admin allowed on social-work/mortuary');
['Lab Technician', 'Pharmacist', 'Finance', 'Insurance', 'Blood Bank', 'Radiologist'].forEach(r =>
    assert(!roleAllowed(r, ['him', 'nursing']), `${r} DENIED on social-work/mortuary`));

// ---- H-7: employee CREATE/DELETE = hr (Admin/HR). employees GET stays OPEN (no requireRole, per
// committed decision bc24a47) BUT compensation fields are projected out for non-HR/Admin (Option C).
console.log(`${BOLD}[H-7] employee create/delete role gate (hr)${RESET}`);
assert(roleAllowed('HR', ['hr']), 'HR allowed on employee create/delete');
assert(roleAllowed('Admin', ['hr']), 'Admin allowed on employee create/delete');
['Doctor', 'Nurse', 'Lab Technician', 'Finance', 'Reception'].forEach(r =>
    assert(!roleAllowed(r, ['hr']), `${r} DENIED on employee create/delete`));

// ---- H-7 Option C: employees GET field filtering (salary hidden from non-HR/Admin) ----
console.log(`${BOLD}[H-7 Option C] employees GET compensation field filtering${RESET}`);
// behavioral: replay isHrOrAdmin() decision using the real ROLE_PERMISSIONS
function isHrOrAdmin(role) {
    if (role === 'Admin') return true;
    const perms = ROLE_PERMISSIONS[role];
    return Array.isArray(perms) && perms.includes('hr');
}
assert(isHrOrAdmin('HR') === true, 'HR => full employee record (salary visible)');
assert(isHrOrAdmin('Admin') === true, 'Admin => full employee record (salary visible)');
['Doctor', 'Nurse', 'Lab Technician', 'Finance', 'Reception', 'IT', 'Pharmacist'].forEach(r =>
    assert(isHrOrAdmin(r) === false, `${r} => directory projection only (salary HIDDEN)`));
// static: the directory projection constant excludes compensation fields, and GET uses it
const dirMatch = src.match(/const EMPLOYEE_DIRECTORY_COLS = '([^']*)';/);
assert(!!dirMatch, 'EMPLOYEE_DIRECTORY_COLS constant exists');
const dirCols = dirMatch ? dirMatch[1] : '';
['salary', 'commission_type', 'commission_value'].forEach(f =>
    assert(!new RegExp('\\b' + f + '\\b').test(dirCols), `directory projection EXCLUDES ${f}`));
['id', 'name', 'role'].forEach(f =>
    assert(new RegExp('\\b' + f + '\\b').test(dirCols), `directory projection INCLUDES safe field ${f}`));
const getEmp = (src.match(/app\.get\('\/api\/employees',[\s\S]{0,800}?\n\}\);/) || [''])[0];
assert(/isHrOrAdmin\(/.test(getEmp) && /EMPLOYEE_DIRECTORY_COLS/.test(getEmp), 'employees GET branches on isHr
OrAdmin + uses directory projection');
assert(!/SELECT \* FROM employees/.test(getEmp), 'employees GET no longer hardcodes SELECT * (role-gated proje
ction instead)');

// ---- static: sensitive routes are no longer requireAuth-only ----
console.log(`${BOLD}[static] route signatures carry requireRole (+ tenant scope)${RESET}`);
function sig(re) { const m = src.match(re); return m ? m[0] : ''; }
const checks = [
    [/app\.get\('\/api\/audit-trail',[^\n]*/, "requireRole('settings')", 'requireTenantScope', 'audit-trail'],
    [/app\.post\('\/api\/employees',[^\n]*/, "requireRole('hr')", null, 'employees POST'],
    [/app\.delete\('\/api\/employees\/:id',[^\n]*/, "requireRole('hr')", null, 'employees DELETE'],
```

```
    [/app\.get\('\/api\/cosmetic\/cases',[^\n]*/, "requireRole('doctor', 'surgery')", 'requireTenantScope', 'cosmetic/cases'],
    [/app\.put\('\/api\/cosmetic\/cases\/:id',[^\n]*/, "requireRole('doctor', 'surgery')", 'requireTenantScope', 'cosmetic/cases PUT'],
    [/app\.get\('\/api\/social-work\/cases',[^\n]*/, "requireRole('him', 'nursing')", 'requireTenantScope', 'social-work/cases'],
    [/app\.put\('\/api\/social-work\/cases\/:id',[^\n]*/, "requireRole('him', 'nursing')", 'requireTenantScope', 'social-work PUT'],
    [/app\.get\('\/api\/mortuary\/cases',[^\n]*/, "requireRole('him', 'nursing')", 'requireTenantScope', 'mortuary/cases'],
    [/app\.put\('\/api\/mortuary\/cases\/:id',[^\n]*/, "requireRole('him', 'nursing')", 'requireTenantScope', 'mortuary PUT'],
];
for (const [re, role, scope, label] of checks) {
    const s = sig(re);
    assert(s.includes('requireAuth') && s.includes(role), `${label} carries ${role}`, s.slice(0, 90));
    if (scope) assert(s.includes(scope), `${label} carries ${scope}`);
    assert(!/requireAuth,\s*async/.test(s), `${label} is NOT requireAuth-only`);
}

console.log(`\n${BOLD}Result: ${passed} passed, ${failed} failed${RESET}`);
if (failed > 0) { console.log(`${RED}Failures:${RESET}`); failures.forEach(f => console.log(`  - ${f.name}: ${f.details}`)); process.exit(1); }
process.exit(0);



========================================
FILE: README.md
========================================

# ? jumanaMedical ERP ? ????? ?????

???? ????? ?????????? ?????? | Comprehensive Hospital Management System

## ? ???????? | Features

- ? ???? ???? ?? ???? ?????? | Dashboard with Charts
- ? ??????? ?????? ???? | Reception & Patient Registration
- ???? ???? ???? ????? | Full Doctor Station
- ? ????? + ???? + ?????? | Lab + Radiology + Pharmacy
- ? ????? + ????? + ?????? | Finance + Insurance + Billing
- ? 44 ??? ?????? | 44 Integrated Departments
- ? ???? + ??????? | Arabic + English
- ? ?????? ?? ???????? | Mobile Responsive

## ? ??????? ?????? | Quick Setup

### ????????? | Prerequisites

- [Node.js](https://nodejs.org) v18+
- [PostgreSQL](https://www.postgresql.org/download/) v14+

### ???? ????? ???! | One Step Only
```

```bash
git clone https://github.com/iceman18ice-sketch/namaweb3.git
cd namaweb3
setup.bat
```

> ???????? ???? ?? ??? ????????: ???? ?????????? ???? ????? ????????? ???? ????????? ????? ???????? ???????.

?? ?????? | Or Manually

```bash
git clone https://github.com/iceman18ice-sketch/namaweb3.git
cd namaweb3
copy .env.example .env
npm install
node server.js
```

?? ??????? | Run

```bash
start.bat
```

?? | or

```bash
node server.js
```

?? ???? | Then open: **<http://localhost:3000>**

?????? ?????? ?????????? | Default Login

?????	??????
????????	`admin`
???? ??????	`admin`

? ???? ??????? | Project Structure

```
namaweb3/
??? server.js              # ??????? ??????? + APIs
??? public/
?   ??? index.html         # ?????? ????????
?   ??? login.html         # ???? ??????
?   ??? js/
?   ?   ??? app.js         # ??????? (44 ???)
?   ?   ??? api.js         # ????? API
?   ??? css/styles.css     # ???????
?   ??? consent-forms/     # ????? ???????? (31 ?????)
??? setup.bat              # ????? ??????
```

```
??? start.bat              # ????? ????? ?????
??? .env.example           # ???? ?????????
??? package.json           # ????????
??? README.md              # ??? ?????
```

? ??????? | Departments (44)

?????	Department	?????	Department
???? ??????	Dashboard	?????????	Reception
????????	Appointments	???? ??????	Doctor Station
???????	Laboratory	??????	Radiology
????????	Pharmacy	??????? ???????	HR
???????	Finance	???????	Insurance
???????	Inventory	???????	Nursing
???????	Surgery	???????	Emergency
???????	Inpatient	??????? ???????	ICU
??? ????	Blood Bank	???????	Rehabilitation
?????? ????????	OB/GYN	???????	Cosmetic Surgery
???????	CSSD	???????	Dietary
?????? ??????	Infection Control	??????	Quality
???????	Maintenance	?????	Transport
???? ?? ????	Telemedicine	??? ???????	Pathology
??????? ?????	CME	?????? ??????????	Social Work

?? ???????? | Tech Stack

- **Backend:** Node.js, Express.js
- **Database:** PostgreSQL
- **Frontend:** Vanilla JS, CSS3
- **Security:** Helmet, bcrypt, express-rate-limit

? ??????? | License

MIT License ? Free to use

```
========================================
FILE: restore_backup.js
========================================

#!/usr/bin/env node
/**
 * restore_backup.js ? decrypt an encrypted Nama DB backup (.sql.gz.enc) produced by
 * POST /api/admin/backup back to a plaintext .sql file.
 *
 * On-disk layout written by the server: [salt(16)][iv(12)][authTag(16)][ciphertext]
 *   key = scrypt(BACKUP_ENCRYPTION_KEY, salt, 32);  cipher = AES-256-GCM over gzip(pg_dump output)
 *
 * Usage:
 *   BACKUP_ENCRYPTION_KEY=... node restore_backup.js <file.sql.gz.enc> [out.sql]
 * Then restore (review first!):
```

```
 *   psql "$DATABASE_URL" < out.sql        # or: psql -h .. -U .. -d .. -f out.sql
 *
 * This script ONLY decrypts; it never connects to or modifies any database.
 */
'use strict';
const fs = require('fs');
const crypto = require('crypto');
const zlib = require('zlib');

const keyMaterial = process.env.BACKUP_ENCRYPTION_KEY;
if (!keyMaterial) {
    console.error('ERROR: set BACKUP_ENCRYPTION_KEY (the same value used when the backup was created).');
    process.exit(1);
}
const inFile = process.argv[2];
if (!inFile) {
    console.error('Usage: BACKUP_ENCRYPTION_KEY=... node restore_backup.js <file.sql.gz.enc> [out.sql]');
    process.exit(1);
}
const outFile = process.argv[3] || inFile.replace(/\.gz\.enc$/i, '').replace(/\.enc$/i, '') || 'restored.sql';

let buf;
try { buf = fs.readFileSync(inFile); } catch (e) { console.error('Cannot read', inFile, '-', e.message); proce
ss.exit(1); }
if (buf.length < 44) { console.error('File too small / not a valid encrypted backup.'); process.exit(1); }

const salt = buf.subarray(0, 16);
const iv = buf.subarray(16, 28);
const tag = buf.subarray(28, 44);
const ct = buf.subarray(44);
try {
    const key = crypto.scryptSync(String(keyMaterial), salt, 32);
    const decipher = crypto.createDecipheriv('aes-256-gcm', key, iv);
    decipher.setAuthTag(tag);
    const gz = Buffer.concat([decipher.update(ct), decipher.final()]); // throws if key wrong / tampered
    const sql = zlib.gunzipSync(gz);
    fs.writeFileSync(outFile, sql);
    console.log('Decrypted ->', outFile, '(' + sql.length + ' bytes).');
    console.log('Restore (review the SQL first): psql "$DATABASE_URL" < ' + outFile);
} catch (e) {
    console.error('Decryption FAILED (wrong key or corrupted/tampered file):', e.message);
    process.exit(1);
}
```

```
========================================
FILE: result_loop.js
========================================

/**
 * result_loop.js ? Gate 3 pure engine: order?result closed loop + acknowledgement policy.
 *
 * shouldCloseLegacyOrder: decides whether reporting a lab result may close its
```

```
 * originating legacy order (lab_radiology_orders). Fail-CLOSED: a patient mismatch,
 * a missing patient id, a missing order, or an already-terminal order all refuse the
 * close ? closing the WRONG order (another patient's pending work disappearing from
 * the worklist) is a patient-safety hazard, so refusal is always the safe direction.
 *
 * ackRequirement: physician-acknowledgement policy for a verified result. Fail-CLOSED:
 * critical results (is_critical or HH/LL flags) always require acknowledgement; an
 * UNKNOWN/missing abnormal flag also requires it ? an unreviewed result must never be
 * silently classified as needing no review.
 *
 * ???? ??????? ???????? ?????? ?????? (???? ??? ?????? fail-closed).
 */
'use strict';

// Legacy order statuses that still represent open work. Anything else (Completed,
// Cancelled, unknown custom states) is terminal-or-ambiguous => refuse to close.
const LEGACY_OPEN_STATUSES = new Set([
    'Requested', 'In Progress', 'InProgress', 'Sample Collected', 'Pending', 'Paid',
]);

function shouldCloseLegacyOrder(sample, order) {
    if (!sample || sample.patient_id === null || sample.patient_id === undefined) {
        return { close: false, reason: 'sample missing patient_id (fail-closed)' };
    }
    if (!order) return { close: false, reason: 'order not found' };
    if (order.patient_id === null || order.patient_id === undefined ||
        Number(order.patient_id) !== Number(sample.patient_id)) {
        return { close: false, reason: 'patient mismatch between sample and order ? refusing to close' };
    }
    const status = typeof order.status === 'string' ? order.status.trim() : '';
    if (!LEGACY_OPEN_STATUSES.has(status)) {
        return { close: false, reason: `order status '${status}' is not open` };
    }
    return { close: true, reason: 'sample and order agree on patient; order open' };
}

// Abnormal-flag taxonomy used by lis.js: N normal, H/L abnormal, HH/LL critical.
const CRITICAL_FLAGS = new Set(['HH', 'LL']);
const ABNORMAL_FLAGS = new Set(['H', 'L']);

function ackRequirement(result) {
    if (!result || typeof result !== 'object') {
        return { required: true, level: 'unknown', reason: 'result missing ? fail-closed: requires review' };
    }
    const flag = typeof result.abnormal_flag === 'string' ? result.abnormal_flag.trim().toUpperCase() : '';
    const critical = result.is_critical === 1 || result.is_critical === true || CRITICAL_FLAGS.has(flag);
    if (critical) return { required: true, level: 'critical', reason: 'critical result requires physician ackn
owledgement' };
    if (ABNORMAL_FLAGS.has(flag)) return { required: true, level: 'abnormal', reason: 'abnormal result require
s physician acknowledgement' };
    if (flag === 'N') return { required: false, level: 'none', reason: 'normal result' };
    return { required: true, level: 'unknown', reason: `abnormal_flag '${flag}' unknown ? fail-closed: require
s review` };
}
```

```
module.exports = { shouldCloseLegacyOrder, ackRequirement, LEGACY_OPEN_STATUSES };



========================================
FILE: result_loop_test.js
========================================

/**
 * result_loop_test.js ? PURE-ENGINE unit test for Gate 3 (order?result closed loop
 * + result acknowledgement requirement).
 * Run: node result_loop_test.js   (no DB; deterministic)
 *
 * Safety invariants:
 *  - A legacy order is closed ONLY when the sample and order agree on the patient
 *    (guard against closing the wrong patient's order) and the order is still open.
 *  - Acknowledgement requirement is fail-closed: a critical result ALWAYS requires
 *    physician acknowledgement; unknown/missing flags never silently mean 'none'.
 */
'use strict';
const R = require('./result_loop');

const GREEN = '\x1b[32m', RED = '\x1b[31m', RESET = '\x1b[0m', BOLD = '\x1b[1m';
let passed = 0, failed = 0; const fails = [];
function assert(cond, name, det = '') {
  if (cond) { console.log(`  ${GREEN}PASS${RESET} ${name}`); passed++; }
  else { console.log(`  ${RED}FAIL${RESET} ${name}${det ? ' | ' + det : ''}`); failed++; fails.push(name); }
}

console.log(`${BOLD}Gate 3 ? order?result loop engine (pure unit test)${RESET}\n`);

// ---- shouldCloseLegacyOrder ----
console.log('[1] shouldCloseLegacyOrder ? patient guard, open-status guard');
let d = R.shouldCloseLegacyOrder({ patient_id: 7 }, { id: 12, patient_id: 7, status: 'Requested' });
assert(d.close === true, 'same patient + open order -> close', JSON.stringify(d));
d = R.shouldCloseLegacyOrder({ patient_id: 7 }, { id: 12, patient_id: 9, status: 'Requested' });
assert(d.close === false && /patient/i.test(d.reason), 'PATIENT MISMATCH -> never close', JSON.stringify(d));
d = R.shouldCloseLegacyOrder({ patient_id: 7 }, { id: 12, patient_id: 7, status: 'Completed' });
assert(d.close === false, 'already Completed -> no re-close', JSON.stringify(d));
d = R.shouldCloseLegacyOrder({ patient_id: 7 }, { id: 12, patient_id: 7, status: 'Cancelled' });
assert(d.close === false, 'Cancelled -> never close', JSON.stringify(d));
d = R.shouldCloseLegacyOrder({ patient_id: 7 }, null);
assert(d.close === false, 'order not found -> no close', JSON.stringify(d));
d = R.shouldCloseLegacyOrder({ patient_id: null }, { id: 12, patient_id: 7, status: 'Requested' });
assert(d.close === false, 'sample missing patient -> fail-closed no close', JSON.stringify(d));
// status naming variants used by legacy UI
d = R.shouldCloseLegacyOrder({ patient_id: 7 }, { id: 12, patient_id: 7, status: 'In Progress' });
assert(d.close === true, "'In Progress' counts as open", JSON.stringify(d));

// ---- ackRequirement ----
console.log('\n[2] ackRequirement ? critical always, abnormal flags, fail-closed');
let a = R.ackRequirement({ is_critical: 1, abnormal_flag: 'HH', status: 'verified' });
assert(a.required === true && a.level === 'critical', 'critical -> required critical', JSON.stringify(a));
```

```
a = R.ackRequirement({ is_critical: 0, abnormal_flag: 'H', status: 'verified' });
assert(a.required === true && a.level === 'abnormal', 'H flag -> required abnormal', JSON.stringify(a));
a = R.ackRequirement({ is_critical: 0, abnormal_flag: 'LL', status: 'verified' });
assert(a.required === true && a.level === 'critical', 'LL (critical-low) -> critical level', JSON.stringify(a)
);
a = R.ackRequirement({ is_critical: 0, abnormal_flag: 'N', status: 'verified' });
assert(a.required === false && a.level === 'none', 'normal -> not required', JSON.stringify(a));
a = R.ackRequirement({ is_critical: false, abnormal_flag: '', status: 'verified' });
assert(a.required === true && a.level === 'unknown', 'MISSING flag -> fail-closed required (unknown)', JSON.st
ringify(a));
a = R.ackRequirement(null);
assert(a.required === true && a.level === 'unknown', 'null result -> fail-closed required', JSON.stringify(a))
;
a = R.ackRequirement({ is_critical: 0, abnormal_flag: 'HH', status: 'verified' });
assert(a.required === true && a.level === 'critical', 'HH flag alone -> critical even if is_critical unset', J
SON.stringify(a));

console.log(`\n${BOLD}Result:${RESET} ${passed} passed, ${failed} failed`);
if (failed) { console.log(`${RED}FAILURES:${RESET} ${fails.join(', ')}`); process.exit(1); }
process.exit(0);
```

```
========================================
FILE: rheumatology_integration_test.js
========================================

/**
 * rheumatology_integration_test.js ? Integration test for the Rheumatology module HTTP endpoints.
 */
'use strict';

process.env.NODE_ENV = 'staging';

const { spawn } = require('child_process');
const http = require('http');
const assert = require('assert');
const bcrypt = require('bcryptjs');
const { pool } = require('./db_postgres');

const TEST_PORT = 3012;
const TEST_USERNAME = 'rheum_doctor';
const TEST_PASSWORD = 'RHEUM_PASSWORD_PLACEHOLDER';

let serverProcess;

function makeRequest(method, path, body, headers = {}) {
    return new Promise((resolve, reject) => {
        const payload = body ? JSON.stringify(body) : '';
        const req = http.request({
            hostname: 'localhost',
            port: TEST_PORT,
            path,
            method,
```

```
            headers: {
                'Content-Type': 'application/json',
                'Content-Length': Buffer.byteLength(payload),
                ...headers
            }
        }, (res) => {
            let data = '';
            res.on('data', chunk => data += chunk);
            res.on('end', () => {
                try {
                    const parsed = data ? JSON.parse(data) : {};
                    resolve({ statusCode: res.statusCode, headers: res.headers, body: parsed });
                } catch (e) {
                    resolve({ statusCode: res.statusCode, headers: res.headers, rawBody: data });
                }
            });
        });
        req.on('error', reject);
        if (payload) req.write(payload);
        req.end();
    });
}

async function runTests() {
    console.log('--- STARTING RHEUMATOLOGY MODULE INTEGRATION TESTS ---');

    const patientId = 9991;
    const doctorUserId = 9992;
    const client = await pool.connect();

    try {
        console.log('Setting up test data...');
        await client.query("SET app.tenant_id = '1'");

        // Clean up old test data
        await client.query('DELETE FROM joint_assessments WHERE patient_id = $1', [patientId]);
        await client.query('DELETE FROM patients WHERE id = $1', [patientId]);
        await client.query('DELETE FROM user_tenants WHERE user_id = $1', [doctorUserId]);
        await client.query('DELETE FROM system_users WHERE id = $1', [doctorUserId]);

        // Insert patient
        await client.query(
            'INSERT INTO patients (id, name_en, name_ar, tenant_id) VALUES ($1, $2, $3, 1)',
            [patientId, 'Rheum Test Patient', '???? ??? ??????????']
        );

        // Insert doctor user
        const hashedPassword = await bcrypt.hash(TEST_PASSWORD, 10);
        await client.query(
            'INSERT INTO system_users (id, username, password_hash, display_name, role, speciality, permission
s, is_active) VALUES ($1, $2, $3, $4, $5, $6, $7, 1)',
            [doctorUserId, TEST_USERNAME, hashedPassword, 'Dr. Rheum Consultant', 'Doctor', 'Rheumatology', '[
"patients", "prescriptions"]']
        );
```

```
// Associate doctor with tenant 1
await client.query(
    'INSERT INTO user_tenants (user_id, tenant_id, is_active) VALUES ($1, 1, true)',
    [doctorUserId]
);

console.log('Spawning test server...');
serverProcess = spawn('node', ['server.js'], {
    env: { ...process.env, PORT: TEST_PORT, NODE_ENV: 'staging', SKIP_DB_INIT: 'true' }
});

// Pipe stdout/stderr for logging
serverProcess.stdout.on('data', (data) => {
    console.log(`[Server STDOUT] ${data.toString().trim()}`);
});
serverProcess.stderr.on('data', (data) => {
    console.error(`[Server STDERR] ${data.toString().trim()}`);
});

// Wait 3 seconds for server to boot
await new Promise(resolve => setTimeout(resolve, 3000));

console.log('Logging in...');
const loginRes = await makeRequest('POST', '/api/auth/login', {
    username: TEST_USERNAME,
    password: TEST_PASSWORD
});

assert.strictEqual(loginRes.statusCode, 200, 'Login should succeed');
console.log('? Logged in successfully.');

// Extract session cookie
const setCookie = loginRes.headers['set-cookie'];
assert.ok(setCookie, 'Should receive session cookie');
const cookie = setCookie[0].split(';')[0];

// 1. Test POST /api/rheumatology/joints (Create joint assessment)
console.log('Testing create joint assessment...');
const jointCreateRes = await makeRequest('POST', '/api/rheumatology/joints', {
    patient_id: patientId,
    tender_joint_count: 4,
    swollen_joint_count: 2,
    vas_pain: 45,
    das28_score: 2.15,
    notes: 'Rheumatoid arthritis evaluation.'
}, { 'Cookie': cookie });

assert.strictEqual(jointCreateRes.statusCode, 200, 'Should return 200 OK');
assert.strictEqual(jointCreateRes.body.success, true, 'Should return success true');
assert.ok(jointCreateRes.body.id, 'Should return joint assessment ID');
const assessmentId = jointCreateRes.body.id;
console.log(`? Joint assessment created successfully. ID: ${assessmentId}`);
```

```
        // 2. Test GET /api/rheumatology/joints/patient/:patient_id
        console.log('Testing get patient joint assessments...');
        const jointGetRes = await makeRequest('GET', `/api/rheumatology/joints/patient/${patientId}`, null, {
'Cookie': cookie });
        assert.strictEqual(jointGetRes.statusCode, 200, 'Should return 200 OK');
        assert.strictEqual(jointGetRes.body.length, 1, 'Should return exactly 1 joint assessment');
        assert.strictEqual(jointGetRes.body[0].id, assessmentId, 'Joint assessment ID should match');
        assert.strictEqual(parseInt(jointGetRes.body[0].tender_joint_count), 4, 'Tender joint count should mat
ch');
        assert.strictEqual(parseInt(jointGetRes.body[0].swollen_joint_count), 2, 'Swollen joint count should m
atch');
        assert.strictEqual(parseInt(jointGetRes.body[0].vas_pain), 45, 'VAS pain should match');
        assert.strictEqual(parseFloat(jointGetRes.body[0].das28_score), 2.15, 'DAS28 score should match');
        console.log('? Patient joint assessments retrieved successfully.');

    } finally {
        console.log('Cleaning up test data...');
        await client.query("SET app.tenant_id = '1'");
        await client.query('DELETE FROM joint_assessments WHERE patient_id = $1', [patientId]);
        await client.query('DELETE FROM patients WHERE id = $1', [patientId]);
        await client.query('DELETE FROM user_tenants WHERE user_id = $1', [doctorUserId]);
        await client.query('DELETE FROM system_users WHERE id = $1', [doctorUserId]);
        client.release();

        if (serverProcess) {
            console.log('Killing test server...');
            serverProcess.kill();
        }
    }

    console.log('? Rheumatology Module Integration Tests passed successfully!\n');
}

if (require.main === module) {
    runTests().catch(err => {
        console.error('? Test failed:', err);
        if (serverProcess) serverProcess.kill();
        process.exit(1);
    });
}

module.exports = { runTests };
```

```
========================================
FILE: rls_local_dry_run_3_tables.js
========================================

/**
 * rls_local_dry_run_3_tables.js
 * ============================================================================
 * ????? ??????? ???????? ?????? ?? RLS ??? 3 ?????: patients, invoices, appointments
 *
```

```
 * ???????:
 * 1. ?????? ?? ???? ?????? ?????? ????? ????????.
 * 2. ??? ???? ???????? ??????? ???????? pg_dump.
 * 3. ????? RLS ?????? ???????? ??? ??????? ??????? ???.
 * 4. ????? ??? ?????? ??? ???? (test_rls_user) ?????? ?? ????? ????????.
 * 5. ????? ???????? ?????? ?? ??? ?????????? (?????? ?????? ?????? ?????) ??? ??? test_rls_user.
 * 6. ??????? ?????? (Rollback) ???? ???????? ?????? RLS ?????? ????? ??????????.
 * 7. ?????? ??????? ?? ??? ????? ???????? ?? ?? ?????? ??????.
 * ============================================================================
 */

const { Client } = require('pg');
const { execSync } = require('child_process');
const fs = require('fs');
const path = require('path');
const dotenv = require('dotenv');

// ????? ????????? ?? ??? .env
dotenv.config({ path: path.join(__dirname, '.env') });

const DB_HOST = process.env.DB_HOST || 'localhost';
const DB_PORT = process.env.DB_PORT || '5432';
const DB_NAME = process.env.DB_NAME || 'nama_medical_web';
const DB_USER = process.env.DB_USER || 'postgres';
const DB_PASSWORD = process.env.DB_PASSWORD || 'postgres';

// ????? ????? ??????? ??????? ????????
const RED = '\x1b[31m';
const GREEN = '\x1b[32m';
const YELLOW = '\x1b[33m';
const BLUE = '\x1b[34m';
const RESET = '\x1b[0m';
const BOLD = '\x1b[1m';

console.log(`\n${BOLD}${BLUE}============================================================${RESET}`);
console.log(`${BOLD}${BLUE}  ??????? ???????? ?? RLS ??? ????? ?????? ????????? ?????????${RESET}`);
console.log(`${BOLD}${BLUE}  NamaMedical ? RLS Local Dry-Run (3 Tables Only)${RESET}`);
console.log(`${BOLD}${BLUE}============================================================${RESET}\n`);

// ----------------------------------------------------------------------------
// 1. ?????? ?? ???? ??????
// ----------------------------------------------------------------------------
function checkEnvironment() {
    const isLocal = ['localhost', '127.0.0.1', '::1'].includes(DB_HOST.toLowerCase());
    const isProdName = ['prod', 'production', 'live'].some(keyword => DB_NAME.toLowerCase().includes(keyword))
;

    console.log(`[1] ??? ????? ?????? ?????? ????? ????????...`);
    console.log(`  - ??????: ${DB_HOST}`);
    console.log(`  - ????? ????????: ${DB_NAME}`);

    if (!isLocal || isProdName) {
        console.error(`\n${BOLD}${RED}? ??? ????: ?????? ?? ???? ????? ?? ????!${RESET}`);
        console.error(`???? ????? ??? ??????? ??? ????? ??????? ?? ??????. ?? ????? ???????.`);
```

```
        process.exit(1);
    }
    console.log(`  ${GREEN}? ?????? ???? ?????? ??????.${RESET}`);
}

// ---------------------------------------------------------------------------
// 2. ??? ?????? ?????????? ????????
// ---------------------------------------------------------------------------
function getPgDumpExecutable() {
    // 1. Check env variable
    if (process.env.PG_DUMP_PATH && fs.existsSync(process.env.PG_DUMP_PATH)) {
        return `"${process.env.PG_DUMP_PATH}"`;
    }

    // 2. Check if pg_dump is globally available in PATH
    try {
        execSync('pg_dump --version', { stdio: 'ignore' });
        return 'pg_dump';
    } catch (e) {
        // Not in PATH globally
    }

    // 3. Check known Windows installation paths
    const winPaths = [
        'C:\\Program Files\\PostgreSQL\\16\\bin\\pg_dump.exe',
        'C:\\Program Files\\PostgreSQL\\17\\bin\\pg_dump.exe',
        'C:\\Program Files\\PostgreSQL\\15\\bin\\pg_dump.exe',
        'C:\\Program Files\\PostgreSQL\\14\\bin\\pg_dump.exe'
    ];
    for (const p of winPaths) {
        if (fs.existsSync(p)) {
            return `"${p}"`;
        }
    }

    // 4. Default fallback
    return 'pg_dump';
}

function runBackup() {
    console.log(`\n[2] ????? ???? ???????? ?????? ???????? ??????? ??? ?????...`);
    const dateStr = new Date().toISOString().replace(/T/, '_').replace(/\..+/, '').replace(/:/g, '-');
    const backupsDir = path.join(__dirname, 'backups');

    if (!fs.existsSync(backupsDir)) {
        fs.mkdirSync(backupsDir);
    }

    const backupFileName = `rls_local_dry_run_before_${dateStr}.backup`;
    const backupPath = path.join(backupsDir, backupFileName);

    try {
        // ????? ???? ?????? ?? pg_dump ?????? ?????? ??????? ??????
        process.env.PGPASSWORD = DB_PASSWORD;
```

```
        const pgDumpExe = getPgDumpExecutable();
        console.log(`  - ??????? ???? pg_dump ??: ${pgDumpExe}`);
        const backupCommand = `${pgDumpExe} -h ${DB_HOST} -p ${DB_PORT} -U ${DB_USER} -d ${DB_NAME} -F c -f "${backupPath}"`;

        execSync(backupCommand, { stdio: 'inherit' });
        console.log(`  ${GREEN}? ?? ??? ?????? ?????????? ????? ?????? ??:${RESET}`);
        console.log(`  ${backupPath}`);
        return backupPath;
    } catch (error) {
        console.error(`\n${BOLD}${RED}? ???: ??? ????? pg_dump ???? ?????? ??????????!${RESET}`);
        console.error(`?????: ${error.message}`);
        console.error(`?? ???? ????????? ??? ???? ??? ???? ????????. ?? ????? ???????.`);
        process.exit(1);
    }
}

// ---------------------------------------------------------------------------
// 3. ????? ?????? RLS ????????? ???????? ????
// ---------------------------------------------------------------------------
async function executeDryRun() {
    const client = new Client({
        host: DB_HOST,
        port: DB_PORT,
        database: DB_NAME,
        user: DB_USER,
        password: DB_PASSWORD
    });

    try {
        await client.connect();
        console.log(`\n[3] ??????? ?????? ???????? ?????.`);

        // ?????? ?? ???? ??????? ???????
        const colCheck = await client.query(`
            SELECT table_name, column_name
            FROM information_schema.columns
            WHERE table_name IN ('patients', 'invoices', 'appointments')
              AND column_name = 'tenant_id'
        `);
        if (colCheck.rows.length < 3) {
            console.error(`\n${RED}? ???: ???? tenant_id ????? ?? ??? ??????? ???????!${RESET}`);
            process.exit(1);
        }

        // ????? RLS ?????? ????????
        console.log(`\n[4] ????? RLS ?????? ???????? ??? ??????? ??????? ??????...`);
        await client.query(`
            -- ????? RLS
            ALTER TABLE patients ENABLE ROW LEVEL SECURITY;
            ALTER TABLE patients FORCE ROW LEVEL SECURITY;

            ALTER TABLE invoices ENABLE ROW LEVEL SECURITY;
            ALTER TABLE invoices FORCE ROW LEVEL SECURITY;
```

```
    ALTER TABLE appointments ENABLE ROW LEVEL SECURITY;
    ALTER TABLE appointments FORCE ROW LEVEL SECURITY;

    -- ????? ?????? ?????
    DROP POLICY IF EXISTS dry_run_patients_policy ON patients;
    CREATE POLICY dry_run_patients_policy ON patients
        FOR ALL
        USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
        WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

    DROP POLICY IF EXISTS dry_run_invoices_policy ON invoices;
    CREATE POLICY dry_run_invoices_policy ON invoices
        FOR ALL
        USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
        WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

    DROP POLICY IF EXISTS dry_run_appointments_policy ON appointments;
    CREATE POLICY dry_run_appointments_policy ON appointments
        FOR ALL
        USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
        WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

    -- ????? ?? ??? ???? ?????????? ?????? ?? ????? ??? ????????
    DO $$
    BEGIN
        IF EXISTS (SELECT FROM pg_roles WHERE rolname = 'test_rls_user') THEN
            REVOKE ALL ON patients, invoices, appointments FROM test_rls_user;
            REVOKE ALL ON ALL SEQUENCES IN SCHEMA public FROM test_rls_user;
            DROP ROLE test_rls_user;
        END IF;
    END $$;

    -- ????? ??? ?????? ??? ???? ??????? ??????????
    CREATE ROLE test_rls_user WITH LOGIN;
    GRANT SELECT, INSERT, UPDATE, DELETE ON patients, invoices, appointments TO test_rls_user;
    GRANT USAGE, SELECT ON ALL SEQUENCES IN SCHEMA public TO test_rls_user;
`);

console.log(`  ${GREEN}? ?? ????? RLS ?????? ??? ?????? ???????? (test_rls_user).${RESET}`);

// ????? ?? ?????? ??????? ????? ???? ??????? ??????? ????? ???????
await client.query(`
    DELETE FROM appointments WHERE patient_name IN ('TEST_PATIENT_T1', 'TEST_PATIENT_T2');
    DELETE FROM invoices WHERE invoice_number IN ('INV-T1-DRY', 'INV-T2-DRY');
    DELETE FROM patients WHERE name_ar IN ('TEST_PATIENT_T1', 'TEST_PATIENT_T2');
`);

// ????? ?????? ??? ??? ?????? ???????? ??? ??????
await client.query("SET ROLE test_rls_user");

// ?????? 1: ????? ?????? ??????? 1 ??? ????? ?????
console.log(`\n[5.1] ?????? ????? ?????? ???????? 1 (tenant_id = 1)...`);
await client.query("SET app.tenant_id = '1'");
```

```
        await client.query("INSERT INTO patients (name_ar, tenant_id) VALUES ('TEST_PATIENT_T1', 1)");
        await client.query("INSERT INTO invoices (invoice_number, amount, tenant_id) VALUES ('INV-T1-DRY', 100
, 1)");
        await client.query("INSERT INTO appointments (patient_name, appt_date, tenant_id) VALUES ('TEST_PATIEN
T_T1', '2026-06-20', 1)");
        console.log(`  ${GREEN}? ?? ????? ?????? ?????? 1 ?????.${RESET}`);

        // ?????? 2: ????? ?????? ??????? 2 ??? ????? ?????
        console.log(`\n[5.2] ?????? ????? ?????? ???????? 2 (tenant_id = 2)...`);
        await client.query("SET app.tenant_id = '2'");
        await client.query("INSERT INTO patients (name_ar, tenant_id) VALUES ('TEST_PATIENT_T2', 2)");
        await client.query("INSERT INTO invoices (invoice_number, amount, tenant_id) VALUES ('INV-T2-DRY', 200
, 2)");
        await client.query("INSERT INTO appointments (patient_name, appt_date, tenant_id) VALUES ('TEST_PATIEN
T_T2', '2026-06-21', 2)");
        console.log(`  ${GREEN}? ?? ????? ?????? ?????? 2 ?????.${RESET}`);

        // ?????? 3: ?????? ?? ??????? ?????? ??????? 1
        console.log(`\n[5.3] ?????? ?? ??? ??????? ???????? 1...`);
        await client.query("SET app.tenant_id = '1'");

        const resPatientsT1 = await client.query("SELECT * FROM patients WHERE name_ar LIKE 'TEST_PATIENT%'");
        const resInvoicesT1 = await client.query("SELECT * FROM invoices WHERE invoice_number LIKE '%DRY'");
        const resApptsT1 = await client.query("SELECT * FROM appointments WHERE patient_name LIKE 'TEST_PATIEN
T%'");

        if (resPatientsT1.rows.length === 1 && resPatientsT1.rows[0].name_ar === 'TEST_PATIENT_T1' &&
            resInvoicesT1.rows.length === 1 && resInvoicesT1.rows[0].invoice_number === 'INV-T1-DRY' &&
            resApptsT1.rows.length === 1 && resApptsT1.rows[0].patient_name === 'TEST_PATIENT_T1') {
            console.log(`  ${GREEN}? ???: ???????? 1 ??? ??????? ?????? ??? ??? ????? ?? ?????? ???????? 2.${R
ESET}`);
        } else {
            throw new Error(`??? ??? ??????? ???????? 1. ??????? ?????????: ${JSON.stringify({ patients: resPa
tientsT1.rows, invoices: resInvoicesT1.rows, appointments: resApptsT1.rows })}`);
        }

        // ?????? 4: ?????? ?? ??????? ?????? ??????? 2
        console.log(`\n[5.4] ?????? ?? ??? ??????? ???????? 2...`);
        await client.query("SET app.tenant_id = '2'");

        const resPatientsT2 = await client.query("SELECT * FROM patients WHERE name_ar LIKE 'TEST_PATIENT%'");
        const resInvoicesT2 = await client.query("SELECT * FROM invoices WHERE invoice_number LIKE '%DRY'");
        const resApptsT2 = await client.query("SELECT * FROM appointments WHERE patient_name LIKE 'TEST_PATIEN
T%'");

        if (resPatientsT2.rows.length === 1 && resPatientsT2.rows[0].name_ar === 'TEST_PATIENT_T2' &&
            resInvoicesT2.rows.length === 1 && resInvoicesT2.rows[0].invoice_number === 'INV-T2-DRY' &&
            resApptsT2.rows.length === 1 && resApptsT2.rows[0].patient_name === 'TEST_PATIENT_T2') {
            console.log(`  ${GREEN}? ???: ???????? 2 ??? ??????? ?????? ??? ??? ????? ?? ?????? ???????? 1.${R
ESET}`);
        } else {
            throw new Error(`??? ??? ??????? ???????? 2. ??????? ?????????: ${JSON.stringify({ patients: resPa
tientsT2.rows, invoices: resInvoicesT2.rows, appointments: resApptsT2.rows })}`);
        }
```

```
        // ?????? 5: ??? ????? ??? ?????? (INSERT Mismatch Prevention)
        console.log(`\n[5.5] ?????? ?? ??? ????? ????? ??? ??????? ?? ???? ?????? (IDOR Prevention)...`);
        await client.query("SET app.tenant_id = '1'");
        try {
            await client.query("INSERT INTO patients (name_ar, tenant_id) VALUES ('TEST_PATIENT_FRAUD', 2)");
            throw new Error('?? ?????? ?????? ??? ?????? 2 ????? ????? ?????? ??????? 1!');
        } catch (err) {
            if (err.message.includes('row-level security policy') || err.code === '42501') {
                console.log(`  ${GREEN}? ???: ?? ????? ?????? ??????? ??? ?????? ?? ????? ?? ????? ????????.${
RESET}`);
            } else {
                throw err;
            }
        }

        // ?????? 6: ??? ????? ??? ??? ?????? ??? (UPDATE Isolation)
        console.log(`\n[5.6] ?????? ?? ??? ????? ????? ???? ???????? ???...`);
        await client.query("SET app.tenant_id = '1'");
        const updateRes = await client.query("UPDATE patients SET name_ar = 'HACKED' WHERE name_ar = 'TEST_PAT
IENT_T2'");
        if (updateRes.rowCount === 0) {
            console.log(`  ${GREEN}? ???: ?? ???? ??????? ??? ?? ????? ?????? ?????? ??? (???? ??? 0 ????).${R
ESET}`);
        } else {
            throw new Error('?? ????? ??? ?????? 2 ????? ?? ??? ?????? 1!');
        }

        // ?????? 7: ??? ????? ????????? (Aggregates check)
        console.log(`\n[5.7] ?????? ?? ??? ???????? ????????? (SUM/COUNT)...`);
        await client.query("SET app.tenant_id = '1'");
        const aggregateRes = await client.query("SELECT COUNT(*) as cnt, SUM(amount) as total FROM invoices WH
ERE invoice_number LIKE '%DRY'");
        const cnt = parseInt(aggregateRes.rows[0].cnt);
        const total = parseFloat(aggregateRes.rows[0].total);
        if (cnt === 1 && total === 100.0) {
            console.log(`  ${GREEN}? ???: ???????? ????????? ????? ??? ??? ?????? ???????? ?????? (?????: 1? ?
??????: 100).${RESET}`);
        } else {
            throw new Error(`???? ???????? ????????? ????????. ?????: ${cnt}? ???????: ${total}`);
        }

        // ????? ?????? ??? ??? ???? ???? ??????? ?????? ???????? ?????? RLS
        await client.query("RESET ROLE");

        // ????? ?????? ???????? ???????
        await client.query(`
            DELETE FROM appointments WHERE patient_name IN ('TEST_PATIENT_T1', 'TEST_PATIENT_T2');
            DELETE FROM invoices WHERE invoice_number IN ('INV-T1-DRY', 'INV-T2-DRY');
            DELETE FROM patients WHERE name_ar IN ('TEST_PATIENT_T1', 'TEST_PATIENT_T2');
        `);

        // ------------------------------------------------------------------------
        // 4. ??????? ?????? (Rollback)
```

```
        // ------------------------------------------------------------------------
        console.log(`\n[6] ??? ????? ??????? ?????? RLS (Local Rollback)...`);
        await client.query(`
            ALTER TABLE patients NO FORCE ROW LEVEL SECURITY;
            ALTER TABLE invoices NO FORCE ROW LEVEL SECURITY;
            ALTER TABLE appointments NO FORCE ROW LEVEL SECURITY;

            ALTER TABLE patients DISABLE ROW LEVEL SECURITY;
            ALTER TABLE invoices DISABLE ROW LEVEL SECURITY;
            ALTER TABLE appointments DISABLE ROW LEVEL SECURITY;

            DROP POLICY IF EXISTS dry_run_patients_policy ON patients;
            DROP POLICY IF EXISTS dry_run_invoices_policy ON invoices;
            DROP POLICY IF EXISTS dry_run_appointments_policy ON appointments;

            DO $$
            BEGIN
                IF EXISTS (SELECT FROM pg_roles WHERE rolname = 'test_rls_user') THEN
                    REVOKE ALL ON patients, invoices, appointments FROM test_rls_user;
                    REVOKE ALL ON ALL SEQUENCES IN SCHEMA public FROM test_rls_user;
                    DROP ROLE test_rls_user;
                END IF;
            END $$;
        `);
        console.log(`  ${GREEN}? ?? ????? ????? RLS ???? ???? ???????? ????????? ?????? ??????.${RESET}`);

        // ------------------------------------------------------------------------
        // 5. ?????? ??? ???????
        // ------------------------------------------------------------------------
        console.log(`\n[7] ?????? ??????? ?? ????? RLS...`);
        // ???? ?????? ????? ????????? ??????? ??? ????? ?????? ?????? ?? ???? ?????? ??????? ????????
        await client.query("INSERT INTO patients (name_ar, tenant_id) VALUES ('TEST_PATIENT_T1', 1)");
        await client.query("INSERT INTO patients (name_ar, tenant_id) VALUES ('TEST_PATIENT_T2', 2)");

        const finalCheck = await client.query("SELECT * FROM patients WHERE name_ar LIKE 'TEST_PATIENT%'");
        if (finalCheck.rows.length === 2) {
            console.log(`  ${GREEN}? ???: RLS ???? ?????? ????? ??????? ?????? ???? ???? ??????? ??? ???? ????
????.${RESET}`);
        } else {
            throw new Error(`??? ?????? ?? ????? RLS. ??? ???????: ${finalCheck.rows.length}`);
        }

        // ????? ?????
        await client.query("DELETE FROM patients WHERE name_ar LIKE 'TEST_PATIENT%'");

    } catch (error) {
        console.error(`\n${BOLD}${RED}? ??? ??? ????? ????? ???????!${RESET}`);
        console.error(error);

        console.log(`\n${YELLOW}?? ?????? ????? ????? ????? (Emergency Rollback) ?????? ????? ????????...${RES
ET}`);
        try {
            await client.query("RESET ROLE");
            await client.query(`
```

```
                ALTER TABLE patients NO FORCE ROW LEVEL SECURITY;
                ALTER TABLE invoices NO FORCE ROW LEVEL SECURITY;
                ALTER TABLE appointments NO FORCE ROW LEVEL SECURITY;

                ALTER TABLE patients DISABLE ROW LEVEL SECURITY;
                ALTER TABLE invoices DISABLE ROW LEVEL SECURITY;
                ALTER TABLE appointments DISABLE ROW LEVEL SECURITY;

                DROP POLICY IF EXISTS dry_run_patients_policy ON patients;
                DROP POLICY IF EXISTS dry_run_invoices_policy ON invoices;
                DROP POLICY IF EXISTS dry_run_appointments_policy ON appointments;

                DO $$
                BEGIN
                    IF EXISTS (SELECT FROM pg_roles WHERE rolname = 'test_rls_user') THEN
                        REVOKE ALL ON patients, invoices, appointments FROM test_rls_user;
                        REVOKE ALL ON ALL SEQUENCES IN SCHEMA public FROM test_rls_user;
                        DROP ROLE test_rls_user;
                    END IF;
                END $$;
            `);
            console.log(`  ${GREEN}? ??? ????? ??????? ??? ????? ???? ???????.${RESET}`);
        } catch (rollbackErr) {
            console.error(`  ${RED}? ??? ????? ???????! ?? ???? ????? ???????? ?? ???? ??? ??????.${RESET}`, r
ollbackErr);
        }
        process.exit(1);
    } finally {
        await client.end();
        console.log(`\n${BOLD}${GREEN}? ????? ?????? RLS ???????? ?????? ????? 100% ??? ????? ????? ??????? ??
??????.${RESET}\n`);
    }
}

// ????? ????????
checkEnvironment();
runBackup();
executeDryRun();



========================================
FILE: route_schemas.js
========================================

/**
 * route_schemas.js ? accurate validation schemas for high-value financial/clinical routes (GATE3-H1).
 *
 * These schemas are derived by reading the ACTUAL req.body usage of each route, so they are
 * NON-BREAKING: every field a route currently accepts is allowed; money fields that are already
 * validated server-side by ./billing_integrity (parseMoney/enforceDiscountCap) and engine-validated
 * structures (journal lines via finance_engine.validateBalancedEntry) are intentionally LEFT to those
 * authorities and not duplicated here.
 *
```

```
 * Usage (deferred to staging integration test ? PHASE 1.2):
 *   const { validateBody } = require('./validation');
 *   const S = require('./route_schemas');
 *   app.post('/api/invoices', requireAuth, requireRole('invoices','accounts'), validateBody(S.invoiceCreate),
 handler)
 *
 * Each schema uses ./validation specs. Most fields are OPTIONAL (the routes accept partial bodies and
 * the permissive DB schema defaults blanks) ? we enforce TYPE, LENGTH, ENUM, and DATE validity only,
 * which strictly hardens without rejecting any currently-valid request.
 */
'use strict';

// POST /api/invoices ? body: patient_id?, patient_name?, description?, service_type?, payment_method?,
// discount_reason?  (total/discount validated by billing_integrity.parseMoney; not duplicated here)
const invoiceCreate = {
    patient_id:      { type: 'id',  required: false },
    patient_name:    { type: 'str', required: false, max: 200 },
    description:     { type: 'str', required: false, max: 1000 },
    service_type:    { type: 'str', required: false, max: 100 },
    payment_method:  { type: 'str', required: false, max: 40 },
    discount_reason: { type: 'str', required: false, max: 500 }
};

// POST /api/finance/journal ? body: entry_date, description?, reference?, source_type?
// (lines[] balance is validated by finance_engine.validateBalancedEntry; not duplicated here)
const journalCreate = {
    entry_date:  { type: 'dateStr', required: true },
    description: { type: 'str', required: false, max: 1000 },
    reference:   { type: 'str', required: false, max: 200 },
    source_type: { type: 'enumOf', allowed: ['MANUAL', 'INVOICE', 'SYSTEM'], required: false }
};

// POST /api/invoices/:id/refund ? body: amount (validated by billing_integrity), reason?
const invoiceRefund = {
    reason: { type: 'str', required: false, max: 500 }
};

// POST /api/patients ? common PHI registration fields. national_id is LENIENT (non-Saudi patients use
// iqama/passport which are NOT 10 digits), so it is a bounded string, not the strict 10-digit validator.
const patientCreate = {
    name_ar:     { type: 'str', required: false, max: 200 },
    name_en:     { type: 'str', required: false, max: 200 },
    national_id: { type: 'str', required: false, max: 30 },   // lenient: iqama/passport allowed
    phone:       { type: 'phone', required: false },
    gender:      { type: 'enumOf', allowed: ['Male', 'Female', 'male', 'female', 'M', 'F', '???', '????', ''],
 required: false },
    dob:         { type: 'str', required: false, max: 30 }
};

module.exports = { invoiceCreate, journalCreate, invoiceRefund, patientCreate };



========================================
```

FILE: route_schemas_test.js

```
/**
 * route_schemas_test.js ? pure unit tests for ./route_schemas via ./validation (no DB).
 * Verifies the schemas accept real valid payloads and reject malformed ones, and that they are
 * NON-BREAKING (a minimal/partial body still passes). Run: node route_schemas_test.js
 */
'use strict';
const { validate, ValidationError } = require('./validation');
const S = require('./route_schemas');

let pass = 0, fail = 0;
const ok = (n, c) => { if (c) pass++; else { fail++; console.error('  FAIL:', n); } };
const accepts = (n, schema, body) => { try { validate(body, schema); ok(n, true); } catch (e) { ok(n + ' (' +
e.message + ')', false); } };
const rejects = (n, schema, body) => { try { validate(body, schema); ok(n + ' (should reject)', false); } catc
h (e) { ok(n, e instanceof ValidationError); } };

// invoiceCreate ? valid full + partial + empty (non-breaking)
accepts('invoice full', S.invoiceCreate, { patient_id: 12, patient_name: 'Ahmad', description: 'CBC', service_
type: 'lab', payment_method: 'cash', discount_reason: 'loyalty' });
accepts('invoice partial', S.invoiceCreate, { patient_name: 'Walk-in' });
accepts('invoice empty (non-breaking)', S.invoiceCreate, {});
rejects('invoice bad patient_id', S.invoiceCreate, { patient_id: 'abc' });
rejects('invoice patient_id zero', S.invoiceCreate, { patient_id: 0 });
rejects('invoice over-long name', S.invoiceCreate, { patient_name: 'x'.repeat(201) });

// journalCreate ? entry_date required + valid; source_type enum
accepts('journal valid', S.journalCreate, { entry_date: '2026-06-30', description: 'd', reference: 'JV-1', sou
rce_type: 'MANUAL' });
accepts('journal minimal', S.journalCreate, { entry_date: '2026-06-30' });
rejects('journal missing date', S.journalCreate, { description: 'no date' });
rejects('journal bad date', S.journalCreate, { entry_date: 'yesterday' });
rejects('journal bad source_type', S.journalCreate, { entry_date: '2026-06-30', source_type: 'HACK' });

// invoiceRefund ? reason optional
accepts('refund with reason', S.invoiceRefund, { reason: 'duplicate charge' });
accepts('refund empty', S.invoiceRefund, {});
rejects('refund over-long reason', S.invoiceRefund, { reason: 'x'.repeat(501) });

// patientCreate ? lenient national_id (non-Saudi), phone validated, gender enum
accepts('patient saudi id', S.patientCreate, { name_ar: '????', national_id: '1234567890', phone: '+9665012345
67', gender: 'Female' });
accepts('patient passport id (non-Saudi)', S.patientCreate, { name_en: 'John', national_id: 'P1234567', gender
: 'Male' });
accepts('patient minimal', S.patientCreate, {});
rejects('patient bad phone', S.patientCreate, { phone: 'abc' });
rejects('patient bad gender', S.patientCreate, { gender: 'unknown-x' });
rejects('patient over-long national_id', S.patientCreate, { national_id: 'x'.repeat(31) });

console.log(`route_schemas_test: ${pass} passed, ${fail} failed`);
process.exit(fail === 0 ? 0 : 1);
```

```
========================================
FILE: run_all_tests.js
========================================

const { execSync } = require('child_process');
const fs = require('fs');
const path = require('path');

const testDir = __dirname;
const hasDocs = fs.existsSync(path.join(__dirname, '../docs'));

// Filter out tests requiring full parent repo (docs) or database schema initialization if docs is missing
const files = fs.readdirSync(testDir).filter(f => {
  if (!f.endsWith('test.js') || f === 'e2e_local_smoke_test.js') return false;
  if (!hasDocs) {
    if (f === 'cross_tenant_catalog_override_test.js' || f === 'cross_tenant_surgery_or_test.js') {
      console.log(`Skipping ${f} (production environment, missing docs schema)`);
      return false;
    }
  }
  return true;
});

console.log(`Found ${files.length} test files to run.`);

let passed = 0;
let failed = 0;
const failures = [];

for (const file of files) {
  try {
    execSync(`node "${path.join(testDir, file)}"`, { stdio: 'ignore', env: process.env });
    passed++;
  } catch (err) {
    failed++;
    failures.push(file);
  }
}

console.log('\n--- TEST SUMMARY ---');
console.log(`Total test files run: ${files.length}`);
console.log(`Passed: ${passed}`);
console.log(`Failed: ${failed}`);
if (failures.length > 0) {
  console.log('Failing files:');
  failures.forEach(f => console.log(` - ${f}`));
  process.exit(1);
} else {
  console.log('All tests passed successfully!');
  process.exit(0);
}
```

```
========================================
FILE: run_nahdi_scrape.py
========================================

import time
import sqlite3
import undetected_chromedriver as uc
from selenium.webdriver.common.by import By

def run_scraper():
    print("Starting Nahdi Scraper...")
    try:
        conn = sqlite3.connect("database.db")
        cur = conn.cursor()

        # Ensure tables exist
        cur.execute("""
        CREATE TABLE IF NOT EXISTS medications (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            name TEXT NOT NULL,
            active_ingredient TEXT,
            stock_quantity INTEGER DEFAULT 0,
            price REAL DEFAULT 0.0,
            created_at DATETIME DEFAULT CURRENT_TIMESTAMP
        )""")

        options = uc.ChromeOptions()
        # Headless mode is usually detected by Cloudflare, but we can try False
        options.headless = False
        driver = uc.Chrome(options=options)

        base_url = "https://www.nahdionline.com/ar-sa/plp/nahdi-global?page="

        print("Driver started. Bypassing WAF...")
        driver.get("https://www.nahdionline.com/ar-sa/plp/nahdi-global")
        time.sleep(10)

        total_inserted = 0

        for page in range(1, 201):
            print(f"Scraping page {page} of 200...")
            driver.get(f"{base_url}{page}")
            time.sleep(6) # Wait for Next.js to render

            # Scroll to trigger lazy loading
            for _ in range(5):
                driver.execute_script("window.scrollBy(0, 1000);")
                time.sleep(1)

            products = driver.find_elements(By.CSS_SELECTOR, ".product-item-info")
            found = 0
```

```
            for p in products:
                try:
                    name_el = p.find_element(By.CSS_SELECTOR, ".product-item-link")
                    name = name_el.text.strip()

                    price_str = p.find_element(By.CSS_SELECTOR, ".price").text.replace("?.?", "").strip()
                    price = float(price_str) if price_str else 0.0
                    cost = round(price * 0.7, 2)
                    stock = 50
                    category = "???"
                    active = "??? ????"

                    if name and price >= 0:
                        # Check exist
                        cur.execute("SELECT id FROM medications WHERE name = ?", (name,))
                        if not cur.fetchone():
                            cur.execute("INSERT INTO medications (name, active_ingredient, stock_quantity, pri
ce) VALUES (?, ?, ?, ?)", (name, active, stock, price))

                        cur.execute("SELECT id FROM pharmacy_drug_catalog WHERE drug_name = ?", (name,))
                        if not cur.fetchone():
                            cur.execute("INSERT INTO pharmacy_drug_catalog (drug_name, active_ingredient, cate
gory, selling_price, cost_price, stock_qty) VALUES (?, ?, ?, ?, ?, ?)", (name, active, category, price, cost,
stock))
                            found += 1
                            total_inserted += 1
                except Exception as e:
                    pass

            conn.commit()
            print(f"Page {page} done! Inserted {found} new drugs. Total inserted so far: {total_inserted}")

    except Exception as e:
        print(f"Error occurred: {e}")
    finally:
        try:
            driver.quit()
        except:
            pass
        try:
            conn.close()
        except:
            pass

if __name__ == "__main__":
    run_scraper()



========================================
FILE: run_safe_tests.js
========================================

/**
```

```
 * run_safe_tests.js ? runs ONLY the DB-free unit tests (no database/server/network), so it is safe to
 * run anywhere (local dev machine where the only DB is production, or CI with no DB). Complements
 * run_all_tests.js (which needs a provisioned DB for the cross-tenant/RLS/e2e integration tests).
 *
 * A test is treated as DB/server-dependent (and SKIPPED here) if it statically references the DB layer,
 * pg, the server, http, or a spawned process. Everything else is a pure unit test and is executed.
 *
 * Safety net: DB_NAME is pointed at a non-existent guard database and PGCONNECT_TIMEOUT is tiny, so if
 * a test were ever mis-classified it fails fast against a non-existent DB instead of touching production.
 *
 * Exit code 0 only if every executed test passes.
 */
'use strict';
const { execFileSync } = require('child_process');
const fs = require('fs');
const path = require('path');

const dir = __dirname;
const DB_SIGNALS = /require\(['"]\.\/(db_postgres|database)['"]\)|require\(['"]pg['"]\)|require\(['"]\.\/serve
r['"]\)|require\(['"]http['"]\)|require\(['"]supertest['"]\)|\bpool\b|withTenant|tenantStore|initDatabase|spaw
n|app\.listen|http:\/\/localhost|127\.0\.0\.1/;

const all = fs.readdirSync(dir).filter(f => f.endsWith('_test.js') && f !== 'run_safe_tests.js');
const safe = [], skipped = [];
for (const f of all) {
    const src = fs.readFileSync(path.join(dir, f), 'utf8');
    (DB_SIGNALS.test(src) ? skipped : safe).push(f);
}

// Shield: any accidental DB connection hits a non-existent DB, never production.
const env = { ...process.env, DB_NAME: 'nama_safe_guard_nonexistent', PGDATABASE: 'nama_safe_guard_nonexistent
', PGCONNECT_TIMEOUT: '2', NODE_ENV: 'test' };

console.log(`run_safe_tests: ${safe.length} DB-free tests to run, ${skipped.length} skipped (need DB/server).`
);
let passed = 0, failed = 0; const failures = [];
for (const f of safe) {
    try {
        execFileSync(process.execPath, [f], { cwd: dir, env, stdio: 'pipe', timeout: 120000, maxBuffer: 16 * 1
024 * 1024 });
        passed++;
    } catch (e) {
        failed++; failures.push(f);
        const tail = String((e.stdout || '') + (e.stderr || '')).split('\n').filter(Boolean).slice(-3).join('
| ');
        console.error(`FAIL  ${f}  [Err: ${e.message}]  ${tail}`);
    }
}
console.log(`\nrun_safe_tests: ${passed} passed, ${failed} failed (of ${safe.length}).`);
if (skipped.length) console.log(`skipped (need provisioned DB): ${skipped.length} ? run via run_all_tests.js o
n an isolated DB.`);
process.exit(failed === 0 ? 0 : 1);
```

```
========================================
FILE: safety_crosscheck_test.js
========================================

/**
 * safety_crosscheck_test.js ? Integration test for the Drug-Allergy cross-check safety endpoint.
 */
'use strict';

process.env.NODE_ENV = 'staging';

const { spawn } = require('child_process');
const http = require('http');
const assert = require('assert');
const bcrypt = require('bcryptjs');
const { pool } = require('./db_postgres');

const TEST_PORT = 3020;
const TEST_USERNAME = 'safety_doc';
const TEST_PASSWORD = 'SAFETY_PASSWORD_PLACEHOLDER';

let serverProcess;

function makeRequest(method, path, body, headers = {}) {
    return new Promise((resolve, reject) => {
        const payload = body ? JSON.stringify(body) : '';
        const req = http.request({
            hostname: 'localhost',
            port: TEST_PORT,
            path,
            method,
            headers: {
                'Content-Type': 'application/json',
                'Content-Length': Buffer.byteLength(payload),
                ...headers
            }
        }, (res) => {
            let data = '';
            res.on('data', chunk => data += chunk);
            res.on('end', () => {
                try {
                    const parsed = data ? JSON.parse(data) : {};
                    resolve({ statusCode: res.statusCode, headers: res.headers, body: parsed });
                } catch (e) {
                    resolve({ statusCode: res.statusCode, headers: res.headers, rawBody: data });
                }
            });
        });
        req.on('error', reject);
        if (payload) req.write(payload);
        req.end();
    });
}
```

```
async function runTests() {
    console.log('--- STARTING DRUG-ALLERGY SAFETY CROSS-CHECK INTEGRATION TESTS ---');

    const patientId = 9961;
    const doctorUserId = 9962;
    const client = await pool.connect();

    try {
        console.log('Setting up test data...');
        await client.query("SET app.tenant_id = '1'");

        // Clean up old test data
        await client.query('DELETE FROM patients WHERE id = $1', [patientId]);
        await client.query('DELETE FROM user_tenants WHERE user_id = $1', [doctorUserId]);
        await client.query('DELETE FROM system_users WHERE id = $1', [doctorUserId]);

        // Insert patient with allergy documented in notes
        await client.query(
            "INSERT INTO patients (id, name_en, name_ar, notes, tenant_id) VALUES ($1, $2, $3, 'Allergic to Pe
nicillin and Sulfa drugs', 1)",
            [patientId, 'Allergy Test Patient', '???? ??? ????????']
        );

        // Insert doctor user
        const hashedPassword = await bcrypt.hash(TEST_PASSWORD, 10);
        await client.query(
            "INSERT INTO system_users (id, username, password_hash, display_name, role, speciality, permission
s, is_active) VALUES ($1, $2, $3, $4, $5, $6, $7, 1)",
            [doctorUserId, TEST_USERNAME, hashedPassword, 'Dr. Safety Officer', 'Doctor', 'General Medicine',
'["patients", "prescriptions"]']
        );

        // Associate doctor with tenant 1
        await client.query(
            'INSERT INTO user_tenants (user_id, tenant_id, is_active) VALUES ($1, 1, true)',
            [doctorUserId]
        );

        console.log('Spawning test server...');
        serverProcess = spawn('node', ['server.js'], {
            env: { ...process.env, PORT: TEST_PORT, NODE_ENV: 'staging', SKIP_DB_INIT: 'true' }
        });

        serverProcess.stdout.on('data', (data) => {
            console.log(`[Server STDOUT] ${data.toString().trim()}`);
        });
        serverProcess.stderr.on('data', (data) => {
            console.error(`[Server STDERR] ${data.toString().trim()}`);
        });

        // Wait for server to start
        await new Promise(resolve => setTimeout(resolve, 3000));
```

```
        console.log('Logging in...');
        const loginRes = await makeRequest('POST', '/api/auth/login', {
            username: TEST_USERNAME,
            password: TEST_PASSWORD
        });
        assert.strictEqual(loginRes.statusCode, 200, 'Login should succeed');
        const cookie = loginRes.headers['set-cookie'][0];

        // Test 1: Conflict expected for Penicillin
        console.log('Testing safety check for allergic drug (Penicillin)...');
        const check1 = await makeRequest('POST', '/api/clinical/safety-check', {
            patient_id: patientId,
            drug_name: 'Penicillin'
        }, { Cookie: cookie });

        assert.strictEqual(check1.statusCode, 200, 'Allergy check failed: ' + JSON.stringify(check1.body || ch
eck1.rawBody));
        assert.strictEqual(check1.body.alert, true, 'Alert should be true for Penicillin');
        assert.ok(check1.body.message.includes('Potential allergy conflict'), 'Message should contain warning'
);
        console.log('? Penicillin conflict detected successfully.');

        // Test 2: No conflict expected for Aspirin
        console.log('Testing safety check for non-allergic drug (Aspirin)...');
        const check2 = await makeRequest('POST', '/api/clinical/safety-check', {
            patient_id: patientId,
            drug_name: 'Aspirin'
        }, { Cookie: cookie });

        assert.strictEqual(check2.statusCode, 200);
        assert.strictEqual(check2.body.alert, false, 'Alert should be false for Aspirin');
        console.log('? Aspirin check cleared successfully.');

        console.log('Cleaning up test data...');
        await client.query('DELETE FROM patients WHERE id = $1', [patientId]);
        await client.query('DELETE FROM user_tenants WHERE user_id = $1', [doctorUserId]);
        await client.query('DELETE FROM system_users WHERE id = $1', [doctorUserId]);

        console.log('Killing test server...');
        serverProcess.kill();
        console.log('? Drug-Allergy Safety Cross-Check Integration Tests passed successfully!');
        process.exit(0);

    } catch (e) {
        console.error('? Test failed:', e);
        if (serverProcess) serverProcess.kill();
        process.exit(1);
    } finally {
        client.release();
    }
}

runTests();
```

```
========================================
FILE: saudi_compliance_test.js
========================================

/**
 * saudi_compliance_test.js
 * Integration test for Saudi regulatory compliance upgrades (ZATCA Phase 2, CBAHI OVR, PDPL Consent).
 */

const { spawn } = require('child_process');
const http = require('http');
const assert = require('assert');
const bcrypt = require('bcryptjs');
const { pool } = require('./db_postgres');

const TEST_PORT = 3009;
const TEST_USERNAME = 'compliance_admin';
const TEST_PASSWORD = 'COMPLIANCE_PASSWORD_PLACEHOLDER';

let server;

function makeRequest(method, path, body, headers = {}) {
    return new Promise((resolve, reject) => {
        const payload = body ? JSON.stringify(body) : '';
        const req = http.request({
            hostname: 'localhost',
            port: TEST_PORT,
            path,
            method,
            headers: {
                'Content-Type': 'application/json',
                'Content-Length': Buffer.byteLength(payload),
                ...headers
            }
        }, (res) => {
            let data = '';
            res.on('data', chunk => data += chunk);
            res.on('end', () => {
                try {
                    const parsed = data ? JSON.parse(data) : {};
                    resolve({ statusCode: res.statusCode, headers: res.headers, body: parsed });
                } catch (e) {
                    resolve({ statusCode: res.statusCode, headers: res.headers, rawBody: data });
                }
            });
        });
        req.on('error', reject);
        if (payload) req.write(payload);
        req.end();
    });
}
```

```
async function runTests() {
    console.log('--- STARTING SAUDI COMPLIANCE INTEGRATION TESTS ---');

    // 1. Setup compliance test data
    const patientId = 8881;
    const adminUserId = 8882;
    const client = await pool.connect();

    try {
        await client.query("SET app.tenant_id = '1'");

        // Clean up
        await client.query('DELETE FROM invoices WHERE patient_id = $1', [patientId]);
        await client.query('DELETE FROM patients WHERE id = $1', [patientId]);
        await client.query('DELETE FROM quality_incidents WHERE reported_by = $1', ['Compliance Test Reporter'
]);
        await client.query('DELETE FROM user_tenants WHERE user_id = $1', [adminUserId]);
        await client.query('DELETE FROM system_users WHERE id = $1', [adminUserId]);

        // Insert patient
        await client.query(
            'INSERT INTO patients (id, name_en, name_ar, phone, tenant_id) VALUES ($1, $2, $3, $4, 1)',
            [patientId, 'Compliance Patient', '???? ????????', '+966555555555']
        );

        // Insert admin user
        const hashedPassword = await bcrypt.hash(TEST_PASSWORD, 10);
        await client.query(
            'INSERT INTO system_users (id, username, password_hash, display_name, role, speciality, permission
s, is_active) VALUES ($1, $2, $3, $4, $5, $6, $7, 1)',
            [adminUserId, TEST_USERNAME, hashedPassword, 'Compliance Admin', 'Admin', 'General', '["patients",
 "invoices", "quality"]']
        );

        // Associate user with tenant 1
        await client.query('INSERT INTO user_tenants (user_id, tenant_id, is_active) VALUES ($1, 1, true)', [a
dminUserId]);

    } finally {
        client.release();
    }

    // 2. Start local server
    server = spawn('node', ['server.js'], {
        env: { ...process.env, PORT: TEST_PORT, SKIP_DB_INIT: '1' }
    });

    server.stdout.on('data', (data) => {
        // console.log('SERVER:', data.toString());
    });
    server.stderr.on('data', (data) => {
        console.error('SERVER ERR:', data.toString());
    });
```

```
    // Wait for server to boot
    await new Promise(resolve => setTimeout(resolve, 2500));

    try {
        // Log in
        console.log('Logging in...');
        const loginRes = await makeRequest('POST', '/api/auth/login', {
            username: TEST_USERNAME,
            password: TEST_PASSWORD
        });
        assert.strictEqual(loginRes.statusCode, 200, 'Login should succeed');
        const setCookie = loginRes.headers['set-cookie'];
        assert.ok(setCookie, 'Should receive set-cookie header');
        const cookie = setCookie[0].split(';')[0];
        const authHeaders = { 'Cookie': cookie };

        // --- Test Case 1: PDPL Privacy Consent ---
        console.log('Testing Case 1: PDPL Patient Privacy Consent...');
        const consentRes = await makeRequest('POST', `/api/patients/${patientId}/consent`, {}, authHeaders);
        assert.strictEqual(consentRes.statusCode, 200, 'Consent signing should succeed');
        assert.strictEqual(consentRes.body.success, true);

        // Verify database state
        const dbPatient = (await pool.query('SELECT privacy_consent_signed, privacy_consent_date FROM patients
 WHERE id = $1', [patientId])).rows[0];
        assert.strictEqual(dbPatient.privacy_consent_signed, true, 'privacy_consent_signed should be true');
        assert.ok(dbPatient.privacy_consent_date, 'privacy_consent_date should be populated');
        console.log('? Patient privacy consent successfully signed and verified in DB.');

        // --- Test Case 2: ZATCA Phase 2 Invoice Chaining & Hashing ---
        console.log('Testing Case 2: ZATCA Phase 2 Invoice Chaining & Hashing...');

        // Generate Invoice 1
        const inv1Res = await makeRequest('POST', '/api/invoices/generate', {
            patient_id: patientId,
            items: [{ description: 'Consultation', amount: '100.00' }]
        }, authHeaders);
        assert.strictEqual(inv1Res.statusCode, 200, 'Invoice 1 generation should succeed');
        const inv1 = inv1Res.body;
        assert.ok(inv1.invoice_hash, 'Invoice 1 should have a hash');
        assert.strictEqual(inv1.previous_invoice_hash, '000000000000000000000000000000000000000000000000000000000
0000000000', 'First invoice previous hash should be zero sentinel');

        // Generate Invoice 2
        const inv2Res = await makeRequest('POST', '/api/invoices/generate', {
            patient_id: patientId,
            items: [{ description: 'Follow-up', amount: '50.00' }]
        }, authHeaders);
        assert.strictEqual(inv2Res.statusCode, 200, 'Invoice 2 generation should succeed');
        const inv2 = inv2Res.body;
        assert.ok(inv2.invoice_hash, 'Invoice 2 should have a hash');
        assert.strictEqual(inv2.previous_invoice_hash, inv1.invoice_hash, 'Invoice 2 previous_invoice_hash mus
t exactly match Invoice 1 invoice_hash');
        console.log('? Invoice chaining verified. Chaining sequence: Inv1 Hash -> Inv2 PrevHash.');
```

```
        // --- Test Case 3: CBAHI OVR Incident Reporting ---
        console.log('Testing Case 3: CBAHI OVR Incident Reporting...');
        const incidentRes = await makeRequest('POST', '/api/quality/incidents', {
            incident_type: 'medication_error',
            severity: 'medium',
            harm_level: 'Mild',
            department: 'Pharmacy',
            location: 'Main Pharmacy',
            description: 'Wrong dosage of aspirin dispensed but caught before administration.',
            immediate_action: 'Dispensed correct dose, notified supervisor.'
        }, authHeaders);
        if (incidentRes.statusCode !== 200) {
            console.error('FAILED RESPONSE:', incidentRes.statusCode, incidentRes.body || incidentRes.rawBody)
;
        }
        assert.strictEqual(incidentRes.statusCode, 200, 'Incident reporting should succeed');
        assert.ok(incidentRes.body.id, 'Should return the generated incident ID');

        // Fetch incidents
        const getIncidentsRes = await makeRequest('GET', '/api/quality/incidents', {}, authHeaders);
        assert.strictEqual(getIncidentsRes.statusCode, 200, 'Fetching incidents should succeed');
        const found = getIncidentsRes.body.find(i => i.id === incidentRes.body.id);
        assert.ok(found, 'Should find the reported incident in the list');
        assert.strictEqual(found.department, 'Pharmacy');
        assert.strictEqual(found.severity, 'medium');
        console.log('? CBAHI OVR incident successfully reported and retrieved.');

        console.log('? All Saudi Regulatory Compliance Integration Tests passed successfully!');
    } catch (e) {
        console.error('? Test failed:', e);
        process.exitCode = 1;
    } finally {
        // Clean up
        server.kill();
        const cleanClient = await pool.connect();
        try {
            await cleanClient.query("SET app.tenant_id = '1'");
            await cleanClient.query('DELETE FROM invoices WHERE patient_id = $1', [patientId]);
            await cleanClient.query('DELETE FROM patients WHERE id = $1', [patientId]);
            await cleanClient.query('DELETE FROM quality_incidents WHERE reported_by = $1', ['Compliance Test
Reporter']);
            await cleanClient.query('DELETE FROM user_tenants WHERE user_id = $1', [adminUserId]);
            await cleanClient.query('DELETE FROM system_users WHERE id = $1', [adminUserId]);
        } finally {
            cleanClient.release();
        }
        await pool.end();
        console.log('? Cleanup complete');
    }
}

runTests();
```

```
========================================
FILE: SECURITY.md
========================================

# Security Policy ? NamaMedical

NamaMedical is a multi-tenant HIS/EMR that stores Protected Health Information (PHI) and financial
records. Security issues are treated as the highest priority.

## Reporting a vulnerability
- **Do NOT** open a public issue for a security vulnerability.
- Email the maintainer privately with: affected endpoint/file, reproduction steps, and impact.
- Allow reasonable time for a fix before any disclosure.

## Security posture (enforced)
- **Tenant isolation:** PostgreSQL FORCE Row-Level Security; the app runs as a non-superuser role
  (`nama_medical_app`, `bypassrls=false`) with per-request `app.tenant_id` binding.
- **AuthN:** bcrypt password hashing (no plaintext fallback), account lockout, optional TOTP MFA with
  one-time recovery codes; secrets required in production (the app refuses to boot without them).
- **AuthZ:** layered `requireAuth` ? `requireRole` ? DB-matrix `requirePermission` (fail-closed).
- **PHI at rest:** stored outside the web root, served only via an authenticated, tenant-scoped,
  traversal-denied, audited endpoint; optional envelope encryption (DPAPI KEK).
- **Transport/headers:** Helmet, CORS allowlist (no reflect-any-origin), CSRF Origin checks,
  Permissions-Policy, CSP (report-only ? enforce on staging first).
- **Input:** money/clinical engines are fail-closed; central `validation.js` for input schemas.
- **Audit:** `audit_trail` + optional automatic audit middleware (HIPAA §164.312(b)).

## Automated checks (CI)
- Dependency audit (`npm audit`, fail on high), CodeQL SAST, gitleaks secret scan, DB-free unit tests.
- Weekly Dependabot updates.

## Handling secrets
- Never commit real secrets. Only `*.example` env templates are tracked.
- If a secret is ever exposed: rotate immediately, then purge history. Public remotes are push-disabled.



========================================
FILE: seed_clinical_specialties.js
========================================

/**
 * seed_clinical_specialties.js
 * Database seeder for the 100+ clinical subspecialties and EMR templates.
 */

const { pool } = require('./db_postgres');

const DEPARTMENTS = [
    // I. Internal Medicine & Subspecialties
    { code: 'CARDIOLOGY', name_en: 'General Cardiology', name_ar: '?? ????? ?????' },
    { code: 'INTERVENTIONAL_CARDIOLOGY', name_en: 'Interventional Cardiology', name_ar: '?? ????? ????????' },
```

```
    { code: 'ELECTROPHYSIOLOGY', name_en: 'Electrophysiology', name_ar: '?? ????? ??????????????????' },
    { code: 'PREVENTIVE_CARDIOLOGY', name_en: 'Preventive Cardiology', name_ar: '?? ????? ???????' },
    { code: 'CARDIAC_CATH', name_en: 'Cardiac Catheterization Lab', name_ar: '????? ?????' },
    { code: 'PULMONOLOGY', name_en: 'Pulmonology', name_ar: '?? ????? ??????? ???????' },
    { code: 'SLEEP_MEDICINE', name_en: 'Sleep Medicine', name_ar: '?? ????? ????????? ??????' },
    { code: 'GASTROENTEROLOGY', name_en: 'Gastroenterology', name_ar: '?? ?????? ??????' },
    { code: 'HEPATOLOGY', name_en: 'Hepatology', name_ar: '????? ?????' },
    { code: 'NEPHROLOGY', name_en: 'General Nephrology', name_ar: '?? ????? ?????' },
    { code: 'RENAL_DIALYSIS', name_en: 'Dialysis Unit', name_ar: '???? ???? ?????' },
    { code: 'ONCOLOGY', name_en: 'Medical Oncology', name_ar: '?? ??????? ?????' },
    { code: 'HEMATOLOGY', name_en: 'Clinical Hematology', name_ar: '????? ????' },
    { code: 'ENDOCRINOLOGY', name_en: 'Endocrinology & Diabetology', name_ar: '????? ?????? ???????' },
    { code: 'RHEUMATOLOGY', name_en: 'Rheumatology', name_ar: '??????? ???????????' },
    { code: 'INFECTIOUS_DISEASES', name_en: 'Infectious Diseases', name_ar: '??????? ???????' },
    { code: 'DERMATOLOGY', name_en: 'Dermatology', name_ar: '??????? ???????' },

    // II. Surgical Departments
    { code: 'GENERAL_SURGERY', name_en: 'General Surgery', name_ar: '??????? ??????' },
    { code: 'SURGICAL_ONCOLOGY', name_en: 'Surgical Oncology', name_ar: '????? ??????? ??????' },
    { code: 'BARIATRIC_SURGERY', name_en: 'Bariatric Surgery', name_ar: '????? ?????? ???????' },
    { code: 'CARDIOTHORACIC_SURGERY', name_en: 'Cardiothoracic Surgery', name_ar: '????? ????? ??????' },
    { code: 'VASCULAR_SURGERY', name_en: 'Vascular Surgery', name_ar: '????? ??????? ???????' },
    { code: 'NEUROSURGERY', name_en: 'Neurosurgery', name_ar: '????? ???? ????????' },
    { code: 'SPINE_SURGERY', name_en: 'Spine Surgery', name_ar: '????? ?????? ??????' },
    { code: 'ORTHOPEDIC_SURGERY', name_en: 'Orthopedic Surgery', name_ar: '????? ?????? ??????' },
    { code: 'OPHTHALMOLOGY', name_en: 'Ophthalmology', name_ar: '?? ?????? ??????' },
    { code: 'ENT', name_en: 'Otolaryngology (ENT)', name_ar: '????? ?????? ????????' },
    { code: 'UROLOGY', name_en: 'Urology', name_ar: '????? ??????? ???????' },
    { code: 'PLASTIC_SURGERY', name_en: 'Plastic & Reconstructive Surgery', name_ar: '????? ??????? ????????'
},

    // III. Obstetrics, Gynecology & Pediatrics
    { code: 'OBGYN', name_en: 'Obstetrics & Gynecology', name_ar: '?????? ???????? ?????' },
    { code: 'FETAL_MEDICINE', name_en: 'Maternal-Fetal Medicine', name_ar: '?? ???? ???????' },
    { code: 'IVF_LAB', name_en: 'Reproductive Endocrinology & IVF', name_ar: '??????? ?????? ????????' },
    { code: 'PEDIATRICS', name_en: 'General Pediatrics', name_ar: '?? ??????? ?????' },
    { code: 'NEONATOLOGY_NICU', name_en: 'Neonatal ICU (NICU)', name_ar: '??????? ??????? ?????? ???????' },
    { code: 'PEDIATRIC_CARDIOLOGY', name_en: 'Pediatric Cardiology', name_ar: '??? ???????' },

    // IV. Advanced Diagnostics
    { code: 'RADIOLOGY', name_en: 'Diagnostic Radiology', name_ar: '?????? ????????? ????????' },
    { code: 'INTERVENTIONAL_RAD', name_en: 'Interventional Radiology', name_ar: '?????? ?????????' },
    { code: 'PATHOLOGY', name_en: 'Clinical Pathology & Histopathology', name_ar: '????????? ???????????' },
    { code: 'GENETICS', name_en: 'Medical Genetics', name_ar: '??? ??????? ??????' }
];

// Sample Form Structures for dynamic rendering
const CARDIOLOGY_TEMPLATE = {
    fields: [
        { name: 'chest_pain', type: 'select', label_en: 'Chest Pain Type', label_ar: '??? ??? ?????', options:
 ['Typical Angina', 'Atypical Angina', 'Non-anginal', 'None'] },
        { name: 'bp_systolic', type: 'number', label_en: 'BP Systolic (mmHg)', label_ar: '????? ?????????' },
        { name: 'bp_diastolic', type: 'number', label_en: 'BP Diastolic (mmHg)', label_ar: '????? ?????????' }
,
```

```
        { name: 'ecg_finding', type: 'text', label_en: 'ECG Findings', label_ar: '????? ????? ?????' },
        { name: 'ejec_fraction', type: 'number', label_en: 'Ejection Fraction (%)', label_ar: '????? ?????? ??
???' }
    ]
};

const PEDIATRICS_NICU_TEMPLATE = {
    fields: [
        { name: 'birth_weight', type: 'number', label_en: 'Birth Weight (kg)', label_ar: '??? ??????? (???)' }
,
        { name: 'apgar_1m', type: 'number', label_en: 'APGAR Score (1 min)', label_ar: '????? ????? ?????' },
        { name: 'apgar_5m', type: 'number', label_en: 'APGAR Score (5 min)', label_ar: '????? ????? 5 ?????' }
,
        { name: 'o2_saturation', type: 'number', label_en: 'O2 Saturation (%)', label_ar: '???? ???????? ?????
' },
        { name: 'ventilator_mode', type: 'select', label_en: 'Ventilator Mode', label_ar: '??? ???? ??????', o
ptions: ['None', 'CPAP', 'SIMV', 'HFOV'] }
    ]
};

const SURGICAL_COUNT_TEMPLATE = {
    fields: [
        { name: 'sponge_count_pre', type: 'number', label_en: 'Pre-incision Sponge Count', label_ar: '??? ????
? ??? ?????' },
        { name: 'sponge_count_post', type: 'number', label_en: 'Post-closure Sponge Count', label_ar: '??? ???
?? ??? ???????' },
        { name: 'instrument_count_pre', type: 'number', label_en: 'Pre-incision Instrument Count', label_ar: '
??? ??????? ??? ?????' },
        { name: 'instrument_count_post', type: 'number', label_en: 'Post-closure Instrument Count', label_ar:
'??? ??????? ??? ???????' },
        { name: 'sharp_count_pre', type: 'number', label_en: 'Pre-incision Sharps Count', label_ar: '??? ?????
 ??????? ?????? ??? ?????' },
        { name: 'sharp_count_post', type: 'number', label_en: 'Post-closure Sharps Count', label_ar: '??? ????
? ??????? ?????? ??? ???????' },
        { name: 'override_reason', type: 'text', label_en: 'Override Reason (if mismatch)', label_ar: '??? ???
???? (?? ??? ??? ???????)' }
    ]
};

const BRADEN_TEMPLATE = {
    fields: [
        { name: 'sensory_perception', type: 'number', label_en: 'Sensory Perception (1-4)', label_ar: '???????
 ????? (1-4)' },
        { name: 'moisture', type: 'number', label_en: 'Moisture (1-4)', label_ar: '??????? (1-4)' },
        { name: 'activity', type: 'number', label_en: 'Activity (1-4)', label_ar: '?????? (1-4)' },
        { name: 'mobility', type: 'number', label_en: 'Mobility (1-4)', label_ar: '?????? (1-4)' },
        { name: 'nutrition', type: 'number', label_en: 'Nutrition (1-4)', label_ar: '??????? (1-4)' },
        { name: 'friction_shear', type: 'number', label_en: 'Friction & Shear (1-3)', label_ar: '???????? ????
? (1-3)' }
    ]
};

const MORSE_TEMPLATE = {
    fields: [
```

```
        { name: 'history_of_falls', type: 'number', label_en: 'History of Falls (0 or 25)', label_ar: '????? ?????? (0 ?? 25)' },
        { name: 'secondary_diagnosis', type: 'number', label_en: 'Secondary Diagnosis (0 or 15)', label_ar: '???? ????? (0 ?? 15)' },
        { name: 'ambulatory_aid', type: 'number', label_en: 'Ambulatory Aid (0, 15, or 30)', label_ar: '????? ????? (0 ?? 15 ?? 30)' },
        { name: 'iv_heparin_lock', type: 'number', label_en: 'IV/Heparin Lock (0 or 20)', label_ar: '?????? ??????/???? ???????? (0 ?? 20)' },
        { name: 'gait_transferring', type: 'number', label_en: 'Gait/Transferring (0, 10, or 20)', label_ar: '?????? ???????? (0 ?? 10 ?? 20)' },
        { name: 'mental_status', type: 'number', label_en: 'Mental Status (0 or 15)', label_ar: '?????? ??????? (0 ?? 15)' }
    ]
};

const APGAR_TEMPLATE = {
    fields: [
        { name: 'apgar_1m_appearance', type: 'number', label_en: '1-Min Appearance (0-2)', label_ar: '?????? ??? ????? (0-2)' },
        { name: 'apgar_1m_pulse', type: 'number', label_en: '1-Min Pulse (0-2)', label_ar: '????? ??? ????? (0-2)' },
        { name: 'apgar_1m_grimace', type: 'number', label_en: '1-Min Grimace (0-2)', label_ar: '????????? ????? ??? ????? (0-2)' },
        { name: 'apgar_1m_activity', type: 'number', label_en: '1-Min Activity (0-2)', label_ar: '?????? ?????? ??? ????? (0-2)' },
        { name: 'apgar_1m_respiration', type: 'number', label_en: '1-Min Respiration (0-2)', label_ar: '?????? ??? ????? (0-2)' },
        { name: 'apgar_5m_appearance', type: 'number', label_en: '5-Min Appearance (0-2)', label_ar: '?????? ??? 5 ????? (0-2)' },
        { name: 'apgar_5m_pulse', type: 'number', label_en: '5-Min Pulse (0-2)', label_ar: '????? ??? 5 ????? (0-2)' },
        { name: 'apgar_5m_grimace', type: 'number', label_en: '5-Min Grimace (0-2)', label_ar: '????????? ????? ??? 5 ????? (0-2)' },
        { name: 'apgar_5m_activity', type: 'number', label_en: '5-Min Activity (0-2)', label_ar: '?????? ?????? ??? 5 ????? (0-2)' },
        { name: 'apgar_5m_respiration', type: 'number', label_en: '5-Min Respiration (0-2)', label_ar: '?????? ??? 5 ????? (0-2)' }
    ]
};

async function seed() {
    console.log('Starting specialties database seeding...');
    const client = await pool.connect();
    try {
        await client.query('BEGIN');
        await client.query("SET app.tenant_id = '1'");

        // 1. Insert Departments
        for (const dept of DEPARTMENTS) {
            await client.query(
                `INSERT INTO clinical_departments (tenant_id, code, name_en, name_ar, category)
                 VALUES (1, $1, $2, $3, $4)
                 ON CONFLICT (tenant_id, code) DO UPDATE
                 SET name_en = EXCLUDED.name_en, name_ar = EXCLUDED.name_ar, category = EXCLUDED.category`,
```

```
                [dept.code, dept.name_en, dept.name_ar, dept.category || 'General']
            );
        }
        console.log(`? Seeded ${DEPARTMENTS.length} clinical departments.`);

        // Check is_active type dynamically
        const colTypeRes = await client.query(`
            SELECT data_type FROM information_schema.columns
            WHERE table_name = 'clinical_templates' AND column_name = 'is_active'
        `);
        const isBool = colTypeRes.rows.length && colTypeRes.rows[0].data_type === 'boolean';
        const isActiveVal = isBool ? true : 1;

        // Helper to insert templates safely and idempotently
        async function insertTemplate(deptCode, nameEn, nameAr, structure) {
            const deptRes = await client.query('SELECT id FROM clinical_departments WHERE code = $1', [deptCod
e]);
            if (!deptRes.rows.length) return;
            const deptId = deptRes.rows[0].id;

            const check = await client.query(
                "SELECT id FROM clinical_templates WHERE department_id = $1 AND template_name_en = $2 AND tena
nt_id = 1",
                [deptId, nameEn]
            );
            if (check.rowCount === 0) {
                await client.query(
                    `INSERT INTO clinical_templates (department_id, template_name_en, template_name_ar, versio
n, form_structure, is_active, tenant_id)
                     VALUES ($1, $2, $3, '1.0.0', $4, $5, 1)`,
                    [deptId, nameEn, nameAr, JSON.stringify(structure), isActiveVal]
                );
            }
        }

        // 2. Insert Templates
        await insertTemplate('CARDIOLOGY', 'Cardiology Evaluation', '????? ????? ??????', CARDIOLOGY_TEMPLATE)
;
        await insertTemplate('NEONATOLOGY_NICU', 'NICU Admission & Vitals', '???? ??????? ??????? ??????? ????
?? ???????', PEDIATRICS_NICU_TEMPLATE);

        // Seed F1 Templates
        await insertTemplate('GENERAL_SURGERY', 'Surgical Count Sheet', '??? ??? ????? ???????', SURGICAL_COUN
T_TEMPLATE);
        await insertTemplate('PEDIATRICS', 'Braden Scale Assessment', '????? ????? ???? ??????', BRADEN_TEMPLA
TE);
        await insertTemplate('PEDIATRICS', 'Morse Fall Risk Assessment', '????? ???? ?????? ??????', MORSE_TEM
PLATE);
        await insertTemplate('NEONATOLOGY_NICU', 'Neonatal Apgar Score', '????? ????? ?????? ???????', APGAR_T
EMPLATE);

        await client.query('COMMIT');
        console.log('? Seeding clinical templates complete.');
    } catch (e) {
```

```
        await client.query('ROLLBACK');
        console.error('? Seeding failed:', e);
    } finally {
        client.release();
    }
}

seed();



========================================
FILE: seed_data_pg.js
========================================

// seed_data_pg.js ? Populate lab catalog, radiology catalog, medical services, sample drugs & patients
const { pool } = require('./db_postgres');

async function insertSampleData() {
    const res = await pool.query("SELECT setting_value FROM company_settings WHERE setting_key='sample_data_in
serted'");
    if (res.rows.length && res.rows[0].setting_value === '1') return;
    const pc = await pool.query('SELECT COUNT(*) as cnt FROM patients');
    if (parseInt(pc.rows[0].cnt) > 0) {
        await pool.query("UPDATE company_settings SET setting_value='1' WHERE setting_key='sample_data_inserte
d'");
        return;
    }
    // Patients
    await pool.query(`INSERT INTO patients (file_number,name_ar,name_en,national_id,phone,status) VALUES (1001
,'???? ????','Ahmed Mohammed','1012345678','0551234567','With Doctor'),(1002,'???? ?????????','Sarah Abdulrahm
an','1098765432','0559876543','Waiting'),(1003,'???? ???????','Faisal Al-Otaibi','1054321098','0553456789','Wa
iting')`);
    // Employees
    await pool.query(`INSERT INTO employees (name,name_ar,name_en,role,department_ar,department_en,status,sala
ry) VALUES ('Dr. Khaled Marwan','?. ???? ?????','Dr. Khaled Marwan','Doctor','????? ?????','Medical Dept.','Ac
tive',25000),('Sarah Al-Ahmad','???? ??????','Sarah Al-Ahmad','Nurse','???????','Nursing','Active',12000),('Om
ar Saleh','??? ????','Omar Saleh','Admin','????? ?????????','IT Dept.','On Leave',9500)`);
    // Invoices
    await pool.query(`INSERT INTO invoices (patient_name,total,paid) VALUES ('Ahmed Mohammed',575.00,1),('Yass
er Khaled',1240.00,0),('Sarah Ali',890.50,1)`);
    // Insurance claims
    await pool.query(`INSERT INTO insurance_claims (patient_name,insurance_company,claim_amount,status) VALUES
 ('Ahmed Mohammed','Bupa Arabia',2500.00,'Approved'),('Yasser Khaled','Tawuniya',4200.00,'Pending'),('Sarah Al
i','MedGulf',1800.00,'Rejected')`);
    await pool.query("UPDATE company_settings SET setting_value='1' WHERE setting_key='sample_data_inserted'")
;
    console.log('  ? Sample data inserted');
}

async function populateLabCatalog() {
    const r = await pool.query('SELECT COUNT(*) as cnt FROM lab_tests_catalog');
    if (parseInt(r.rows[0].cnt) > 0) return;
    const tests = [
```

['CBC - Complete Blood Count', 'Hematology', 'See components', 100], ['WBC - White Blood Cell Count', 'Hematology', '4.5-11.0 x10^9/L', 50], ['RBC - Red Blood Cell Count', 'Hematology', 'M:4.7-6.1 F:4.2-5.4 x10^12/L', 50], ['Hemoglobin (Hgb)', 'Hematology', 'M:13.5-17.5 F:12.0-16.0 g/dL', 50], ['Hematocrit (Hct)', 'Hematology', 'M:38.3-48.6% F:35.5-44.9%', 50], ['MCV - Mean Corpuscular Volume', 'Hematology', '80-100 fL', 40], ['MCH - Mean Corpuscular Hemoglobin', 'Hematology', '27-33 pg', 40], ['MCHC', 'Hematology', '31.5-35.7 g/dL', 40], ['RDW - Red Cell Distribution Width', 'Hematology', '11.5-14.5%', 40], ['Platelet Count', 'Hematology', '150-400 x10^9/L', 50], ['MPV - Mean Platelet Volume', 'Hematology', '7.5-11.5 fL', 40], ['ESR - Erythrocyte Sedimentation Rate', 'Hematology', 'M:0-15 F:0-20 mm/hr', 60], ['Reticulocyte Count', 'Hematology', '0.5-2.5%', 80], ['Peripheral Blood Smear', 'Hematology', 'Normal morphology', 120], ['Hemoglobin Electrophoresis', 'Hematology', 'HbA >95%', 200], ['G6PD', 'Hematology', '4.6-13.5 U/g Hb', 150], ['Sickle Cell Screen', 'Hematology', 'Negative', 100], ['Direct Coombs Test (DAT)', 'Hematology', 'Negative', 100], ['Indirect Coombs Test (IAT)', 'Hematology', 'Negative', 100], ['Haptoglobin', 'Hematology', '30-200 mg/dL', 120], ['CD4 Count (Flow Cytometry)', 'Hematology', '500-1500 cells/uL', 250], ['CD8 Count (Flow Cytometry)', 'Hematology', '150-1000 cells/uL', 250],

['PT - Prothrombin Time', 'Coagulation', '11.0-13.5 seconds', 80], ['INR', 'Coagulation', '0.8-1.1', 80], ['aPTT', 'Coagulation', '25-35 seconds', 80], ['D-Dimer', 'Coagulation', '<0.50 mg/L FEU', 120], ['Fibrinogen', 'Coagulation', '200-400 mg/dL', 100], ['Thrombin Time', 'Coagulation', '14-19 seconds', 100], ['Bleeding Time', 'Coagulation', '2-7 minutes', 60], ['Factor V Leiden Mutation', 'Coagulation', 'Not detected', 300], ['Protein C Activity', 'Coagulation', '70-140%', 250], ['Protein S Activity', 'Coagulation', '60-140%', 250], ['Antithrombin III', 'Coagulation', '80-120%', 200], ['Lupus Anticoagulant', 'Coagulation', 'Negative', 200], ['von Willebrand Factor Antigen', 'Coagulation', '50-150%', 250],

['Glucose, Fasting', 'Chemistry', '70-100 mg/dL', 50], ['Glucose, Random', 'Chemistry', '70-140 mg/dL', 50], ['Glucose, 2-Hour Postprandial', 'Chemistry', '<140 mg/dL', 60], ['Oral Glucose Tolerance Test (OGTT)', 'Chemistry', '<140 mg/dL at 2hr', 120], ['BUN - Blood Urea Nitrogen', 'Chemistry', '7-20 mg/dL', 50], ['Creatinine, Serum', 'Chemistry', 'M:0.7-1.3 F:0.6-1.1 mg/dL', 50], ['eGFR', 'Chemistry', '>60 mL/min/1.73m2', 50], ['Uric Acid', 'Chemistry', 'M:3.4-7.0 F:2.4-6.0 mg/dL', 60], ['Total Protein, Serum', 'Chemistry', '6.0-8.3 g/dL', 50], ['Albumin, Serum', 'Chemistry', '3.5-5.5 g/dL', 50], ['Globulin', 'Chemistry', '2.0-3.5 g/dL', 50], ['BMP - Basic Metabolic Panel', 'Chemistry', 'See components', 150], ['CMP - Comprehensive Metabolic Panel', 'Chemistry', 'See components', 200], ['Ammonia Level', 'Chemistry', '15-45 mcg/dL', 100], ['Lactate (Lactic Acid)', 'Chemistry', '0.5-2.2 mmol/L', 80], ['LDH - Lactate Dehydrogenase', 'Chemistry', '140-280 U/L', 70], ['CPK - Creatine Phosphokinase', 'Chemistry', 'M:39-308 F:26-192 U/L', 80], ['Amylase', 'Chemistry', '28-100 U/L', 80], ['Lipase', 'Chemistry', '0-160 U/L', 80],

['ALT (SGPT)', 'Liver Function', '7-56 U/L', 60], ['AST (SGOT)', 'Liver Function', '10-40 U/L', 60], ['ALP - Alkaline Phosphatase', 'Liver Function', '44-147 U/L', 60], ['GGT', 'Liver Function', 'M:9-48 F:9-36 U/L', 60], ['Total Bilirubin', 'Liver Function', '0.1-1.2 mg/dL', 60], ['Direct Bilirubin', 'Liver Function', '0.0-0.3 mg/dL', 60], ['Indirect Bilirubin', 'Liver Function', '0.1-0.9 mg/dL', 50], ['LFT - Liver Function Panel', 'Liver Function', 'See components', 150], ['Alpha-Fetoprotein (Liver)', 'Liver Function', '<10 ng/mL', 150],

['Total Cholesterol', 'Lipid Panel', '<200 mg/dL', 60], ['LDL Cholesterol', 'Lipid Panel', '<100 mg/dL optimal', 60], ['HDL Cholesterol', 'Lipid Panel', 'M:>40 F:>50 mg/dL', 60], ['Triglycerides', 'Lipid Panel', '<150 mg/dL', 60], ['VLDL Cholesterol', 'Lipid Panel', '5-40 mg/dL', 60], ['Lipid Panel (Complete)', 'Lipid Panel', 'See components', 120], ['Apolipoprotein B', 'Lipid Panel', '<90 mg/dL', 150], ['Lipoprotein(a)', 'Lipid Panel', '<30 mg/dL', 180],

['Sodium (Na)', 'Electrolytes', '136-145 mEq/L', 50], ['Potassium (K)', 'Electrolytes', '3.5-5.0 mEq/L', 50], ['Chloride (Cl)', 'Electrolytes', '98-106 mEq/L', 50], ['CO2 (Bicarbonate)', 'Electrolytes', '22-29 mEq/L', 50], ['Calcium, Total', 'Electrolytes', '8.6-10.2 mg/dL', 50], ['Calcium, Ionized', 'Electrolytes', '4.5-5.6 mg/dL', 80], ['Phosphorus', 'Electrolytes', '2.5-4.5 mg/dL', 50], ['Magnesium', 'Electrolytes', '1.7-2.2 mg/dL', 60], ['Zinc, Serum', 'Electrolytes', '60-120 mcg/dL', 100], ['Copper, Serum', 'Electrolytes', '70-140 mcg/dL', 100],

['TSH', 'Endocrinology', '0.27-4.20 mIU/L', 100], ['Free T4', 'Endocrinology', '0.93-1.70 ng/dL', 100], ['Free T3', 'Endocrinology', '2.0-4.4 pg/mL', 100], ['Total T4', 'Endocrinology', '4.5-12.0 mcg/dL', 80], ['Total T3', 'Endocrinology', '80-200 ng/dL', 80], ['Anti-TPO', 'Endocrinology', '<35 IU/mL', 120], ['Anti-Thyroglobulin Antibody', 'Endocrinology', '<40 IU/mL', 120], ['HbA1c', 'Endocrinology', '4.0-5.6%', 100], ['Fasting Insulin', 'Endocrinology', '2.6-24.9 mIU/L', 120], ['C-Peptide', 'Endocrinology', '1.1-4.4 ng/mL', 150], ['Co

rtisol, Morning', 'Endocrinology', '6.2-19.4 mcg/dL', 120], ['ACTH', 'Endocrinology', '7.2-63.3 pg/mL', 180], ['Aldosterone', 'Endocrinology', '<21 ng/dL upright', 180], ['PTH - Parathyroid Hormone', 'Endocrinology', '15-65 pg/mL', 150], ['Growth Hormone', 'Endocrinology', 'M:<5 F:<10 ng/mL', 180], ['IGF-1', 'Endocrinology', 'Age-dependent', 180], ['Prolactin', 'Endocrinology', 'M:4-15 F:4-23 ng/mL', 120], ['DHEA-S', 'Endocrinology', 'Age/sex-dependent', 120], ['Metanephrines, Plasma', 'Endocrinology', '<0.90 nmol/L', 250],

['CRP', 'Immunology', '<3.0 mg/L', 80], ['hs-CRP', 'Immunology', '<1.0 mg/L low risk', 100], ['RF - Rheumatoid Factor', 'Immunology', '<14 IU/mL', 80], ['Anti-CCP Antibodies', 'Immunology', '<20 U/mL', 150], ['ANA - Antinuclear Antibody', 'Immunology', 'Negative (<1:40)', 120], ['Anti-dsDNA', 'Immunology', '<30 IU/mL', 150], ['ENA Panel', 'Immunology', 'Negative', 250], ['Complement C3', 'Immunology', '90-180 mg/dL', 100], ['Complement C4', 'Immunology', '10-40 mg/dL', 100], ['IgG', 'Immunology', '700-1600 mg/dL', 100], ['IgA', 'Immunology', '70-400 mg/dL', 100], ['IgM', 'Immunology', '40-230 mg/dL', 100], ['IgE, Total', 'Immunology', '<100 IU/mL', 120], ['ANCA', 'Immunology', 'Negative', 200], ['ASO', 'Immunology', '<200 IU/mL', 80], ['SPEP', 'Immunology', 'See pattern', 200],

['Blood Culture, Aerobic', 'Microbiology', 'No growth', 150], ['Blood Culture, Anaerobic', 'Microbiology', 'No growth', 150], ['Urine Culture & Sensitivity', 'Microbiology', '<10,000 CFU/mL', 120], ['Wound Culture & Sensitivity', 'Microbiology', 'See report', 120], ['Throat Culture', 'Microbiology', 'Normal flora', 100], ['Sputum Culture', 'Microbiology', 'See report', 120], ['Stool Culture', 'Microbiology', 'No pathogen', 120], ['CSF Culture', 'Microbiology', 'No growth', 150], ['Fungal Culture', 'Microbiology', 'No growth', 150], ['AFB Culture (TB)', 'Microbiology', 'No growth', 200], ['AFB Smear', 'Microbiology', 'Negative', 80], ['Gram Stain', 'Microbiology', 'See report', 60], ['H. pylori Antigen, Stool', 'Microbiology', 'Negative', 120], ['H. pylori Antibody', 'Microbiology', 'Negative', 100], ['H. pylori Breath Test', 'Microbiology', 'Negative', 150], ['C. difficile Toxin', 'Microbiology', 'Negative', 150], ['MRSA Screen', 'Microbiology', 'Not detected', 150],

['Urinalysis, Complete', 'Urinalysis', 'See components', 60], ['Urine Dipstick', 'Urinalysis', 'See components', 40], ['Urine Microscopy', 'Urinalysis', 'See report', 50], ['Urine Albumin/Creatinine Ratio', 'Urinalysis', '<30 mg/g', 100], ['24-Hour Urine Protein', 'Urinalysis', '<150 mg/24hr', 100], ['24-Hour Creatinine Clearance', 'Urinalysis', 'M:97-137 F:88-128 mL/min', 120], ['Urine Drug Screen', 'Urinalysis', 'Negative', 150],

['Urine Drug Screen Panel', 'Toxicology', 'Negative', 200], ['Acetaminophen Level', 'Toxicology', '10-30 mcg/mL', 100], ['Ethanol Level', 'Toxicology', '0 mg/dL', 80], ['Digoxin Level', 'Toxicology', '0.8-2.0 ng/mL', 120], ['Lithium Level', 'Toxicology', '0.6-1.2 mEq/L', 100], ['Vancomycin Trough', 'Toxicology', '15-20 mcg/mL', 150], ['Tacrolimus Level', 'Toxicology', '5-15 ng/mL', 200], ['Lead Level, Blood', 'Toxicology', '<5 mcg/dL', 150],

['PSA', 'Tumor Markers', '<4.0 ng/mL', 120], ['AFP - Alpha-Fetoprotein', 'Tumor Markers', '<10 ng/mL', 150], ['CEA', 'Tumor Markers', '<3.0 ng/mL', 150], ['CA-125', 'Tumor Markers', '<35 U/mL', 150], ['CA 19-9', 'Tumor Markers', '<37 U/mL', 150], ['CA 15-3', 'Tumor Markers', '<30 U/mL', 150], ['Beta-hCG (Tumor Marker)', 'Tumor Markers', '<5 mIU/mL', 120], ['NSE', 'Tumor Markers', '<16.3 ng/mL', 180], ['Chromogranin A', 'Tumor Markers', '<93 ng/mL', 200], ['Calcitonin', 'Tumor Markers', 'M:<8.4 F:<5.0 pg/mL', 200],

['Blood Group & Rh Type', 'Blood Bank', 'A/B/AB/O, Rh+/-', 50], ['Antibody Screen', 'Blood Bank', 'Negative', 80], ['Crossmatch', 'Blood Bank', 'Compatible', 100], ['Direct Antiglobulin Test', 'Blood Bank', 'Negative', 80], ['Cold Agglutinins', 'Blood Bank', '<1:64', 120],

['Vitamin D, 25-Hydroxy', 'Vitamins', '30-100 ng/mL', 120], ['Vitamin B12', 'Vitamins', '200-900 pg/mL', 100], ['Folate, Serum', 'Vitamins', '>3.0 ng/mL', 80], ['Iron, Serum', 'Vitamins', 'M:60-170 F:40-150 mcg/dL', 60], ['TIBC', 'Vitamins', '250-400 mcg/dL', 60], ['Transferrin Saturation', 'Vitamins', '20-50%', 60], ['Ferritin', 'Vitamins', 'M:12-300 F:12-150 ng/mL', 80], ['Vitamin A', 'Vitamins', '30-65 mcg/dL', 150], ['Vitamin C', 'Vitamins', '0.2-2.0 mg/dL', 120], ['Vitamin E', 'Vitamins', '5.5-17.0 mg/L', 150], ['Vitamin B1 (Thiamine)', 'Vitamins', '70-180 nmol/L', 150], ['Vitamin B6', 'Vitamins', '5-50 mcg/L', 150],

['Troponin I', 'Cardiac Markers', '<0.04 ng/mL', 120], ['Troponin T, High Sensitivity', 'Cardiac Markers', '<14 ng/L', 150], ['BNP', 'Cardiac Markers', '<100 pg/mL', 150], ['NT-proBNP', 'Cardiac Markers', '<125 pg/mL', 180], ['CK-MB', 'Cardiac Markers', '<5.0 ng/mL', 100], ['Myoglobin', 'Cardiac Markers', '<90 ng/mL', 100], ['Homocysteine', 'Cardiac Markers', '5-15 umol/L', 120],

['Total IgE', 'Allergy', '<100 IU/mL', 100], ['Specific IgE - Dust Mite', 'Allergy', '<0.35 kU/L', 120], ['Specific IgE - Cat Dander', 'Allergy', '<0.35 kU/L', 120], ['Specific IgE - Grass Pollen', 'Allergy', '<0.35 kU/L', 120], ['Specific IgE - Milk', 'Allergy', '<0.35 kU/L', 120], ['Specific IgE - Egg White', 'Allergy', '<0.35 kU/L', 120], ['Specific IgE - Peanut', 'Allergy', '<0.35 kU/L', 120], ['Specific IgE - Wheat', 'Aller

```
gy', '<0.35 kU/L', 120], ['Food Allergy Panel (Top 8)', 'Allergy', 'See components', 400], ['Inhalant Allergy Panel', 'Allergy', 'See components', 400],
        ['Stool Analysis, Complete', 'Stool Analysis', 'See components', 80], ['Stool Occult Blood (FOBT)', 'Stool Analysis', 'Negative', 50], ['FIT - Fecal Immunochemical', 'Stool Analysis', 'Negative', 80], ['Stool Ova & Parasites', 'Stool Analysis', 'No parasites', 80], ['Fecal Calprotectin', 'Stool Analysis', '<50 mcg/g', 200], ['Fecal Elastase', 'Stool Analysis', '>200 mcg/g', 180],
        ['COVID-19 PCR', 'Molecular', 'Not detected', 200], ['COVID-19 Rapid Antigen', 'Molecular', 'Negative', 100], ['Influenza A/B PCR', 'Molecular', 'Not detected', 200], ['RSV PCR', 'Molecular', 'Not detected', 200], ['Respiratory Pathogen Panel', 'Molecular', 'See components', 500], ['TB QuantiFERON (IGRA)', 'Molecular', 'Negative', 250], ['Hepatitis B PCR (HBV DNA)', 'Molecular', 'Not detected', 300], ['Hepatitis C PCR (HCV RNA)', 'Molecular', 'Not detected', 300], ['HIV-1 RNA Viral Load', 'Molecular', 'Not detected', 350], ['CMV PCR', 'Molecular', 'Not detected', 250], ['EBV PCR', 'Molecular', 'Not detected', 250], ['HPV DNA Test', 'Molecular', 'Not detected', 200],
        ['CSF Analysis', 'Body Fluids', 'See components', 200], ['CSF Protein', 'Body Fluids', '15-45 mg/dL', 80], ['CSF Glucose', 'Body Fluids', '40-70 mg/dL', 60], ['CSF Cell Count', 'Body Fluids', '0-5 WBC/uL', 80], ['Pleural Fluid Analysis', 'Body Fluids', 'See components', 200], ['Synovial Fluid Analysis', 'Body Fluids', 'See components', 200], ['Ascitic Fluid Analysis', 'Body Fluids', 'See components', 200],
        ['Estradiol (E2)', 'Reproductive Hormones', 'Phase-dependent', 120], ['Progesterone', 'Reproductive Hormones', 'Phase-dependent', 120], ['Testosterone, Total', 'Reproductive Hormones', 'M:264-916 ng/dL', 120], ['Testosterone, Free', 'Reproductive Hormones', 'M:8.7-25.1 pg/mL', 150], ['FSH', 'Reproductive Hormones', 'Phase/sex-dependent', 120], ['LH', 'Reproductive Hormones', 'Phase/sex-dependent', 120], ['AMH', 'Reproductive Hormones', 'Age-dependent', 200], ['Beta-hCG (Pregnancy)', 'Reproductive Hormones', '<5 mIU/mL non-pregnant', 100], ['SHBG', 'Reproductive Hormones', 'M:10-57 F:18-114 nmol/L', 150],
        ['HBsAg', 'Infectious Disease', 'Negative', 80], ['HBsAb', 'Infectious Disease', '>10 mIU/mL immune', 80], ['HBcAb', 'Infectious Disease', 'Negative', 80], ['HCV Ab', 'Infectious Disease', 'Negative', 80], ['HIV 1/2 Ag/Ab Combo', 'Infectious Disease', 'Non-reactive', 100], ['RPR/VDRL (Syphilis)', 'Infectious Disease', 'Non-reactive', 60], ['FTA-ABS', 'Infectious Disease', 'Non-reactive', 100], ['Rubella IgG', 'Infectious Disease', '>10 IU/mL immune', 80], ['Rubella IgM', 'Infectious Disease', 'Negative', 80], ['CMV IgG', 'Infectious Disease', 'See interpretation', 80], ['CMV IgM', 'Infectious Disease', 'Negative', 80], ['Toxoplasma IgG', 'Infectious Disease', 'See interpretation', 80], ['Toxoplasma IgM', 'Infectious Disease', 'Negative', 80], ['EBV Panel', 'Infectious Disease', 'See interpretation', 200], ['Brucella Agglutination', 'Infectious Disease', '<1:80', 80], ['Widal Test (Typhoid)', 'Infectious Disease', '<1:80', 60], ['Dengue NS1 Antigen', 'Infectious Disease', 'Negative', 120], ['Malaria Smear', 'Infectious Disease', 'No parasites seen', 80], ['Malaria Rapid Test', 'Infectious Disease', 'Negative', 80], ['Mono Spot Test', 'Infectious Disease', 'Negative', 60], ['Varicella-Zoster IgG', 'Infectious Disease', 'See interpretation', 80], ['Measles IgG', 'Infectious Disease', 'See interpretation', 80],
        ['Anti-tTG IgA (Celiac)', 'Autoimmune', '<20 U/mL', 150], ['Anti-Endomysial Ab', 'Autoimmune', 'Negative', 200], ['Anti-GBM Antibodies', 'Autoimmune', '<20 U/mL', 200], ['Anti-Smooth Muscle Ab', 'Autoimmune', '<1:40', 150], ['Anti-Mitochondrial Ab', 'Autoimmune', 'Negative', 150], ['HLA-B27', 'Autoimmune', 'Negative/Positive', 200], ['Anti-Jo-1 Antibodies', 'Autoimmune', 'Negative', 150], ['Anti-Scl-70 Antibodies', 'Autoimmune', 'Negative', 150],
        ['ABG - Arterial Blood Gas', 'Blood Gas', 'See components', 100], ['pH, Arterial', 'Blood Gas', '7.35-7.45', 50], ['pCO2, Arterial', 'Blood Gas', '35-45 mmHg', 50], ['pO2, Arterial', 'Blood Gas', '80-100 mmHg', 50], ['HCO3, Arterial', 'Blood Gas', '22-26 mEq/L', 50], ['O2 Saturation, Arterial', 'Blood Gas', '95-100%', 40], ['VBG - Venous Blood Gas', 'Blood Gas', 'See components', 80],
    ];
    const client = await pool.connect();
    try {
        await client.query('BEGIN');
        for (const [n, c, r, p] of tests) {
            await client.query('INSERT INTO lab_tests_catalog (test_name,category,normal_range,price) VALUES ($1,$2,$3,$4)', [n, c, r, p]);
        }
        await client.query('COMMIT');
```

```
    } catch (e) { await client.query('ROLLBACK'); throw e; } finally { client.release(); }
    console.log(`  ? Lab catalog: ${tests.length} tests`);
}

async function populateRadiologyCatalog() {
    const r = await pool.query('SELECT COUNT(*) as cnt FROM radiology_catalog');
    if (parseInt(r.rows[0].cnt) > 0) return;
    const rads = [
        ['X-Ray', 'X-Ray Chest (PA)', '', 100], ['X-Ray', 'X-Ray Chest (Lateral)', '', 100], ['X-Ray', 'X-Ray Abdomen (KUB)', '', 100], ['X-Ray', 'X-Ray Abdomen (Erect)', '', 100], ['X-Ray', 'X-Ray Cervical Spine (AP/Lat)', '', 120], ['X-Ray', 'X-Ray Thoracic Spine', '', 120], ['X-Ray', 'X-Ray Lumbar Spine (AP/Lat)', '', 120], ['X-Ray', 'X-Ray Lumbosacral Spine', '', 120], ['X-Ray', 'X-Ray Pelvis (AP)', '', 100], ['X-Ray', 'X-Ray Hip (AP/Lat)', '', 100], ['X-Ray', 'X-Ray Shoulder', '', 100], ['X-Ray', 'X-Ray Elbow', '', 100], ['X-Ray', 'X-Ray Wrist', '', 100], ['X-Ray', 'X-Ray Hand', '', 100], ['X-Ray', 'X-Ray Fingers', '', 80], ['X-Ray', 'X-Ray Knee (AP/Lat)', '', 100], ['X-Ray', 'X-Ray Ankle', '', 100], ['X-Ray', 'X-Ray Foot', '', 100], ['X-Ray', 'X-Ray Toes', '', 80], ['X-Ray', 'X-Ray Skull (AP/Lat)', '', 120], ['X-Ray', 'X-Ray Facial Bones', '', 120], ['X-Ray', 'X-Ray Nasal Bones', '', 100], ['X-Ray', 'X-Ray Sinuses (Waters View)', '', 100], ['X-Ray', 'X-Ray Mandible', '', 100], ['X-Ray', 'X-Ray Panoramic (OPG)', '', 150], ['X-Ray', 'X-Ray Clavicle', '', 100], ['X-Ray', 'X-Ray Ribs', '', 100], ['X-Ray', 'X-Ray Sacrum/Coccyx', '', 100], ['X-Ray', 'X-Ray Both Knees (Standing)', '', 150], ['X-Ray', 'X-Ray Scapula', '', 100], ['X-Ray', 'X-Ray Forearm', '', 100], ['X-Ray', 'X-Ray Humerus', '', 100], ['X-Ray', 'X-Ray Femur', '', 100], ['X-Ray', 'X-Ray Tibia/Fibula', '', 100],
        ['CT', 'CT Brain (Non-contrast)', '', 500], ['CT', 'CT Brain (With Contrast)', '', 700], ['CT', 'CT Brain (With & Without Contrast)', '', 800], ['CT', 'CT Orbits', '', 500], ['CT', 'CT Sinuses', '', 400], ['CT', 'CT Temporal Bones', '', 500], ['CT', 'CT Neck', '', 500], ['CT', 'CT Chest (Non-contrast)', '', 500], ['CT', 'CT Chest (With Contrast)', '', 700], ['CT', 'CT Chest High Resolution (HRCT)', '', 600], ['CT', 'CT Abdomen (Non-contrast)', '', 500], ['CT', 'CT Abdomen (With Contrast)', '', 700], ['CT', 'CT Pelvis', '', 500], ['CT', 'CT Abdomen & Pelvis (With Contrast)', '', 800], ['CT', 'CT KUB (Renal Stone Protocol)', '', 500], ['CT', 'CT Cervical Spine', '', 500], ['CT', 'CT Thoracic Spine', '', 500], ['CT', 'CT Lumbar Spine', '', 500], ['CT', 'CT Angiography - Brain (CTA)', '', 900], ['CT', 'CT Angiography - Neck', '', 900], ['CT', 'CT Angiography - Chest (PE Protocol)', '', 900], ['CT', 'CT Angiography - Abdominal Aorta', '', 900], ['CT', 'CT Angiography - Lower Limbs', '', 900], ['CT', 'CT Angiography - Coronary (CCTA)', '', 1200], ['CT', 'CT Angiography - Renal', '', 900], ['CT', 'CT Enterography', '', 800], ['CT', 'CT Colonography (Virtual Colonoscopy)', '', 800], ['CT', 'CT Urography', '', 700], ['CT', 'CT Guided Biopsy', '', 1000], ['CT', 'CT 3D Reconstruction', '', 400],
        ['MRI', 'MRI Brain (Non-contrast)', '', 800], ['MRI', 'MRI Brain (With Contrast)', '', 1000], ['MRI', 'MRI Brain & MRA (Angiography)', '', 1200], ['MRI', 'MRI Orbits', '', 800], ['MRI', 'MRI Internal Auditory Canal (IAC)', '', 800], ['MRI', 'MRI Pituitary', '', 800], ['MRI', 'MRI Temporomandibular Joint (TMJ)', '', 800], ['MRI', 'MRI Neck', '', 800], ['MRI', 'MRI Cervical Spine', '', 800], ['MRI', 'MRI Thoracic Spine', '', 800], ['MRI', 'MRI Lumbar Spine', '', 800], ['MRI', 'MRI Whole Spine', '', 1500], ['MRI', 'MRI Sacroiliac Joints', '', 800], ['MRI', 'MRI Shoulder', '', 800], ['MRI', 'MRI Elbow', '', 800], ['MRI', 'MRI Wrist', '', 800], ['MRI', 'MRI Hand', '', 800], ['MRI', 'MRI Hip', '', 800], ['MRI', 'MRI Knee', '', 800], ['MRI', 'MRI Ankle', '', 800], ['MRI', 'MRI Foot', '', 800], ['MRI', 'MRI Abdomen', '', 900], ['MRI', 'MRI Pelvis', '', 900], ['MRI', 'MRI Liver (Hepatocyte-specific)', '', 1000], ['MRI', 'MRI MRCP (Biliary)', '', 1000], ['MRI', 'MRI Prostate (Multiparametric)', '', 1200], ['MRI', 'MRI Breast (Bilateral)', '', 1200], ['MRI', 'MRI Cardiac (CMR)', '', 1500], ['MRI', 'MRI Enterography', '', 1000], ['MRI', 'MRI Fetal', '', 1000], ['MRI', 'MRI Brachial Plexus', '', 800], ['MRI', 'MRA - Head (Intracranial)', '', 1000], ['MRI', 'MRA - Neck (Carotid)', '', 1000], ['MRI', 'MRA - Abdominal', '', 1000], ['MRI', 'MRA - Lower Limbs', '', 1000], ['MRI', 'MRV - Brain (Venography)', '', 1000],
        ['Ultrasound', 'US Abdomen (Complete)', '', 200], ['Ultrasound', 'US Abdomen (Limited)', '', 150], ['Ultrasound', 'US Pelvis (Transabdominal)', '', 200], ['Ultrasound', 'US Pelvis (Transvaginal)', '', 250], ['Ultrasound', 'US Thyroid', '', 200], ['Ultrasound', 'US Breast (Bilateral)', '', 250], ['Ultrasound', 'US Breast (Unilateral)', '', 200], ['Ultrasound', 'US Obstetric (1st Trimester)', '', 200], ['Ultrasound', 'US Obstetric (2nd/3rd Trimester)', '', 250], ['Ultrasound', 'US Obstetric (Growth Scan)', '', 300], ['Ultrasound', 'US Obstetric (Anomaly Scan - Level II)', '', 400], ['Ultrasound', 'US Renal', '', 200], ['Ultrasound', 'US Bladder (Pre/Post Void)', '', 200], ['Ultrasound', 'US Scrotal', '', 200], ['Ultrasound', 'US Soft Tissue', '', 150], [
```

```
'Ultrasound', 'US Musculoskeletal', '', 200], ['Ultrasound', 'US Joint', '', 200], ['Ultrasound', 'US Neonatal Brain (Cranial)', '', 250], ['Ultrasound', 'US Hip (Infant)', '', 200], ['Ultrasound', 'US Guided Biopsy', '', 500], ['Ultrasound', 'US Guided Aspiration', '', 400], ['Ultrasound', 'Doppler - Carotid', '', 300], ['Ultrasound', 'Doppler - Lower Limb Arterial', '', 300], ['Ultrasound', 'Doppler - Lower Limb Venous (DVT)', '', 300], ['Ultrasound', 'Doppler - Upper Limb', '', 300], ['Ultrasound', 'Doppler - Renal', '', 300], ['Ultrasound', 'Doppler - Portal Vein/Hepatic', '', 300], ['Ultrasound', 'Doppler - Testicular', '', 250], ['Ultrasound', 'Doppler - Fetal', '', 300], ['Ultrasound', 'US Elastography (Liver)', '', 350],
        ['Mammography', 'Mammography (Screening - Bilateral)', '', 400], ['Mammography', 'Mammography (Diagnostic)', '', 450], ['Mammography', 'Tomosynthesis (3D Mammography)', '', 500], ['Mammography', 'Mammography with Spot Compression', '', 450], ['Mammography', 'Stereotactic Breast Biopsy', '', 1000],
        ['DEXA', 'DEXA Bone Densitometry (Spine & Hip)', '', 350], ['DEXA', 'DEXA Forearm', '', 250], ['DEXA', 'DEXA Whole Body Composition', '', 400],
        ['Echo', 'Echocardiography (TTE - Transthoracic)', '', 400], ['Echo', 'Echocardiography (TEE - Transesophageal)', '', 800], ['Echo', 'Stress Echocardiography', '', 600], ['Echo', 'Fetal Echocardiography', '', 500],
        ['Fluoroscopy', 'Barium Swallow', '', 300], ['Fluoroscopy', 'Barium Meal', '', 350], ['Fluoroscopy', 'Barium Enema', '', 400], ['Fluoroscopy', 'Small Bowel Follow Through', '', 350], ['Fluoroscopy', 'Voiding Cystourethrogram (VCUG)', '', 350], ['Fluoroscopy', 'Hysterosalpingography (HSG)', '', 500], ['Fluoroscopy', 'IVP/IVU (Intravenous Pyelogram)', '', 400], ['Fluoroscopy', 'Fistulography', '', 400], ['Fluoroscopy', 'Arthrography', '', 500],
        ['Nuclear Medicine', 'Bone Scan (Whole Body)', '', 600], ['Nuclear Medicine', 'Thyroid Scan & Uptake', '', 500], ['Nuclear Medicine', 'Renal Scan (DTPA/MAG3)', '', 500], ['Nuclear Medicine', 'DMSA Renal Scan', '', 500], ['Nuclear Medicine', 'Cardiac Perfusion Scan (SPECT)', '', 1000], ['Nuclear Medicine', 'Lung Perfusion/Ventilation Scan (V/Q)', '', 600], ['Nuclear Medicine', 'Hepatobiliary Scan (HIDA)', '', 600], ['Nuclear Medicine', 'GI Bleeding Scan', '', 600], ['Nuclear Medicine', 'Gastric Emptying Study', '', 500], ['Nuclear Medicine', 'Parathyroid Scan (Sestamibi)', '', 600], ['Nuclear Medicine', 'Sentinel Lymph Node Scan', '', 600], ['Nuclear Medicine', 'Gallium Scan', '', 700],
        ['PET/CT', 'PET/CT (FDG - Whole Body)', '', 3000], ['PET/CT', 'PET/CT (Brain)', '', 2500], ['PET/CT', 'PET/CT (Cardiac)', '', 2500],
        ['Interventional', 'Angiography (Diagnostic)', '', 2000], ['Interventional', 'Angioplasty', '', 5000], ['Interventional', 'Image-Guided Drainage', '', 1500], ['Interventional', 'Embolization', '', 5000], ['Interventional', 'Port-a-Cath Insertion', '', 3000], ['Interventional', 'PICC Line Insertion', '', 1500], ['Interventional', 'Nephrostomy', '', 2000], ['Interventional', 'Biliary Drainage (PTBD)', '', 3000], ['Interventional', 'Vertebroplasty', '', 5000], ['Interventional', 'Radiofrequency Ablation (RFA)', '', 5000], ['Interventional', 'TIPS Procedure', '', 8000], ['Interventional', 'Uterine Fibroid Embolization', '', 6000],
    ];
    const client = await pool.connect();
    try {
        await client.query('BEGIN');
        for (const [m, n, t, p] of rads) {
            await client.query('INSERT INTO radiology_catalog (modality,exact_name,default_template,price) VALUES ($1,$2,$3,$4)', [m, n, t, p]);
        }
        await client.query('COMMIT');
    } catch (e) { await client.query('ROLLBACK'); throw e; } finally { client.release(); }
    console.log(`  ? Radiology catalog: ${rads.length} exams`);
}

module.exports = { insertSampleData, populateLabCatalog, populateRadiologyCatalog };
```

```
========================================
FILE: seed_extra_catalog.js
```

```
========================================

// seed_extra_catalog.js ? Add comprehensive lab tests and radiology exams
const { pool } = require('./db_postgres');

async function addExtraLabTests() {
    const existing = (await pool.query('SELECT COUNT(*) as cnt FROM lab_tests_catalog')).rows[0].cnt;
    console.log(`  ? Current lab tests: ${existing}`);

    const extraTests = [
        // ===== GENETICS / CYTOGENETICS =====
        ['Karyotype Analysis', 'Genetics', 'Normal 46,XX or 46,XY', 500],
        ['FISH (Fluorescence In Situ Hybridization)', 'Genetics', 'See report', 600],
        ['BRCA1/BRCA2 Gene Test', 'Genetics', 'No mutation detected', 1200],
        ['Cystic Fibrosis Gene Panel', 'Genetics', 'No mutation detected', 800],
        ['Hemoglobin S Gene Test', 'Genetics', 'Not detected', 400],
        ['Thalassemia Gene Panel', 'Genetics', 'Not detected', 600],
        ['Fragile X Syndrome Test', 'Genetics', 'Normal CGG repeats', 500],
        ['Prader-Willi/Angelman Syndrome', 'Genetics', 'Normal methylation', 600],
        ['Spinal Muscular Atrophy (SMA) Test', 'Genetics', 'Normal SMN1 copies', 500],
        ['Prenatal Cell-Free DNA (NIPT)', 'Genetics', 'Low risk', 1500],
        ['Newborn Screening Panel', 'Genetics', 'Normal', 300],
        ['Pharmacogenomics Panel', 'Genetics', 'See report', 800],
        ['Chromosomal Microarray', 'Genetics', 'Normal', 1000],
        ['Whole Exome Sequencing', 'Genetics', 'See report', 3000],
        ['Lynch Syndrome Panel (MLH1/MSH2/MSH6/PMS2)', 'Genetics', 'No mutation', 1000],
        ['Factor V Leiden Gene Test', 'Genetics', 'Not detected', 300],
        ['Prothrombin G20210A Mutation', 'Genetics', 'Not detected', 300],
        ['MTHFR Gene Mutation', 'Genetics', 'Not detected', 250],
        ['JAK2 V617F Mutation', 'Genetics', 'Not detected', 400],
        ['BCR-ABL Quantitative (CML)', 'Genetics', 'Not detected', 600],
        ['EGFR Mutation (Lung Cancer)', 'Genetics', 'Not detected', 500],
        ['BRAF V600E Mutation', 'Genetics', 'Not detected', 500],
        ['HER2/neu Gene Amplification', 'Genetics', 'Not amplified', 500],
        ['Microsatellite Instability (MSI)', 'Genetics', 'Stable', 400],
        ['PDL1 Expression', 'Genetics', 'See report', 400],
        ['Next-Gen Sequencing - Tumor Panel', 'Genetics', 'See report', 2500],
        ['FMR1 Gene Analysis', 'Genetics', 'Normal', 500],
        ['Duchenne/Becker Muscular Dystrophy', 'Genetics', 'Not detected', 600],
        ['Hereditary Breast/Ovarian Cancer Panel', 'Genetics', 'No mutation', 1500],
        ['Familial Hypercholesterolemia Panel', 'Genetics', 'No mutation', 800],

        // ===== DERMATOLOGY =====
        ['Skin Biopsy Pathology', 'Dermatology', 'See report', 300],
        ['Fungal Scrape (KOH Preparation)', 'Dermatology', 'Negative', 60],
        ['Nail Clipping Fungal Culture', 'Dermatology', 'No growth', 100],
        ['Woods Lamp Examination', 'Dermatology', 'Normal fluorescence', 50],
        ['Patch Test (Contact Allergy)', 'Dermatology', 'See results', 300],
        ['Tzanck Smear', 'Dermatology', 'No multinucleated giant cells', 80],
        ['Direct Immunofluorescence (DIF) - Skin', 'Dermatology', 'Negative', 400],
        ['Scabies Scraping', 'Dermatology', 'Negative', 60],

        // ===== ADDITIONAL MICROBIOLOGY =====
        ['GBS Culture (Group B Strep)', 'Microbiology', 'Negative', 100],
```

```
['Chlamydia PCR', 'Microbiology', 'Not detected', 200],
['Gonorrhea PCR', 'Microbiology', 'Not detected', 200],
['Chlamydia/Gonorrhea Combo PCR', 'Microbiology', 'Not detected', 250],
['Trichomonas PCR', 'Microbiology', 'Not detected', 150],
['BV (Bacterial Vaginosis) Panel', 'Microbiology', 'Negative', 150],
['Mycoplasma Culture', 'Microbiology', 'No growth', 150],
['Ureaplasma Culture', 'Microbiology', 'No growth', 150],
['Legionella Urinary Antigen', 'Microbiology', 'Negative', 150],
['Strep A Rapid Test', 'Microbiology', 'Negative', 50],
['Cryptococcal Antigen', 'Microbiology', 'Negative', 150],
['Aspergillus Galactomannan', 'Microbiology', 'Negative', 200],
['Beta-D-Glucan', 'Microbiology', '<60 pg/mL', 250],
['Parasite Blood Smear (Thick/Thin)', 'Microbiology', 'No parasites', 80],
['Pinworm Test (Scotch Tape)', 'Microbiology', 'Negative', 50],
['Giardia Antigen', 'Microbiology', 'Negative', 100],
['Clostridium botulinum Toxin', 'Microbiology', 'Not detected', 300],
['Norovirus PCR', 'Microbiology', 'Not detected', 200],
['Rotavirus Antigen', 'Microbiology', 'Negative', 100],
['Adenovirus Antigen', 'Microbiology', 'Negative', 100],

// ===== ADDITIONAL ENDOCRINOLOGY =====
['Insulin Antibodies', 'Endocrinology', 'Negative', 200],
['Anti-GAD65 Antibodies', 'Endocrinology', '<5 U/mL', 200],
['IA-2 Antibodies', 'Endocrinology', 'Negative', 200],
['Fructosamine', 'Endocrinology', '200-285 umol/L', 100],
['1,25-Dihydroxy Vitamin D', 'Endocrinology', '18-72 pg/mL', 200],
['Catecholamines, Plasma', 'Endocrinology', 'See ranges', 250],
['24-Hour Urine Catecholamines', 'Endocrinology', 'See ranges', 250],
['24-Hour Urine 5-HIAA', 'Endocrinology', '2-8 mg/24hr', 200],
['24-Hour Urine Cortisol', 'Endocrinology', '10-100 mcg/24hr', 180],
['Renin Activity, Plasma', 'Endocrinology', '0.5-3.5 ng/mL/hr', 200],
['Aldosterone/Renin Ratio', 'Endocrinology', '<30', 250],
['17-OH Progesterone', 'Endocrinology', 'Age/sex-dependent', 150],
['Androstenedione', 'Endocrinology', 'Age/sex-dependent', 150],
['Sex Hormone Binding Globulin (SHBG)', 'Endocrinology', 'M:10-57 F:18-114 nmol/L', 150],
['Insulin-like Growth Factor Binding Protein 3', 'Endocrinology', 'Age-dependent', 200],
['Chromogranin A', 'Endocrinology', '<93 ng/mL', 200],
['Serotonin, Serum', 'Endocrinology', '50-200 ng/mL', 180],

// ===== ADDITIONAL IMMUNOLOGY =====
['Cryoglobulins', 'Immunology', 'Negative', 200],
['Beta-2 Microglobulin', 'Immunology', '<2.0 mg/L', 150],
['Serum Free Light Chains', 'Immunology', 'Kappa:3.3-19.4 Lambda:5.7-26.3', 250],
['Immunofixation Electrophoresis (IFE)', 'Immunology', 'No monoclonal protein', 300],
['IgG Subclasses', 'Immunology', 'See ranges', 300],
['Mannose-Binding Lectin', 'Immunology', '>100 ng/mL', 200],
['CH50 (Total Complement)', 'Immunology', '60-144 CAE Units', 150],
['Anti-Cardiolipin Antibodies (IgG/IgM)', 'Immunology', '<20 GPL/MPL', 200],
['Anti-Beta2 Glycoprotein I', 'Immunology', '<20 U/mL', 200],
['Tissue Transglutaminase IgA', 'Immunology', '<20 U/mL', 150],
['Deamidated Gliadin Peptide', 'Immunology', '<20 U/mL', 150],

// ===== ADDITIONAL CHEMISTRY =====
['Procalcitonin', 'Chemistry', '<0.1 ng/mL', 200],
```

```
['Presepsin', 'Chemistry', '<317 pg/mL', 250],
['Ceruloplasmin', 'Chemistry', '20-35 mg/dL', 120],
['Copper, 24-Hour Urine', 'Chemistry', '<40 mcg/24hr', 150],
['Alpha-1 Antitrypsin', 'Chemistry', '100-200 mg/dL', 150],
['Angiotensin Converting Enzyme (ACE)', 'Chemistry', '8-52 U/L', 150],
['Osmolality, Serum', 'Chemistry', '275-295 mOsm/kg', 80],
['Osmolality, Urine', 'Chemistry', '300-900 mOsm/kg', 80],
['Specific Gravity, Urine', 'Chemistry', '1.005-1.030', 30],
['Cystatin C', 'Chemistry', '0.55-1.15 mg/L', 200],
['Bile Acids, Total', 'Chemistry', '<10 umol/L', 150],
['Galactose-1-Phosphate', 'Chemistry', '<1 mg/dL', 200],
['Pyruvate', 'Chemistry', '0.3-0.9 mg/dL', 100],
['Organic Acids, Urine', 'Chemistry', 'Normal pattern', 400],
['Amino Acids, Plasma Panel', 'Chemistry', 'Normal pattern', 400],
['Carnitine Profile', 'Chemistry', 'Normal pattern', 300],
['Acylcarnitine Profile', 'Chemistry', 'Normal pattern', 350],
['Biotinidase Activity', 'Chemistry', '>5.0 nmol/min/mL', 200],
['Sweat Chloride Test', 'Chemistry', '<30 mmol/L normal', 150],

// ===== POINT OF CARE =====
['Rapid Strep A Test', 'Point of Care', 'Negative', 40],
['Rapid Flu A/B Test', 'Point of Care', 'Negative', 60],
['Rapid RSV Test', 'Point of Care', 'Negative', 60],
['Urine Pregnancy Test (hCG)', 'Point of Care', 'Negative', 30],
['Rapid HIV Test', 'Point of Care', 'Non-reactive', 50],
['Rapid Dengue IgG/IgM', 'Point of Care', 'Negative', 80],
['Blood Glucose (Fingerstick)', 'Point of Care', '70-140 mg/dL', 20],
['Hemoglobin A1c (Point of Care)', 'Point of Care', '<5.7%', 50],
['INR (Point of Care)', 'Point of Care', '0.8-1.1', 40],
['Troponin I (Point of Care)', 'Point of Care', '<0.04 ng/mL', 80],
['CRP (Point of Care)', 'Point of Care', '<3 mg/L', 40],
['D-Dimer (Point of Care)', 'Point of Care', '<0.5 mg/L', 80],
['Procalcitonin (Point of Care)', 'Point of Care', '<0.1 ng/mL', 100],
['Lactate (Point of Care)', 'Point of Care', '<2.2 mmol/L', 50],

// ===== ADDITIONAL HEMATOLOGY =====
['Osmotic Fragility Test', 'Hematology', 'Normal', 150],
['Platelet Function Assay (PFA-100)', 'Hematology', 'Col/EPI <165s Col/ADP <120s', 200],
['Bone Marrow Biopsy/Aspirate', 'Hematology', 'See report', 500],
['Flow Cytometry - Leukemia Panel', 'Hematology', 'See report', 500],
['Hemoglobin HPLC', 'Hematology', 'Normal pattern', 200],
['Heinz Body Preparation', 'Hematology', 'Negative', 80],
['Ham Test (Acidified Serum)', 'Hematology', 'Negative', 150],
['Sugar Water Test', 'Hematology', 'Negative', 80],
['Methemoglobin Level', 'Hematology', '<1.5%', 100],
['Carboxyhemoglobin', 'Hematology', '<3% non-smoker', 100],
['RBC Folate', 'Hematology', '>280 ng/mL', 120],
['Immature Platelet Fraction', 'Hematology', '1.1-6.1%', 100],
['Thromboelastography (TEG)', 'Hematology', 'See report', 300],

// ===== THERAPEUTIC DRUG MONITORING =====
['Phenytoin Level', 'Therapeutic Drug Monitoring', '10-20 mcg/mL', 120],
['Carbamazepine Level', 'Therapeutic Drug Monitoring', '4-12 mcg/mL', 120],
['Valproic Acid Level', 'Therapeutic Drug Monitoring', '50-125 mcg/mL', 120],
```

```
['Phenobarbital Level', 'Therapeutic Drug Monitoring', '15-40 mcg/mL', 120],
['Lamotrigine Level', 'Therapeutic Drug Monitoring', '2.5-15 mcg/mL', 150],
['Levetiracetam Level', 'Therapeutic Drug Monitoring', '12-46 mcg/mL', 150],
['Gentamicin Level (Peak/Trough)', 'Therapeutic Drug Monitoring', 'Peak:5-10 Trough:<2', 150],
['Amikacin Level', 'Therapeutic Drug Monitoring', 'Peak:20-30 Trough:<8', 150],
['Cyclosporine Level', 'Therapeutic Drug Monitoring', '100-400 ng/mL', 200],
['Sirolimus Level', 'Therapeutic Drug Monitoring', '4-12 ng/mL', 200],
['Mycophenolate Level', 'Therapeutic Drug Monitoring', '1-3.5 mg/L', 200],
['Theophylline Level', 'Therapeutic Drug Monitoring', '10-20 mcg/mL', 120],
['Methotrexate Level', 'Therapeutic Drug Monitoring', 'Time-dependent', 200],
['Salicylate Level', 'Therapeutic Drug Monitoring', '15-30 mg/dL', 80],

// ===== ADDITIONAL TUMOR MARKERS =====
['HE4 (Ovarian)', 'Tumor Markers', '<140 pmol/L premenopausal', 200],
['ROMA Index', 'Tumor Markers', 'See interpretation', 250],
['Thyroglobulin', 'Tumor Markers', '<40 ng/mL', 150],
['SCC Antigen', 'Tumor Markers', '<2.0 ng/mL', 150],
['CYFRA 21-1', 'Tumor Markers', '<3.3 ng/mL', 150],
['S-100 Protein', 'Tumor Markers', '<0.12 mcg/L', 200],
['Lactate Dehydrogenase (LDH) Isoenzymes', 'Tumor Markers', 'See pattern', 200],
['Free PSA / Total PSA Ratio', 'Tumor Markers', '>25% low risk', 150],
['PCA3 Urine Test', 'Tumor Markers', '<25 normal', 400],
['Pepsinogen I/II', 'Tumor Markers', 'PGI/PGII >3', 200],

// ===== ADDITIONAL ALLERGY =====
['Specific IgE - Cockroach', 'Allergy', '<0.35 kU/L', 120],
['Specific IgE - Dog Dander', 'Allergy', '<0.35 kU/L', 120],
['Specific IgE - Mold Mix', 'Allergy', '<0.35 kU/L', 120],
['Specific IgE - Tree Pollen Mix', 'Allergy', '<0.35 kU/L', 120],
['Specific IgE - Weed Pollen', 'Allergy', '<0.35 kU/L', 120],
['Specific IgE - Latex', 'Allergy', '<0.35 kU/L', 120],
['Specific IgE - Fish', 'Allergy', '<0.35 kU/L', 120],
['Specific IgE - Shellfish', 'Allergy', '<0.35 kU/L', 120],
['Specific IgE - Soy', 'Allergy', '<0.35 kU/L', 120],
['Specific IgE - Sesame', 'Allergy', '<0.35 kU/L', 120],
['Specific IgE - Bee Venom', 'Allergy', '<0.35 kU/L', 120],
['Specific IgE - Wasp Venom', 'Allergy', '<0.35 kU/L', 120],
['Comprehensive Food Panel (20+ items)', 'Allergy', 'See components', 600],
['Comprehensive Inhalant Panel (20+ items)', 'Allergy', 'See components', 600],
['Drug Allergy - Penicillin IgE', 'Allergy', '<0.35 kU/L', 120],
['Drug Allergy - Cephalosporin IgE', 'Allergy', '<0.35 kU/L', 120],
['Tryptase (Mast Cell Activation)', 'Allergy', '<11.4 ng/mL', 200],
['Eosinophil Cationic Protein', 'Allergy', '<24 mcg/L', 150],

// ===== ADDITIONAL INFECTIOUS DISEASE =====
['Hepatitis A IgM', 'Infectious Disease', 'Negative', 80],
['Hepatitis A Total Antibody', 'Infectious Disease', 'See interpretation', 80],
['Hepatitis B e-Antigen', 'Infectious Disease', 'Negative', 80],
['Hepatitis B e-Antibody', 'Infectious Disease', 'See interpretation', 80],
['Hepatitis D Antibody', 'Infectious Disease', 'Negative', 100],
['Hepatitis E IgM', 'Infectious Disease', 'Negative', 100],
['Herpes Simplex IgG Type 1', 'Infectious Disease', 'See interpretation', 100],
['Herpes Simplex IgG Type 2', 'Infectious Disease', 'See interpretation', 100],
['Herpes Simplex PCR', 'Infectious Disease', 'Not detected', 200],
```

```
        ['Parvovirus B19 IgM', 'Infectious Disease', 'Negative', 120],
        ['Leishmania Antibody', 'Infectious Disease', 'Negative', 150],
        ['Schistosoma Antibody', 'Infectious Disease', 'Negative', 100],
        ['Echinococcus Antibody', 'Infectious Disease', 'Negative', 120],
        ['Lyme Disease Antibody (IgG/IgM)', 'Infectious Disease', 'Negative', 200],
        ['Q Fever Antibody', 'Infectious Disease', 'Negative', 150],
        ['Rickettsial Antibody Panel', 'Infectious Disease', 'Negative', 200],
        ['Chikungunya Antibody', 'Infectious Disease', 'Negative', 150],
        ['Zika Virus IgM', 'Infectious Disease', 'Negative', 200],
        ['HTLV I/II Antibody', 'Infectious Disease', 'Non-reactive', 150],
    ];

    if (parseInt(existing) >= 400) {
        console.log('  ? Lab catalog already has sufficient tests');
        return;
    }

    const client = await pool.connect();
    let added = 0;
    try {
        await client.query('BEGIN');
        for (const [n, c, r, p] of extraTests) {
            const exists = (await client.query('SELECT 1 FROM lab_tests_catalog WHERE test_name=$1', [n])).row
s.length;
            if (!exists) {
                await client.query('INSERT INTO lab_tests_catalog (test_name,category,normal_range,price) VALU
ES ($1,$2,$3,$4)', [n, c, r, p]);
                added++;
            }
        }
        await client.query('COMMIT');
    } catch (e) { await client.query('ROLLBACK'); console.error('Lab seed error:', e.message); } finally { cli
ent.release(); }
    const newTotal = (await pool.query('SELECT COUNT(*) as cnt FROM lab_tests_catalog')).rows[0].cnt;
    console.log(`  ? Added ${added} extra lab tests ? Total: ${newTotal}`);
}

async function addExtraRadiology() {
    const existing = (await pool.query('SELECT COUNT(*) as cnt FROM radiology_catalog')).rows[0].cnt;
    console.log(`  ? Current radiology exams: ${existing}`);

    const extraRads = [
        // ===== ADDITIONAL X-RAY =====
        ['X-Ray', 'X-Ray Sternum', '', 100],
        ['X-Ray', 'X-Ray Acromioclavicular Joint', '', 100],
        ['X-Ray', 'X-Ray Whole Spine (Scoliosis)', '', 200],
        ['X-Ray', 'X-Ray Bone Age (Left Hand)', '', 150],
        ['X-Ray', 'X-Ray Soft Tissue Neck (Lateral)', '', 100],
        ['X-Ray', 'X-Ray Mastoid (Towne View)', '', 120],
        ['X-Ray', 'X-Ray Orbit', '', 120],
        ['X-Ray', 'X-Ray Zygomatic Arch', '', 100],
        ['X-Ray', 'X-Ray TMJ', '', 120],
        ['X-Ray', 'X-Ray Both Hips (Frog Leg)', '', 150],
        ['X-Ray', 'X-Ray Leg Length (Scanogram)', '', 200],
```

```
['X-Ray', 'X-Ray Chest (AP Portable)', '', 100],
['X-Ray', 'X-Ray Chest (Decubitus)', '', 120],
['X-Ray', 'X-Ray Abdomen (Supine & Erect)', '', 150],
['X-Ray', 'X-Ray Calcaneus', '', 80],
['X-Ray', 'X-Ray Patella', '', 80],

// ===== ADDITIONAL CT =====
['CT', 'CT Maxillofacial', '', 500],
['CT', 'CT Jaw (Dental CT / Cone Beam)', '', 400],
['CT', 'CT Chest/Abdomen/Pelvis (Triple Phase)', '', 1000],
['CT', 'CT Liver (Triple Phase)', '', 800],
['CT', 'CT Pancreas Protocol', '', 700],
['CT', 'CT Adrenal Protocol', '', 600],
['CT', 'CT Shoulder', '', 500],
['CT', 'CT Hip', '', 500],
['CT', 'CT Knee', '', 500],
['CT', 'CT Ankle/Foot', '', 500],
['CT', 'CT Wrist/Hand', '', 500],
['CT', 'CT Elbow', '', 500],
['CT', 'CT Calcium Scoring (Heart)', '', 500],
['CT', 'CT Perfusion - Brain (Stroke)', '', 1000],
['CT', 'CT Aortography', '', 900],
['CT', 'CT Whole Body (Trauma Protocol)', '', 1200],
['CT', 'CT Cisternography', '', 800],

// ===== ADDITIONAL MRI =====
['MRI', 'MRI Plexus (Lumbosacral)', '', 800],
['MRI', 'MRI Peripheral Nerve', '', 800],
['MRI', 'MRI Rectal', '', 900],
['MRI', 'MRI Pancreas', '', 900],
['MRI', 'MRI Adrenal', '', 800],
['MRI', 'MRI Arthrogram - Shoulder', '', 1200],
['MRI', 'MRI Arthrogram - Hip', '', 1200],
['MRI', 'MRI Arthrogram - Wrist', '', 1200],
['MRI', 'MRI Diffusion Tensor Imaging (DTI)', '', 1200],
['MRI', 'MRI Spectroscopy (Brain)', '', 1500],
['MRI', 'MRI Functional (fMRI)', '', 1500],
['MRI', 'MRI Small Bowel', '', 900],
['MRI', 'MRI Defecography', '', 800],
['MRI', 'MRI Spleen', '', 800],
['MRI', 'MRI Whole Body (Screening)', '', 2000],
['MRI', 'MRA - Renal', '', 1000],
['MRI', 'MRA - Upper Limbs', '', 1000],
['MRI', 'MRA - Aorta', '', 1200],

// ===== ADDITIONAL ULTRASOUND =====
['Ultrasound', 'US Appendix', '', 200],
['Ultrasound', 'US Gallbladder', '', 150],
['Ultrasound', 'US Liver', '', 150],
['Ultrasound', 'US Spleen', '', 150],
['Ultrasound', 'US Pancreas', '', 200],
['Ultrasound', 'US Lymph Nodes', '', 200],
['Ultrasound', 'US Salivary Glands', '', 200],
['Ultrasound', 'US Parathyroid', '', 200],
```

```
['Ultrasound', 'US Chest Wall', '', 150],
['Ultrasound', 'US Penile Doppler', '', 350],
['Ultrasound', 'US Transfontanelle', '', 250],
['Ultrasound', 'US Pyloric Stenosis', '', 200],
['Ultrasound', 'US Umbilical Doppler', '', 300],
['Ultrasound', 'US Uterine Artery Doppler', '', 300],
['Ultrasound', 'US Middle Cerebral Artery Doppler', '', 300],
['Ultrasound', 'US Biophysical Profile (BPP)', '', 300],
['Ultrasound', 'US Cervical Length', '', 200],
['Ultrasound', 'US Nuchal Translucency', '', 300],
['Ultrasound', 'US 4D/3D Obstetric', '', 400],
['Ultrasound', 'US Endoanal/Endorectal', '', 400],
['Ultrasound', 'US Guided Thyroid FNA', '', 500],
['Ultrasound', 'US Guided Breast FNA', '', 500],
['Ultrasound', 'US Guided Liver Biopsy', '', 600],
['Ultrasound', 'US Guided Renal Biopsy', '', 600],
['Ultrasound', 'Doppler - AV Fistula Mapping', '', 350],
['Ultrasound', 'Doppler - Abdominal Aorta', '', 300],
['Ultrasound', 'Doppler - Transcranial (TCD)', '', 400],

// ===== ADDITIONAL SPECIAL PROCEDURES =====
['Mammography', 'Mammography with CAD (Computer-Aided)', '', 500],
['Mammography', 'Breast MRI Guided Biopsy', '', 1500],
['Mammography', 'Galactography', '', 400],
['Mammography', 'Ductography', '', 400],

['DEXA', 'DEXA Body Fat Analysis', '', 300],
['DEXA', 'DEXA Pediatric', '', 300],
['DEXA', 'DEXA Vertebral Fracture Assessment', '', 350],

['Echo', 'Dobutamine Stress Echo', '', 700],
['Echo', 'Contrast Echocardiography', '', 600],
['Echo', '3D Echocardiography', '', 600],
['Echo', 'Strain Echocardiography', '', 500],

// ===== ADDITIONAL NUCLEAR MEDICINE =====
['Nuclear Medicine', 'WBC Labeled Scan (Infection)', '', 700],
['Nuclear Medicine', 'Octreotide Scan (Neuroendocrine)', '', 800],
['Nuclear Medicine', 'MIBG Scan', '', 800],
['Nuclear Medicine', 'Meckel Scan', '', 500],
['Nuclear Medicine', 'Lymphoscintigraphy', '', 600],
['Nuclear Medicine', 'Brain Perfusion SPECT', '', 800],
['Nuclear Medicine', 'DaTSCAN (Dopamine Transport)', '', 1500],
['Nuclear Medicine', 'I-131 Whole Body Scan', '', 700],
['Nuclear Medicine', 'I-131 Therapy (Thyroid)', '', 2000],
['Nuclear Medicine', 'Ra-223 Therapy (Bone Mets)', '', 5000],
['Nuclear Medicine', 'Lu-177 PSMA Therapy', '', 8000],
['Nuclear Medicine', 'Cisternography (CSF Leak)', '', 700],
['Nuclear Medicine', 'Salivary Gland Scintigraphy', '', 500],

// ===== ADDITIONAL PET =====
['PET/CT', 'PET/CT (PSMA - Prostate)', '', 3500],
['PET/CT', 'PET/CT (Ga-68 DOTATATE - NET)', '', 3500],
['PET/CT', 'PET/CT (Amyloid Brain)', '', 3000],
```

```
        ['PET/CT', 'PET/CT (F-18 NaF Bone)', '', 2500],
        ['PET/CT', 'PET/MRI', '', 4000],

        // ===== ADDITIONAL INTERVENTIONAL =====
        ['Interventional', 'Transjugular Liver Biopsy', '', 3000],
        ['Interventional', 'Carotid Stenting', '', 8000],
        ['Interventional', 'Dialysis Access Creation', '', 3000],
        ['Interventional', 'Thrombolysis (IR)', '', 5000],
        ['Interventional', 'IVC Filter Placement', '', 4000],
        ['Interventional', 'IVC Filter Retrieval', '', 3000],
        ['Interventional', 'Thoracentesis (IR-guided)', '', 1000],
        ['Interventional', 'Paracentesis (IR-guided)', '', 1000],
        ['Interventional', 'Joint Injection (IR-guided)', '', 800],
        ['Interventional', 'Epidural Injection (IR-guided)', '', 1500],
        ['Interventional', 'Facet Joint Injection', '', 1500],
        ['Interventional', 'Nerve Block (IR-guided)', '', 1200],
        ['Interventional', 'Kyphoplasty', '', 6000],
        ['Interventional', 'Cryoablation', '', 5000],
        ['Interventional', 'Microwave Ablation', '', 5000],
        ['Interventional', 'Chemoembolization (TACE)', '', 8000],
        ['Interventional', 'Radioembolization (Y-90)', '', 10000],
        ['Interventional', 'Sclerotherapy', '', 2000],
        ['Interventional', 'Varicocele Embolization', '', 4000],
        ['Interventional', 'Pelvic Congestion Embolization', '', 5000],
    ];

    if (parseInt(existing) >= 280) {
        console.log('  ? Radiology catalog already has sufficient exams');
        return;
    }

    const client = await pool.connect();
    let added = 0;
    try {
        await client.query('BEGIN');
        for (const [m, n, t, p] of extraRads) {
            const exists = (await client.query('SELECT 1 FROM radiology_catalog WHERE exact_name=$1', [n])).ro
ws.length;
            if (!exists) {
                await client.query('INSERT INTO radiology_catalog (modality,exact_name,default_template,price)
 VALUES ($1,$2,$3,$4)', [m, n, t, p]);
                added++;
            }
        }
        await client.query('COMMIT');
    } catch (e) { await client.query('ROLLBACK'); console.error('Radiology seed error:', e.message); } finally
 { client.release(); }
    const newTotal = (await pool.query('SELECT COUNT(*) as cnt FROM radiology_catalog')).rows[0].cnt;
    console.log(`  ? Added ${added} extra radiology exams ? Total: ${newTotal}`);
}

module.exports = { addExtraLabTests, addExtraRadiology };

// Run directly if executed
```

```
if (require.main === module) {
    (async () => {
        await addExtraLabTests();
        await addExtraRadiology();
        process.exit(0);
    })();
}



========================================
FILE: seed_services_pg.js
========================================

// seed_services_pg.js ? Medical services (338) + base drug catalog (108)
const { pool } = require('./db_postgres');

async function populateMedicalServices() {
    const r = await pool.query('SELECT COUNT(*) as cnt FROM medical_services');
    if (parseInt(r.rows[0].cnt) > 0) return;
    const svcs = [
        ['General Consultation', '??????? ????', 'General Practice', 'Consultation', 150], ['Follow-up Visit',
 '????? ??????', 'General Practice', 'Consultation', 100], ['Comprehensive Checkup', '??? ????', 'General Prac
tice', 'Consultation', 300], ['Pre-employment Medical', '??? ?? ??? ???????', 'General Practice', 'Consultatio
n', 250], ['Wound Dressing', '????? ????', 'General Practice', 'Procedure', 80], ['Wound Suturing', '????? ???
', 'General Practice', 'Procedure', 200], ['Abscess Drainage', '????? ????', 'General Practice', 'Procedure',
300], ['Foreign Body Removal', '????? ??? ????', 'General Practice', 'Procedure', 250], ['Cauterization', '??'
, 'General Practice', 'Procedure', 200], ['IV Fluid Administration', '????? ?????? ??????', 'General Practice'
, 'Procedure', 150], ['IM/SC Injection', '??? ???? / ??? ?????', 'General Practice', 'Procedure', 50], ['Nebul
ization', '???? ?????', 'General Practice', 'Procedure', 80], ['ECG', '????? ???', 'General Practice', 'Diagno
stic', 100], ['Blood Pressure Monitoring', '?????? ??? ????', 'General Practice', 'Diagnostic', 50], ['Blood S
ugar Test (POCT)', '??? ??? ????', 'General Practice', 'Diagnostic', 30], ['Medical Report', '????? ???', 'Gen
eral Practice', 'Service', 100], ['Sick Leave Certificate', '????? ?????', 'General Practice', 'Service', 50],
 ['Fitness Certificate', '????? ?????', 'General Practice', 'Service', 100], ['Vaccination - Adult', '????? ??
????', 'General Practice', 'Procedure', 150], ['Allergy Test (Skin Prick)', '??? ?????? ????', 'General Practi
ce', 'Diagnostic', 200],
        ['Dental Consultation', '??????? ?????', 'Dentistry', 'Consultation', 150], ['Dental Follow-up', '????
?? ?????', 'Dentistry', 'Consultation', 80], ['Dental X-Ray (Periapical)', '???? ??', 'Dentistry', 'Diagnostic
', 80], ['Panoramic X-Ray (OPG)', '???? ????????', 'Dentistry', 'Diagnostic', 150], ['Dental Cleaning (Scaling
)', '????? ?????', 'Dentistry', 'Procedure', 200], ['Dental Polishing', '????? ?????', 'Dentistry', 'Procedure
', 100], ['Simple Tooth Extraction', '??? ?? ????', 'Dentistry', 'Procedure', 200], ['Surgical Tooth Extractio
n', '??? ?? ?????', 'Dentistry', 'Procedure', 500], ['Wisdom Tooth Extraction', '??? ??? ???', 'Dentistry', 'P
rocedure', 600], ['Composite Filling (1 surface)', '???? ????? ??? ????', 'Dentistry', 'Procedure', 200], ['Co
mposite Filling (2 surfaces)', '???? ????? ?????', 'Dentistry', 'Procedure', 300], ['Composite Filling (3 surf
aces)', '???? ????? ???? ????', 'Dentistry', 'Procedure', 400], ['Amalgam Filling', '???? ?????', 'Dentistry',
 'Procedure', 150], ['Temporary Filling', '???? ?????', 'Dentistry', 'Procedure', 100], ['Root Canal - Anterio
r', '???? ??? ?????', 'Dentistry', 'Procedure', 800], ['Root Canal - Premolar', '???? ??? ????', 'Dentistry',
 'Procedure', 1000], ['Root Canal - Molar', '???? ??? ???', 'Dentistry', 'Procedure', 1200], ['Root Canal Re-tr
eatment', '????? ???? ???', 'Dentistry', 'Procedure', 1500], ['Post & Core', '????? ???', 'Dentistry', 'Proced
ure', 500], ['PFM Crown', '??? ??? ?????', 'Dentistry', 'Procedure', 800], ['Zirconia Crown', '??? ???????', '
Dentistry', 'Procedure', 1200], ['E-Max Crown', '??? ??????', 'Dentistry', 'Procedure', 1400], ['Temporary Cro
wn', '??? ????', 'Dentistry', 'Procedure', 200], ['Dental Bridge (per unit)', '??? ????? (??????)', 'Dentistry
', 'Procedure', 800], ['Porcelain Veneer', '???? ?????', 'Dentistry', 'Procedure', 1500], ['Composite Veneer',
 '???? ?????', 'Dentistry', 'Procedure', 600], ['Complete Denture (Upper)', '??? ????? ???? ????', 'Dentistry'
```

, 'Procedure', 2000], ['Complete Denture (Lower)', '??? ????? ???? ????', 'Dentistry', 'Procedure', 2000], ['P
artial Denture (Acrylic)', '??? ???? ???????', 'Dentistry', 'Procedure', 1200], ['Partial Denture (Metal Frame
)', '??? ???? ?????', 'Dentistry', 'Procedure', 2500], ['Dental Implant (Single)', '????? ?? ????', 'Dentistry
', 'Procedure', 4000], ['Implant Abutment', '????? ????', 'Dentistry', 'Procedure', 1500], ['Implant Crown', '
??? ??? ????', 'Dentistry', 'Procedure', 1500], ['Gingivectomy', '?? ???', 'Dentistry', 'Procedure', 400], ['G
um Treatment (Curettage)', '???? ??? (???)', 'Dentistry', 'Procedure', 300], ['Frenectomy', '??? ????', 'Denti
stry', 'Procedure', 400], ['Teeth Whitening (Office)', '????? ????? ?????', 'Dentistry', 'Procedure', 1000], [
'Teeth Whitening (Home Kit)', '????? ????? ?????', 'Dentistry', 'Procedure', 500], ['Fluoride Application', '?
???? ???????', 'Dentistry', 'Procedure', 100], ['Sealant (per tooth)', '???? ???? (????)', 'Dentistry', 'Proce
dure', 100], ['Orthodontic Consultation', '??????? ?????', 'Dentistry', 'Consultation', 200], ['Metal Braces (
Full)', '????? ????? ????', 'Dentistry', 'Procedure', 8000], ['Ceramic Braces (Full)', '????? ???? ????', 'Den
tistry', 'Procedure', 10000], ['Clear Aligners (Invisalign)', '????? ????', 'Dentistry', 'Procedure', 15000],
['Retainer', '???? ?????', 'Dentistry', 'Procedure', 500], ['Space Maintainer', '???? ?????', 'Dentistry', 'Pr
ocedure', 400], ['Pulpotomy (Pediatric)', '??? ?? (?????)', 'Dentistry', 'Procedure', 300], ['Stainless Steel
Crown (Pediatric)', '??? ????? ?????', 'Dentistry', 'Procedure', 400], ['Night Guard', '???? ????', 'Dentistry
', 'Procedure', 600], ['Sport Guard', '???? ?????', 'Dentistry', 'Procedure', 400], ['TMJ Treatment', '???? ??
?? ????', 'Dentistry', 'Procedure', 500], ['Incision & Drainage (Dental)', '?? ?????? ???', 'Dentistry', 'Proc
edure', 300],

['Internal Medicine Consultation', '??????? ??????', 'Internal Medicine', 'Consultation', 200], ['Foll
ow-up Visit', '????? ?????? ??????', 'Internal Medicine', 'Consultation', 150], ['Diabetes Management', '?????
??????', 'Internal Medicine', 'Consultation', 200], ['Hypertension Management', '????? ??? ????', 'Internal M
edicine', 'Consultation', 200], ['Thyroid Assessment', '????? ????? ???????', 'Internal Medicine', 'Consultati
on', 200], ['Liver Disease Management', '????? ????? ?????', 'Internal Medicine', 'Consultation', 250], ['Kidn
ey Disease Management', '????? ????? ?????', 'Internal Medicine', 'Consultation', 250], ['Rheumatology Consult
ation', '??????? ????????', 'Internal Medicine', 'Consultation', 250], ['Holter Monitor Setup', '????? ?????',
'Internal Medicine', 'Diagnostic', 300], ['Spirometry', '??? ????? ?????', 'Internal Medicine', 'Diagnostic',
200], ['Pleural Tap (Thoracentesis)', '??? ????', 'Internal Medicine', 'Procedure', 500], ['Ascitic Tap (Para
centesis)', '??? ????', 'Internal Medicine', 'Procedure', 500], ['Joint Aspiration', '??? ????', 'Internal Med
icine', 'Procedure', 400], ['Bone Marrow Biopsy', '???? ???? ???', 'Internal Medicine', 'Procedure', 800],

['Cardiology Consultation', '??????? ???', 'Cardiology', 'Consultation', 300], ['Cardiology Follow-up'
, '?????? ???', 'Cardiology', 'Consultation', 200], ['ECG (12-Lead)', '????? ??? ???????', 'Cardiology', 'Diag
nostic', 100], ['Echocardiography', '???? ???', 'Cardiology', 'Diagnostic', 500], ['Stress ECG (Treadmill)', '
????? ??? ????????', 'Cardiology', 'Diagnostic', 400], ['Holter Monitor (24h)', '????? 24 ????', 'Cardiology',
'Diagnostic', 400], ['Ambulatory BP Monitor (24h)', '?????? ??? ?????', 'Cardiology', 'Diagnostic', 300], ['C
ardiac Catheterization', '????? ?????', 'Cardiology', 'Procedure', 5000], ['Pacemaker Check', '??? ???? ?????'
, 'Cardiology', 'Diagnostic', 300],

['Dermatology Consultation', '??????? ?????', 'Dermatology', 'Consultation', 200], ['Dermatology Follo
w-up', '?????? ?????', 'Dermatology', 'Consultation', 150], ['Skin Biopsy', '???? ???', 'Dermatology', 'Proced
ure', 400], ['Cryotherapy', '???? ????????', 'Dermatology', 'Procedure', 300], ['Electrocautery', '?? ???????'
, 'Dermatology', 'Procedure', 300], ['Mole Removal', '????? ????', 'Dermatology', 'Procedure', 500], ['Wart Re
moval', '????? ?????', 'Dermatology', 'Procedure', 300], ['Skin Tag Removal', '????? ????? ?????', 'Dermatolog
y', 'Procedure', 200], ['Acne Treatment', '???? ?? ??????', 'Dermatology', 'Procedure', 300], ['Chemical Peeli
ng', '????? ???????', 'Dermatology', 'Procedure', 500], ['Laser Treatment', '???? ???????', 'Dermatology', 'Pr
ocedure', 800], ['Laser Hair Removal (Session)', '????? ??? ???????', 'Dermatology', 'Procedure', 500], ['PRP
Injection (Skin/Hair)', '??? ??????', 'Dermatology', 'Procedure', 800], ['Botox Injection', '??? ??????', 'Der
matology', 'Procedure', 1200], ['Filler Injection', '??? ????', 'Dermatology', 'Procedure', 1500], ['Mesothera
py', '??????????', 'Dermatology', 'Procedure', 600], ['Vitiligo Treatment', '???? ????', 'Dermatology', 'Proce
dure', 400], ['Psoriasis Treatment', '???? ?????', 'Dermatology', 'Procedure', 400], ['Eczema Treatment', '???
? ??????', 'Dermatology', 'Procedure', 300], ['Fungal Infection Treatment', '???? ??????', 'Dermatology', 'Pro
cedure', 200], ['Patch Test (Allergy)', '??? ???? ??????', 'Dermatology', 'Diagnostic', 300], ['Wood Lamp Exam
ination', '??? ????? ???', 'Dermatology', 'Diagnostic', 100], ['Dermoscopy', '??? ??????????', 'Dermatology',
'Diagnostic', 150],

['Ophthalmology Consultation', '??????? ????', 'Ophthalmology', 'Consultation', 200], ['Ophthalmology
Follow-up', '?????? ????', 'Ophthalmology', 'Consultation', 150], ['Comprehensive Eye Exam', '??? ???? ????',

'Ophthalmology', 'Diagnostic', 300], ['Refraction Test', '??? ???', 'Ophthalmology', 'Diagnostic', 100], ['Tonometry (IOP)', '???? ??? ?????', 'Ophthalmology', 'Diagnostic', 100], ['Visual Field Test', '??? ???? ??????', 'Ophthalmology', 'Diagnostic', 200], ['Fundoscopy', '??? ??? ?????', 'Ophthalmology', 'Diagnostic', 150], ['OCT Scan', '????? ????? ?????', 'Ophthalmology', 'Diagnostic', 300], ['Fluorescein Angiography', '????? ????? ?????', 'Ophthalmology', 'Diagnostic', 400], ['Slit Lamp Examination', '??? ??????? ?????', 'Ophthalmology', 'Diagnostic', 100], ['Contact Lens Fitting', '????? ????? ?????', 'Ophthalmology', 'Service', 200], ['Foreign Body Removal (Eye)', '????? ??? ???? ?? ?????', 'Ophthalmology', 'Procedure', 200], ['Chalazion Excision', '???? ??? ???????', 'Ophthalmology', 'Procedure', 500], ['Pterygium Surgery', '????? ????', 'Ophthalmology', 'Procedure', 2000], ['Cataract Surgery (Phaco)', '????? ??? (????)', 'Ophthalmology', 'Procedure', 5000], ['LASIK Consultation', '???????? ????', 'Ophthalmology', 'Consultation', 300], ['LASIK Surgery', '????? ????', 'Ophthalmology', 'Procedure', 8000], ['Intravitreal Injection', '??? ???? ?????', 'Ophthalmology', 'Procedure', 2000], ['Glaucoma Screening', '??? ?????????', 'Ophthalmology', 'Diagnostic', 200], ['Diabetic Eye Screening', '??? ??? ????? ??????', 'Ophthalmology', 'Diagnostic', 250], ['Lacrimal Duct Probing', '????? ???? ?????', 'Ophthalmology', 'Procedure', 800], ['Eyelid Surgery', '????? ???', 'Ophthalmology', 'Procedure', 3000],

['ENT Consultation', '??????? ??? ???? ??????', 'ENT', 'Consultation', 200], ['ENT Follow-up', '?????? ??? ????', 'ENT', 'Consultation', 150], ['Audiometry', '??? ???', 'ENT', 'Diagnostic', 200], ['Tympanometry', '???? ???? ?????', 'ENT', 'Diagnostic', 150], ['Nasal Endoscopy', '????? ???', 'ENT', 'Diagnostic', 300], ['Laryngoscopy', '????? ?????', 'ENT', 'Diagnostic', 400], ['Ear Wax Removal (Irrigation)', '????? ??? ?????', 'ENT', 'Procedure', 150], ['Ear Wax Removal (Micro-suction)', '????? ??? ??????', 'ENT', 'Procedure', 200], ['Foreign Body Removal (Ear)', '????? ??? ???? ?? ?????', 'ENT', 'Procedure', 200], ['Foreign Body Removal (Nose)', '????? ??? ???? ?? ?????', 'ENT', 'Procedure', 200], ['Nasal Cauterization', '?? ????', 'ENT', 'Procedure', 250], ['Anterior Nasal Packing', '??? ???? ?????', 'ENT', 'Procedure', 200], ['Tonsillectomy', '??????? ??????', 'ENT', 'Procedure', 3000], ['Adenoidectomy', '??????? ???????', 'ENT', 'Procedure', 2500], ['Septoplasty', '????? ?????? ??????', 'ENT', 'Procedure', 5000], ['Turbinate Reduction', '????? ????????', 'ENT', 'Procedure', 3000], ['Myringotomy with Tube', '????? ????', 'ENT', 'Procedure', 2000], ['Tympanoplasty', '????? ????', 'ENT', 'Procedure', 5000], ['Hearing Aid Fitting', '????? ?????', 'ENT', 'Service', 500], ['Speech Therapy Session', '???? ???? ???', 'ENT', 'Therapy', 200], ['Vertigo Assessment', '????? ??????', 'ENT', 'Diagnostic', 250], ['Sleep Study Referral', '????? ????? ???', 'ENT', 'Service', 200],

['Orthopedics Consultation', '??????? ????', 'Orthopedics', 'Consultation', 200], ['Orthopedics Follow-up', '?????? ????', 'Orthopedics', 'Consultation', 150], ['Cast Application', '?????', 'Orthopedics', 'Procedure', 300], ['Cast Removal', '????? ???', 'Orthopedics', 'Procedure', 100], ['Splint Application', '????? ????', 'Orthopedics', 'Procedure', 200], ['Fracture Reduction (Closed)', '?? ??? ????', 'Orthopedics', 'Procedure', 800], ['Joint Injection', '??? ????', 'Orthopedics', 'Procedure', 400], ['Trigger Point Injection', '??? ??? ??????', 'Orthopedics', 'Procedure', 300], ['PRP Injection (Joint)', '??? ?????? ???????', 'Orthopedics', 'Procedure', 1000], ['Knee Aspiration', '??? ????', 'Orthopedics', 'Procedure', 400], ['Carpal Tunnel Release', '????? ??? ?????', 'Orthopedics', 'Procedure', 3000], ['Trigger Finger Release', '????? ???? ?????', 'Orthopedics', 'Procedure', 2000], ['ACL Reconstruction', '????? ???? ???? ?????', 'Orthopedics', 'Procedure', 15000], ['Meniscus Surgery', '????? ?????', 'Orthopedics', 'Procedure', 8000], ['Hip Replacement', '??????? ???? ???', 'Orthopedics', 'Procedure', 30000], ['Knee Replacement', '??????? ???? ????', 'Orthopedics', 'Procedure', 25000], ['Arthroscopy', '????? ????', 'Orthopedics', 'Procedure', 5000], ['Physical Therapy Session', '???? ???? ?????', 'Orthopedics', 'Therapy', 200], ['TENS Therapy', '???? ???????', 'Orthopedics', 'Therapy', 150], ['Ultrasound Therapy', '???? ????????', 'Orthopedics', 'Therapy', 150], ['Spinal Injection', '??? ?????? ??????', 'Orthopedics', 'Procedure', 1500], ['Bone Density Test (Referral)', '????? ??? ????? ???', 'Orthopedics', 'Service', 100],

['OB/GYN Consultation', '??????? ???? ??????', 'Obstetrics', 'Consultation', 200], ['OB/GYN Follow-up', '?????? ????', 'Obstetrics', 'Consultation', 150], ['Prenatal Visit', '????? ???', 'Obstetrics', 'Consultation', 200], ['Postpartum Visit', '????? ?? ??? ???????', 'Obstetrics', 'Consultation', 200], ['Pap Smear', '??? ??? ?????', 'Obstetrics', 'Diagnostic', 150], ['Obstetric Ultrasound', '????? ???', 'Obstetrics', 'Diagnostic', 250], ['Fetal Heart Monitoring (NST)', '?????? ??? ??????', 'Obstetrics', 'Diagnostic', 200], ['Colposcopy', '????? ??? ?????', 'Obstetrics', 'Diagnostic', 400], ['IUD Insertion', '????? ????', 'Obstetrics', 'Procedure', 500], ['IUD Removal', '????? ????', 'Obstetrics', 'Procedure', 300], ['Contraceptive Implant', '???? ??? ???', 'Obstetrics', 'Procedure', 800], ['Hormonal Injection (Contraceptive)', '???? ??? ???', 'Obstetrics', 'Procedure', 150], ['Cervical Biopsy', '???? ??? ?????', 'Obstetrics', 'Procedure', 500], ['Endometrial Biopsy', '???? ????? ?????', 'Obstetrics', 'Procedure', 600], ['Hysteroscopy', '????? ????', 'Obstetrics', 'Procedure'

, 3000], ['D&C (Dilation & Curettage)', '??? ???', 'Obstetrics', 'Procedure', 2000], ['Cesarean Section', '??? ?? ??????', 'Obstetrics', 'Procedure', 8000], ['Normal Delivery', '????? ??????', 'Obstetrics', 'Procedure', 5000], ['Episiotomy Repair', '????? ?? ?????', 'Obstetrics', 'Procedure', 500], ['Breast Examination', '??? ???', 'Obstetrics', 'Diagnostic', 150], ['Fertility Consultation', '??????? ?????', 'Obstetrics', 'Consultation', 300], ['Ovulation Induction', '????? ?????', 'Obstetrics', 'Procedure', 500], ['Polycystic Ovary Treatment', '???? ???? ???????', 'Obstetrics', 'Consultation', 250],

['Pediatric Consultation', '??????? ?????', 'Pediatrics', 'Consultation', 150], ['Pediatric Follow-up', '?????? ?????', 'Pediatrics', 'Consultation', 100], ['Well-Baby Visit', '????? ??? ????', 'Pediatrics', 'Consultation', 150], ['Newborn Examination', '??? ???? ?????', 'Pediatrics', 'Consultation', 200], ['Vaccination (Standard)', '????? ?????', 'Pediatrics', 'Procedure', 100], ['Vaccination (Optional)', '????? ???????', 'Pediatrics', 'Procedure', 150], ['Growth Assessment', '????? ???', 'Pediatrics', 'Diagnostic', 100], ['Developmental Screening', '??? ????', 'Pediatrics', 'Diagnostic', 150], ['Pediatric Nebulization', '????? ?????', 'Pediatrics', 'Procedure', 80], ['Pediatric IV Fluid', '?????? ?????? ?????', 'Pediatrics', 'Procedure', 150], ['Circumcision', '????', 'Pediatrics', 'Procedure', 1000], ['Tongue Tie Release', '??? ???? ??????', 'Pediatrics', 'Procedure', 500], ['Allergy Testing (Pediatric)', '??? ?????? ?????', 'Pediatrics', 'Diagnostic', 300], ['Hearing Screening (Newborn)', '??? ??? ????? ???????', 'Pediatrics', 'Diagnostic', 200], ['Jaundice Screening', '??? ?????', 'Pediatrics', 'Diagnostic', 100],

['Neurology Consultation', '??????? ?????', 'Neurology', 'Consultation', 300], ['Neurology Follow-up', '?????? ?????', 'Neurology', 'Consultation', 200], ['EEG (Electroencephalogram)', '????? ????', 'Neurology', 'Diagnostic', 400], ['EMG / NCS', '????? ????? ??????', 'Neurology', 'Diagnostic', 500], ['Lumbar Puncture', '??? ????', 'Neurology', 'Procedure', 800], ['Botox for Migraine', '?????? ?????? ??????', 'Neurology', 'Procedure', 1500], ['Nerve Block', '???? ????', 'Neurology', 'Procedure', 600], ['Epilepsy Management', '????? ?????', 'Neurology', 'Consultation', 250], ['Stroke Assessment', '????? ???? ??????', 'Neurology', 'Diagnostic', 400], ['Memory Assessment', '????? ?????', 'Neurology', 'Diagnostic', 300], ['Headache Clinic', '????? ????', 'Neurology', 'Consultation', 250],

['Psychiatry Consultation', '??????? ?????', 'Psychiatry', 'Consultation', 300], ['Psychiatry Follow-up', '?????? ?????', 'Psychiatry', 'Consultation', 200], ['Psychological Assessment', '????? ????', 'Psychiatry', 'Diagnostic', 500], ['Cognitive Behavioral Therapy', '???? ????? ?????', 'Psychiatry', 'Therapy', 300], ['Psychotherapy Session', '???? ???? ????', 'Psychiatry', 'Therapy', 300], ['Couple/Family Therapy', '???? ????', 'Psychiatry', 'Therapy', 400], ['ADHD Assessment', '????? ??? ??????', 'Psychiatry', 'Diagnostic', 400], ['Addiction Counseling', '??????? ?????', 'Psychiatry', 'Therapy', 300], ['Psychiatric Report', '????? ????', 'Psychiatry', 'Service', 200],

['Urology Consultation', '??????? ????? ?????', 'Urology', 'Consultation', 200], ['Urology Follow-up', '?????? ?????', 'Urology', 'Consultation', 150], ['Cystoscopy', '????? ?????', 'Urology', 'Diagnostic', 1500], ['Urodynamic Study', '????? ????????? ?????', 'Urology', 'Diagnostic', 800], ['Prostate Exam (DRE)', '??? ??????', 'Urology', 'Diagnostic', 150], ['Urethral Dilation', '????? ?????', 'Urology', 'Procedure', 500], ['Catheter Insertion/Removal', '?????/????? ?????', 'Urology', 'Procedure', 200], ['Circumcision (Adult)', '??????', 'Urology', 'Procedure', 1500], ['Vasectomy', '??? ????? ?????', 'Urology', 'Procedure', 3000], ['ESWL (Kidney Stone)', '????? ?????', 'Urology', 'Procedure', 3000], ['Kidney Stone Management', '????? ????? ?????', 'Urology', 'Consultation', 250],

['Endocrinology Consultation', '??????? ??? ????', 'Endocrinology', 'Consultation', 250], ['Endocrinology Follow-up', '?????? ???', 'Endocrinology', 'Consultation', 200], ['Diabetes Education', '????? ????', 'Endocrinology', 'Service', 150], ['Insulin Pump Assessment', '????? ???? ???????', 'Endocrinology', 'Diagnostic', 400], ['Thyroid Nodule FNA', '???? ???? ?????', 'Endocrinology', 'Procedure', 800], ['Continuous Glucose Monitor', '???? ??? ?????', 'Endocrinology', 'Service', 500], ['Growth Hormone Assessment', '????? ????? ???', 'Endocrinology', 'Diagnostic', 300], ['Osteoporosis Management', '????? ????? ??????', 'Endocrinology', 'Consultation', 200], ['Adrenal Assessment', '????? ??? ?????', 'Endocrinology', 'Diagnostic', 300], ['Pituitary Assessment', '????? ??? ??????', 'Endocrinology', 'Diagnostic', 300],

['GI Consultation', '??????? ???? ????', 'Gastroenterology', 'Consultation', 250], ['GI Follow-up', '????? ???? ????', 'Gastroenterology', 'Consultation', 180], ['Upper GI Endoscopy (OGD)', '????? ???? ????', 'Gastroenterology', 'Procedure', 2000], ['Colonoscopy', '????? ?????', 'Gastroenterology', 'Procedure', 2500], ['Liver Biopsy', '???? ???', 'Gastroenterology', 'Procedure', 1500], ['H. Pylori Breath Test', '??? ??? ?????? ??????', 'Gastroenterology', 'Diagnostic', 200], ['FibroScan', '?????????? ???', 'Gastroenterology', 'Diagnostic', 500], ['Hemorrhoid Treatment', '???? ??????', 'Gastroenterology', 'Procedure', 1000], ['Polypectomy', '??

```
????? ?????', 'Gastroenterology', 'Procedure', 1500], ['PEG Tube Insertion', '????? ????? ?????', 'Gastroenterology', 'Procedure', 3000], ['IBS Management', '????? ??????? ??????', 'Gastroenterology', 'Consultation', 200], ['Celiac Disease Screening', '??? ?????? ?????', 'Gastroenterology', 'Diagnostic', 250],
        ['Pulmonology Consultation', '??????? ?????', 'Pulmonology', 'Consultation', 250], ['Pulmonology Follow-up', '?????? ?????', 'Pulmonology', 'Consultation', 180], ['Spirometry (PFT)', '??? ????? ???', 'Pulmonology', 'Diagnostic', 200], ['Bronchoscopy', '????? ????', 'Pulmonology', 'Procedure', 2500], ['Chest Tube Insertion', '????? ????? ????', 'Pulmonology', 'Procedure', 1500], ['Pleural Biopsy', '???? ?????', 'Pulmonology', 'Procedure', 1000], ['Asthma Management', '????? ???', 'Pulmonology', 'Consultation', 200], ['COPD Management', '????? ?????? ????', 'Pulmonology', 'Consultation', 200], ['Sleep Study Interpretation', '????? ????? ???', 'Pulmonology', 'Diagnostic', 400], ['Oxygen Therapy Assessment', '????? ???? ??????', 'Pulmonology', 'Diagnostic', 200],
        ['Nephrology Consultation', '??????? ???', 'Nephrology', 'Consultation', 250], ['Nephrology Follow-up', '?????? ???', 'Nephrology', 'Consultation', 180], ['Dialysis Access Assessment', '????? ???? ????', 'Nephrology', 'Diagnostic', 300], ['Kidney Biopsy', '???? ???', 'Nephrology', 'Procedure', 2000], ['Dialysis Session', '???? ???? ???', 'Nephrology', 'Procedure', 1000], ['Peritoneal Dialysis Setup', '????? ???? ???????', 'Nephrology', 'Procedure', 1500], ['Electrolyte Management', '????? ???????', 'Nephrology', 'Consultation', 200],
        ['Surgery Consultation', '??????? ?????', 'Surgery', 'Consultation', 200], ['Surgery Follow-up', '????? ?????', 'Surgery', 'Consultation', 150], ['Minor Surgery', '????? ????', 'Surgery', 'Procedure', 1000], ['Lipoma Excision', '??????? ??? ????', 'Surgery', 'Procedure', 1500], ['Sebaceous Cyst Excision', '??????? ??? ????', 'Surgery', 'Procedure', 1000], ['Hernia Repair', '????? ???', 'Surgery', 'Procedure', 5000], ['Appendectomy', '??????? ?????', 'Surgery', 'Procedure', 5000], ['Cholecystectomy (Lap)', '??????? ????? ????????', 'Surgery', 'Procedure', 8000], ['Hemorrhoidectomy', '??????? ??????', 'Surgery', 'Procedure', 3000], ['Anal Fissure Surgery', '????? ??? ????', 'Surgery', 'Procedure', 2000], ['Pilonidal Sinus Surgery', '????? ??? ????', 'Surgery', 'Procedure', 3000], ['Breast Lump Excision', '??????? ???? ???', 'Surgery', 'Procedure', 3000], ['Thyroidectomy', '??????? ??? ?????', 'Surgery', 'Procedure', 8000], ['Wound Debridement', '????? ??? ?????', 'Surgery', 'Procedure', 500], ['Drain Insertion/Removal', '?????/????? ?????', 'Surgery', 'Procedure', 300], ['Skin Graft', '????? ???', 'Surgery', 'Procedure', 4000], ['Varicose Vein Surgery', '????? ?????', 'Surgery', 'Procedure', 5000],
        ['Oncology Consultation', '??????? ?????', 'Oncology', 'Consultation', 400], ['Oncology Follow-up', '????? ?????', 'Oncology', 'Consultation', 300], ['Chemotherapy Session', '???? ??????', 'Oncology', 'Procedure', 3000], ['Tumor Marker Review', '?????? ?????? ?????', 'Oncology', 'Diagnostic', 200], ['Port-a-Cath Care', '????? ???? ?????', 'Oncology', 'Procedure', 300], ['Bone Marrow Aspirate', '??? ???? ???', 'Oncology', 'Procedure', 1000], ['Cancer Screening Package', '???? ??? ?????', 'Oncology', 'Diagnostic', 500],
        ['Physiotherapy Assessment', '????? ???? ?????', 'Physiotherapy', 'Consultation', 200], ['Physiotherapy Session', '???? ???? ?????', 'Physiotherapy', 'Therapy', 200], ['Manual Therapy', '???? ????', 'Physiotherapy', 'Therapy', 200], ['Hydrotherapy', '???? ????', 'Physiotherapy', 'Therapy', 250], ['Post-Surgical Rehab', '????? ??? ?????', 'Physiotherapy', 'Therapy', 250], ['Sports Injury Rehab', '????? ?????? ??????', 'Physiotherapy', 'Therapy', 250], ['Back Pain Program', '?????? ???? ?????', 'Physiotherapy', 'Therapy', 200], ['Neck Pain Program', '?????? ???? ??????', 'Physiotherapy', 'Therapy', 200], ['Stroke Rehabilitation', '????? ???? ????', 'Physiotherapy', 'Therapy', 300],
        ['Nutrition Consultation', '??????? ?????', 'Nutrition', 'Consultation', 200], ['Nutrition Follow-up', '?????? ?????', 'Nutrition', 'Consultation', 150], ['Weight Management Program', '?????? ????? ???', 'Nutrition', 'Service', 300], ['Diabetes Diet Program', '?????? ????? ??????', 'Nutrition', 'Service', 250], ['Sports Nutrition Plan', '????? ??????', 'Nutrition', 'Service', 250], ['Body Composition Analysis', '????? ????? ????', 'Nutrition', 'Diagnostic', 100],
        ['Emergency Consultation', '??????? ?????', 'Emergency', 'Consultation', 300], ['Resuscitation', '????', 'Emergency', 'Procedure', 1000], ['Fracture Splinting', '????? ???? ?????', 'Emergency', 'Procedure', 300], ['Laceration Repair', '????? ??? ?????', 'Emergency', 'Procedure', 300], ['Burn Dressing', '???? ????', 'Emergency', 'Procedure', 200], ['Poisoning Management', '????? ????', 'Emergency', 'Procedure', 500], ['Snake/Insect Bite Treatment', '???? ?????', 'Emergency', 'Procedure', 300],
    ];
    const client = await pool.connect();
    try {
        await client.query('BEGIN');
```

```
        for (const [en, ar, sp, cat, pr] of svcs) {
            await client.query('INSERT INTO medical_services (name_en,name_ar,specialty,category,price) VALUES ($1,$2,$3,$4,$5)', [en, ar, sp, cat, pr]);
        }
        await client.query('COMMIT');
    } catch (e) { await client.query('ROLLBACK'); throw e; } finally { client.release(); }
    console.log(`  ? Medical services: ${svcs.length} procedures`);
}

async function populateBaseDrugs() {
    const r = await pool.query('SELECT COUNT(*) as cnt FROM pharmacy_drug_catalog');
    if (parseInt(r.rows[0].cnt) > 0) return;
    const drugs = [
        ['Panadol 500mg (Paracetamol)', 'Analgesic', 8, 200], ['Fevadol 500mg (Paracetamol)', 'Analgesic', 6, 200], ['Brufen 400mg (Ibuprofen)', 'NSAID', 14, 150], ['Brufen 600mg (Ibuprofen)', 'NSAID', 20, 100], ['Profenal 400mg (Ibuprofen)', 'NSAID', 15, 120], ['Voltaren 50mg (Diclofenac)', 'NSAID', 19, 130], ['Voltaren Emulgel 1% 100gm', 'NSAID', 26, 80], ['Cataflam 50mg (Diclofenac Potassium)', 'NSAID', 18, 100], ['Catafast 50mg Sachet 9pcs', 'NSAID', 18, 90], ['Rapidus 50mg (Diclofenac Potassium)', 'NSAID', 29, 80], ['Ponstan-Forte 500mg (Mefenamic Acid)', 'NSAID', 15, 90], ['Aspirin Protect 100mg', 'Blood Thinner', 10, 200], ['Jusprin 81mg (Aspirin)', 'Blood Thinner', 8, 250], ['Roxonin 60mg', 'NSAID', 30, 60], ['Advil Liquid Caps 200mg', 'NSAID', 27, 70], ['Panadol Extra Tablet 24pcs', 'Pain Relief', 8, 150], ['Panadol Night 20 Caplets', 'Pain Relief', 12, 100], ['Panadol Advance 24 Tablets', 'Pain Relief', 6, 200], ['Panadol Actifast 20 Tablets', 'Pain Relief', 9, 120], ['Panadol Extend 24 Tablets', 'Pain Relief', 16, 100], ['Panadol Cold & Flu Night 24 Caplets', 'Cough & Cold', 12, 100], ['Panadol Cold & Flu Sinus 24 Caplets', 'Cough & Cold', 14, 100], ['Panadol Cold & Flu All In One 24s', 'Cough & Cold', 27, 80], ['Solpadeine Soluble Tablet 20pcs', 'Pain Relief', 13, 80], ['Solpadeine Capsule 20pcs', 'Pain Relief', 13, 80], ['Adol Paracetamol 500mg 24 Caplets', 'Pain Relief', 5, 200], ['Adol-Extra Caplet 24pcs', 'Pain Relief', 6, 150], ['Fevadol-Extra Tablet 20pcs', 'Pain Relief', 6, 150], ['Fevadol-Plus Tablet 20pcs', 'Pain Relief', 10, 120], ['Salonpas Patches Small 20pcs', 'Pain Relief', 13, 100], ['Salonpas Patches Large 2pcs', 'Pain Relief', 11, 100], ['Relaxon Capsule 30pcs', 'Pain Relief', 28, 60], ['Reparil 20mg Tablet 40pcs', 'Pain Relief', 21, 70], ['Nexium 40mg (Esomeprazole)', 'PPI / Antacid', 65, 80], ['Pariet 20mg (Rabeprazole)', 'PPI / Antacid', 55, 70], ['Gaviscon Advance (Sodium Alginate)', 'Antacid', 22, 100], ['Ezora 40mg Esomeprazole 28 Caps', 'PPI / Antacid', 66, 60], ['Augmentin 1g (Amoxicillin/Clavulanate)', 'Antibiotic', 45, 100], ['Klavox 1g (Amoxicillin/Clavulanate)', 'Antibiotic', 40, 100], ['Amoxil 500mg (Amoxicillin)', 'Antibiotic', 15, 200], ['Suprax 400mg (Cefixime)', 'Antibiotic', 55, 80], ['Zinnat 500mg (Cefuroxime)', 'Antibiotic', 50, 80], ['Zithromax 500mg (Azithromycin)', 'Antibiotic', 35, 100], ['Ciprofloxacin 500mg', 'Antibiotic', 20, 120], ['Tavanic 500mg (Levofloxacin)', 'Antibiotic', 65, 60], ['Flagyl 500mg (Metronidazole)', 'Antiprotozoal', 12, 150], ['Lipitor 20mg (Atorvastatin)', 'Cholesterol', 45, 80], ['Crestor 10mg (Rosuvastatin)', 'Cholesterol', 55, 80], ['Glucophage 500mg (Metformin)', 'Diabetes', 15, 200], ['Diamicron MR 60mg (Gliclazide)', 'Diabetes', 35, 100], ['Januvia 100mg (Sitagliptin)', 'Diabetes', 95, 60], ['Amaryl 2mg (Glimepiride)', 'Diabetes', 25, 100], ['Concor 5mg (Bisoprolol)', 'Blood Pressure', 30, 100], ['Diovan 160mg (Valsartan)', 'Blood Pressure', 55, 80], ['Exforge 5/160mg (Amlodipine/Valsartan)', 'Blood Pressure', 65, 60], ['Micardis 80mg (Telmisartan)', 'Blood Pressure', 55, 80], ['Amlor 5mg (Amlodipine)', 'Blood Pressure', 20, 120], ['Lasix 40mg (Furosemide)', 'Diuretic', 8, 200], ['Zyrtec 10mg (Cetirizine)', 'Antihistamine', 15, 150], ['Clarinex 5mg (Desloratadine)', 'Antihistamine', 20, 120], ['Aerius 5mg (Desloratadine)', 'Antihistamine', 25, 100], ['Telfast 120mg (Fexofenadine)', 'Antihistamine', 22, 120], ['Singulair 10mg (Montelukast)', 'Asthma', 40, 80], ['Symbicort 160/4.5 (Budesonide/Formoterol)', 'Asthma Inhaler', 95, 50], ['Ventolin Evohaler 100mcg (Salbutamol)', 'Asthma Inhaler', 18, 150], ['Eltroxin 50mcg (Thyroxine)', 'Thyroid', 12, 200], ['Cortiment 9mg (Budesonide)', 'Corticosteroid', 60, 50], ['Predo 5mg (Prednisolone)', 'Corticosteroid', 10, 150], ['Rinza (Paracetamol/Pseudoephedrine)', 'Cold & Flu', 15, 100], ['Fludrex Tablet 24pcs', 'Cold & Flu', 12, 120], ['Prof Cold & Flu Caplet 20pcs', 'Cold & Flu', 12, 100], ['Flutab Tablet 30pcs', 'Cold & Flu', 15, 100], ['Flutab-Sinus Tablet 20pcs', 'Cold & Flu', 9, 100], ['Beatswell Probiotic 60 Capsules', 'Digestive Care', 29, 50], ['Bio Gaia Protectis Baby Drops 5ml', 'Digestive Care', 64, 40], ['Beatswell Multivitamins for Adults 60 Gummies', 'Vitamins & Supplements', 49, 60], ['Beatswell Kids Multivitamins 60 Gummies', 'Vitamins & Supplements', 37, 70], ['Sanotact Multivitamin 20 Effervescent', 'Vitamins & Supplements', 25, 80], ['Voltaren 100mg Suppository 5pcs', 'NSAID', 18, 60], [
```

```
'Voltaren 50mg Suppository 10pcs', 'NSAID', 20, 60], ['Procto Glyvenol Cream 30gm', 'Hemorrhoids', 35, 50], ['Disprin 81mg Tablet 100pcs', 'Blood Thinner', 25, 80], ['Panadrex Paracetamol 500mg 48 Tablets', 'Pain Relief', 9, 100], ['Divido 75mg Capsule 20pcs', 'NSAID', 31, 60], ['Rofenac 50mg Tablet 20pcs', 'NSAID', 20, 80], ['Rofenac-D 50mg Dispersable 20pcs', 'NSAID', 30, 70], ['Emifenac 50mg Tablet 20pcs', 'NSAID', 23, 80], ['Sapofen 400mg Tablet 30pcs', 'NSAID', 14, 90], ['Sapofen 600mg Tablet 30pcs', 'NSAID', 17, 80], ['Fast Flam 50mg Tablet 20pcs', 'NSAID', 27, 70],
    ];
    const client = await pool.connect();
    try {
        await client.query('BEGIN');
        for (const [n, c, p, s] of drugs) {
            await client.query('INSERT INTO pharmacy_drug_catalog (drug_name,category,selling_price,stock_qty) VALUES ($1,$2,$3,$4)', [n, c, p, s]);
        }
        await client.query('COMMIT');
    } catch (e) { await client.query('ROLLBACK'); throw e; } finally { client.release(); }
    console.log(`  ? Base drugs: ${drugs.length} items`);
}

module.exports = { populateMedicalServices, populateBaseDrugs };
```